



116421 字 | 292 分钟

大模型技术30讲（英文&中文批注）

大模型技术30讲（原版），30 Essential Questions and Answers on Machine Learning and AI

1.背景

买了一本《大模型技术30讲》，简单阅读了下，要点突出，对于入门、加深关键点理解，很有用。

但是，也存在问题 & 未满足的诉求：

- 1.《大模型技术30讲》纸质版（2025年3月第2次印刷），印刷质量，偏差；
- 2.大部分 术语，都翻译为中文，不利于中英对比，特别是 AI 领域基本都是英文的，需要我们熟悉 英文术语。
- 3.希望有 电子版 资料，方便利用 AI 工具辅助理解，提升效率。

因此，找到 [原始文档：Machine Learning Q and AI](#)，反复消化原始信息。

然后，将原始信息，转录为 md 格式，并且，存储在 github 上。

2.项目介绍

本项目是系统性学习大模型技术要点的教程，基于原始的《30 Essential Questions and Answers on Machine Learning and AI》，进行中文批注，增强可读性。

2.1.你将收获什么？

1. 系统性学习：大模型技术要点的教程
2. 熟悉核心的术语（英文 + 中文）
3. 共同维护，中文批注，增强可读性，贡献给开源社区



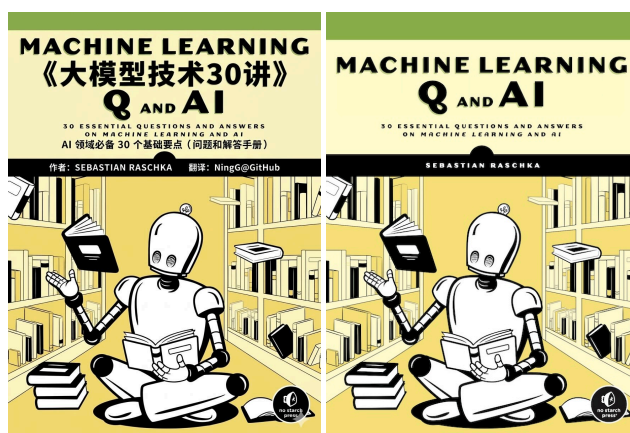
2.2.迭代计划

1. 首版：中文批注:
2. 导出 pdf 文件:
3. 同步到多个开源社区: TODO

3.在线阅读

在线阅读: [《大模型技术30讲》](#) , pdf 文件: [《大模型技术30讲-PDF版本》](#)

深入理解 LLM 核心原理, 直击要点



4.如何贡献

我们欢迎任何形式的贡献!

- 🐛 报告 Bug - 发现问题请提交 [Issue](#)
- 💡 功能建议 - 有好想法就告诉我们
- 📝 内容完善 - 帮助改进教程内容

Note: 后续资料会附上贡献者名单.

30 Essential Questions and Answers on Machine Learning and AI



Machine learning and AI are moving at a rapid pace. Researchers and practitioners are constantly struggling to keep up with the breadth of concepts and techniques. This book provides bite-sized bits of knowledge for your journey from machine learning beginner to expert, covering topics from various machine learning areas. Even experienced machine learning researchers and practitioners will encounter something new that they can add to their arsenal of techniques.

What People Are Saying

"Sebastian has a gift for distilling complex, AI-related topics into practical takeaways that can be understood by anyone. His new book, Machine Learning Q and AI, is another great resource for AI practitioners of any level."? ""Cameron R. Wolfe, Writer of Deep (Learning) Focus

"Sebastian uniquely combines academic depth, engineering agility, and the ability to demystify complex ideas. He can go deep into any theoretical topics, experiment to validate new ideas, then explain them all to you in simple words. If you're starting your journey into machine learning, Sebastian is your guide."? ""Chip Huyen, Author of Designing Machine Learning Systems

"One could hardly ask for a better guide than Sebastian, who is, without exaggeration, the best machine learning educator currently in the field. On each page, Sebastian not only imparts his extensive knowledge but also shares the passion and curiosity that mark true expertise."? ""Chris Albon, Director of Machine Learning, The Wikimedia Foundation

"Sebastian Raschka's new book, Machine Learning Q and AI, is a one-stop shop for overviews of crucial AI topics beyond the core covered in most introductory courses"|If you have already stepped into the world of AI via deep neural networks, then this book will give you what you need to locate and understand the next level."? ""Ronald T. Kneusel, author of How AI Works



Table of Contents

- [Introduction](#)

Part I: Neural Networks and Deep Learning

- [Chapter 1: Embeddings, Latent Space, and Representations](#)
- [Chapter 2: Self-Supervised Learning](#)
- [Chapter 3: Few-Shot Learning](#)
- [Chapter 4: The Lottery Ticket Hypothesis](#)
- [Chapter 5: Reducing Overfitting with Data](#)
- [Chapter 6: Reducing Overfitting with Model Modifications](#)
- [Chapter 7: Multi-GPU Training Paradigms](#)
- [Chapter 8: The Success of Transformers](#)
- [Chapter 9: Generative AI Models](#)
- [Chapter 10: Sources of Randomness](#)

Part II: Computer Vision

- [Chapter 11: Calculating the Number of Parameters](#)
- [Chapter 12: Fully Connected and Convolutional Layers](#)
- [Chapter 13: Large Training Sets for Vision Transformers](#)

Part III: Natural Language Processing

- [Chapter 14: The Distributional Hypothesis](#)
- [Chapter 15: Data Augmentation for Text](#)
- [Chapter 16: Self-Attention](#)
- [Chapter 17: Encoder- and Decoder-Style Transformers](#)
- [Chapter 18: Using and Fine-Tuning Pretrained Transformers](#)
- [Chapter 19: Evaluating Generative Large Language Models](#)

Part IV: Production and Deployment

- [Chapter 20: Stateless and Stateful Training](#)
- [Chapter 21: Data-Centric AI](#)



- [Chapter 22: Speeding Up Inference](#)
- [Chapter 23: Data Distribution Shifts](#)

Part V: Predictive Performance and Model Evaluation

- [Chapter 24: Poisson and Ordinal Regression](#)
- [Chapter 25: Confidence Intervals](#)
- [Chapter 26: Confidence Intervals vs. Conformal Predictions](#)
- [Chapter 27: Proper Metrics](#)
- [Chapter 28: The k in k-Fold Cross-Validation](#)
- [Chapter 29: Training and Test Set Discordance](#)
- [Chapter 30: Limited Labeled Data](#)



Introduction

本页交代本书的写作背景、适合的读者、问答体例与阅读建议，并概述全书五大部分及各章主题，最后给出在线代码资源链接。

Thanks to rapid advancements in deep learning, we have seen a significant expansion of machine learning and AI in recent years.

近年来深度学习快速发展，机器学习与 AI 的应用范围也随之大幅扩展。

This progress is exciting if we expect these advancements to create new industries, transform existing ones, and improve the quality of life for people around the world. On the other hand, the constant emergence of new techniques can make it challenging and time-consuming to keep abreast of the latest developments. Nonetheless, staying current is essential for professionals and organizations that use these technologies.

若这些进展能催生新产业、改造旧业态并改善全球生活质量，那无疑是令人振奋的。另一方面，新技术层出不穷，跟进最新进展既费时又费力；但对使用这些技术的从业者与机构而言，保持更新仍然十分必要。

I wrote this book as a resource for readers and machine learning practitioners who want to advance their expertise in the field and learn about techniques that I consider useful and significant but that are often overlooked in traditional and introductory textbooks and classes. I hope you'll find this book a valuable resource for obtaining new insights and discovering new techniques you can implement in your work.

本书面向希望提升功力、并了解那些在传统入门教材中常被忽略、却十分重要的技术的读者与实践者。希望它能帮助你获得新洞见，并在工作中落地新技巧。

Tips: 本书会突出 **核心概念**，并且，会给出 **示例**，辅助理解。

Who Is This Book For?

本节对比市面上入门书与偏重数学专著的两极，说明本书的定位、预备知识要求，以及书中「机器学习」一词的用法。



Navigating the world of AI and machine learning literature can often feel like walking a tightrope, with most books positioned at either end: broad beginner's introductions or deeply mathematical treatises. This book illustrates and discusses important developments in these fields while staying approachable and not requiring an advanced math or coding background.

AI 与机器学习读物往往两极：要么面向初学者的泛览，要么高度数学化的专著。本书在介绍重要进展的同时保持可读，不要求高等数学或编程背景。

Tips: 本书，并不要求读者有高等数学知识、也无需编码背景。简单来说，普通的高中毕业，也可以流畅阅读。

This book is for people with some experience with machine learning who want to learn new concepts and techniques. It's ideal for those who have taken a beginner course in machine learning or deep learning or have read an equivalent introductory book on the topic. (Throughout this book, I will use *machine learning* as an umbrella term for machine learning, deep learning, and AI.)

本书适合已具备一定机器学习基础、希望学习新概念与技术的读者；修过一门入门课或读过同等入门书尤为合适。（全书用 *machine learning* 统称机器学习、深度学习与 AI。）

本书中，会使用 *机器学习* 作为 **统称**，包括机器学习、深度学习、AI。

What Will You Get Out of This Book?

本节说明本书的问答体例、插图与练习安排、所涉主题广度，以及本书并非数学或编程教材的定位。

This book adopts a unique Q&A style, where each brief chapter is structured around a central question related to fundamental concepts in machine learning, deep learning, and AI. Every question is followed by an explanation, with several illustrations and figures, as well as exercises to test your understanding. Many



chapters also include references for further reading. These bite-sized nuggets of information provide an enjoyable jumping-off point on your journey from machine learning beginner to expert.

本书采用问答体例：每章围绕机器学习、深度学习与 AI 中的一个核心问题展开，配有解释、多幅插图与自测练习，许多章末附延伸阅读。这些信息块便于你从入门走向更熟练。

The book covers a wide range of topics. It includes new insights about established architectures, such as convolutional networks, that allow you to utilize these technologies more effectively. It also discusses more advanced techniques, such as the inner workings of large language models (LLMs) and vision transformers. Even experienced machine learning researchers and practitioners will encounter something new to add to their arsenal of techniques.

全书主题覆盖面广：既有对卷积网络等成熟架构的新视角，也讨论 LLM、视觉 Transformer 等更前沿内容；即便有经验的研究者与实践者也能从中拾取可纳入工具箱的新知。

Tips: 本书，会介绍 **AI 领域** 的典型概念、知识，但不是数学或编码书籍。阅读时，无需证明或编码、突出易读性。

While this book will expose you to new concepts and ideas, it's not a math or coding book. You won't need to solve any proofs or run any code while reading. In other words, this book is a perfect travel companion or something you can read on your favorite reading chair with your morning coffee or tea.

本书重在概念与思想，不是数学或编程书：阅读时不必写证明或跑代码，适合旅途或晨间咖啡时随手翻阅。

How to Read This Book

本节说明各章可独立阅读与建议顺序阅读的理由、练习与参考文献的位置，以及全书五大部分的划分逻辑。

Each chapter of this book is designed to be self-contained, offering you the freedom to jump between topics as you wish. When a concept from one chapter is



explained in more detail in another, I've included chapter references you can follow to fill in gaps in your understanding.

各章自成一体，可按兴趣跳转；若某概念在别章有更细讲解，文中会给出章节引用以便补齐背景。

本书每个章节，都是独立的，你可以跳过一些章节，直接阅读你感兴趣的章节。

However, there's a strategic sequence to the chapters. For example, the early chapter on embeddings sets the stage for later discussions on self-supervised learning and few-shot learning. For the easiest reading experience and the most comprehensive grasp of the content, my recommendation is to approach the book from start to finish.

但章节之间仍有策略性顺序：例如先读嵌入有助于理解后续自监督与少样本学习。若要最省力地建立整体图景，建议从头到尾通读。

然而，本书的章节，是有 **顺序** 的，建议从前往后阅读；因为，把 **最通用的概念**，放在了最前章节。

Each chapter is accompanied by optional exercises for readers who want to test their understanding, with an answer key located at the end of the book. In addition, for any papers referenced in a chapter or further reading on that chapter's topic, you can find the complete citation information in that chapter's "References" section.

每章配有可选练习，书末附有答案要点；章内引用论文的完整书目信息见各章「References」。

The book is structured into five main parts centered on the most important topics in machine learning and AI today.

全书按当今机器学习与 AI 最重要的主题分为五大部分。

整体内容，分为 5 个部分，都是 **AI 领域** 的 **最重要的主题**。

Tips: 下面第一部分，是 **神经网络** 和 **深度学习** 的 **通用概念**，包含 嵌入、自监督学习、少样本学习、彩票假设、过拟合、多 GPU 训练范式等。



Part I: Neural Networks and Deep Learning covers questions about deep neural networks and deep learning that are not specific to a particular subdomain. For example, we discuss alternatives to supervised learning and techniques for reducing overfitting, which is a common problem when using machine learning models for real-world problems where data is limited.

第一部分：神经网络与深度学习 讨论不局限于某一子领域的深度网络与共性问题，例如监督学习之外的范式，以及数据有限时常见的过拟合与应对思路。

Chapter **[ch01]**: Embeddings, Latent Space, and Representations

Delves into the distinctions and similarities between embedding vectors, latent vectors, and representations. Elucidates how these concepts help encode information in the context of machine learning.

第 1 章：嵌入、潜空间与表示——辨析嵌入向量、潜向量与表示的异同，以及它们在机器学习中表示信息的方式。

Chapter **[ch02]**: Self-Supervised Learning

Focuses on self-supervised learning, a method that allows neural networks to utilize large, unlabeled datasets in a supervised manner.

第 2 章：自监督学习——让神经网络以「类监督」方式利用大规模无标签数据。

Chapter **[ch03]**: Few-Shot Learning

Introduces few-shot learning, a specialized supervised learning technique tailored for small training datasets.

第 3 章：少样本学习——面向极小训练集的专门监督学习设定。

Chapter **[ch04]**: The Lottery Ticket Hypothesis

Explores the idea that randomly initialized neural networks contain smaller, efficient subnetworks.

第 4 章：彩票假设——随机初始化的网络中可能存在更小却高效的子网络。

Chapter **[ch05]**: Reducing Overfitting with Data

Addresses the challenge of overfitting in machine learning, discussing strategies



centered on data augmentation and the use of unlabeled data to reduce overfitting.

第 5 章：用数据缓解过拟合——数据增强与利用无标签数据等策略。

Chapter [\[ch06\]](#): Reducing Overfitting with Model Modifications

Extends the conversation on overfitting, focusing on model-related solutions like regularization, opting for simpler models, and ensemble techniques.

第 6 章：用模型与训练改动缓解过拟合——正则、更小模型与集成等。

Chapter [\[ch07\]](#): Multi-GPU Training Paradigms

Explains various training paradigms for multi-GPU setups to accelerate model training, including data and model parallelism.

第 7 章：多 GPU 训练范式——数据并行、模型并行等加速手段。

Chapter [\[ch08\]](#): The Success of Transformers

Explores the popular transformer architecture, highlighting features like attention mechanisms, parallelization ease, and high parameter counts.

第 8 章：Transformer 的成功——注意力、易并行与高参数量等特点。

Chapter [\[ch09\]](#): Generative AI Models

Provides a comprehensive overview of deep generative models, which are used to produce various media forms, including images, text, and audio. Discusses the strengths and weaknesses of each model type.

第 9 章：生成式 AI 模型——图像、文本、音频等媒介的深度生成模型概览与优劣。

Chapter [\[ch10\]](#): Sources of Randomness

Addresses the various sources of randomness in the training of deep neural networks that may lead to inconsistent and non-reproducible results during both training and inference. While randomness can be accidental, it can also be intentionally introduced by design.

第 10 章：随机性的来源——训练与推理中导致结果不一致的多种随机因素，以及有意与无意引入的随机性。

Tips: 下面第二部分，是 **计算机视觉** 的 **典型概念**，包含 卷积神经网络、视觉变换器。



Part II: Computer Vision focuses on topics mainly related to deep learning but specific to computer vision, many of which cover convolutional neural networks and vision transformers.

第二部分：计算机视觉聚焦视觉任务中的深度学习，多涉及卷积网络与视觉Transformer。

Chapter **[ch11]**: Calculating the Number of Parameters

Explains the

procedure for determining the parameters in a convolutional neural network, which is useful for gauging a model's storage and memory requirements.

第 11 章：参数数量计算——估算卷积网络的存储与显存需求。

Chapter **[ch12]**: Fully Connected and Convolutional Layers

Illustrates the circumstances in which convolutional layers can seamlessly replace fully connected layers, which can be useful for hardware optimization or simplifying implementations.

第 12 章：全连接与卷积层——何种情形下卷积可替代全连接以利于硬件或实现简化。

Chapter **[ch13]**: Large Training Sets for Vision Transformers

Probes the rationale behind vision transformers requiring more extensive training sets compared to conventional convolutional neural networks.

第 13 章：视觉 Transformer 为何需要更大训练集——相对传统 CNN 的数据规模要求。

Tips: 下面第三部分，文本相关，是 **自然语言处理** 的 **典型概念**，包含 分布式假设、数据增强、自注意力、编码器-解码器式变换器、使用和微调预训练变换器、评估生成式大语言模型等。

Part III: Natural Language Processing covers topics around working with text, many of which are related to transformer architectures and self-attention.



第三部分：自然语言处理围绕文本与 Transformer、自注意力等主题。

Chapter [ch14]: The Distributional Hypothesis

Delves into the distributional hypothesis, a linguistic theory suggesting that words appearing in the same contexts tend to possess similar meanings, which has useful implications for training machine learning models.

第 14 章：分布式假设——语境相近则语义相近的语言学思想及其对模型训练的启示。

Chapter [ch15]: Data Augmentation for Text

Highlights the significance of data augmentation for text, a technique used to artificially increase dataset sizes, which can help with improving model performance.

第 15 章：文本数据增强——人工扩充语料以提升性能。

Chapter [ch16]: Self-Attention

Introduces self-attention, a mechanism allowing each segment of a neural network's input to refer to other parts. Self-attention is a key mechanism in modern large language models.

第 16 章：自注意力——输入各部分互相关注的机制，是现代 LLM 的核心组件之一。

Chapter [ch17]: Encoder- and Decoder-Style Transformers

Describes the nuances of encoder and decoder transformer architectures and explains which type of architecture is most useful for each language processing task.

第 17 章：编码器式与解码器式 Transformer——结构差异与适用任务。

Chapter [ch18]: Using and Fine-Tuning Pretrained Transformers

Explains different methods for fine-tuning pretrained large language models and discusses their strengths and weaknesses.

第 18 章：使用与微调预训练 Transformer——多种微调路径及其利弊。

Chapter [ch19]: Evaluating Generative Large Language Models

Lists prominent evaluation metrics for language models like Perplexity, BLEU, ROUGE, and BERTScore.



第 19 章：评估生成式大语言模型——Perplexity、BLEU、ROUGE、BERTScore 等常见指标。

Tips: 下面**第四部分**，是 **生产** 和 **部署** 的 **典型概念**，包含 无状态和有状态训练、数据分布偏移等。

Part IV: Production and Deployment covers questions pertaining to practical scenarios, such as increasing inference speeds and various types of distribution shifts.

第四部分：生产与部署涵盖推理加速、数据分布偏移等落地问题。

Chapter **[ch20]**: Stateless and Stateful Training

Distinguishes between stateless and stateful training methodologies used in deploying models.

第 20 章：无状态与有状态训练——部署场景下的两种训练/更新范式。

Chapter **[ch21]**: Data-Centric AI

Explores data-centric AI, which prioritizes refining datasets to enhance model performance. This approach contrasts with the conventional model-centric approach, which emphasizes improving model architectures or methods.

第 21 章：以数据为中心的 AI——通过打磨数据集提升性能，与以模型为中心路线的对照。

Chapter **[ch22]**: Speeding Up Inference

Introduces techniques to enhance the speed of model inference without tweaking the model's architecture or compromising accuracy.

第 22 章：加速推理——在不改架构、不明显牺牲精度的前提下提速。

Chapter **[ch23]**: Data Distribution Shifts

Post-deployment, AI models

may face discrepancies between training data and real-world data distributions, known as data distribution shifts. These shifts can deteriorate model performance. This chapter categorizes and elaborates on common shifts like covariate shift, concept drift, label shift, and domain shift.



第 23 章：数据分布偏移——上线后训练分布与真实分布不一致及其对性能的影响，并梳理协变量偏移、概念漂移、标签偏移、域偏移等。

Tips: 下面**第五部分**，是 **预测性能** 和 **模型评估** 的 **典型概念**，包含 泊松回归、置信区间、置信区间与一致性预测、交叉验证、训练和测试集不一致、有限标签数据等。

Part V: Predictive Performance and Model Evaluation dives deeper into various aspects of squeezing out predictive performance, such as changing the loss function, setting up k -fold cross-validation, and dealing with limited labeled data.

第五部分：预测性能与模型评估深入损失设计、 k 折交叉验证、标签稀缺等榨取预测性能的主题。

Chapter **[ch24]**: Poisson and Ordinal Regression

Highlights the differences between Poisson and ordinal regression. Poisson regression is suitable for count data that follows a Poisson distribution, like the number of colds contracted on an airplane. In contrast, ordinal regression caters to ordered categorical data without assuming equidistant categories, such as disease severity.

第 24 章：泊松回归与序数回归——计数数据与有序分类各自适用的设定与区别。

Chapter **[ch25]**: Confidence Intervals

Delves into methods for constructing confidence intervals for machine learning classifiers. Reviews the purpose of confidence intervals, discusses how they estimate unknown population parameters, and introduces techniques such as normal approximation intervals, bootstrapping, and retraining with various random seeds.

第 25 章：置信区间——分类器上构造置信区间的方法、意义及正态近似、自助法、多随机种子重训等。

Chapter **[ch26]**: Confidence Intervals vs. Conformal Predictions

Discusses the distinction between confidence intervals and conformal predictions and describes the latter as a tool for creating prediction intervals that cover actual outcomes with specific probability.



第 26 章：置信区间与 conformal prediction——二者差异及后者构造具有覆盖保证的预测区间。

Chapter [ch27]: Proper Metrics

Focuses on the essential properties of a proper metric in mathematics and computer science. Examines whether commonly used loss functions in machine learning, such as mean squared error and cross-entropy loss, satisfy these properties.

第 27 章：proper scoring rule（恰当评分）——数学与机器学习损失（如 MSE、交叉熵）是否满足恰当性。

Chapter [ch28]: The k in k -Fold Cross-Validation

Explores the role of the k in k -fold cross-validation and provides insight into the advantages and disadvantages of selecting a large k .

第 28 章： k 折交叉验证中的 k ——取大或取小的利弊。

Chapter [ch29]: Training and Test Set Discordance

Addresses the scenario where a model performs better on a test dataset than the training dataset. Offers strategies to discover and address discrepancies between training and test datasets, introducing the concept of adversarial validation.

第 29 章：训练集与测试集表现倒挂——如何发现与处理分布差异，并介绍对抗验证思路。

Chapter [ch30]: Limited Labeled Data

Introduces various techniques to enhance model performance in situations where data is limited. Covers data labeling, bootstrapping, and paradigms such as transfer learning, active learning, and multimodal learning.

第 30 章：标签数据有限——标注、自助法以及迁移学习、主动学习、多模态等范式。

Online Resources

本节给出作者在 GitHub 上提供的补充代码与深度学习材料链接，便于与正文对照练习。

I've provided optional supplementary materials on GitHub with code examples for certain chapters to enhance your learning experience (see <https://github.com/rasbt/MachineLearning-QandAI-book>). These materials are designed as practical extensions and deep dives into topics covered in the book. You can use them alongside each chapter or explore them after reading to solidify and expand your knowledge.



作者在 GitHub 上提供了部分章节的代码示例与补充材料 (<https://github.com/rasbt/MachineLearning-QandAI-book>)，可作为正文的实践延伸与深挖；可边读边用，也可读后再做以巩固。

Without further ado, let's dive in.

闲话少叙，开始阅读吧。

Chapter 1: Embeddings, Latent Space, and Representations



本章围绕嵌入向量、潜空间与表示三者的联系与区别展开，说明它们在机器学习中表示与编码信息的方式，并配有练习与参考文献。

In deep learning, we often use the terms *embedding vectors*, *representations*, and *latent space*. What do these concepts have in common, and how do they differ?

在深度学习中，我们常说 *embedding vectors*（嵌入向量）、*representations*（表示）和 *latent space*（潜空间）。这些概念有什么共同点、又有什么不同？

While these three terms are often used interchangeably, we can make subtle distinctions between them:

这三个术语常被混用，但仍可作出细致区分：

- **Embedding vectors** are representations of input data where similar items are close to each other.
- **Latent vectors** are intermediate representations of input data.
- **Representations** are encoded versions of the original input.
- **Embedding vectors**（嵌入向量）是对输入数据的表示，相似项在空间中彼此靠近。
- **Latent vectors**（潜向量）是输入的中间表示。
- **Representations**（表示）则是对原始输入的编码形式。

The following sections explore the relationship between embeddings, latent vectors, and representations and how each functions to encode information in machine learning contexts.

下文将探讨嵌入、潜向量与表示之间的关系，以及它们在机器学习语境中各自如何承担编码信息的角色。



Embeddings

本节定义嵌入向量、说明其与 one-hot 及神经网络层输出的关系，并讨论相似性、结构保持与数学上的单射映射。

Embedding vectors, or *embeddings* for short, encode relatively high-dimensional data into relatively low-dimensional vectors.

嵌入向量（简称 *embeddings*）把相对高维的数据编码到相对低维的向量中。

Tips: 嵌入向量，简称 **嵌入**，是输入数据的一种表示形式，相似的输入、对应的嵌入向量 **彼此接近**；通常，将高维数据，转换为低维嵌入向量。

We can apply embedding methods to create a continuous dense (non-sparse) vector from a (sparse) **one-hot** encoding.

我们可以用嵌入方法，从（稀疏的）**one-hot** 编码得到连续、稠密（非稀疏）的向量。

One-hot encoding is a method used to represent categorical data as binary vectors, where each category is mapped to a vector containing 1 in the position corresponding to the category's index, and 0 in all other positions.

One-hot encoding（独热编码）用二进制向量表示分类数据：每个类别在对应索引位置为 1，其余位置为 0。

This ensures that the categorical values are represented in a way that certain machine learning algorithms can process. For example, if we have a categorical variable Color with three categories, Red, Green, and Blue, the one-hot encoding would represent Red as [1, 0, 0], Green as [0, 1, 0], and Blue as [0, 0, 1]. These one-hot encoded categorical variables can then be mapped into continuous embedding vectors by utilizing the learned weight matrix of an embedding layer or module.

这样分类值就能被某些机器学习算法处理。例如变量 Color 有红、绿、蓝三类，则红为 [1, 0, 0]，绿为 [0, 1, 0]，蓝为 [0, 0, 1]。这些 one-hot 向量再通过嵌入层或模块学到的权重矩阵映射为连续嵌入向量。

独热编码：one-hot 编码，是一种将分类数据转换为二进制向量的方法，其中每个类别映射到包含 1 的向量，在对应类别的索引位置为1，其他位置为0。

We can also use embedding methods for dense data such as images. For example, the last layers of a convolutional neural network may yield embedding vectors, as illustrated in Figure 1.1 .

嵌入同样可用于图像等稠密数据；例如卷积神经网络的靠后层可产生嵌入向量，见 Figure 1.1。

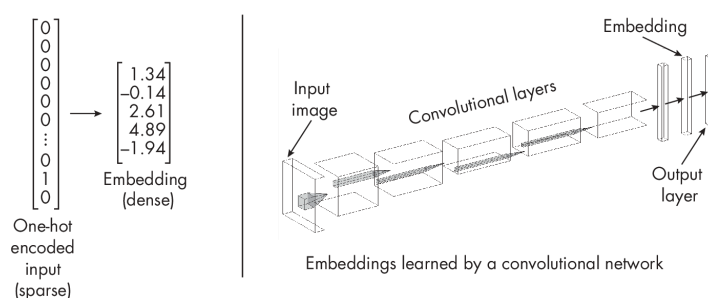


Figure 1.1

To be technically correct, all intermediate layer outputs of a neural network could yield embedding vectors. Depending on the training objective, the output layer may also produce useful embedding vectors. For the sake of simplicity, the convolutional neural network in Figure 1.1

严格来说，神经网络任意中间层的输出都可视为嵌入向量；视训练目标而定，输出层也可能给出有用的嵌入。为简明起见，Figure 1.1 中的卷积网络

Embeddings can have higher or lower numbers of dimensions than the original input. For instance, using embeddings methods for extreme expression, we can encode data into two-dimensional dense and continuous representations for visualization purposes and clustering analysis, as illustrated in Figure 1.2.

嵌入的维数可以高于或低于原始输入；例如为可视化与聚类，可把数据压到二维稠密连续表示，见 Figure 1.2。

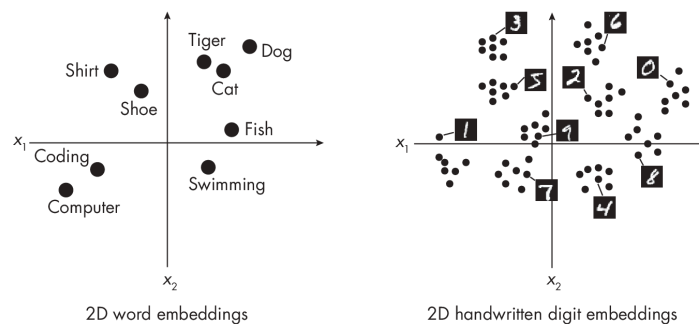


Figure 1.2

A **fundamental property** of embeddings is that they encode *distance* or *similarity*. This means that embeddings capture the semantics of the data such that similar inputs are close in the embeddings space.

嵌入的一个**根本性质**是编码**距离**或**相似性**：相似输入在嵌入空间中彼此靠近，从而承载数据的语义。

Tips: 嵌入向量，具有一个重要的性质，即编码 **距离相近** 或 **相似性**。这意味着嵌入向量能够捕捉数据的语义，使得相似的输入在嵌入空间中彼此接近。这也称为 **结构保持** structure-preserving 特性。

For readers interested in a more formal explanation using mathematical terminology, an embedding is an **injective** and **structure-preserving** map between an input space X and the embedding space Y . This implies that similar inputs will be located at points in close proximity within the embedding space, which can be seen as the "structure-preserving" characteristic of the embedding.

若用更正式的数学语言，嵌入是从输入空间 X 到嵌入空间 Y 的一个 **injective**（单射）且 **structure-preserving**（结构保持）的映射：相似输入在嵌入空间中位置接近，即所谓「结构保持」。

Tips: 嵌入向量，是输入空间 X 和嵌入空间 Y 之间的一个**单向**和**结构保持**映射。这意味着相似的输入，在嵌入空间中彼此接近，这就是 **结构保持** 特性。

Latent Space

本节说明潜空间与嵌入空间的关系、潜特征与自编码器瓶颈，以及自编码器目标如何促使相似输入在潜空间中靠近。



Latent space is typically used synonymously with *embedding space*, the space into which embedding vectors are mapped.

Latent space（潜空间）通常与 *embedding space*（嵌入空间）同义，即嵌入向量被映射到的空间。

Tips: 隐空间 \ 潜空间，通常与嵌入空间同义，是 **嵌入向量** 映射到的空间。

Similar items can appear close in the latent space; however, this is not a strict requirement. More loosely, we can think of the latent space as any feature space that contains features, often compressed versions of the original input features. These latent space features can be learned by a neural network, such as an **autoencoder** that reconstructs input images, as shown in Figure 1.3.

相似样本在潜空间中可以靠近，但并非硬性要求。更宽泛地，潜空间可理解为包含（常为压缩后的）输入特征的任何特征空间；这些潜特征可由网络学习，例如用 **autoencoder** 重建输入图像，见 Figure 1.3。

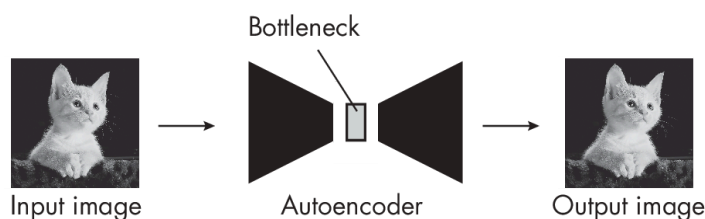


Figure 1.3

The **bottleneck** in Figure 1.3 represents a small, intermediate neural network layer that encodes or maps the input image into a lower-dimensional representation. We can think of the target space of this mapping as a latent space. The training objective of the autoencoder is to reconstruct the input image, that is, to minimize the distance between the input and output images. In order to optimize the training objective, the autoencoder may learn to place the encoded features of similar inputs (for example, pictures of cats) close to each other in the latent space, thus creating useful embedding vectors where similar inputs are close in the embedding (latent) space.



Figure 1.3 中的**瓶颈层**是较小的中间层，把输入图像编码/映射到低维表示，其目标空间可视为潜空间。自编码器的训练目标是最小化输入与重建输出之间的距离；为优化该目标，网络可能学会把相似输入（例如猫图）的编码在潜空间中拉近，从而在嵌入（潜）空间中形成「相似近邻」的有用嵌入。

Representation

本节界定 representation 与嵌入、潜向量的关系，并说明简单流程（如 one-hot）也可产生表示及其在后续分析中的作用。

A *representation* is an encoded, typically intermediate form of an input. For instance, an embedding vector or vector in the latent space is a representation of the input, as previously discussed. However, representations can also be produced by simpler procedures. For example, one-hot encoded vectors are considered representations of an input.

Representation（表示）是输入的编码形式，常为中间形式；嵌入向量或潜空间中的向量都是表示。表示也可由更简单流程产生，例如 one-hot 向量也是一种输入表示。

The key idea is that the representation captures some essential features or characteristics of the original data to make it useful for further analysis or processing.

关键在于表示要抓住原始数据的某些本质特征或特性，以便后续分析或处理。

Tips: 表示/表征 representation，是输入的一种编码形式，通常是中间形式。关键点是，它能够捕捉输入的一些 **本质特征** 或 **特性**，可用于后续分析。

Exercises

本节为两道练习题：其一讨论 AlexNet 式网络中哪些层适合作为 embedding；其二列举非嵌入类型的输入表示。



1-1. Suppose we're training a convolutional network with five convolutional layers followed by three fully connected (FC) layers, similar to AlexNet (<https://en.wikipedia.org/wiki/AlexNet>), as illustrated in Figure 1.4.

1-1. 假设我们训练一个五层卷积 + 三层全连接（FC）的网络，结构类似 AlexNet (<https://en.wikipedia.org/wiki/AlexNet>)，如 Figure 1.4 所示。

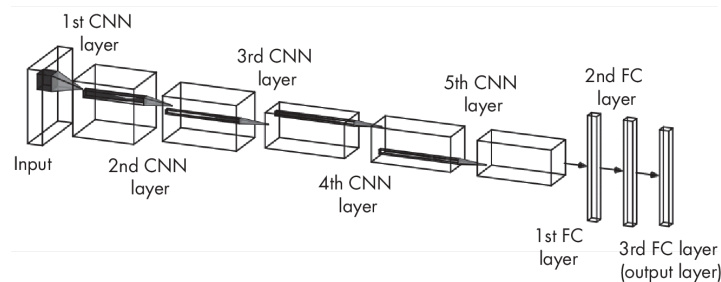
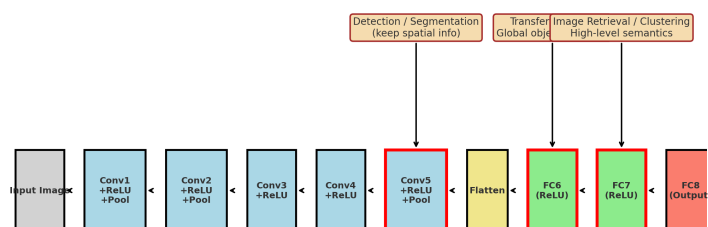


Figure 1.4

We can think of these fully connected layers as two hidden layers and an output layer in a multilayer perceptron. Which of the neural network layers can be utilized to produce useful embeddings? Interested readers can find more details about the AlexNet architecture and implementation in the original publication by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton.

可把三层 FC 看作多层感知机中的两个隐层加一个输出层。哪些层适合用来提取有用的 embedding？细节见 Alex Krizhevsky、Ilya Sutskever 与 Geoffrey Hinton 的原始论文。

AlexNet Embedding Extraction Points and Their Applications



Answer 1-1

AlexNet 的完整流程：

- 输入图像 → 卷积层堆叠 (Conv1–Conv5) → 展平 → 全连接层 (FC6–FC8)
- 标红的 Conv5、FC6、FC7 是常用来提取有用 **embedding** 特征的层。
 - Conv5 → 检测 / 分割（保留空间位置信息）
 - FC6 → 迁移学习（整体结构特征）



- FC7 → 图像检索 / 聚类（高级语义表示）

Answer:

asking: in an AlexNet-style convolutional neural network (five conv layers + three >FC layers), which layers can provide useful embeddings that capture semantic >information about the input.

Let's break it down:

1. Convolutional Layers (early to mid layers)

- **Early conv layers (conv1–conv2):** These capture **low-level features** like edges, corners, textures, and simple >shapes. → Not very useful as "semantic embeddings," but good for transfer to tasks >requiring fine textures (e.g., medical images, style).
- **Mid-depth conv layers (conv3–conv5):** These capture **mid- to high-level features** (object parts, motifs, hierarchical >compositions). → Often very useful as **general embeddings** for vision tasks like detection, >segmentation, or retrieval. (In fact, features from conv5 of AlexNet were widely used in early transfer >learning.)

2. Fully Connected Layers

- **First two FC layers (fc6, fc7):** These act like **dense "semantic bottlenecks."**
- **fc6** embeddings capture global object structure.
- **fc7** embeddings are particularly popular for transfer learning and >representation learning because they're **highly discriminative yet not tied to the >final class labels**. → Both are excellent choices for embeddings.
- **Last FC layer (fc8, the classifier):** This outputs class scores (e.g., 1000-dim for ImageNet). → Too task-specific (logits for specific labels). Not generally useful as >embeddings unless you remove the softmax and use the logits for re-scoring.



3. Best Practices

- If you want **general embeddings** (for clustering, retrieval, transfer learning):
 - Use `conv5` (after flatten + pooling) or `fc6` / `fc7` .
 - `fc7` is most common, since it captures high-level semantics.
- If you want **task-specific representations**:
 - Use `fc8` logits.
- If you want **visual features for downstream CV models (e.g., detection, >segmentation)**:
 - Use convolutional feature maps (conv3–conv5), since they preserve spatial >structure.

4. Summary:

Useful embeddings can be taken from the last convolutional layer (conv5) or the >penultimate fully connected layers (fc6, fc7). The final output layer (fc8) is >usually not used for embeddings since it's tied to a specific classification task.

1-2. Name some types of input representations that are not embeddings.

1-2. 请举出一些属于输入表示、但不是 embedding 的类型。

References

本节列出本章涉及的 AlexNet 原始论文链接，供延伸阅读。

- The original paper describing the AlexNet architecture and implementation: Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, "ImageNet Classification with Deep Convolutional Neural Networks"? (2012), <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>.
- AlexNet 架构与实现的原始论文：Alex Krizhevsky、Ilya Sutskever、Geoffrey Hinton, 《ImageNet Classification with Deep Convolutional Neural Networks》(2012), <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>。





Chapter 2: Self-Supervised Learning

本章定义自监督学习及其与迁移学习的关系，说明如何利用无标签数据，并介绍自预测与对比式自监督两大范式及练习、延伸阅读。

What is self-supervised learning, when is it useful, and what are the main approaches to implementing it?

什么是自监督学习、何时有用、以及实现它的主要途径有哪些？

Self-supervised learning is a pretraining procedure that lets neural networks leverage large, unlabeled datasets in a supervised fashion. This chapter compares self-supervised learning to transfer learning, a related method for pretraining neural networks, and discusses the practical applications of self-supervised learning. Finally, it outlines the main categories of self-supervised learning.

自监督学习是一种预训练流程，使神经网络能以「监督」的方式利用大规模无标签数据。本章将自监督学习与迁移学习对照，讨论其实用场景，并概括自监督学习的主要类别。

Tips: 自监督学习，是一种预训练方法，让神经网络利用 **无标签** 的大数据集，进行 **监督学习**。其实，使用 **无标签** 数据，自动构造了 **伪标签**（例如：遮挡部分内容、预测缺失内容），也是一种 **监督学习**。

Self-Supervised Learning vs. Transfer Learning

本节对比迁移学习（有标签 ImageNet 预训练再微调）与自监督学习（从无标签数据构造伪标签的 pretext task），并说明二者在「标签来源」上的根本差异。

Self-supervised learning is related to **transfer learning**, a technique in which a model pretrained on one task is reused as the starting point for a model on a second task. For example, suppose we are interested in training an image classifier to classify bird species. In transfer learning, we would pretrain a



convolutional neural network on the ImageNet dataset, a large, labeled image dataset with many different categories, including various objects and animals. After pretraining on the general ImageNet dataset, we would take that pretrained model and train it on the smaller, more specific target dataset that contains the bird species of interest. (Often, we just have to change the class-specific output layer, but we can otherwise adopt the pretrained network as is.)

自监督学习与 **transfer learning**（迁移学习）相关：后者把在任务 A 上预训练的模型作为任务 B 的起点。例如先在大型有标签数据集 ImageNet 上预训练卷积网络，再在小规模鸟类数据集上微调（常只需替换最后的分类头，其余权重可沿用）。

Figure 2.1 illustrates the process of transfer learning.

Figure 2.1 示意传统迁移学习的流程。

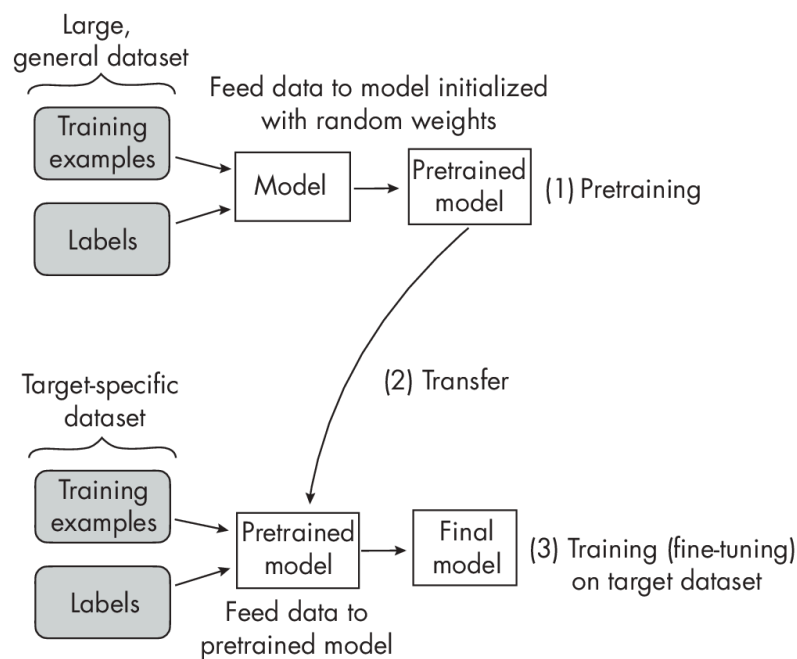


Figure 2.1

Tips: 自监督学习，与 迁移学习 是相关的。

- 相同点在于，都是使用 **预训练** 的模型，然后进行 **微调**。
- 差异在于，迁移学习是使用 **有标签** 的数据，而自监督学习使用 **无标签** 的数据。



Self-supervised learning is an alternative approach to transfer learning in which the model is pretrained not on labeled data but on *unlabeled* data. We consider an unlabeled dataset for which we do not have label information, and then we find a way to obtain labels from the dataset's structure to formulate a prediction task for the neural network, as illustrated in Figure 2.2. These self-supervised training tasks are also called *pretext tasks*.

自监督学习则改为在无标签数据上预训练：利用数据结构「自造」标签，把预测任务交给网络，见 Figure 2.2；这类任务也称 *pretext tasks*（前置/代理任务）。

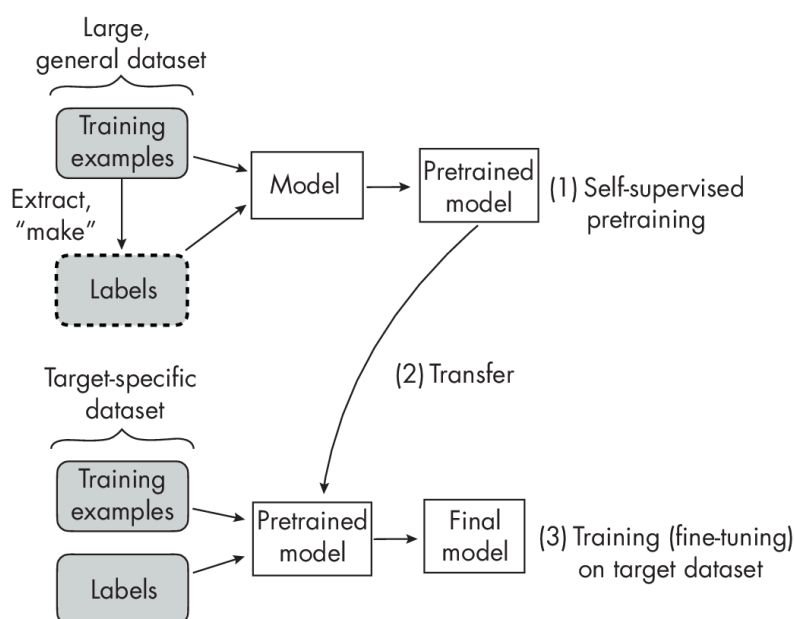


Figure 2.2

The main difference between transfer learning and self-supervised learning lies in how we obtain the labels during step 1 in Figures 2.1 and 2.2. In transfer learning, we assume that the labels are provided along with the dataset; they are typically created by human labelers. In self-supervised learning, the labels can be directly derived from the training examples.

迁移学习与自监督学习的关键差别在于 Figure 2.1 与 2.2 中第 1 步的标签从何而来：前者依赖人工标注；后者标签可直接由样本本身推导。

Tips: 自监督学习中，数据集的标签，可以 **直接** 从训练样本中 **推导** 出来。

A self-supervised learning task could be a missing-word prediction in a natural language processing context. For example, given the sentence "It is beautiful and



sunny outside,"? we can mask out the word *sunny*, feed the network the input "It is beautiful and [MASK] outside,"? and have the network predict the missing word in the "[MASK]"? location. Similarly, we could remove image patches in a computer vision context and have the neural network fill in the blanks. These are just two examples of self-supervised learning tasks; many more methods and paradigms for this type of learning exist.

例如在 NLP 中可遮住词让模型预测；在视觉中可挖去图像块让网络补全。这里仅举两例，自监督任务的形式还有很多。

In sum, we can think of self-supervised learning on the pretext task as *representation learning*. We can take the pretrained model to fine-tune it on the target task (also known as the *downstream* task).

总之，可在代理任务上的自监督预训练视为 *representation learning*（表示学习）；随后在目标（*downstream*）任务上微调即可。

Leveraging Unlabeled Data

本节说明自监督在大模型与 ViT 等架构中的必要性，以及在小 MLP 或树模型上通常不适用、也不兼容迁移的原因。

Large neural network architectures require large amounts of labeled data to perform and generalize well. However, for many problem areas, we don't have access to large labeled datasets. With self-supervised learning, we can leverage unlabeled data. Hence, self-supervised learning is likely to be useful when working with large neural networks and with a limited quantity of labeled training data.

大网络往往需要大量有标签数据才能泛化好；许多领域却缺乏标注。自监督让我们利用无标签数据，因而在「大网络 + 有标签数据少」的场景特别有价值。

Transformer-based architectures that form the basis of LLMs and vision transformers are known to require self-supervised learning for pretraining to perform well.

LLM 与视觉 Transformer 等架构的预训练普遍依赖自监督才能发挥性能。



For small neural network models such as multilayer perceptrons with two or three layers, self-supervised learning is typically considered neither useful nor necessary.

对仅两三层的小 MLP，自监督通常既不必要也不划算。

Tips: 对于 **小型** 的神经网络模型，如具有两到三层的 **多层感知器**，**自监督学习** 在这种情况下 **不实用也 不必要**。

Self-supervised learning likewise isn't useful in traditional machine learning with nonparametric models such as tree-based random forests or gradient boosting. Conventional tree-based methods do not have a fixed parameter structure (in contrast to the weight matrices, for example). Thus, conventional tree-based methods are not capable of transfer learning and are incompatible with self-supervised learning.

在随机森林、梯度提升等非参数树模型上，自监督同样不适用：树方法没有固定参数矩阵式的表示，难以做迁移，也与自监督流程不兼容。

Tips: 对于 **非参数模型**，如基于树的随机森林或梯度提升，**自监督学习** 通常不适用。

传统的基于树的方法没有固定的参数结构（与权重矩阵相比），因此传统的基于树的方法无法进行迁移学习，也不兼容自监督学习。

FIXME 没理解???

Self-Prediction and Contrastive Self-Supervised Learning

本节区分自预测（掩码重建、去噪等）与对比式自监督（拉近正样本、拉远负样本），并介绍 Siamese 设定及样本对比与维度对比两类变体。

There are two main categories of self-supervised learning: **self-prediction** and **contrastive self-supervised** learning. In *self-prediction*, illustrated in Figure [2.3](#), we typically change or hide parts of the input and train the model to



reconstruct the original inputs, such as by using a perturbation mask that obfuscates certain pixels in an image.

自监督大致分两类：`self-prediction` 与 `contrastive self-supervised`。前者常遮挡或扰动输入的一部分，再训练模型重建原输入，见 Figure 2.3。

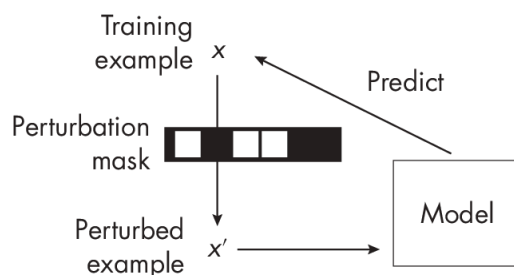


Figure 2.3

A classic example is a denoising autoencoder that learns to remove noise from an input image. Alternatively, consider a masked autoencoder that reconstructs the missing parts of an image, as shown in Figure 2.4.

经典例子包括去噪自编码器，以及 Figure 2.4 所示的掩码自编码器（MAE）重建缺失区域。

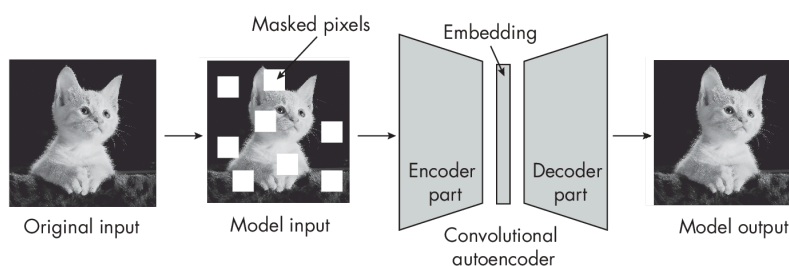


Figure 2.4

Missing (`masked`) input self-prediction methods are also commonly used in natural language processing contexts. Many generative LLMs, such as GPT, are trained on a next-word prediction pretext task (GPT will be discussed at greater length in Chapters [ch14] and [ch17]. Here, we feed the network text fragments, where it has to predict the next word in the sequence (as we'll discuss further in Chapter [ch17]).

NLP 中也广泛使用掩码或缺失输入式的自预测；许多生成式 LLM（如 GPT）以下一词预测为代理任务（详见第 14、17 章）。



In *contrastive self-supervised learning*, we train the neural network to learn an embedding space where similar inputs are close to each other and dissimilar inputs are far apart. In other words, we train the network to produce embeddings that minimize the distance between similar training inputs and maximize the distance between dissimilar training examples.

在 *contrastive self-supervised learning* 中，我们训练网络使相似输入的嵌入彼此靠近、不相似输入彼此远离。

Let's discuss contrastive learning using concrete example inputs. Suppose we have a dataset consisting of random animal images. First, we draw a random image of a cat (the network does not know the label, because we assume that the dataset is unlabeled). We then augment, corrupt, or perturb this cat image, such as by adding a random noise layer and cropping it differently, as shown in Figure 2.5.

下面用猫图举例：从无标签动物图中随机抽一张猫，对其加噪、裁剪等得到扰动版本，见 Figure 2.5。

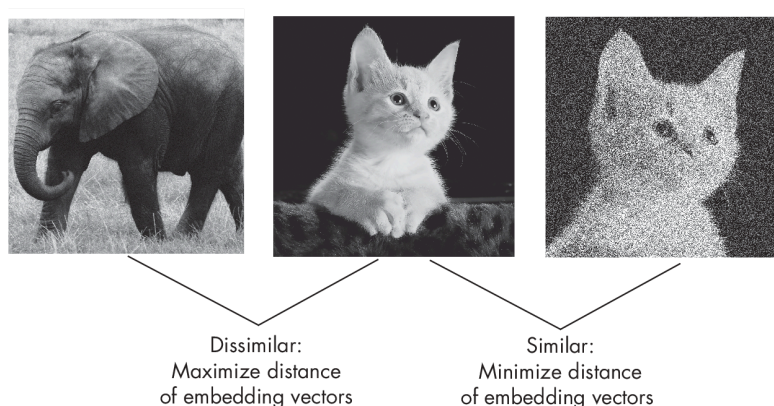


Figure 2.5

The perturbed cat image in this figure still shows the same cat, so we want the network to produce a similar embedding vector. We also consider a random image drawn from the training set (for example, an elephant, but again, the network doesn't know the label).

扰动后的猫仍应视为同一只猫，故希望嵌入相近；再抽一张例如大象的图作为负样本，网络不知道类别标签。

For the cat-elephant pair, we want the network to produce dissimilar embeddings. This way, we implicitly force the network to capture the image's core content



while being somewhat agnostic to small differences and noise. For example, the simplest form of a contrastive loss is the L_2 -norm (Euclidean distance) between the embeddings produced by model $M(\cdot)$. Let's say we update the model weights to decrease the distance $\|M(cat) - M(cat')\|_2$ and increase the distance $\|M(cat) - M(elephant)\|_2$.

对猫-象对则希望嵌入远离，从而迫使网络抓住语义主体而对小幅差异不敏感。最简单的对比损失可用嵌入间 L_2 距离表示：缩小 $\|M(cat) - M(cat')\|_2$ 、增大 $\|M(cat) - M(elephant)\|_2$ 。

Figure 2.6 summarizes the central concept behind **contrastive learning** for the perturbed image scenario. The model is shown twice, which is known as a *siamese network* setup. Essentially, the same model is utilized in two instances: first, to generate the embedding for the original training example, and second, to produce the embedding for the perturbed version of the sample.

Figure 2.6 概括扰动图像场景下的对比学习：同一模型接两份输入，称为 *siamese network*（孪生网络）设定。

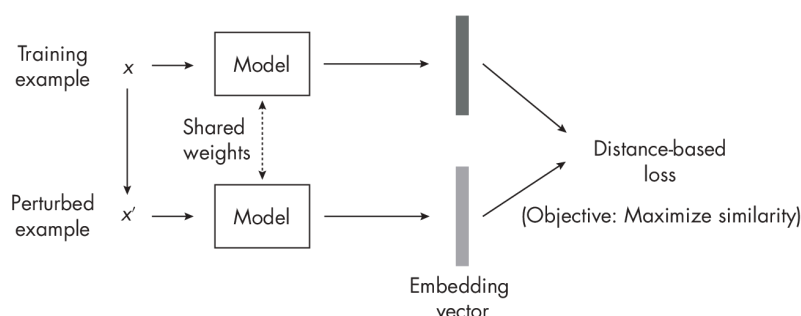


Figure 2.6

This example outlines the main idea behind contrastive learning, but many subvariants exist. Broadly, we can categorize these into *sample* contrastive and *dimension* contrastive methods. The elephant-cat example in Figure 2.6 illustrates a sample contrastive method, where we focus on learning embeddings to minimize and maximize distances between training pairs. In *dimension*-contrastive approaches, on the other hand, we focus on making only certain variables in the embedding representations of similar training pairs appear close to each other while maximizing the distance of others.

对比学习还有许多子变体，可粗分为 *sample contrastive*（样本对距离）与 *dimension contrastive*（只让嵌入的某些维度接近、其余维度推远）。

Tips: 对比学习，可以分为 **样本对比** 和 **维度对比** 两种方法。

- 样本对比，关注于学习嵌入，以最小化/最大化 **训练对** 之间的距离。
- 维度对比，关注于使相似 **训练对** 中的**某些变量**接近，同时最大化其他变量的距离。



Exercises

本节提出两道思考题：自监督如何用于视频；以及表格数据上是否可行、如何设计。

2-1. How could we apply self-supervised learning to video data?

2-1. 如何把自监督学习用到视频数据上？

2-2. Can self-supervised learning be used for tabular data represented as rows and columns? If so, how could we approach this?

2-2. 行列表格数据能否做自监督？若可以，可如何设计？

Talk

Talk 1: 自监督学习，使用了无标签数据，进行预训练，本质也是一种有监督训练，理解对吗？

你的理解非常接近，但里面有一点细微差别，我帮你梳理下：

1. 传统监督学习

- 有人工标注的标签 y 。
- 目标函数直接基于 (x, y) 优化（例如图像分类：输入图片，标签是「猫/狗」）。

2. 自监督学习 (Self-Supervised Learning, SSL)

- 使用无标签数据（比如大量文本、图片、语音）。



- 但它会 自动生成“伪标签” (pretext task)，来构造监督信号。例子：
 - NLP 里 **掩码预测 (masked language modeling)**：给模型输入 “我去 [MASK] 吃饭”，伪标签就> 是被遮住的词。
 - CV 里 **对比学习**：从同一张图生成两个不同增强视图，伪标签就是“这俩是同一个对象”。

换句话说，**自监督 ≠ 无监督**。它其实是监督学习的一种特例，只不过监督信号不是人手标注，而是 **从数据本身挖掘出来的监督信息**。

3. 本质理解

- **是监督学习**：因为它还是通过“预测某个目标”来训练，损失函数需要“标签”（哪怕是自生成的）。
- **区别在于标签的来源**：
 - 监督学习 → 外部人工提供。
 - 自监督学习 → 数据内部挖掘。

4. 结论

所以你的说法「自监督学习，本质也是一种有监督训练」是正确的，只是要更精确地说：

- 它是一种 “**利用数据自身生成监督信号的监督学习**”。

References

本节列出 ImageNet、SimCLR、VICRegL、自监督 cookbook 及表格数据相关论文链接。

- For more on the ImageNet dataset: <https://en.wikipedia.org/wiki/ImageNet>.
- 关于 ImageNet 数据集: <https://en.wikipedia.org/wiki/ImageNet>.
- An example of a contrastive self-supervised learning method: Ting Chen et al., "A Simple Framework for Contrastive Learning of Visual Representations"?



(2020), <https://arxiv.org/abs/2002.05709>.

- 对比式自监督示例：Ting Chen 等，SimCLR (2020), <https://arxiv.org/abs/2002.05709>。
- An example of a dimension-contrastive method: Adrien Bardes, Jean Ponce, and Yann LeCun, "VICRegL: Self-Supervised Learning of Local Visual Features"? (2022), <https://arxiv.org/abs/2210.01571>.
- 维度对比示例：VICRegL (2022), <https://arxiv.org/abs/2210.01571>。
- If you plan to employ self-supervised learning in practice: Randall Balestriero et al., "A Cookbook of Self-Supervised Learning"? (2023), <https://arxiv.org/abs/2304.12210>.
- 实践指南：自监督 Cookbook (2023), <https://arxiv.org/abs/2304.12210>。
- A paper proposing a method of transfer learning and self-supervised learning for relatively small multilayer perceptrons on tabular datasets: Dara Bahri et al., "SCARF: Self-Supervised Contrastive Learning Using Random Feature Corruption"? (2021), <https://arxiv.org/abs/2106.15147>.
- 表格数据上小 MLP 的自监督：SCARF (2021), <https://arxiv.org/abs/2106.15147>。
- A second paper proposing such a method: Roman Levin et al., "Transfer Learning with Deep Tabular Models"? (2022), <https://arxiv.org/abs/2206.15306>.
- 另一篇深度表格迁移：Levin 等 (2022), <https://arxiv.org/abs/2206.15306>。



Chapter 3: Few-Shot Learning

本章介绍小样本学习（few-shot）与常规监督在数据组织上的差异，解释 N-way K-shot、支持集与 episode，并简述元学习与基于近邻的嵌入思路及练习。

What is few-shot learning? How does it differ from the conventional training procedure for supervised learning?

什么是小样本学习？它与常规监督学习训练流程有何不同？

Few-shot learning is a type of supervised learning for small training sets with a very small example-to-class ratio. In regular supervised learning, we train models by iterating over a training set where the model always sees a **fixed set** of classes. In few-shot learning, we are working on a **support set** from which we create multiple training tasks to assemble training episodes, where each training task consists of different classes.

Few-shot learning 是在「每类样本极少」设定下的监督学习：常规监督在固定类别集上迭代整个训练集；小样本则从 **support set** 采样多组任务组成 episode，每组任务的类别集合可以不同。

Tips:

- 小样本学习，关注于学习模型，以适应新的任务。
- 在传统的监督学习中，我们通过迭代训练集来训练模型，模型总是看到固定的类集。
- 在小样本学习中，我们从一个 **支持集** 开始，创建 **多个训练任务** 来组装训练集，每个训练任务包含不同的分类。

Datasets and Terminology

本节对比常规训练/测试集与小样本中的 support、query、base 类与 episode，并用图示说明训练与测试阶段类别不重叠的设定及常见元学习策略。



In supervised learning, we fit a model on a **training dataset** and evaluate it on a **test dataset**. The training set typically contains a relatively large number of examples per class. For example, in a supervised learning context, the Iris dataset, which has 50 examples per class, is considered a tiny dataset. For deep learning models, on the other hand, even a dataset like MNIST that has 5,000 training examples per class is considered very small.

常规监督在 **training dataset** 上拟合、在 **test dataset** 上评估；每类样本数通常较多（Iris 每类 50 已算很小；MNIST 每类约 5000 对深度学习仍偏小）。

In **few-shot learning**, the number of examples per class is much smaller. When specifying the few-shot learning task, we typically use the term *N-way K-shot*, where

- N stands for the number of classes
- and K stands for the number of examples per class.

The most common values are $K = 1$ or $K = 5$. For instance, in a 5-way 1-shot problem, there are five classes with only one example each. Figure 3.1 depicts a 3-way 1-shot setting to illustrate the concept with a smaller example.

在 **few-shot learning** 中每类样本极少；常用 **N-way K-shot** 描述： N 为类别数， K 为每类支撑样本数，常见 $K=1$ 或 5。Figure 3.1 用 3-way 1-shot 示意。

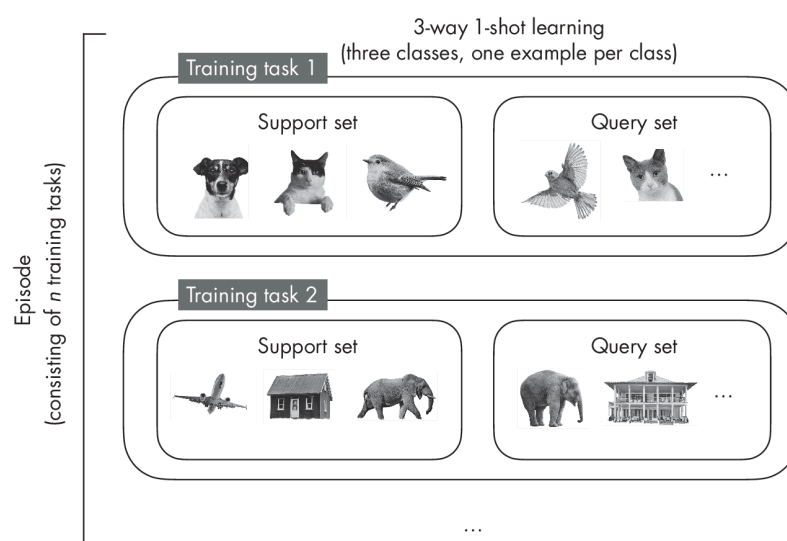


Figure 3.1



Rather than fitting the model to the training dataset, we can think of **few-shot learning** as "learning to learn." In contrast to supervised learning, **few-shot learning** uses not a training dataset but a so-called **support set**, from which we sample training tasks that mimic the use-case scenario during prediction. With each training task comes a query image to be classified. The model is trained on several training tasks from the support set; this is called an **episode**.

也可把 few-shot 理解为「学会学习」：不用传统大训练集，而从 **support set** 反复采样任务；每个任务带待分类的 query；在 support 上串起多轮任务称为一个 **episode**。

Tips: 小样本学习，可以看作是 **学习如何学习**。

与传统的监督学习不同，小样本学习不使用训练集，而是使用所谓的 **支持集**，从中采样训练任务，以模仿预测时的使用场景。

每个训练任务都有一个查询图像需要分类。

模型在支持集的多个训练任务上进行训练；这称为 **一个训练轮次**。

FIXME??? 不理解

Next, during testing, the model receives a new task with classes different from those seen during training. The classes encountered in training are also called **base classes**, and the support set during training is also often called the **base set**. Again, the task is to classify the query images. Test tasks are similar to training tasks, except that none of the classes during testing overlap with those encountered during training, as illustrated in Figure 3.2.

测试时模型见到的新任务类别应与训练时不同；训练阶段见到的类常称 **base classes**，对应 support 也称 **base set**。测试任务形式类似，但类别与训练不重叠，见 Figure 3.2。

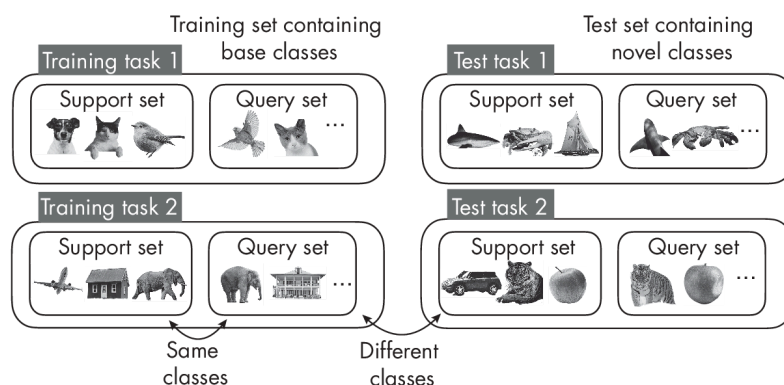


Figure 3.2

As Figure 3.2 shows, the support and query sets contain different images from the same class during training. The same is true during testing. However, notice that the classes in the support and query sets differ from the support and query sets encountered during training.

如图，训练时 support 与 query 虽图像不同但属同一批训练类别；测试时亦然，但测试见到的类别集与训练阶段完全不同。

There are many different types of few-shot learning. In the most common, *meta-learning*, training is essentially about updating the model's parameters such that it can *adapt* well to a new task. On a high level, one few-shot learning strategy is to learn a model that produces embeddings where we can find the target class via a nearest-neighbor search among the images in the support set. Figure 3.3 illustrates this approach.

小样本流派众多；常见的 *meta-learning* 通过更新参数使模型能快速 *adapt* 新任务。一种高层策略是学习嵌入，再在 support 上做最近邻以判定 query 类别，见 Figure 3.3。

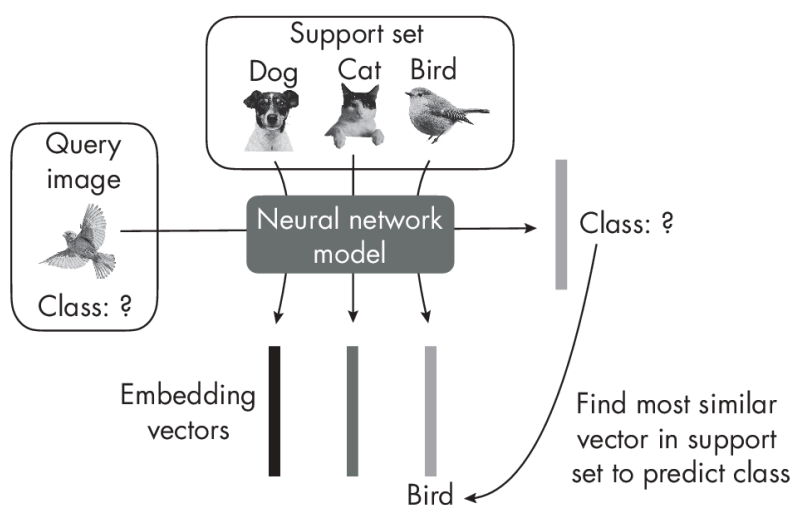


Figure 3.3



The model learns how to produce good embeddings from the support set to classify the query image based on finding the most similar embedding vector.

模型学会从 support 生成利于分类的嵌入，通过最相似嵌入向量判定 query。

Exercises

本节练习：如何把 MNIST 划分成 one-shot 设定；以及 few-shot 的实际应用场景。

3-1. MNIST (https://en.wikipedia.org/wiki/MNIST_database) is a classic and popular machine learning dataset consisting of 50,000 handwritten digits from 10 classes corresponding to the digits 0 to 9. How can we partition the MNIST dataset for a one-shot classification context?

3-1. MNIST (https://en.wikipedia.org/wiki/MNIST_database) 含 10 类手写数字各约 5000 训练样本。若要构造 one-shot 分类实验，应如何划分数据？

3-2. What are some real-world applications or use cases for few-shot learning?

3-2. 小样本学习在现实中有哪些应用或用例？



Chapter 4: The Lottery Ticket Hypothesis

本章解释彩票假设（随机初始化大网中存在可单独训练达到同精度的小子网）、迭代幅度剪枝流程，并讨论其在训练/推理上的潜力与现实限制及练习。

What is the lottery ticket hypothesis, and, if it holds true, how is it useful in practice?

什么是彩票假设？若成立，在实践中能带来什么用处？

The lottery ticket hypothesis is a concept in neural network training that posits that within a randomly initialized neural network, there exists a **subnetwork** (or **winning ticket** ?) that can, when trained separately, achieve the same accuracy on a test set as the full network after being trained for the same number of steps. This idea was first proposed by Jonathan Frankle and Michael Carbin in 2018.

彩票假设认为：在随机初始化的大网络里存在一个 **subnetwork**（「中奖彩票」），单独训练同样步数即可达到与整网相当的测试精度；该思想由 Frankle 与 Carbin 于 2018 年提出。

Tips: 彩票假设 **lottery ticket hypothesis**，是神经网络训练中，一个重要的概念。它指出，在随机初始化的神经网络中，存在一个 **子网络**（或**彩票**），当单独训练时，可以达到与完整网络相同的准确率。

This chapter illustrates the lottery hypothesis step by step, then goes over *weight pruning*, one of the key techniques to create smaller subnetworks as part of the lottery hypothesis methodology. Lastly, it discusses the practical implications and limitations of the hypothesis.

本章逐步演示该假设，再介绍作为核心手段的 *weight pruning*（权重剪枝），最后讨论其现实意义与局限。

Tips: 本章将展示彩票假设的训练过程，然后介绍权重剪枝，这是彩票假设方法论中，创建较小子网络的关键技术。最后，讨论彩票假设的实际应用和局限性。



The Lottery Ticket Training Procedure

本节按 Figure 4.1 的四步说明：训练大网至收敛、结构化/非结构化剪枝、迭代幅度剪枝、重置为原始初始化再训剪枝子网等，直至得到 winning ticket。

Figure 4.1 illustrates the training procedure for the lottery ticket hypothesis in four steps, which we'll discuss one by one to help clarify the concept.

Figure 4.1 用四步说明彩票假设训练流程，下文逐步拆解。

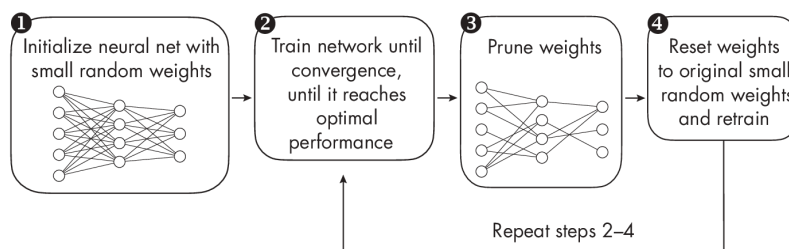


Figure 4.1

In Figure 4.1, we start with a large neural network that we train until convergence, meaning we put in our best efforts to make it perform as well as possible on a target dataset (for example, minimizing training loss and maximizing classification accuracy). This large neural network is initialized as usual using small random weights.

第 1 步：用常规小随机初始化训练大 neural network 至收敛（在目标任务上尽量降低损失、提高精度）。

Next, as shown in Figure 4.1, we prune the neural network's weight parameters, removing them from the network. We can do this by setting the weights to zero to create sparse weight matrices. Here, we can either prune individual weights, known as *unstructured* pruning, or prune larger "chunks" from the network, such as entire convolutional filter channels. This is known as *structured* pruning.



第 2 步：对权重 `prune`，可置零得到稀疏矩阵；可逐权重 *unstructured* 剪枝，也可整通道等 *structured* 剪枝。

Tips: 剪枝时，有两种方式，一种是 `unstructured pruning`，一种是 `structured pruning`。他们的差异是，`unstructured pruning` 是逐个剪枝，而 `structured pruning` 是剪枝整个 卷积核。

The original lottery hypothesis approach follows a concept known as *iterative magnitude pruning*, where the weights with the lowest magnitudes are removed in an iterative fashion. (We will revisit this concept in Chapter [\[ch06\]](#) when discussing techniques to reduce overfitting.)

原始方法采用 *iterative magnitude pruning*: 反复剪掉幅值最小的权重（第 6 章还会从过拟合角度再谈）。

Tips: 迭代剪枝 `iterative magnitude pruning`。

After the pruning step, we reset the weights to the original small random values used in step 1 in Figure [4.1](#) and train the pruned network. It's worth emphasizing that we do not reinitialize the pruned network with any small random weights (as is typical for iterative magnitude pruning), and instead we reuse the weights from step 1.

剪枝后把保留下来的权重重置为第 1 步训练开始时的同一组小随机初值（而非对剪枝后网络全新随机初始化），再训练剪枝子网。

Tips: 剪枝后，我们重置权重为原始小随机值，并训练剪枝后的网络。??? 没理解 FIXME

We then repeat the pruning steps 2 through 4 until we reach the desired network size. For example, in the original lottery ticket hypothesis paper, the authors successfully reduced the network to 10 percent of its original size without sacrificing classification accuracy. As a nice bonus, the pruned (sparse) network, referred to as the *winning ticket*, even demonstrated improved generalization performance compared to the original (large and dense) network.



随后重复 2-4 步直至达到目标规模；原文可将网络缩至约 10% 参数量而不损精度，且稀疏 *winning ticket* 有时泛化更好。

Practical Implications and Limitations

本节讨论：若总能找到小网等价大网，对训练成本与推理部署的意义；以及目前仍需先训大网、初始化敏感等现实障碍。

If it's possible to identify smaller subnetworks that have the same predictive performance as their up-to-10-times-larger counterparts, this can have significant implications for both neural training and inference. Given the ever-growing size of modern neural network architectures, this can help cut training costs and infrastructure.

若能稳定找到与十倍规模大网同等性能的小子网，将显著影响训练与推理成本，对日益膨胀的模型架构尤为重要。

Tips: 如果可以识别出与完整网络具有相同预测性能的较小子网络，这对于神经网络的 **训练** 和 **推理** 都有显著的影响，可以显著 **降低训练成本** 和 **基础设施成本**。

Sound too good to be true? Maybe. If winning tickets can be identified efficiently, this would be very useful in practice. However, at the time of writing, there is no way to find the winning tickets without training the original network. Including the pruning steps would make this even more expensive than a regular training procedure. Moreover, after the publication of the original paper, researchers found that the original weight initialization may not work to find winning tickets for larger-scale networks, and additional experimentation with the initial weights of the pruned networks is required.

但现实是：若不先训练完整大网就难以找到中奖彩票，加上迭代剪枝往往比普通训练更贵；后续研究还发现原始初始化对大模型未必有效，需对剪枝子网初值做更多探索。

The good news is that winning tickets do exist. Even if it's currently not possible to identify them without training their larger neural network counterparts, they can be used for more efficient inference after training.

好消息是 winning ticket 确实存在；即便发现过程仍依赖大网训练，训练完成后仍可用于更高效推理。



Exercises

本节练习：子网效果不佳时的下一步；以及 ReLU 的 $\max(0, x)$ 形式与彩票假设训练的关系。

4-1. Suppose we're trying out the lottery ticket hypothesis approach and find that the performance of the subnetwork is not very good (compared to the original network). What next steps might we try?

4-1. 若按彩票假设得到的子网性能明显弱于原网，可尝试哪些下一步？

4-2. The simplicity and efficiency of the rectified linear unit (ReLU) activation function have made it one of the most popular activation functions in neural network training, particularly in deep learning, where it helps to mitigate problems like the vanishing gradient. The ReLU activation function is defined by the mathematical expression $\max(0, x)$. This means that if the input x is positive, the function returns x , but if the input is negative or 0, the function returns 0. How is the lottery ticket hypothesis related to training a neural network with ReLU activation functions?

4-2. ReLU 定义为 $\max(0, x)$ ，广泛用于缓解梯度消失。它与用 ReLU 训练的网络及彩票假设之间有何联系？

References

本节列出彩票假设原文、结构化剪枝、线性模式连通性后续工作及随机权重子网络等相关文献链接。

- The paper proposing the lottery ticket hypothesis: Jonathan Frankle and Michael Carbin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks"? (2018), <https://arxiv.org/abs/1803.03635>.
- 彩票假设原文 (2018): <https://arxiv.org/abs/1803.03635>。



- The paper proposing structured pruning for removing larger parts, such as entire convolutional filters, from a network: Hao Li et al., "Pruning Filters for Efficient ConvNets"? (2016), <https://arxiv.org/abs/1608.08710>.
 - 结构化剪枝（整卷积核）(2016): <https://arxiv.org/abs/1608.08710>。
 - Follow-up work on the lottery hypothesis, showing that the original weight initialization may not work to find winning tickets for larger-scale networks, and additional experimentation with the initial weights of the pruned networks is required: Jonathan Frankle et al., "Linear Mode Connectivity and the Lottery Ticket Hypothesis"? (2019), <https://arxiv.org/abs/1912.05671>.
 - 线性模式连通性与大网初值问题 (2019): <https://arxiv.org/abs/1912.05671>。
 - An improved lottery ticket hypothesis algorithm that finds smaller networks that match the performance of a larger network exactly: Vivek Ramanujan et al., "What's Hidden in a Randomly Weighted Neural Network"? (2020), <https://arxiv.org/abs/1911.13299>.
 - 随机初始化网络中隐藏小网 (2020): <https://arxiv.org/abs/1911.13299>。
-



Chapter 5: Reducing Overfitting with Data

本章围绕如何通过获取更多或改造训练数据来减轻神经网络监督学习中的过拟合现象展开说明。

Suppose we train a neural network classifier in a supervised fashion and notice that it suffers from overfitting. What are some of the common ways to reduce overfitting in neural networks through the use of altered or additional data?

假设我们用监督方式训练神经网络分类器时发现出现过拟合，那么在仅靠改动或补充数据的前提下，有哪些常用手段可以减轻这种现象？

Overfitting, a common problem in machine learning, occurs when a model fits the training data too closely, learning its noise and outliers rather than the underlying pattern. As a result, the model performs well on the training data but poorly on unseen or test data. While it is ideal to prevent overfitting, it's often not possible to completely eliminate it. Instead, we aim to reduce or minimize overfitting as much as possible.

过拟合是机器学习中的常见问题：模型与训练数据贴得过紧，把噪声和异常也学进去，而不是学到背后的规律，因而在训练集上表现好、在未见过或测试数据上表现差。尽管理想情况是彻底防止过拟合，但往往无法完全消除，只能尽量减轻。

The most successful techniques for reducing overfitting revolve around collecting more high-quality labeled data. However, if collecting more labeled data is not feasible, we can **augment** the existing data or leverage unlabeled data for **pretraining**.

减轻过拟合最成功的做法仍是收集更多高质量标注数据；若做不到，则可以对现有数据做 **augment**（增强），或利用无标注数据做 **pretraining**（预训练）。

Tips: 减少 **过拟合**，最有效的技术是收集更多 **高质量的标签数据**；此外，还可以使用 **数据增强** 和 **预训练** 等技术。



Common Methods

本节将数据集相关的经典抗过拟合手段归为获取更多数据、数据增强与预训练几类并加以概述。

This chapter summarizes the most prominent examples of dataset-related techniques that have stood the test of time, grouping them into the following categories: `collecting more data`, `data augmentation`, and `pretraining`.

本章归纳经受住时间考验的数据集层面技术，并把它们归为：`collecting more data`（收集更多数据）、`data augmentation`（数据增强）和 `pretraining`（预训练）。

Collecting More Data

本节说明为何扩充训练数据能改善泛化，以及如何通过学习曲线判断模型是否还能从更多数据中获益。

One of the best ways to reduce overfitting is to collect more (good-quality) data. We can plot learning curves to find out whether a given model would benefit from more data. To construct a learning curve, we train the model to different training set sizes (10 percent, 20 percent, and so on) and evaluate the trained model on the same fixed-size validation or test set. As shown in Figure [5.1](#), the validation accuracy increases as the training set sizes increase. This indicates that we can improve the model's performance by collecting more data.

减轻过拟合的途径之一是收集更多（高质量）数据。可通过绘制学习曲线判断模型是否还能从更多数据中获益：在不同训练集比例（10%、20% 等）下训练同一模型，并在固定大小的验证集或测试集上评估；如图 [5.1](#) 所示，验证准确率随训练规模增大而上升，说明收集更多数据有望提升性能。

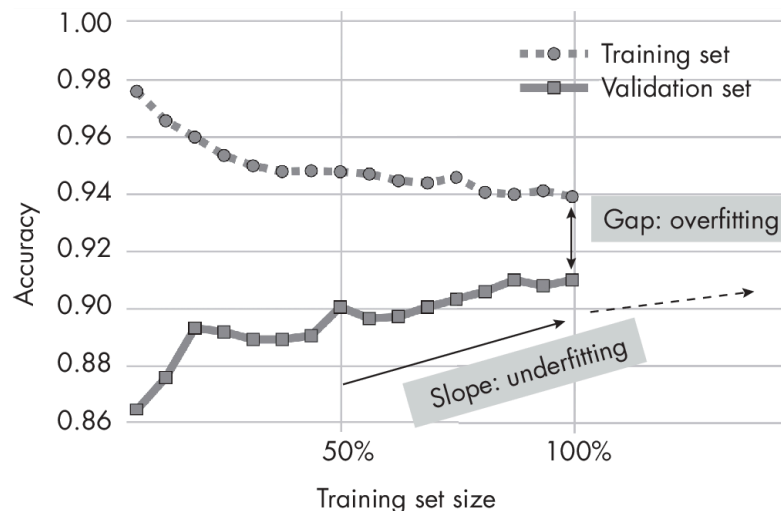


Figure 5.1

The gap between training and validation performance indicates the degree of overfitting--the more extensive the gap, the more overfitting occurs. Conversely, the slope indicating an improvement in the validation performance suggests the model is **underfitting** and can benefit from more data. Typically, **additional data** can decrease both **underfitting** and **overfitting**.

训练与验证表现的差距反映过拟合程度：差距越大过拟合越严重；反之，验证性能仍有改善空间则说明模型处于 **underfitting**（欠拟合），还能从更多数据中获益。一般而言，**additional data**（额外数据）可同时缓解 **underfitting** 与 **overfitting**。

Data Augmentation

本节介绍数据增强的含义、图像领域常见做法，以及与合成数据生成的关系。

Data augmentation refers to generating new data records or features based on existing data. It allows for the expansion of a dataset without additional data collection.

Data augmentation（数据增强）指在现有数据基础上生成新的样本或特征，从而在不另行采集的情况下扩大数据集规模。

Tips: 数据增强 **data augmentation**，是一种常用的技术，用于增加数据集的大小和多样性。它通过 **生成新的数据**，来扩展数据集，而不需要额外的数据收集。

Data augmentation allows us to create different versions of the original input data, which can improve the model's generalization performance. Why? Augmented data can help the model improve its ability to generalize, since it makes it harder to memorize spurious information via training examples or features--or, in the case of image data, exact pixel values for specific pixel locations.

数据增强能基于原始输入构造多种变体，从而提升泛化；因为增强后的数据让模型更难靠死记硬背训练样本、特征或（对图像而言）特定位置的像素值来记住虚假规律。

Tips: 数据增强 **data augmentation**，可以提高模型的泛化性能。为什么？因为增强后的数据，使模型忽略 **虚假信息**，典型场景：在 **图像数据** 中，会弱化 **特定像素** 的 **像素值**。

Figure 5.2 highlights common image data augmentation techniques, including **increasing brightness**, **flipping**, and **cropping**.

图 5.2 展示了常见的图像数据增强手段，包括 **increasing brightness**（提高亮度）、**flipping**（翻转）和 **cropping**（裁剪）。

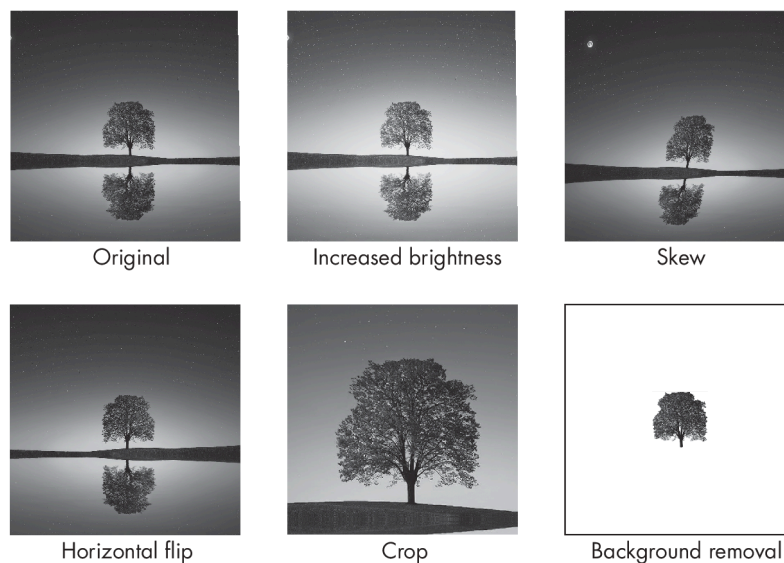


Figure 5.2



Data augmentation is usually standard for image data (see Figure 5.2) and text data (discussed further in Chapter [ch15], but data augmentation methods for tabular data also exist.

对图像数据（见图 5.2）和文本数据（第十五章将进一步论述）而言，数据增强常常是标配；表格数据也存在相应增强方法。

Instead of collecting more data or augmenting existing data, it is also possible to generate new, **synthetic data**. While more common for image data and text, generating synthetic data is also possible for tabular datasets.

除采集更多数据或增强现有数据外，还可生成新的 **synthetic data**（合成数据）；这在图像与文本上更常见，但表格数据集同样可以合成。

Tips:数据增强 **data augmentation**，是图像数据和文本数据的标准技术。除了 **数据增强**，还可以 **生成合成数据**。

Pretraining

本节说明如何利用大规模无标注数据的自监督预训练，以及迁移学习与少样本学习在缓解小数据集过拟合中的作用。

As discussed in Chapter [ch02], self-supervised learning lets us leverage large, unlabeled datasets to pretrain neural networks. This can also help reduce overfitting on the smaller target datasets.

如第二章所述，自监督学习可利用大规模无标注数据预训练神经网络，从而在较小的目标任务数据上减轻过拟合。

As an alternative to **self-supervised learning**, **traditional transfer learning** on large labeled datasets is also an option. Transfer learning is most effective if the labeled dataset is closely related to the target domain. For instance, if we train a model to classify bird species, we can pretrain a network on a large, general animal classification dataset. However, if such a large animal classification dataset is unavailable, we can also pretrain the model on the relatively broad ImageNet dataset.



除 `self-supervised learning`（自监督学习）外，也可在大规模有标注数据上做 `traditional transfer learning`（传统迁移学习）；源域与目标域越接近效果越好。例如训练鸟类分类器时，可先在大规模通用动物分类数据上预训练；若没有这类数据，也可用覆盖面更广的 ImageNet 预训练。

A dataset may be extremely small and unsuitable for supervised learning—for example, if it contains only a handful of labeled examples per class. If our classifier needs to operate in a context where the collection of additional labeled data is not feasible, we may also consider `few-shot learning`.

某些数据集可能极小，以至于不适合监督学习——例如每类只有屈指可数的标注样本；若分类器必须在难以继续获取标注的场景下运行，也可考虑 `few-shot learning`（少样本学习）。

Other Methods

本节列举若干其他与数据或训练设置相关的抗过拟合思路，并预告下一章将从模型角度继续讨论。

The previous sections covered the main approaches to using and modifying datasets to reduce overfitting. However, this is not an exhaustive list. Other common techniques include:

前文涵盖了利用与改造数据集缓解过拟合的主要路径，但并非穷举。其他常见手段包括：

- Feature engineering and normalization

特征工程与特征归一化。

- The inclusion of adversarial examples and label or feature noise

引入对抗样本以及标签或特征噪声。

- Label smoothing

标签平滑（label smoothing）。

- Smaller batch sizes



更小的批量大小。

- Data augmentation techniques such as Mixup, Cutout, and CutMix

Mixup、Cutout、CutMix 等数据增强类方法。

Tips: 减弱 **过度拟合**，还可以使用下述技术：

- **特征工程** 和 **归一化**：改进特征选择和标准化数据
- **对抗样本** 和 **标签或特征噪声**：添加对抗样本或噪声来增强模型的鲁棒性
- **标签平滑**：软化标签，避免模型对训练标签过于自信
- **更小的批量大小**：使用较小的batch size来增加训练的随机性
- **数据增强** 技术，如 **Mixup**、**Cutout** 和 **CutMix**
 - **Mixup**：混合不同样本的数据
 - **Cutout**：随机遮挡图像的部分区域
 - **CutMix**：将一张图像的一部分替换为另一张图像的对应部分

The next chapter covers additional techniques to reduce overfitting from a model perspective, and it concludes by discussing which regularization techniques we should consider in practice.

下一章将从模型与训练角度补充更多抗过拟合技术，并讨论实践中应如何选择正则化策略。

Exercises

本章习题围绕迁移学习可行性以及不当增强导致精度下降的原因展开思考。

5-1. Suppose we train an XGBoost model to classify images based on manually extracted features obtained from collaborators. The dataset of labeled training examples is relatively small, but fortunately, our collaborators also have a labeled training set from an older project on a related domain. We're considering implementing a transfer learning approach to train the XGBoost model. Is this a feasible option? If so, how could we do it? (Assume we are allowed to use only XGBoost and not another classification algorithm or model.)



习题 5-1：假设我们用合作方提供的手工特征训练 XGBoost 做图像分类，当前标注训练集较小，但对方在相近领域还有一个旧项目的标注集；我们想在仅用 XGBoost、不能用其他算法的前提下做迁移学习，这是否可行？若可行应如何做？

5-2. Suppose we're working on the image classification problem of implementing MNIST-based handwritten digit recognition. We've added a decent amount of data augmentation to try to reduce overfitting. Unfortunately, we find that the classification accuracy is much worse than it was before the augmentation. What are some potential reasons for this?

习题 5-2：在 MNIST 手写数字识别任务中，我们加入了较多数据增强以期减轻过拟合，却发现准确率反而明显下降，可能有哪些原因？

References

本节列出表格数据增强、合成数据、预处理、噪声标签、批量与学习率、对抗训练、标签平滑与 Mixup 等相关文献与资料链接。

- A paper on data augmentation for tabular data: Derek Snow, "DeltaPy: A Framework for Tabular Data Augmentation in Python"? (2020), <https://github.com/firmai/deltapy>.

Snow 等人关于表格数据增强框架 DeltaPy 的介绍（2020），仓库见链接。

- The paper proposing the GReaT method for generating synthetic tabular data using an auto-regressive generative large language model: Vadim Borisov et al., "Language Models Are Realistic Tabular Data Generators"? (2022), <https://arxiv.org/abs/2210.06280>.

Borisov 等提出用自回归生成式大语言模型合成表格数据的 GReaT 方法（2022）。

- The paper proposing the TabDDPM method for generating synthetic tabular data using a diffusion model: "Modelling Tabular Data with Diffusion Models"? (2022), <https://arxiv.org/abs/2209.15421>.

Kotelnikov 等提出用扩散模型生成合成表格数据的 TabDDPM（2022）。



- Scikit-learn's user guide offers a section on preprocessing data, featuring techniques like feature scaling and normalization that can enhance your model's performance: <https://scikit-learn.org/stable/modules/preprocessing.html>.

Scikit-learn 用户指南中的数据预处理章节，涵盖特征缩放与归一化等有助于模型性能的技术。

- A survey on methods for robustly training deep models with noisy labels that explores techniques to mitigate the impact of incorrect or misleading target values: Bo Han et al., "A Survey of Label-noise Representation Learning: Past, Present and Future"? (2020), <https://arxiv.org/abs/2011.04406>.

Han 等关于在噪声标签下稳健训练深度模型的综述（2020）。

- Theoretical and empirical evidence to support the idea that controlling the ratio of batch size to learning rate in stochastic gradient descent is crucial for good modeling performance in deep neural networks: Fengxiang He, Tongliang Liu, and Dacheng Tao, "Control Batch Size and Learning Rate to Generalize Well: Theoretical and Empirical Evidence"? (2019), <https://dl.acm.org/doi/abs/10.5555/3454287.3454390>.

He、Liu、Tao 关于随机梯度下降中批量大小与学习率比例对深度网络泛化至关重要的理论与实证研究（2019）。

- Inclusion of adversarial examples, which are input samples designed to mislead the model, can improve prediction performance by making the model more robust: Cihang Xie et al., "Adversarial Examples Improve Image Recognition"? (2019), <https://arxiv.org/abs/1911.09665>.

Xie 等表明引入旨在误导模型的对抗样本可通过增强鲁棒性改善预测性能（2019）。

- Label smoothing is a regularization technique that mitigates the impact of potentially incorrect labels in the dataset by replacing hard 0 and 1 classification targets with softened values: Rafael Müller, Simon Kornblith, and Geoffrey Hinton, "When Does Label Smoothing Help"? (2019), <https://arxiv.org/abs/1906.02629>.

Müller、Kornblith、Hinton 提出标签平滑：用软化后的目标替代硬 0/1 标签以减轻错误标注影响（2019）。



- Mixup, a popular method that trains neural networks on blended data pairs to improve generalization and robustness: Hongyi Zhang et al., "Mixup: Beyond Empirical Risk Minimization"? (2018), <https://arxiv.org/abs/1710.09412>.

Zhang 等提出的 Mixup：在成对混合样本上训练网络以提升泛化与鲁棒性（2018）。



Chapter 6: Reducing Overfitting with Model Modifications

本章说明在已运用数据层面手段之后，如何通过修改模型结构与训练流程进一步抑制过拟合。

Suppose we train a neural network classifier in a supervised fashion and already employ various dataset-related techniques to mitigate overfitting. How can we change the model or make modifications to the training loop to further reduce the effect of overfitting?

假设我们已在监督训练中采用了多种与数据集相关的抗过拟合措施，还能如何通过改动模型或训练环节来进一步削弱过拟合的影响？

The most successful approaches against overfitting include **regularization** techniques like **dropout** and **weight decay**. As a rule of thumb, models with a larger number of parameters require more training data to generalize well. Hence, decreasing the model size and capacity can sometimes also help reduce overfitting. Lastly, building ensemble models is among the most effective ways to combat overfitting, but it comes with increased computational expense.

对抗过拟合较成功的路径包括 **dropout**、**weight decay** 等 **regularization**（正则化）技术；经验上参数量越大往往需要越多训练数据才能泛化良好，因此适度减小模型容量有时也有帮助；此外，集成多个模型效果很好，但计算开销更高。

Tips: 减少 **过拟合**，最有效的技术是 **正则化**，包括 **dropout** 和 **权重衰减**；此外，还可以 **减小模型大小** 和 **构建集成模型**。

This chapter outlines the key ideas and techniques for several categories of reducing overfitting with model modifications and then compares them to one another. It concludes by discussing how to choose between all types of overfitting reduction methods, including those discussed in the previous chapter.

本章先按若干类别梳理通过模型修改减轻过拟合的核心思路与技术并相互对照，最后讨论如何在包含上一章数据手段在内的全部方法之间做出取舍。



Common Methods

本节将模型与训练相关的抗过拟合技术归为正则化、选用更小模型与构建集成模型三大类。

The various model- and training-related techniques to reduce overfitting can be grouped into three broad categories: (1) adding **regularization**, (2) choosing **smaller models**, and (3) building **ensemble models**.

与模型和训练相关的减拟合手段可粗略归为三类：（1）加入 **regularization**（正则化），（2）选择 **smaller models**（更小的模型），（3）构建 **ensemble models**（集成模型）。

Regularization

本节解释正则化及其典型实现，并概述 **dropout** 与早停等作用机制。

We can interpret regularization as a penalty against complexity. Classic regularization techniques for neural networks include L_2 regularization and the related weight decay method. We implement L_2 regularization by adding a penalty term to the loss function that is minimized during training. This added term represents the size of the weights, such as the squared sum of the weights. The following formula shows an L_2 regularized loss

可将正则化理解为对模型复杂度的惩罚；神经网络的经典做法包括 L_2 正则化及相关的权重衰减：在训练最小化的损失上加上刻画权重大小（如权重平方和）的惩罚项。下列公式给出带 L_2 正则的损失：

$$RegularizedLoss = Loss + \frac{\lambda}{n} \sum_j w_j^2$$

where λ is a **hyperparameter** that controls the **regularization strength**.

其中 λ 为控制 **regularization strength**（正则化强度）的 **hyperparameter**（超参数）。



During backpropagation, the optimizer minimizes the modified loss, now including the additional penalty term, which leads to smaller model weights and can improve generalization to unseen data.

反向传播中最小化加入惩罚项后的总损失，会使权重趋向更小，从而有望改善对未见数据的泛化。

Tips: 正则化 **regularization**，通过 **添加惩罚项**，来减少模型的 **权重**。

Weight decay is similar to L_2 regularization but is applied to the optimizer directly rather than modifying the loss function. Since weight decay has the same effect as L_2 regularization, the two methods are often used synonymously, but there may be subtle differences depending on the implementation details and optimizer.

Weight decay（权重衰减）与 L_2 正则类似，但直接作用在优化器而非改写损失；二者效果常被视作等价而混用，具体实现与优化器不同时仍可能有细微差别。

Many other techniques have regularizing effects. For brevity's sake, we'll discuss just two more widely used methods: **dropout** and **early stopping**.

还有许多技术具有正则化效果；为简短起见，这里再介绍两种常用方法：**dropout** 与 **early stopping**（早停）。

Dropout reduces overfitting by randomly setting some of the activations of the hidden units to zero during training. Consequently, the neural network cannot rely on particular neurons to be activated. Instead, it learns to use a larger number of neurons and multiple independent representations of the same data, which helps to reduce overfitting.

Dropout 在训练时随机将部分隐藏单元激活置零，使网络无法依赖少数神经元；因而会动用更多神经元并形成多种独立表征，从而有助于减轻过拟合。

In **early stopping**, we monitor the model's performance on a validation set during training and stop the training process when the performance on the validation set begins to decline, as illustrated in Figure 6.1.

在 **early stopping** 中，我们在训练过程中监控模型在验证集上的表现，并在验证性能开始下滑时终止训练，见图 6.1。

Tips: 早停 **early stopping**，通过 **监控模型在验证集上的性能**，来停止训练过程。

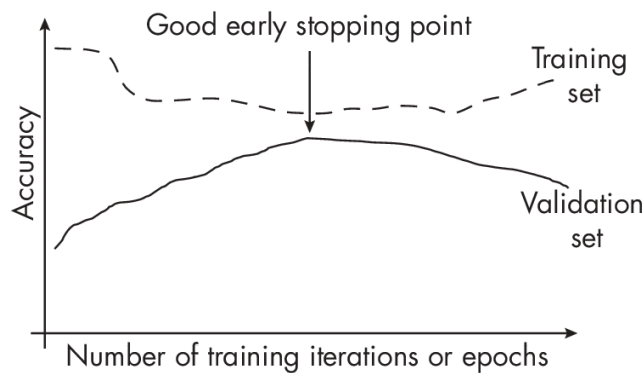


Figure 6.1

In Figure 6.1, we can see that the validation accuracy increases as the training and validation accuracy gap closes. The point where the training and validation accuracy is closest is the point with the least amount of overfitting, which is usually a good point for early stopping.

从图 6.1 可见，随着训练与验证准确率差距收窄，验证准确率上升；二者最接近的点通常过拟合最轻，也是早停的较佳截断点。

Smaller Models

本节讨论剪枝、知识蒸馏等缩小模型规模的思路及其在训练流程中的角色。

Classic bias-variance theory suggests that reducing model size can reduce overfitting. The intuition behind this theory is that, as a general rule of thumb, the smaller the number of model parameters, the smaller its capacity to memorize or overfit to noise in the data. The following paragraphs discuss methods to reduce the model size, including **pruning**, which removes parameters from a model, and **knowledge distillation**, which transfers knowledge to a smaller model.

经典偏差-方差理论认为缩小模型有助于抑制过拟合；直观上参数量越少，记住数据噪声的能力通常越弱。下文讨论减小模型规模的方法，包括移除参数的

pruning（剪枝）以及向更小模型迁移知识的 **knowledge distillation**（知识蒸馏）。



Tips: 减小模型大小，包括 **剪枝** 和 **知识蒸馏**。

Besides reducing the number of layers and shrinking the layers' widths as a hyperparameter tuning procedure, another approach to obtaining smaller models is **iterative pruning**, in which we train a large model to achieve good performance on the original dataset. We then iteratively remove parameters of the model, retraining it on the dataset such that it maintains the same predictive performance as the original model. (The lottery ticket hypothesis, discussed in Chapter [\[ch04\]](#), uses iterative pruning.)

除通过调参减少层数与宽度外，还可采用 **iterative pruning**（迭代剪枝）：先训练大模型在原数据上达到较好性能，再反复删参并在数据集上重训，使其保持与原模型相当的预测能力。（上文括号中所述彩票假设亦采用迭代剪枝。）

Tips: 前置减少模型的参数，包括层数和宽度。后置的 **迭代剪枝**，也是常用方法。

Another common approach to obtaining smaller models is **knowledge distillation**. The general idea behind this approach is to transfer knowledge from a large, more complex model (the *teacher*) to a smaller model (the *student*). Ideally, the student achieves the same predictive accuracy as the teacher, but it does so more efficiently due to the smaller size. As a nice side effect, the smaller student may overfit less than the larger teacher model.

另一条常见路径是 **knowledge distillation**：把大模型（*teacher*）的知识迁移到小模型（*student*）；理想情况下学生以更小体量达到与教师相当的准确率，并往往比大教师更少过拟合。

Figure [6.2](#) diagrams the original knowledge distillation process. Here, the **teacher** is first trained in a regular supervised fashion to classify the examples in the dataset well, using a conventional cross-entropy loss between the predicted scores and ground truth class labels. While the smaller **student** network is trained on the same dataset, the training objective is to minimize both



图 6.2 示意经典蒸馏流程：先用常规监督与交叉熵把 **teacher** 训练好；在同一数据集上训练较小的 **student** 时，目标同时最小化

(a) the cross entropy between the outputs and the class labels and

(a) 学生输出与类别标签之间的交叉熵，以及

(b) the difference between its outputs and the teacher outputs (measured using *Kullback-Leibler* divergence, which quantifies the difference between two probability distributions by calculating how much one distribution diverges from the other in terms of information content).

(b) 学生与教师输出之差（用 *Kullback-Leibler* 散度量两分布的信息差异）。

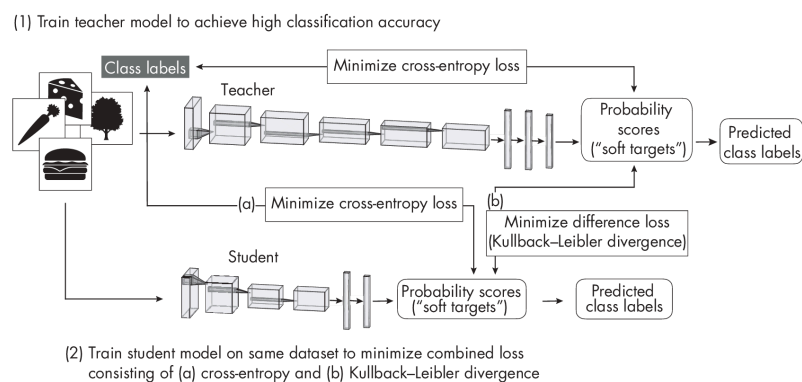


Figure 6.2

By minimizing the Kullback-Leibler divergence--the difference between the teacher and student score distributions--the student learns to mimic the teacher while being smaller and more efficient.

通过最小化师生分数分布间的 KL 散度，学生以更紧凑的结构模仿教师。

Tips: 知识蒸馏 **knowledge distillation**，通过 将知识从大模型，蒸馏到 **小模型**，来提高小模型的性能。

Caveats with Smaller Models

本节澄清剪枝与蒸馏带来的泛化提升并不能简单等同于首选的抗过拟合手段。



While pruning and knowledge distillation can also enhance a model's generalization performance, these techniques are not primary or effective ways of reducing overfitting.

剪枝与知识蒸馏虽也可能改善泛化，但它们并非抑制过拟合的首要或最有效途径。

Tips: 剪枝 和 知识蒸馏，可以 提高模型的泛化性能，但 不是 减少过拟合的 主要 方法。

Early research results indicate that pruning and knowledge distillation can improve the generalization performance, presumably due to smaller model sizes. However, counterintuitively, recent research studying phenomena like double descent and grokking also showed that larger, overparameterized models have improved generalization performance if they are trained beyond the point of overfitting. **Double descent** refers to the phenomenon in which models with either a small or an extremely large number of parameters have good generalization performance, while models with a number of parameters equal to the number of training data points have poor generalization performance. *Grokking* reveals that as the size of a dataset decreases, the need for optimization increases, and generalization performance can improve well past the point of overfitting.

早期研究表明剪枝与蒸馏或因模型变小而改善泛化；但近年关于 double descent、grokking 等现象的工作又指出，过参数化的大模型若在过拟合点之后继续训练，泛化也可能变好。**Double descent**（双降）指参数量很小或极大时泛化较好，而参数量约等于训练样本数时反而较差；*Grokking* 则揭示数据规模缩小时优化需求上升，泛化可在明显过拟合之后仍持续改善。

Tips: 双降 **double descent**，是一种现象，模型参数的数量，在最佳量级之前和之后，都模型泛化效果，都会变差。但是，涌现/顿悟 **grokking** 现象，展示出当模型参数数量超大时，泛化性能又会变好。??? FIXME

How can we reconcile the observation that pruned models can exhibit better generalization performance with contradictory observations from studies of double descent and grokking? Researchers recently showed that the improved training process partly explains the reduction of overfitting due to pruning.



Pruning involves more extended training periods and a replay of learning rate schedules that may be partly responsible for the improved generalization performance.

如何把剪枝模型泛化变好与双降、grokking 的看似矛盾结论统一起来？新近研究认为训练过程的改进部分解释了剪枝带来的过拟合缓解：剪枝往往伴随更长训练与学习率日程的重演，这些都可能促成泛化提升。

Pruning and knowledge distillation remain excellent ways to improve the computational efficiency of a model. However, while they can also enhance a model's generalization performance, these techniques are not primary or effective ways of reducing overfitting.

剪枝与蒸馏仍是提升计算效率的利器；但若论抑制过拟合，它们依然不是首选或最有效的单独手段。

Tips: 剪枝和知识蒸馏，可以提高模型的泛化性能，但不是减少过拟合的主要方法。

Ensemble Methods

本节说明集成学习的基本直觉、常见组合方式及其代价与适用场景。

Ensemble methods combine predictions from multiple models to improve the overall prediction performance. However, the downside of using multiple models is an increased computational cost.

集成方法合并多模型预测以提升整体表现，但使用多模型也意味着更高的计算成本。

We can think of ensemble methods as asking a committee of experts to weigh in on a decision and then combining their judgments in some way to make a final decision. Members in a committee often have different backgrounds and experiences. While they tend to agree on basic decisions, they can overrule bad decisions by majority rule. This doesn't mean that the majority of experts is always right, but there is a good chance that the majority of the committee is more often right, on average, than every single member.



可把集成想象成专家委员会分别表态再汇总结决：成员背景各异，在基础问题上往往一致，又能用多数票否决明显糟糕的判断；多数并不保证永远正确，但平均而言多数意见常常优于任一单个成员。

The most basic example of an ensemble method is majority voting. Here, we train k different classifiers and collect the predicted class label from each of these k models for a given input. We then return the most frequent class label as the final prediction. (Ties are usually resolved using a confidence score, randomly picking a label, or picking the class label with the lowest index.)

最简单的集成是多数投票：训练 k 个分类器，对同一输入收集 k 个类别预测，再取出现次数最多的标签（平局时常靠置信度、随机或取最小索引类别等方式打破）。

Ensemble methods are more prevalent in classical machine learning than deep learning because it is more computationally expensive to employ multiple models than to rely on a single one. In other words, deep neural networks require significant computational resources, making them less suitable for ensemble methods.

集成在传统机器学习中更常见，因为深度学习下单模型已很昂贵，再复制多份更难承受。

Random forests and gradient boosting are popular examples of ensemble methods. However, by using majority voting or stacking, for example, we can combine any group of models: an ensemble may consist of a support vector machine, a multilayer perceptron, and a nearest-neighbor classifier. Here, stacking (also known as *stacked generalization*) is a more advanced variant of majority voting that involves training a new model to combine the predictions of several other models rather than obtaining the label by majority vote.

随机森林与梯度提升是集成范例；也可用多数投票或 stacking 等任意组合模型——例如 SVM、多层感知机与 kNN 并存。stacking（亦称 *stacked generalization*）比简单多数票更进一步，用新模型去学习如何融合若干基模型输出，而不是仅靠 majority vote 表决得到标签。

A popular industry technique is to build models from *k-fold cross-validation*, a model evaluation technique in which we train and evaluate a model on k training folds. We then compute the average performance metric across all k iterations to estimate the overall performance measure of the model. After evaluation, we can



either train the model on the entire training dataset or combine the individual models as an ensemble, as shown in Figure 6.2.

工业界常基于 *k-fold cross-validation* 构造模型：在各训练折上轮流训练与评估，并对 k 轮指标取平均以估计整体性能；评估结束后既可仅用全部训练数据重训单一模型，也可将各折模型组合为集成，见图 6.2。

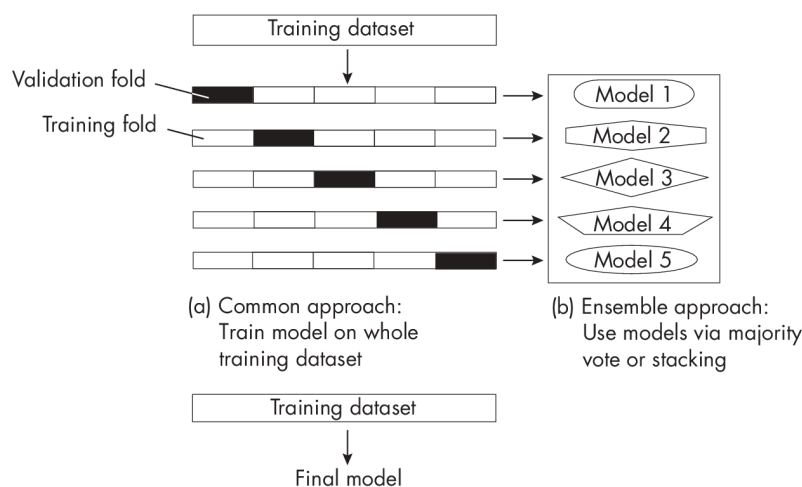


Figure 6.3

As shown in Figure 6.2, the k -fold ensemble approach trains each of the k models on the respective $k - 1$ training folds in each round. After evaluating the models on the validation folds, we can combine them into a majority vote classifier or build an ensemble using stacking, a technique that combines multiple classification or regression models via a meta-model.

如图所示， k 折集成在每轮对各模型使用相应的 $k - 1$ 份训练折；在验证折上评估结束后，可将它们合成多数投票分类器，或通过元模型采用 stacking 融合多个分类或回归模型。

While the ensemble approach can potentially reduce overfitting and improve robustness, this approach is not always suitable. For instance, potential downsides include managing and deploying an ensemble of models, which can be more complex and computationally expensive than using a single model.

集成有望降低过拟合并增强稳健性，但并非总适用：运维与部署多模型往往比单模型更复杂、更耗算力。

Other Methods

本节补充若干并非专为减拟合设计但在实践中常具正则化效果的结构与优化技巧。



So far, this book has covered some of the most prominent techniques to reduce overfitting. Chapter [ch05] covered techniques that aim to reduce overfitting from a data perspective. Additional techniques for reducing overfitting with model modifications include skip-connections (found in residual networks, for example), look-ahead optimizers, stochastic weight averaging, multitask learning, and snapshot ensembles.

本书至此已介绍多种主流抗过拟合手段；第五章从数据视角讨论了相关技术。通过模型修改还能借助残差网络中的跳跃连接、lookahead 优化器、随机权重平均、多任务学习与快照集成等进一步抑制过拟合。

While they are not originally designed to reduce overfitting, layer input normalization techniques such as batch normalization (BatchNorm) and layer normalization (LayerNorm) can stabilize training and often have a regularizing effect that reduces overfitting. Weight normalization, which normalizes the model weights instead of layer inputs, could also lead to better generalization performance. However, this effect is less direct since weight normalization (WeightNorm) doesn't explicitly act as a regularizer like weight decay does.

BatchNorm、LayerNorm 等虽非专为减拟合而设计，却能稳定训练并常带有正则化副作用；WeightNorm 归一化权重而非层输入，也可能改善泛化，但其作用不如权重衰减那样直接充当正则项。

Choosing a Regularization Technique

本节给出在数据与模型两侧多种正则化手段并存时的实践取舍建议。

Improving data quality is an essential first step in reducing overfitting. However, for recent deep neural networks with large numbers of parameters, we need to do more to achieve an acceptable level of overfitting. Therefore, data augmentation and pretraining, along with established techniques such as dropout and weight decay, remain crucial overfitting reduction methods.



提升数据质量是抑制过拟合的首要步骤；但对当今参数众多的深度网络，仅靠数据往往不够，因此数据增强、预训练以及 dropout、权重衰减等成熟手段仍然关键。

In practice, we can and should use multiple methods at once to reduce overfitting for an additive effect. To achieve the best results, treat selecting these techniques as a hyperparameter optimization problem.

实践中通常应叠加多种方法以获得加成效果；具体组合最好视作超参数搜索问题来系统调优。

Exercises

本节习题对比早停与调训练轮数，并反思集成方法的代价。

6-1. Suppose we're using early stopping as a mechanism to reduce overfitting-- in particular, a modern early-stopping variant that creates checkpoints of the best model (for instance, the model with the highest validation accuracy) during training so that we can load it after the training has completed. This mechanism can be enabled in most modern deep learning frameworks. However, a colleague recommends tuning the number of training epochs instead. What are some of the advantages and disadvantages of each approach?

习题 6-1：若用现代早停在训练中保存验证指标最佳检查点并在结束后加载，与同事主张改为调节训练轮数相比，各有何利弊？

6-2. Ensemble models have been established as a reliable and successful method for decreasing overfitting and enhancing the reliability of predictive modeling efforts. However, there's always a trade-off. What are some of the drawbacks associated with ensemble techniques?

习题 6-2：集成模型在降低过拟合与提升预测可靠性方面成效显著，但必有权衡——集成有哪些缺点？

References

本节列出权重衰减、蒸馏、偏差-方差、彩票假设、双降、grokking、剪枝、Dropout 解释及正则“鸡尾酒”调参等相关参考文献。



- For more on the distinction between L_2 regularization and weight decay: Guodong Zhang et al., "Three Mechanisms of Weight Decay Regularization"? (2018), <https://arxiv.org/abs/1810.12281>.

Zhang 等关于 L_2 正则化与 weight decay 区别的论文（2018）。

- Research results indicate that pruning and knowledge distillation can improve generalization performance, presumably due to smaller model sizes: Geoffrey Hinton, Oriol Vinyals, and Jeff Dean, "Distilling the Knowledge in a Neural Network"? (2015), <https://arxiv.org/abs/1503.02531>.

Hinton 等关于知识蒸馏的经典论文（2015）。

- Classic bias-variance theory suggests that reducing model size can reduce overfitting: Jerome H. Friedman, Robert Tibshirani, and Trevor Hastie, "Model Selection and Bias-Variance Tradeoff,"? Chapter 2.9, in *The Elements of Statistical Learning* (Springer, 2009).

《统计学习要素》中关于模型选择与偏差-方差权衡的章节（2009）。

- The lottery ticket hypothesis applies knowledge distillation to find smaller networks with the same predictive performance as the original one: Jonathan Frankle and Michael Carbin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks"? (2018), <https://arxiv.org/abs/1803.03635>.

Frankle 与 Carbin 提出彩票假设的论文（2018）。

- For more on double descent: https://en.wikipedia.org/wiki/Double_descent.

双降现象的维基条目。

- The phenomenon of grokking indicates that generalization performance can improve well past the point of overfitting: Alethea Power et al., "Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets"? (2022), <https://arxiv.org/abs/2201.02177>.

Power 等关于 grokking 的论文（2022）。



- Recent research shows that the improved training process partly explains the reduction of overfitting due to pruning: Tian Jin et al., "Pruning's Effect on Generalization Through the Lens of Training and Regularization"? (2022), <https://arxiv.org/abs/2210.13738>.

Jin 等从训练与正则视角解释剪枝如何影响泛化的研究（2022）。

- Dropout was previously discussed as a regularization technique, but it can also be considered an ensemble method that approximates a weighted geometric mean of multiple networks: Pierre Baldi and Peter J. Sadowski, "Understanding Dropout"? (2013), <https://proceedings.neurips.cc/paper/2013/hash/71f6278d140af599e06ad9bf1ba03cb0-Abstract.html>.

Baldi 与 Sadowski 将 Dropout 解释为近似多网络加权几何均值的工作（2013）。

- Regularization cocktails need to be tuned on a per-dataset basis: Arlind Kadra et al., "Well-Tuned Simple Nets Excel on Tabular Datasets"? (2021), <https://arxiv.org/abs/2106.11189>.

Kadra 等强调正则组合需按数据集调参的研究（2021）。



Chapter 7: Multi-GPU Training Paradigms

本章梳理多卡训练的主要范式及其利弊，并给出在不同规模模型下的选用建议。

What are the different multi-GPU training paradigms, and what are their respective advantages and disadvantages?

多 GPU 训练有哪些典型范式，各自优缺点又是什么？

Multi-GPU training paradigms can be categorized into two groups: dividing data for parallel processing with multiple GPUs and dividing the model among multiple GPUs to handle memory constraints when the model size surpasses that of a single GPU. Data parallelism falls into the first category, while model parallelism and tensor parallelism fall into the second category. Techniques like pipeline parallelism borrow ideas from both categories. In addition, current software implementations such as DeepSpeed, Colossal AI, and others blend multiple approaches into a hybrid technique.

多 GPU 范式大致可分两类：其一是在多块 GPU 间划分数据做并行；其二是在单卡放不下时将模型切分到多卡。数据并行属第一类；模型并行与张量并行属第二类；流水线并行则融合二者思想；DeepSpeed、Colossal AI 等框架还会混合多种策略。

This chapter introduces several training paradigms and provides advice on which to use in practice.

本章将介绍若干训练范式并讨论实践中如何取舍。

This chapter primarily uses the term **GPUs** to describe the hardware utilized for parallel processing. However, the same concepts and techniques discussed can be applied to other specialized hardware devices, such as tensor processing units (TPUs) or other accelerators, depending on the specific architecture and requirements of the system.

本章主要以 **GPUs** 指代并行硬件，但所述概念同样适用于 TPU 等其他加速器，具体取决于系统架构与需求。



The Training Paradigms

以下各小节依次讨论模型并行、数据并行、张量并行与序列并行等范式。

The following sections discuss the model parallelism, data parallelism, tensor parallelism, and sequence parallelism multi-GPU training paradigms.

下面分别阐述模型并行、数据并行、张量并行与序列并行这几类多 GPU 训练范式。

Model Parallelism

本节解释模型（层间）并行如何把大块模型分层摆放及其瓶颈。

Model parallelism, or inter-op parallelism, is a technique in which different sections of a large model are placed on different GPUs and are computed sequentially, with intermediate results passed between the devices. This allows for the training and execution of models that might not fit entirely on a single device, but it can require intricate coordination to manage the dependencies between different parts of the model.

模型并行（亦称 *inter-op parallelism*）把大模型的不同部分放到不同 GPU 上顺序计算并在设备间传递中间结果，从而运行单卡放不下的模型，但需要精细协调各部分依赖。

Model parallelism is perhaps the most intuitive form of parallelization across devices. For example, for a simple neural network that consists of only two layers—a hidden layer and an output layer—we can keep one layer on one GPU and the other layer on another GPU. Of course, this can scale to an arbitrary number of layers and GPUs.

模型并行或许是跨设备并行里最直观的一种：例如仅有隐藏层与输出层两层的小网络，可各放一块 GPU，并可推广到任意多层多卡。

This is a good strategy for dealing with limited GPU memory where the complete network does not fit into one GPU. However, there are more efficient ways of using multiple GPUs, such as tensor parallelism, because the chain-like structure



(layer 1 on GPU 1 → layer 2 on GPU 2 → ...) in model parallelism introduces a bottleneck. In other words, a major disadvantage of model parallelism is that the GPUs have to wait for each other. They cannot efficiently work in parallel, as they depend on one other's outputs.

当显存不足、整网无法装入单卡时这是可行策略；但更高效的用法是张量并行等，因为模型并行中层链（GPU1→GPU2→...）会形成瓶颈——各卡必须互相等待，难以充分并行。

Data Parallelism

本节描述默认的多卡数据并行流程及其“每卡一份完整模型”的限制。

Data parallelism has been the default mode for multi-GPU training for several years. Here, we divide a minibatch into smaller microbatches. Each GPU then processes a microbatch separately to compute the loss and loss gradients for the model weights. After the individual devices process the microbatches, the gradients are combined to compute the weight update for the next round.

数据并行 多年来是多 GPU 训练的默认模式：将小批量拆成微批量，各卡独立算损失与梯度再聚合梯度更新权重。

An advantage of data parallelism over model parallelism is that the GPUs can run in parallel. Each GPU processes a portion of the training minibatch, that is, a microbatch. However, a caveat is that each GPU requires a full copy of the model. This is obviously not feasible if we have large models that don't fit into the GPU's VRAM.

相对模型并行，数据并行能让各卡真正并行处理同一 minibatch 的不同微批量；但每卡都要驻留完整模型，模型超大无法装入显存时即不可行。

Tensor Parallelism

本节说明张量并行如何在单次运算内切分矩阵以实现更高并行度。

Tensor parallelism, or *intra-op parallelism*, is a more efficient form of model parallelism. Here, the weight and activation matrices are spread across the



devices instead of distributing whole layers across devices: the individual matrices are split, so we split an individual matrix multiplication across GPUs.

张量并行 (*intra-op parallelism*) 是更高效的模型并行：按矩阵而非整层切分，把一次矩阵乘分摊到多卡。

We can implement tensor parallelism using basic principles of linear algebra; we can split a matrix multiplication across two GPUs in a row- or column-wise fashion, as illustrated in Figure 7.1 for two GPUs. (This concept can be extended to an arbitrary number of GPUs.)

可借助线性代数把矩阵乘按行或列拆到两块 GPU 上，见图 7.1，并可推广到更多设备。

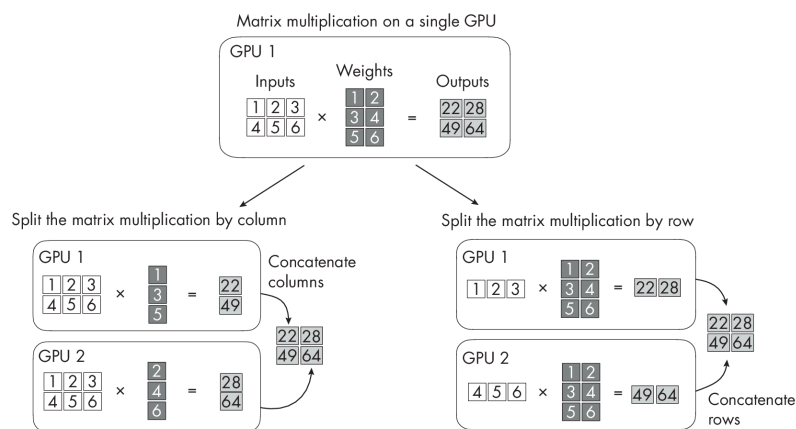


Figure 7.1

Like model parallelism, tensor parallelism allows us to work around memory limitations. At the same time, it also lets us execute operations in parallel, similar to data parallelism.

与模型并行类似，张量并行可绕过显存上限，同时又能像数据并行那样并行执行算子。

A small weakness of tensor parallelism is that it can result in high communication overhead between the multiple GPUs across which the matrices are split or sharded. For instance, tensor parallelism requires frequent synchronization of the model parameters across devices, which can slow down the overall training process.

其弱点在于跨卡通信频繁、开销大，参数同步可能拖慢整体训练。

Figure 7.2 compares model, data, and tensor parallelism.



图 7.2 对照了三种并行方式。

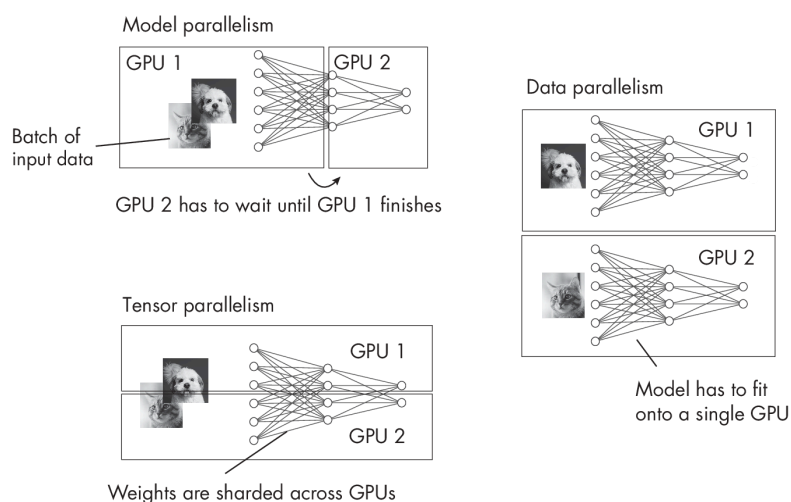


Figure 7.2

In model parallelism, we put different layers onto different GPUs to work around GPU memory limitations. In data parallelism, we split a batch across GPUs to train copies of the model in parallel, averaging gradients for the weight update afterward. In tensor parallelism, we split matrices (inputs and weights) across different GPUs for parallel processing when models are too large to fit into GPU memory.

模型并行把不同层放到不同卡以节省显存；数据并行把批次分到各卡并行训练完整模型副本再平均梯度；张量并行则在模型过大时把输入与权重矩阵切到多卡上并行计算。

Pipeline Parallelism

本节介绍流水线并行如何缓解模型并行的空闲时间及其设计与通信代价。

In *pipeline parallelism*, activations are passed during the forward pass, as in model parallelism. The twist is that the gradients of the input tensor are passed backward to prevent the devices from being idle. In a sense, pipeline parallelism is a sophisticated hybrid version of data and model parallelism.

流水线并行 前向仍像模型并行那样传递激活；区别在于反向传递输入张量的梯度以减少设备空转，可视为数据并行与模型并行的混合升级版。



We can think of pipeline parallelism as a form of model parallelism that tries to minimize the sequential computation bottleneck, enhancing the parallelism between the individual layers sitting on different devices. However, pipeline parallelism also borrows ideas from data parallelism, such as splitting minibatches further into microbatches.

它试图减轻模型并行的顺序瓶颈、提高跨设备层的并行度，同时也借鉴数据并行中再划微批的思想。

Pipeline parallelism is definitely an improvement over model parallelism, though it is not perfect and there will be idle bubbles. A further disadvantage of pipeline parallelism is that it may require significant effort to design and implement the pipeline stages and associated communication patterns. Additionally, the performance gains it generates may not be as substantial as those from other parallelization techniques, such as pure data parallelism, especially for small models or in cases where the communication overhead is high.

流水线并行相对纯模型并行明显更好，但仍会出现流水线气泡；设计与实现各阶段及通信模式成本高，且在小模型或通信昂贵场景下收益未必胜过纯数据并行。

For modern architectures that are too large to fit into GPU memory, it is more common nowadays to use a blend of data parallelism and tensor parallelism techniques instead of pipeline parallelism.

对当今显存放不下的超大架构，更常见的做法是数据并行与张量并行组合，而非单独依赖流水线并行。

Sequence Parallelism

本节聚焦长序列 Transformer 训练中序列切分并行及其局限。

Sequence parallelism aims to address computational bottlenecks when working with long sequences using transformer-based LLMs. More specifically, one shortcoming of transformers is that the self-attention mechanism (the original scaled-dot product attention) scales quadratically with the input sequence length. There are, of course, more efficient alternatives to the original attention mechanism that scale linearly.



序列并行面向基于 Transformer 的长序列场景；原始缩放点积自注意力随序列长度平方增长，虽有线性复杂度的替代注意力，但普及度不同。

However, these efficient self-attention mechanisms are less popular, and most people still prefer the original scaled-dot product attention mechanism as of this writing. Sequence parallelism, illustrated in Figure 7.3, splits the input sequence into smaller chunks to be distributed across GPUs, which aims to reduce computation memory constraints of self-attention mechanisms.

写作本书时主流仍多用原始缩放点积注意力；序列并行（图 7.3）把输入序列切段分布到多卡，以缓解自注意力的算存压力。

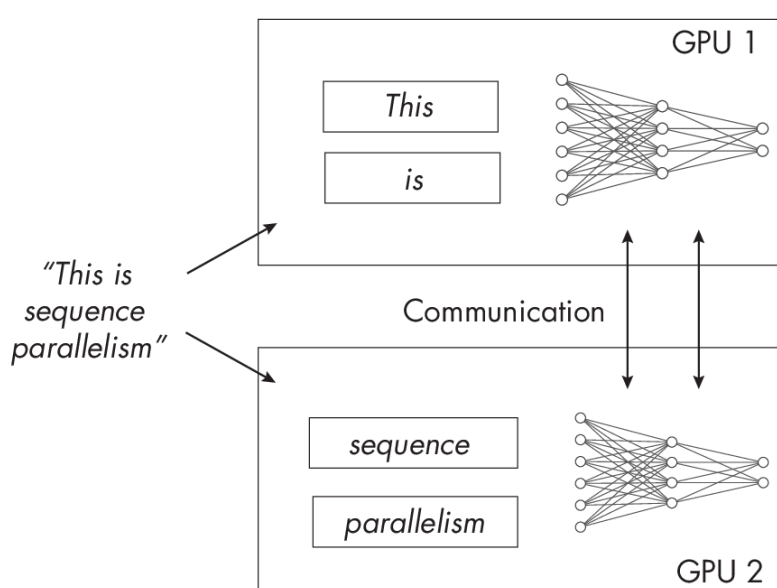


Figure 7.3

How does sequence parallelism relate to the multi-GPU techniques discussed earlier? Sequence parallelism deals specifically with sequential data, tensor parallelism deals with the model's internal structure, and data parallelism deals with how the training data is divided. Theoretically, since each of these parallelism strategies addresses a different aspect of the computational challenge, they can thus be combined in various ways to optimize the training or inference process. Sequence parallelism is not as well studied as other parallelization techniques, however.

序列并行针对序列数据维度的拆分，张量并行针对模型内部结构，数据并行针对样本划分；理论上三者关注点不同可组合优化训练或推理，但序列并行研究相对较少。



While sequence parallelism appears useful in practice, it also introduces additional communication overheads similar to the aforementioned parallelism techniques. Like data parallelism, it requires us to duplicate the model and make sure it fits into the device memory. Another of its disadvantages (depending on the implementation) for multi-GPU training of transformers is that breaking up the input sequence into smaller subsequences can decrease the model's accuracy (mainly when the model is applied to longer sequences).

实践中序列并行有用，但也会引入额外通信；与数据并行类似仍需复制模型并确保单卡可装载。某些实现下把长序列切碎还可能损伤精度（尤其在模型本就面向更长上下文时）。

Recommendations

本节依据模型是否能装入单卡等因素给出多范式组合的实践建议。

Practical recommendations depend on the context. If we train small models that fit onto a single GPU, then data parallelism strategies may be the most efficient. Performance gains from pipeline parallelism may not be as significant as those from other parallelization techniques, such as data parallelism, especially for small models or in cases where the communication overhead is high.

是否选用何种范式取决于场景：小模型可装入单卡时数据并行往往最高效；流水线并行在小模型或通信昂贵时收益有限。

If models are too large to fit into the memory of a single GPU, we need to explore model or tensor parallelism. Tensor parallelism is naturally more efficient; the GPUs can work in parallel since there is no sequential dependency as in model parallelism.

若单卡装不下，则需模型并行或张量并行；张量并行天然 more efficient（原文连写如此），各卡可无模型并行那样的顺序依赖而并行工作。

Modern multi-GPU strategies also typically combine data parallelism and tensor parallelism.

当下主流策略也多将数据并行与张量并行结合使用。



Exercises

习题探讨 Adam 下的显存问题以及在 CPU 上做数据并行的合理性。

7-1. Suppose we are implementing our own version of tensor parallelism, which works great when we train our model with a standard stochastic gradient descent optimizer. However, when we try the Adam optimizer by Diederik P. Kingma and Jimmy Ba, we encounter an out-of-memory device. What problem might explain this issue?

习题 7-1: 自研张量并行配合 vanilla SGD 训练正常, 但换成 Kingma 与 Ba 的 Adam 就出现设备 OOM, 可能原因是什么?

7-2. Suppose we don't have access to a GPU and are considering using data parallelism on the CPU. Is this a good idea?

习题 7-2: 若没有 GPU, 仅在 CPU 上做数据并行是否值得推荐?

References

本节列出 Adam、DeepSpeed、ColossalAI、流水线与序列并行及 Transformer 效率综述等参考文献。

- The original paper on the Adam optimizer: Diederik P. Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization"? (2014), <https://arxiv.org/abs/1412.6980>.

Kingma 与 Ba 提出 Adam 优化器的原始论文 (2014)。

- FormoreonDeepSpeedandColossal-AIformulti-GPUtraining: <https://github.com/microsoft/DeepSpeed> and <https://github.com/hpcaitech/ColossalAI>.

DeepSpeed 与 ColossalAI 的多 GPU 训练资料仓库链接。

- Pipeline parallelism tutorials and research by the DeepSpeed team: <https://www.deepspeed.ai/tutorials/pipeline> and Yanping Huang et al.,



"GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism"? (2018), <https://arxiv.org/abs/1811.06965>.

DeepSpeed 流水线教程及 Huang 等 GPipe 论文（2018）。

- The paper proposing sequence parallelism for transformer-based language models: Shenggui Li et al., "Sequence Parallelism: Long Sequence Training from [a] System[s] Perspective"? (2022), <https://arxiv.org/abs/2105.13120>.

Li 等面向 Transformer 语言模型提出序列并行的论文（2022）。

- The scaled-dot product attention mechanism was proposed with the original transformer architecture: Ashish Vaswani et al., "Attention Is All You Need"? (2017), <https://arxiv.org/abs/1706.03762>.

Vaswani 等在原始 Transformer 论文中提出的缩放点积注意力（2017）。

- A survey covering alternatives to the original self-attention mechanism that scale linearly: Yi Tay et al., "Efficient Transformers: A Survey"? (2020), <https://arxiv.org/abs/2009.06732>.

Tay 等关于高效（含线性复杂度）Transformer 变体的综述（2020）。

- A survey covering additional techniques to improve the training efficiency of transformers: Bohan Zhuang et al., "A Survey on Efficient Training of Transformers"? (2023), <https://arxiv.org/abs/2302.01107>.

Zhuang 等关于提升 Transformer 训练效率的综述（2023）。

- Modern multi-GPU strategies typically combine data parallelism and tensor parallelism. Popular examples include DeepSpeed stages 2 and 3, described in this tutorial on the zero redundancy optimizer: <https://www.deepspeed.ai/tutorials/zero/>.

当代多 GPU 方案常结合数据与张量并行；DeepSpeed ZeRO 阶段 2、3 等的教程链接。



Chapter 8: The Success of Transformers

本章从注意力、预训练、规模与并行性等方面归纳 Transformer 取得成功的关键因素。

What are the main factors that have contributed to the success of transformers?

促成 Transformer 广泛成功的要素主要有哪些？

In recent years, transformers have emerged as the most successful neural network architecture, particularly for various natural language processing tasks. In fact, transformers are now on the cusp of becoming state of the art for computer vision tasks as well. The success of transformers can be attributed to several key factors, including their **attention mechanisms**, ability to be **parallelized easily**, **unsupervised pretraining**, and **high parameter counts**.

近年来 Transformer 成为最成功的神经网络架构之一，尤其在众多 NLP 任务上如此；在计算机视觉领域也接近最前沿。其成功可归因于 **attention mechanisms**（注意力机制）、易于 **parallelized easily**（并行化）、**unsupervised pretraining**（无监督预训练）以及 **high parameter counts**（高参数量）等因素。

Tips:

- 注意力机制，使得模型可以关注到输入序列中的重要部分，从而提高模型性能。
- 模型并行化，从而提高训练速度。
- 自监督预训练，使得模型可以利用大量无标签数据，从而提高模型性能。
- 高参数数量，使得模型可以学习到更复杂的特征，从而提高模型性能。

The Attention Mechanism

本节区分视觉启发式注意力与 Transformer 自注意力，并解释动态权重相对固定层权重的特点。



The self-attention mechanism found in transformers is one of the key design components that make transformer-based LLMs so successful. However, transformers are not the first architecture to utilize attention mechanisms.

Transformer 中的自注意力是驱动大规模语言模型成功的核心构件之一，但它并非首个使用注意力的架构。

Attention mechanisms were first developed in the context of image recognition back in 2010, before being adopted to aid the translation of long sentences in recurrent neural networks. (Chapter [ch16] compares the attention mechanisms found in recurrent neural networks and transformers in greater detail.)

注意力机制最早出现在 2010 年前后的图像识别研究中，随后被用于帮助循环神经网络翻译长句；上文括号中所述第十六章对两类注意力有更细致的对照。

The aforementioned attention mechanism is inspired by human vision, focusing on specific parts of an image (foveal glimpses) at a time to process information hierarchically and sequentially. In contrast, the fundamental mechanism underlying transformers is a self-attention mechanism used for sequence-to-sequence tasks, such as machine translation and text generation. It allows each token in a sequence to attend to all other tokens, thus providing context-aware representations of each token.

前述注意力借用人眼视觉：一次聚焦于图像局部（foveal glimpses）并以层级、顺序方式加工信息。Transformer 的基本单元则是面向序列到序列任务的自注意力：序列中每个 token 可关注其余 token，从而获得上下文相关的表示。

Tips: 人类视觉系统，是分层处理的，先关注到特定部分（**foveal glimpses** 中央凹注视），然后逐步关注到更多部分（以 **分层** 和 **顺序** 的方式处理信息）。

- 人类视觉：像看东西一样，先看重点部分，再逐步看更多细节
- Transformer自注意力：每个词都能"看到"句子中的所有其他词，理解上下文关系

What makes attention mechanisms so unique and useful? For the following illustration, suppose we are using an encoder network on a fixed-length representation of the input sequence or image -- this can be a fully connected, convolutional, or attention-based encoder.



注意力机制的独特用处何在？下文示意中，假设编码器对固定长度的序列或图像表征进行处理——可以是全连接、卷积或注意力型编码器。

Tips: FIXME, 注意力权重是动态的？？？ 因为跟元素相对位置有关？元素就是 token？

- 在Transformer中，编码器使用**自注意力机制**，计算每个输入token相对于序列中其他token的重要性，从而让模型关注输入序列中的相关部分。
- 概念上，注意力机制允许，Transformer关注序列或图像的不同部分。
- 表面上，这听起来非常类似于 **全连接层**，其中每个输入元素与下一个层中的输入元素的权重连接。
- 在注意力机制中，计算注意力权重涉及将每个输入元素与所有其他元素进行比较。
- 通过这种方法获得的注意力权重是 **动态** 的，并且依赖于输入。
- 相比之下，卷积或全连接层的权重在训练后是 **固定的**，如Figure **8.1**所示。

In a transformer, the encoder uses self-attention mechanisms to compute the importance of each input token relative to other tokens in the sequence, allowing the model to focus on relevant parts of the input sequence. Conceptually, attention mechanisms allow the transformers to attend to different parts of a sequence or image. On the surface, this sounds very similar to a fully connected layer where each input element is connected via a weight with the input element in the next layer. In attention mechanisms, the computation of the attention weights involves comparing each input element to all others. The attention weights obtained by this approach are dynamic and input dependent. In contrast, the weights of a convolutional or fully connected layer are fixed after training, as illustrated in Figure **8.1**.

在 Transformer 里，编码器用自注意力衡量各 token 相对重要性，使模型聚焦输入的相关片段；概念上它允许模型“看向”序列或图像的不同区域。表面上这与全连接层类似，但注意力权重通过两两比较输入元素得到，因而动态依赖输入；卷积或全连接权重在训练完成后则固定不变，见图 **8.1**。

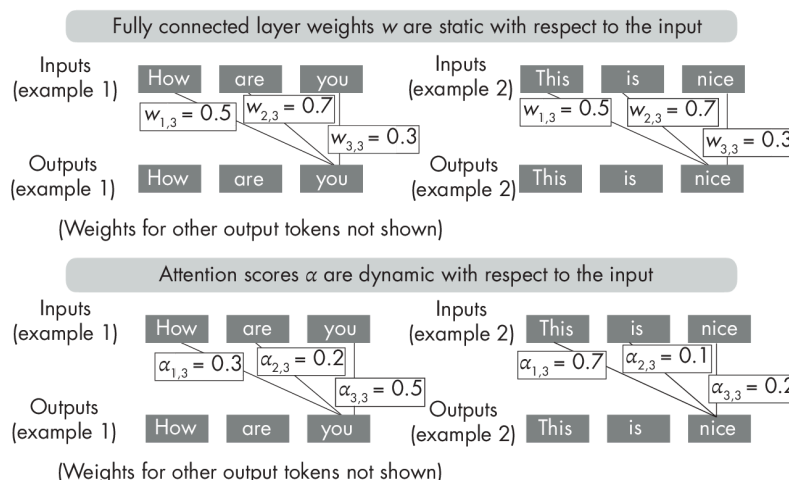


Figure 8.1

As the top part of Figure 8.1 shows, once trained, the weights of fully connected layers remain fixed regardless of the input. In contrast, as shown at the bottom, self-attention weights change depending on the inputs, even after a transformer is trained.

如图 8.1 上部所示，训练完成后全连接权重不随输入改变；下部则表明即便模型已训练完毕，自注意力权重仍会随输入变化。

Tips:

- 注意力机制，允许神经网络**选择性**地对**不同输入特征**的重要性进行**加权**，从而让模型专注于给定任务的输入的**最相关部分**。
- 这提供了对每个词或图像token的**上下文理解**，允许更细致的解释，这是使Transformer如此成功的一个方面。

Attention mechanisms allow a neural network to selectively weigh the importance of different input features, so the model can focus on the most relevant parts of the input for a given task. This provides a contextual understanding of each word or image token, allowing for more nuanced interpretations, which is one of the aspects that can make transformers work so well.

注意力机制让网络按重要性加权不同输入特征，从而使模型专注于任务相关的 most relevant（原文连写如此）区域；这为每个词或图像 token 提供上下文语义与更细粒度解释，也是 Transformer 奏效的原因之一。

Pretraining via Self-Supervised Learning

本节强调大规模无标注数据上的自监督预训练如何为下游任务提供通用语言表征。



Tips:

- 自监督预训练，是Transformer成功的一个重要因素。
- 在自监督预训练中，Transformer模型被训练来预测句子中的缺失词或文档中的下一个句子。
- 通过学习预测这些缺失词或下一个句子，模型被迫学习语言的通用表示，可以针对各种下游任务进行微调。

Pretraining transformers via self-supervised learning on large, unlabeled datasets is another key factor in the success of transformers. During pretraining, the transformer model is trained to predict missing words in a sentence or the next sentence in a document, for example. By learning to predict these missing words or the next sentence, the model is forced to learn general representations of language that can be fine-tuned for a wide range of downstream tasks.

在海量无标注数据上做自监督预训练是 Transformer 成功的又一关键：例如预测句中缺失词或文档的下一句；通过这些代理任务，模型被迫学习可迁移到多种下游任务的通用语言表征。

While unsupervised pretraining has been highly effective for natural language processing tasks, its effectiveness for computer vision tasks is still an active area of research. (Refer to Chapter [\[ch02\]](#) for a more detailed discussion of self-supervised learning.)

无监督预训练在 NLP 上极为有效，但在计算机视觉上的优劣仍是活跃研究方向；上文括号指引第二章查看自监督学习的详细讨论。

Large Numbers of Parameters

本节讨论超大参数量、缩放律以及数据规模与模型规模协同扩展的意义。

One noteworthy characteristic of transformers is their large model sizes. For example, the popular 2020 GPT-3 model consists of 175 billion trainable



parameters, while other transformers, such as switch transformers, have trillions of parameters.

Transformer 的显著特征之一是大规模：例如 2020 年前后知名的 GPT-3 约有 1750 亿可训练参数，Switch Transformer 等则可到万亿量级。

The scale and number of trainable parameters of transformers are essential factors in their modeling performance, particularly for large-scale natural language processing tasks. For instance, **linear scaling laws** suggest that the training loss decreases proportionally with an increase in model size, so a doubling of the model size can halve the training loss.

参数规模对建模能力至关重要，尤其在大规模 NLP 上；例如 **linear scaling laws**（线性缩放律）暗示训练损失随模型增大近似按比例下降，模型体量翻倍或可使训练损失减半。

This, in turn, can lead to better performance on the downstream target task. However, it is essential to scale the model size and the number of training tokens equally. This means the number of training tokens should be doubled for every doubling of model size.

这有助于下游任务表现，但模型体量与训练 token 数需要协同扩展：模型规模每翻倍，训练 token 也应大致翻倍。

Since labeled data is limited, utilizing large amounts of data during unsupervised pretraining is vital.

标注数据有限时，在无监督预训练阶段尽可能利用更多文本尤为关键。

To summarize, large model sizes and large datasets are critical factors in transformers' success. Additionally, using self-supervised learning, the ability to pretrain transformers is closely tied to using large model sizes and large datasets. This combination has been critical in enabling the success of transformers in a wide range of natural language processing tasks.

综上，大模型与大语料是 Transformer 成功的基石；自监督预训练能力又与二者紧密耦合，这一组合支撑了其在众多 NLP 任务中的突破。

Tips: 总而言之，Transformer 的成功，很大程度上归功于其 **大模型** 和 **大数据** 的使用。

- 线性缩放定律：训练损失与模型大小成正比，因此增加模型大小可以减少训练损失。
- 训练tokens数量：训练tokens数量应该与模型大小成正比，因此增加模型大小应该增加训练tokens数量。



Easy Parallelization

本节说明固定长度 token 序列与成对注意力计算为何便于在多核或多机上并行。

Training **large models** on **large datasets** requires **vast computational resources**, and it's key that the computations can be parallelized to utilize these resources.

要用大数据训练大模型需要巨量算力，因此计算能否并行化以吃满硬件至关重要。

Fortunately, transformers are easy to parallelize since they take a fixed-length sequence of word or image tokens as input. For instance, the self-attention mechanism used in most transformer architectures involves computing the weighted sum between a pair of input elements. Furthermore, these pair-wise token comparisons can be computed independently, as illustrated in Figure 8.2, making the self-attention mechanism relatively easy to parallelize across different GPU cores.

幸运的是 Transformer 以固定长度的词或图像 token 序列为输入，较易并行；多数架构中的自注意力需计算输入元素两两之间的加权和，而这些成对比较彼此独立（见图 8.2），因而在多 GPU 核心上相对容易并行。

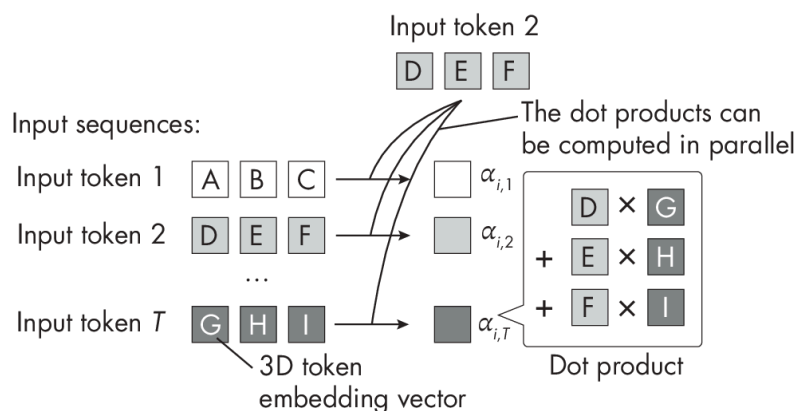




Figure 8.2

In addition, the individual weight matrices used in the self-attention mechanism (not shown in Figure 8.2) can be distributed across different machines for distributed and parallel computing.

此外，自注意力中的各权重矩阵（图 8.2 未画出）也可分布到多台机器以实现分布式并行计算。

Exercises

习题引导读者辨析自注意力“易并行”与整体计算昂贵之间的矛盾，并思考其与特征选择的关系。

8-1. As discussed in this chapter, self-attention is easily parallelizable, yet transformers are considered computationally expensive due to self-attention. How can we explain this contradiction?

习题 8-1：本章称自注意力易于并行，但 Transformer 又因自注意力而计算昂贵，如何理解这一表面矛盾？

8-2. Since self-attention scores represent importance weights for the various input elements, can we consider self-attention to be a form of feature selection?

习题 8-2：若自注意力得分表示各输入元素的重要性权重，能否把它视作一种特征选择？

References

本节列出视觉注意力、Transformer 原始论文、稀疏巨型 Transformer、缩放律与算力最优训练等相关文献与博文链接。

- An example of an attention mechanism in the context of image recognition: Hugo Larochelle and Geoffrey Hinton, "Learning to Combine Foveal Glimpses with a Third-Order Boltzmann Machine"? (2010), <https://dl.acm.org/doi/10.5555/2997189.2997328>.



Larochelle 与 Hinton 在图像语境下使用注意力机制的示例（2010）。

- The paper introducing the self-attention mechanism with the original transformer architecture: Ashish Vaswani et al., "Attention Is All You Need"? (2017), <https://arxiv.org/abs/1706.03762>.

Vaswani 等在原始 Transformer 论文中引入自注意力机制（2017）。

- Transformers can have trillions of parameters: William Fedus, Barret Zoph, and Noam Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"? (2021), <https://arxiv.org/abs/2101.03961>.

Fedus、Zoph、Shazeer 关于可达万亿参数规模的 Switch Transformer（2021）。

- Linear scaling laws suggest that training loss decreases proportionally with an increase in model size: Jared Kaplan et al., "Scaling Laws for Neural Language Models"? (2020), <https://arxiv.org/abs/2001.08361>.

Kaplan 等关于神经语言模型缩放律的论文（2020）。

- Research suggests that in transformer-based language models, the training tokens should be doubled for every doubling of model size: Jordan Hoffmann et al., "Training Compute-Optimal Large Language Models"? (2022), <https://arxiv.org/abs/2203.15556>.

Hoffmann 等指出 Transformer 语言模型在规模翻倍时应同步加倍训练 token 的研究（2022）。

- Formoreabouttheweightssusedinself-attentionandcross-attention mechanisms, check out my blog post: "Understanding and Coding the Self-Attention Mechanism of Large Language Models from Scratch"? at <https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>.

作者关于从零理解并实现自注意力权重的博文链接。



Chapter 9: Generative AI Models

本章界定生成式建模，并按类别综述深度生成模型及其主要优缺点。

What are the popular categories of deep generative models in deep learning (also called *generative AI*), and what are their respective downsides?

深度学习中有哪类主流的深度生成模型类别（也常称生成式 AI），各自的主要缺点又是什么？

Many different types of deep generative models have been applied to generating different types of media: images, videos, text, and audio. Beyond these types of media, models can also be repurposed to generate domain-specific data, such as organic molecules and protein structures. This chapter will first define generative modeling and then outline each type of generative model and discuss its strengths and weaknesses.

已有多类深度生成模型被用来生成不同类型的媒介：图像、视频、文本与音频。除这些媒介外，模型也可改用于生成领域特定数据（如有机分子与蛋白质结构）。本章先定义生成式建模，再逐类概述各类生成模型并讨论其优势与劣势。

Generative vs. Discriminative Modeling

对比生成式与判别式建模：联合分布与条件分布、朴素贝叶斯与逻辑回归的经典对照。

In traditional machine learning, there are two primary approaches to modeling the relationship between input data (x) and output labels (y): **generative models** and **discriminative models**.

在经典机器学习中，刻画输入数据（ x ）与输出标签（ y ）之间的关系主要有两类思路：**生成式模型**与**判别式模型**。

- Generative models** aim to capture the underlying probability distribution of the input data $p(x)$ or the joint distribution $p(x, y)$ between inputs and labels.



- **生成式模型** 力求刻画输入数据的边缘分布 $p(x)$ 或输入与标签的联合分布 $p(x, y)$ 。
- In contrast, **discriminative models** focus on modeling the conditional distribution $p(y | x)$ of the labels given the inputs.
- 相对地，**判别式模型** 关注在给定输入条件下标签的条件分布 $p(y | x)$ 。

A classic example that highlights the differences between these approaches is to compare the **naive Bayes classifier** and the **logistic regression classifier**.

一个经典对照是比较 **朴素贝叶斯分类器** 与 **逻辑回归分类器**。

- Both classifiers estimate the class label probabilities $p(y | x)$ and can be used for classification tasks.
- 两者都估计类别标签概率 $p(y | x)$ ，并可做分类。
- However, **logistic regression** is considered a discriminative model because it directly models the conditional probability distribution $p(y | x)$ of the class labels given the input features without making assumptions about the underlying joint distribution of inputs and labels.
- 但 **逻辑回归** 被视为判别式模型，因其直接建模给定特征时的条件分布 $p(y | x)$ ，而不假设输入与标签的联合分布形式。
- **Naive Bayes**, on the other hand, is considered a generative model because it models the joint probability distribution $p(x, y)$ of the input features x and the output labels y . By learning the joint distribution, a generative model like naive Bayes captures the underlying data generation process, which enables it to generate new samples from the distribution if needed.
- **朴素贝叶斯** 则是生成式模型：它建模特征 x 与标签 y 的联合分布 $p(x, y)$ ；通过学习联合分布，该类生成式模型刻画了数据生成过程，从而在需要时能从分布中采样新样本。

Tips:

- 贝叶斯分类器，假设输入和输出之间存在**联合概率分布**；可以生成新的样本，因为它是生成模型；
- 逻辑回归分类器，假设输入和输出之间存在**条件概率分布**；不能生成新的样本；



Types of Deep Generative Models

概览“深度”生成模型常见类型及下文将讨论的各类方法。

When we speak of *deep* generative models or deep generative AI, we often loosen this definition to include all types of models capable of producing realistic-looking data (typically text, images, videos, and sound). The remainder of this chapter briefly discusses the different types of deep generative models used to generate such data.

说到**深度生成模型**或**深度生成式 AI**，人们常把定义放宽：凡能生成观感逼真的数据（多为文本、图像、视频与声音）的模型都可算入其中。本章余下部分将简要讨论用于生成此类数据的各类深度生成模型。

Energy-Based Models

基于能量的模型（EBM）：能量函数、深度玻尔兹曼机及其在生成中的局限性。

Energy-based models (EBMs) are a class of generative models that learn an energy function, which assigns a scalar value (energy) to each data point. Lower energy values correspond to more likely data points. The model is trained to minimize the energy of real data points while increasing the energy of generated data points.

基于能量的模型（EBM）学习一个能量函数，为每个数据点赋予标量能量；能量越低的数据点越可能。训练时降低真实样本的能量并抬高生成样本的能量。

Examples of EBMs include **deep Boltzmann machines (DBMs)** .

EBM 的例子包括 **深度玻尔兹曼机（DBM）** 。



One of the early breakthroughs in deep learning, DBMs provide a means to learn complex representations of data. You can think of them as a form of unsupervised pretraining, resulting in models that can then be fine-tuned for various tasks.

DBM 曾是深度学习早期的突破之一：可学习数据的复杂表征，可视作一种无监督预训练，得到的模型随后可对各种任务微调。

Somewhat similar to naive Bayes and logistic regression, DBMs and multilayer perceptrons (MLPs) can be thought of as generative and discriminative counterparts, with DBMs focusing on capturing the data generation process and MLPs focusing on modeling the decision boundary between classes or mapping inputs to outputs.

与朴素贝叶斯和逻辑回归的类比类似，DBM 与多层感知机（MLP）可看作生成式与判别式的一对：前者侧重刻画数据生成过程，后者侧重刻画类间决策边界或将输入映射到输出。

A **DBM** consists of multiple layers of hidden nodes, as shown in Figure 9.1. As the figure illustrates, along with the hidden layers, there's usually a visible layer that corresponds to the observable data. This visible layer serves as the input layer where the actual data or features are fed into the network. In addition to using a different learning algorithm than MLPs (contrastive divergence instead of backpropagation), DBMs consist of binary nodes (neurons) instead of continuous ones.

DBM 含多层隐藏节点，见图 9.1。如图所示，除隐藏层外通常还有对应可观测数据的可见层；该层充当输入层，真实数据或特征由此馈入网络。相较 MLP，DBM 使用不同的学习算法（对比散度而非反向传播），且节点为二进制而非连续值。

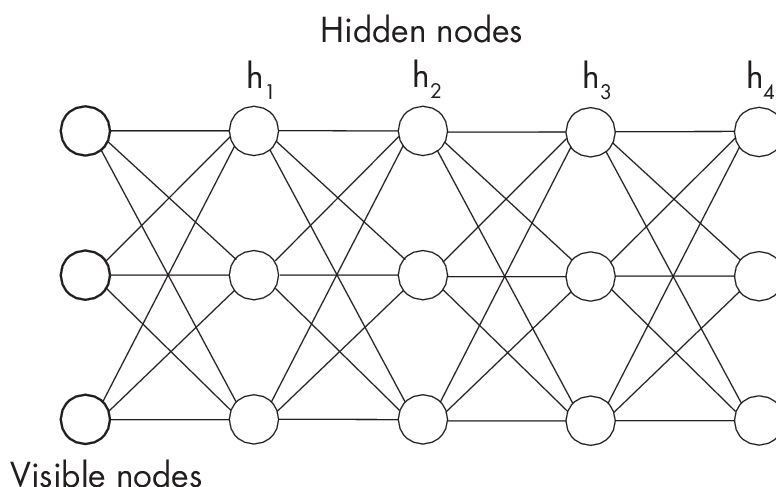


Figure 9.1

Suppose we are interested in generating images. A DBM can learn the joint probability distribution over the pixel values in a simple image dataset like MNIST. To generate new images, the DBM then samples from this distribution by performing a process called *Gibbs sampling*. Here, the visible layer of the DBM represents the input image. To generate a new image, the DBM starts by initializing the visible layer with random values or, alternatively, uses an existing image as a seed. Then, after completing several Gibbs sampling iterations, the final state of the visible layer represents the generated image.

若要生成图像，DBM 可在 MNIST 这类简单数据集上学习像素联合分布；生成时通过 *Gibbs 采样* 从该分布采样。可见层表示输入图像：可随机初始化可见层或以已有图像为种子，经若干步 Gibbs 迭代后，可见层的终态即为生成图像。

DBMs played an important historical role as one of the first deep generative models, but they are no longer very popular for generating data. They are expensive and more complicated to train, and they have lower expressivity compared to the newer models described in the following sections, which generally results in lower-quality generated samples.

DBM 作为早期深度生成模型有重要历史地位，但如今已较少用于生成数据：训练昂贵且复杂，表达力也弱于下文较新模型，通常导致生成样本质量偏低。

Variational Autoencoders

变分自编码器（VAE）：编码器/解码器、ELBO 式损失（重构 + KL）及常见短板。



Variational autoencoders (VAEs) are built upon the principles of variational inference and autoencoder architectures. *Variational inference* is a method for approximating complex probability distributions by optimizing a simpler, tractable distribution to be as close as possible to the true distribution. *Autoencoders* are unsupervised neural networks that learn to compress input data into a low-dimensional representation (encoding) and subsequently reconstruct the original data from the compressed representation (decoding) by minimizing the reconstruction error.

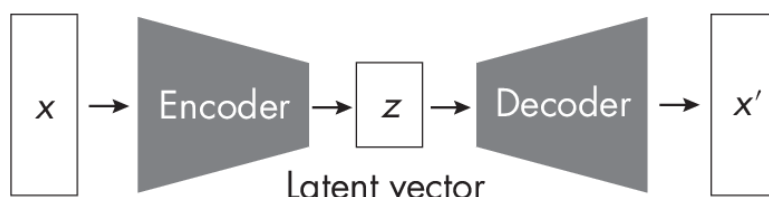
变分自编码器 (VAE) 建立在变分推断与自编码结构之上。变分推断用更易处理的分布去逼近复杂真实分布；自编码器是无监督网络，将输入压缩为低维编码再解码重构，并通过最小化重构误差学习。

The VAE model consists of two main submodules: an encoder network and a decoder network. The encoder network takes, for example, an input image and maps it to a latent space by learning a probability distribution over the latent variables. This distribution is typically modeled as a Gaussian with parameters (mean and variance) that are functions of the input image. The decoder network then takes a sample from the learned latent distribution and reconstructs the input image from this sample. The goal of the VAE is to learn a compact and expressive latent representation that captures the essential structure of the input data while being able to generate new images by sampling from the latent space. (See Chapter [\[ch01\]](#) for more details on latent representations.)

VAE 含编码器与解码器：编码器（如对输入图像）将数据映射到潜变量上的分布（常为高斯，其均值与方差为输入的函数）。解码器从该分布采样并重构输入。目标是学得紧凑而有表达力的潜空间，既抓住数据结构，又能从中采样生成新图像（潜变量更多细节见第 [\[ch01\]](#) 章）。

Figure [9.2](#) illustrates the encoder and decoder submodules of an auto-encoder, where x' represents the reconstructed input x . In a standard variational autoencoder, the latent vector is sampled from a distribution that approximates a standard Gaussian distribution.

图 [9.2](#) 示意自编码器的编解码子模块； x' 表示重构的输入 x 。标准 VAE 中潜向量从逼近标准高斯的分布中采样。



Training a VAE involves optimizing the model's parameters to minimize a loss function composed of two terms: a reconstruction loss and a Kullback -- Leibler-divergence (KL-divergence) regularization term. The reconstruction loss ensures that the decoded samples closely resemble the input images, while the KL-divergence term acts as a surrogate loss that encourages the learned latent distribution to be close to a predefined prior distribution (usually a standard Gaussian). To generate new images, we then sample points from the latent space's prior (standard Gaussian) distribution and pass them through the decoder network, which generates new, diverse images that look similar to the training data.

训练 VAE 需最小化由重构项与 KL 散度正则项构成的损失：重构项使解码结果贴近输入，KL 项促使学到的潜分布接近预设先验（常为标准高斯）。生成新图像时，从先验（标准高斯）采样并经解码器前向，可得到与训练数据相似而又多样的图像。

Disadvantages of VAEs include their complicated loss function consisting of separate terms, as well as their often low expressiveness. The latter can result in blurrier images compared to other models, such as generative adversarial networks.

VAE 的缺点包括损失由多项构成、较为复杂，以及表达力常偏弱——相对生成对抗网络等模型，图像往往更模糊。

Generative Adversarial Networks

生成对抗网络（GAN）：生成器与判别器的对抗训练、不稳定与模式坍塌等问题。

Generative adversarial networks (GANs) are models consisting of interacting subnetworks designed to generate new data samples that are similar to a given



set of input data. While both GANs and VAEs are latent variable models that generate data by sampling from a learned latent space, their architectures and learning mechanisms are fundamentally different.

生成对抗网络 (GAN) 由相互作用的子网络组成，用以生成与训练集相似的新样本。GAN 与 VAE 虽都从潜变量空间采样生成，但其结构与学习机理根本不同。

GANs consist of two neural networks, a generator and a discriminator, that are trained simultaneously in an adversarial manner. The generator takes a random noise vector from the latent space as input and generates a synthetic data sample (such as an image). The discriminator's task is to distinguish between real samples from the training data and fake samples generated by the generator, as illustrated in Figure 9.3.

GAN 包含生成器与判别器两个网络，同时对抗训练：生成器以潜空间中的随机噪声为输入合成样本（如图）；判别器则区分训练集中的真实样本与生成器输出的假样本（见图 9.3）。

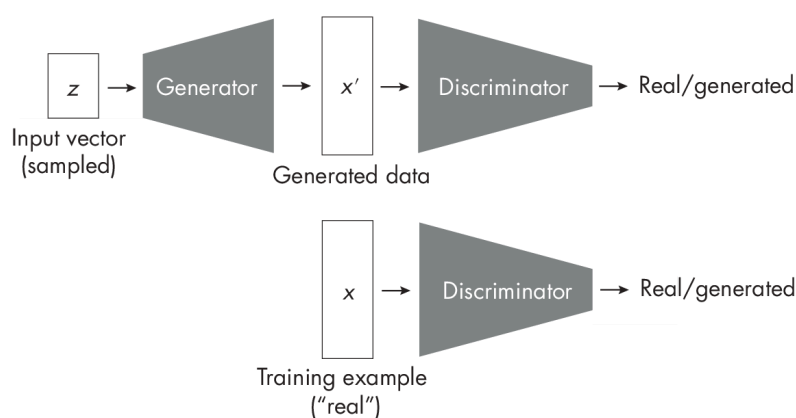


Figure 9.3

The generator in a GAN somewhat resembles the decoder of a VAE in terms of its functionality. During inference, both GAN generators and VAE decoders take random noise vectors sampled from a known distribution (for example, a standard Gaussian) and transform them into synthetic data samples, such as images.

就功能而言，GAN 的生成器与 VAE 的解码器有相似之处：推断时二者都把来自已知分布（如标准高斯）的随机噪声向量变换为图像等合成数据。

One significant disadvantage of GANs is their unstable training due to the adversarial nature of the loss function and learning process. Balancing the



learning rates of the generator and discriminator can be difficult and can often result in oscillations, mode collapse, or non-convergence. The second main disadvantage of GANs is the low diversity of their generated outputs, often due to mode collapse. Here, the generator is able to fool the discriminator successfully with a small set of samples, which are representative of only a small subset of the original training data.

GAN 的主要缺点是训练不稳：对抗目标使生成器与判别器的学习率难以平衡，易出现震荡、模式坍塌或不收敛。第二缺点是输出多样性不足，常源于模式坍塌——生成器用少量即能骗过判别器的样本，只覆盖训练分布的一小部分。

Flow-Based Models

流模型（正则化流）：可逆变换、NICE 思路、似然与其他方法的取舍。

The core concept of *flow-based models*, also known as *normalizing flows*, is inspired by long-standing methods in statistics. The primary goal is to transform a simple probability distribution (like a Gaussian) into a more complex one using invertible transformations.

流模型亦称正则化流，思想源于统计学：用可逆变换将简单分布（如高斯）变形为复杂分布。

Although the concept of **normalizing flows** has been apart of the statistics field for a long time, the implementation of early flow-based deep learning models, particularly for image generation, is a relatively recent development. One of the pioneering models in this area was the *non-linear independent components estimation (NICE)* approach. NICE begins with a simple probability distribution, often something straightforward like a normal distribution. You can think of this as a kind of "random noise," or data with no particular shape or structure. NICE then applies a series of transformations to this simple distribution. Each transformation is designed to make the data look more like the final target (for instance, the distribution of real-world images). These transformations are "invertible," meaning we can always reverse them back to the original simple distribution. After several successive transformations, the simple distribution has morphed into a complex distribution that closely matches the distribution of the

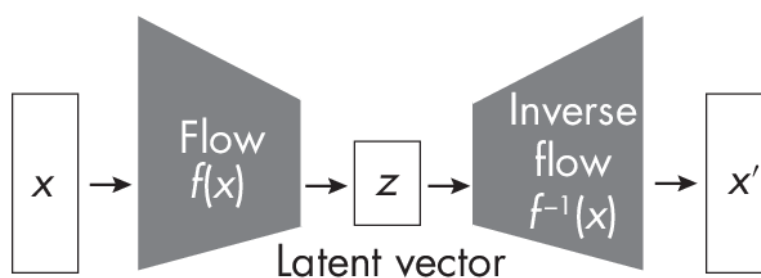


target data (such as images). We can now generate new data that looks like the target data by picking random points from this complex distribution.

正则化流 在统计中由来已久，但早期用于图像生成的流式深度学习实现相对较新。先驱之一是 *NICE*（非线性独立分量估计）：从简单分布（如正态，可类比无结构的“随机噪声”）出发，施加一系列变换，使分布逐步更接近目标（如真实图像分布）；每一步可逆，可回到原始简单分布。经多次变换后，简单分布被塑造成贴近目标数据的复杂分布，再从该分布采样即可生成看起来像目标的数据。

Figure 9.4 illustrates the concept of a flow-based model, which maps the complex input distribution to a simpler distribution and back.

图 9.4 示意流模型如何将复杂输入分布映射到简单分布并可逆还原。



At first glance, the illustration is very similar to the VAE illustration in Figure 9.2. However, while VAEs use neural network encoders like convolutional neural networks, the flow-based model uses simpler decoupling layers, such as simple linear transformations. Additionally, while the decoder in a VAE is independent of the encoder, the data-transforming functions in the flow-based model are mathematically inverted to obtain the outputs.

乍看之下该图与 VAE（图 9.2）很像；但 VAE 编码器常为 CNN 等复杂网络，流模型则用更简单的解耦层（如线性变换）。另外 VAE 的解码器与编码器分离，而流模型的数据变换函数在数学上成对互为逆以实现前向与逆推。

Unlike VAEs and GANs, flow-based models provide exact likelihoods, which gives us insights into how well the generated samples fit the training data distribution. This can be useful in anomaly detection or density estimation, for example. However, the quality of flow-based models for generating image data is usually lower than GANs. Flow-based models also often require more memory and



computational resources than GANs or VAEs since they must store and compute inverses of transformations.

与 VAE、GAN 不同，流模型可给出精确似然，有助判断生成样本与训练分布的贴合度，可用于异常检测或密度估计等。但若论图像生成质量通常不如 GAN；且需存储与计算变换的逆，内存与算力开销常高于 GAN 或 VAE。

Autoregressive Models

自回归模型：逐步预测下一 token/像素、链式分解、优缺点（慢、长程依赖等）。

Autoregressive models are designed to predict the next value based on current (and past) values. LLMs for text generation, like ChatGPT (discussed further in Chapter [ch17]), are one popular example of this type of model.

自回归模型根据当前（及过往）取值预测下一个值；文本生成的 LLM（如 ChatGPT，第 [ch17] 章详述）是典型例子。

Similar to generating one word at a time, in the context of image generation, autoregressive models like **PixelCNN** try to predict one pixel at a time, given the pixels they have seen so far. Such a model might predict pixels from top left to bottom right, in a raster scan order, or in any other defined order.

类比逐词生成，在图像领域 **PixelCNN** 等模型在给定已见像素条件下一次预测一个新像素：可按光栅扫描等任意预定顺序从左到右上至下地进行。

To illustrate how autoregressive models generate an image one pixel at a time, suppose we have an image of size $H \times W$ (where H is the height and W is the width), ignoring the color channel for simplicity's sake. This image consists of N pixels, where $i = 1, \dots, N$. The probability of observing a particular image in the dataset is then $P(\text{Image}) = P(i_1, i_2, \dots, i_N)$. Based on the chain rule of probability in statistics, we can decompose this joint probability into conditional probabilities:

说明自回归图像生成：设图像尺寸为 $H \times W$ （暂忽略颜色通道），共 N 个像素， $i = 1, \dots, N$ 。某张图像的概率为 $P(\text{Image}) = P(i_1, i_2, \dots, i_N)$ 。由概率论的链式法则，可将联合分解为一系列条件概率：



$$\begin{aligned} P(\text{Image}) &= P(i_1, i_2, \dots, i_N) \\ &= P(i_1) \cdot P(i_2 | i_1) \cdot P(i_3 | i_1, i_2) \dots P(i_N | i_1 \dots i_{N-1}) \end{aligned}$$

Here, $P(i_1)$ is the probability of the first pixel, $P(i_2|i_1)$ is the probability of the second pixel given the first pixel, $P(i_3|i_1, i_2)$ is the probability of the third pixel given the first and second pixels, and so on.

其中 $P(i_1)$ 是第一个像素的概率， $P(i_2|i_1)$ 是在第一个已知时第二个像素的概率，依此类推。

In the context of image generation, an autoregressive model essentially tries to predict one pixel at a time, as described earlier, given the pixels it has seen so far.

在图像生成中，自回归模型本质上就是按前述方式，在已见像素的条件下逐步预测下一像素。

Figure 9.5 illustrates this process, where pixels $i_1, \dots, i_{53}$ represent the context and pixel i_{54} is the next pixel to be generated.

图 9.5 中 $i_1, \dots, i_{53}$ 为上下文， i_{54} 为待生成的下一像素。

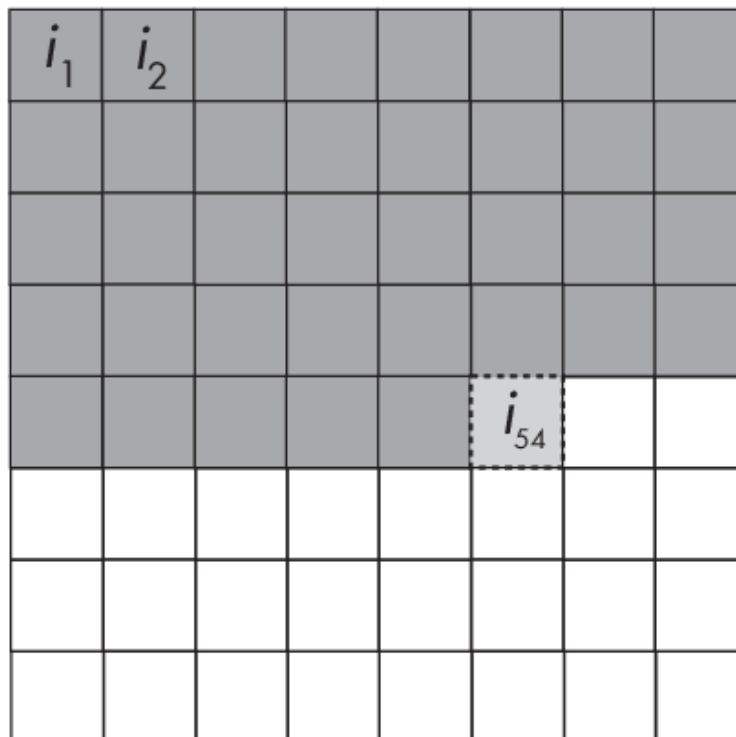


Figure 9.5



The advantage of autoregressive models is that the next-pixel (or word) prediction is relatively straightforward and interpretable. In addition, autoregressive models can compute the likelihood of data exactly, similar to flow-based models, which can be useful for tasks like anomaly detection. Furthermore, autoregressive models are easier to train than GANs as they don't suffer from issues like mode collapse and other training instabilities.

其优点在于下一步（像素或词）预测直观易懂；似然可与流模型一样精确计算，利于异常检测等；且无 GAN 的模式坍缩等问题，通常更易训练。

However, autoregressive models can be slow at generating new samples. This is because they have to generate data one step at a time (for example, pixel by pixel for images), which can be computationally expensive. Autoregressive models may also struggle to capture long-range dependencies because each output is conditioned only on previously generated outputs.

缺点是采样慢：必须逐步生成（如图逐像素），计算成本高；且每步只看已生成部分，长程依赖较难刻画。

In terms of overall image quality, autoregressive models are therefore usually worse than GANs but are easier to train.

就整体图像质量而言，自回归模型通常弱于 GAN，但训练更平易。

Diffusion Models

扩散模型：前向加噪与反向去噪，与流模型的区别及采样速度取舍。

As discussed in the previous section, flow-based models transform a simple distribution (such as a standard normal distribution) into a complex one (the target distribution) by applying a sequence of invertible and differentiable transformations (flows). Like flow-based models, *diffusion models* also apply a series of transformations. However, the underlying concept is fundamentally different.

如上一节所述，流模型用可逆可微的一系列“流”把简单分布变到复杂目标分布；*扩散模型*也施加多步变换，但原理根本不同。



Diffusion models transform the input data distribution into a simple noise distribution over a series of steps using stochastic differential equations. Diffusion is a stochastic process in which noise is progressively added to the data until it resembles a simpler distribution, like Gaussian noise. To generate new samples, the process is then reversed, starting from noise and progressively removing it.

扩散模型通过随机微分方程等，在若干步内把数据分布逐步变成简单噪声分布：扩散即向数据逐步加噪直至接近高斯噪声；生成则反向从噪声出发逐步去噪。

Figure 9.6 outlines the process of adding and removing Gaussian noise from an input image x . During inference, the reverse diffusion process is used to generate a new image x , starting with the noise tensor z_n sampled from a Gaussian distribution.

图 9.6 概括对输入图像 x 加减高斯噪声的过程；推断时用反向扩散从采样得到的高斯噪声张量 z_n 生成新图像 x 。

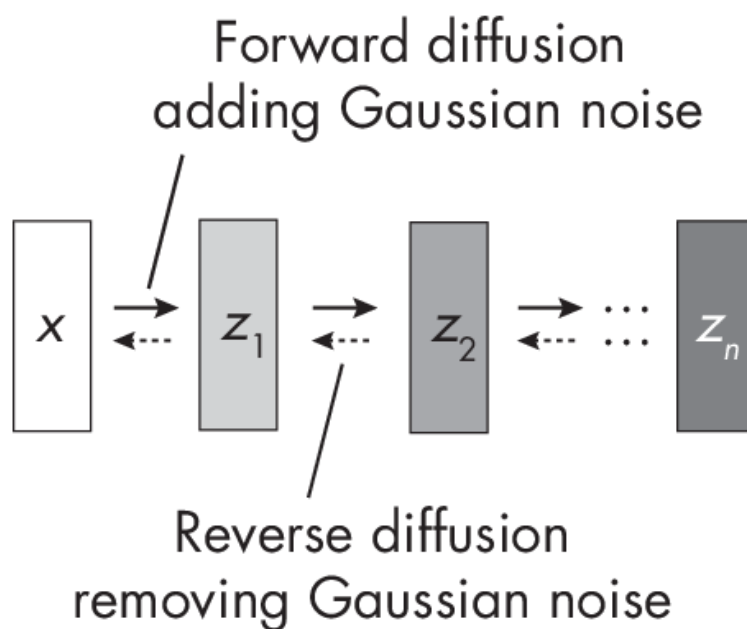


Figure 9.6

While both diffusion models and flow-based models are generative models aiming to learn complex data distributions, they approach the problem from different angles. Flow-based models use deterministic invertible transformations, while diffusion models use the aforementioned stochastic diffusion process.



二者都是学习复杂分布的生成模型，但角度不同：流模型用确定性可逆变换，扩散模型用前述随机扩散过程。

Recent projects have established state-of-the-art performance in generating high-quality images with realistic details and textures. Diffusion models are also easier to train than GANs. The downside of diffusion models, however, is that they are slower to sample from since they require running a series of sequential steps, similar to flow-based models and autoregressive models. This can make diffusion models less practical for some applications requiring fast sampling.

近年扩散模型在高细节、高质感图像生成上可达顶尖水平，且通常比 GAN 更易训练。缺点在于采样需顺序多步运行，速度与流模型、自回归模型类似偏慢，在需要快速生成的场景不够实用。

Consistency Models

一致性模型：单步去噪映射、ODE 轨迹及与扩散在质量与速度上的比较。

Consistency models train a neural network to map a noisy image to a clean one. The network is trained on a dataset of pairs of noisy and clean images and learns to identify patterns in the clean images that are modified by noise. Once the network is trained, it can be used to generate reconstructed images from noisy images in one step.

一致性模型训练网络把含噪图像映射到干净图像：在 noisy/clean 成对数据上学习噪声如何改变干净图中的结构；训练后可一步从噪声图重建或生成。

Consistency model training employs an *ordinary differential equation (ODE)* trajectory, a path that a noisy image follows as it is gradually denoised. The ODE trajectory is defined by a set of differential equations that describe how the noise in the image changes over time, as illustrated in Figure 9.7.

训练借助 常微分方程 (ODE) 轨迹描述图像随时间逐步去噪的路径；微分方程刻画噪声如何演变，见图 9.7。

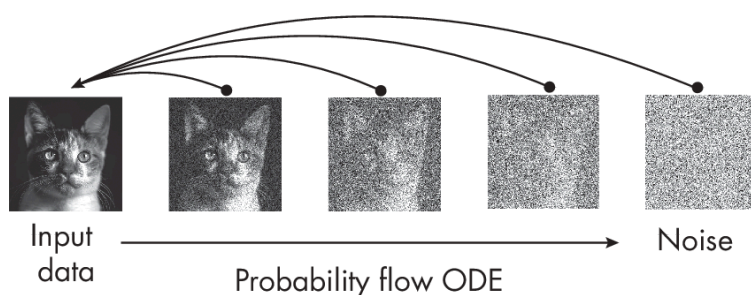


Figure 9.7

As Figure 9.7 demonstrates, we can think of consistency models as models that learn to map any point from a probability flow ODE, which smoothly converts data to noise, to the input.

如图 9.7 所示，可把一致性模型理解为：学习将概率流 ODE 上任意一点（该 ODE 平滑地把数据变成噪声）映射回数据（输入）端。

At the time of writing, consistency models are the most recent type of generative AI model. Based on the original paper proposing this method, consistency models rival diffusion models in terms of image quality. Consistency models are also faster than diffusion models because they do not require an iterative process to generate images; instead, they generate images in a single step.

写作时一致性模型是较新的一类生成式方法；原论文称其在图像质量上可与扩散模型媲美。因无需迭代多步、可单步生成图像，推断快于扩散模型。

However, while consistency models allow for faster inference, they are still expensive to train because they require a large dataset of pairs of noisy and clean images.

但训练成本仍高：需要大量含噪与干净图像的成对数据。

Recommendations

各模型适用场景小结：历史意义、似然估计与当下图像生成主流选择。

Deep Boltzmann machines are interesting from a historical perspective since they were one of the pioneering models to effectively demonstrate the concept of unsupervised learning. Flow-based and autoregressive models may be useful



when you need to estimate exact likelihoods. However, other models are usually the first choice when it comes to generating high-quality images.

深玻尔兹曼机在历史上率先有效展示无监督学习，有其价值。若需精确似然，流模型与自回归模型更有用；但追求高质量图像时，人们通常首选其他架构。

In particular, VAEs and GANs have competed for years to generate the best high-fidelity images. However, in 2022, diffusion models began to take over image generation almost entirely. Consistency models are a promising alternative to diffusion models, but it remains to be seen whether they become more widely adopted to generate state-of-the-art results. The trade-off here is that sampling from diffusion models is generally slower since it involves a sequence of noise-removal steps that must be run in order, similar to autoregressive models. This can make diffusion models less practical for some applications requiring fast sampling.

VAE 与 GAN 曾长期角逐高保真图像；2022 年后扩散模型几乎主导图像生成。一致性模型是有前景的替代，但能否广泛用于 SOTA 尚待观察。折中在于：扩散采样需顺序多步去噪，类似自回归，故在要快速采样的应用中不够方便。

Exercises

习题：评价生成图像质量；如何用一致性模型生成新图像。

9-1. How would we evaluate the quality of the images generated by a generative AI model?

9-1. 如何评价生成式 AI 模型所得图像的质量？

9-2. Given this chapter's description of consistency models, how would we use them to generate new images?

9-2. 根据本章对一致性模型的描述，应如何用其生成新图像？

References

参考文献：VAE、GAN、NICE、PixelCNN、Stable Diffusion 与一致性模型等原始资料与实现链接。



- The original paper proposing variational autoencoders: Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes"? (2013), <https://arxiv.org/abs/1312.6114>.
- 提出变分自编码器的原始论文：Diederik P. Kingma 与 Max Welling, 《Auto-Encoding Variational Bayes》(2013), <https://arxiv.org/abs/1312.6114>。
- The paper introducing generative adversarial networks: Ian J. Goodfellow et al., "Generative Adversarial Networks"? (2014), <https://arxiv.org/abs/1406.2661>.
- 引入生成对抗网络的论文：Ian J. Goodfellow 等, 《Generative Adversarial Networks》(2014), <https://arxiv.org/abs/1406.2661>。
- The paper introducing NICE: Laurent Dinh, David Krueger, and Yoshua Bengio, "NICE: Non-linear Independent Components Estimation"? (2014), <https://arxiv.org/abs/1410.8516>.
- 介绍 NICE 的论文：Laurent Dinh、David Krueger 与 Yoshua Bengio, 《NICE: Non-linear Independent Components Estimation》(2014), <https://arxiv.org/abs/1410.8516>。
- The paper proposing the autoregressive PixelCNN model: Aaron van den Oord et al., "Conditional Image Generation with PixelCNN Decoders"? (2016), <https://arxiv.org/abs/1606.05328>.
- 提出自回归 PixelCNN 的论文：Aaron van den Oord 等, 《Conditional Image Generation with PixelCNN Decoders》(2016), <https://arxiv.org/abs/1606.05328>。
- The paper introducing the popular Stable Diffusion latent diffusion model: Robin Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models"? (2021), <https://arxiv.org/abs/2112.10752>.
- 介绍广泛使用的 Stable Diffusion（潜空间扩散）的论文：Robin Rombach 等, 《High-Resolution Image Synthesis with Latent Diffusion Models》(2021), <https://arxiv.org/abs/2112.10752>。



- The Stable Diffusion code implementation: <https://github.com/CompVis/stable-diffusion>.
 - Stable Diffusion 代码实现: <https://github.com/CompVis/stable-diffusion>。
 - The paper originally proposing consistency models: Yang Song et al., "Consistency Models"? (2023), <https://arxiv.org/abs/2303.01469>.
 - 最初提出一致性模型的论文: Yang Song 等, 《Consistency Models》(2023), <https://arxiv.org/abs/2303.01469>。
-



Chapter 10: Sources of Randomness

本章归类并讨论深度网络训练与推理中导致不可复现结果的各种随机性来源。

What are the common sources of randomness when training deep neural networks that can cause non-reproducible behavior during training and inference?

训练深度神经网络时，哪些常见的随机性来源会导致训练与推理阶段的不可复现行为？

When training or using machine learning models such as deep neural networks, several sources of randomness can lead to different results every time we train or run these models, even though we use the same overall settings. Some of these effects are accidental and some are intended. The following sections categorize and discuss these various sources of randomness.

训练或使用深度神经网络等模型时，即使整体设置相同，多种随机性仍可能使每次训练或运行的结果不同；其中有些是意外，有些是刻意的。以下几节将这些随机性分类并逐一讨论。

Tips:

- 在训练和使用机器学习模型时，如深度神经网络，**随机性**会导致每次 **训练** 或 **运行** 模型时得到不同的结果，即使我们使用相同的配置。
- 这些随机性可能是 **偶然** 的，也可能是 **故意** 的。

Optional hands-on examples for most of these categories are provided in the *supplementary/q10-random-sources* subfolder at <https://github.com/rasbt/MachineLearning-QandAI-book>.

多数类别的动手示例见图书配套仓库中的 *supplementary/q10-random-sources* 子文件夹：<https://github.com/rasbt/MachineLearning-QandAI-book>。

Model Weight Initialization

随机权重初始化、非凸损失下的不同局部极小，以及用随机种子固定初始化。



All common deep neural network frameworks, including TensorFlow and PyTorch, randomly initialize the weights and bias units at each layer by default. This means that the final model will be different every time we start the training. The reason these trained models will differ when we start with different random weights is the nonconvex nature of the loss, as illustrated in Figure 10.1. As the figure shows, the loss will converge to different local minima depending on where the initial starting weights are located.

TensorFlow、PyTorch 等框架默认对各层权重与偏置随机初始化，因此每次开训会得到不同模型。从不同随机权重出发会得到不同训练结果的原因在于损失的非凸性（见图 10.1）：初值位置不同，损失会收敛到不同局部极小。

Tips: 初始化权重，会导致不同的局部最优解。

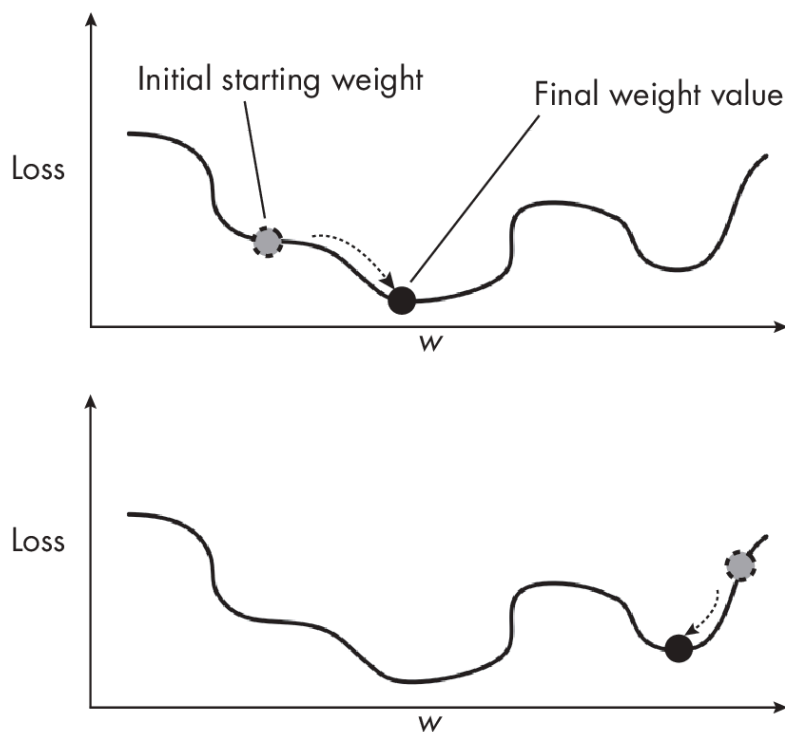


Figure 10.1

In practice, it is therefore recommended to run the training (if the computational resources permit) at least a handful of times; unlucky initial weights can sometimes cause the model not to converge or to converge to a local minimum corresponding to poorer predictive accuracy.



实践中若算力允许，建议至少重复训练数次；糟糕的初值有时会导致不收敛或收敛到预测更差的局部极小。

Tips: 实践中，建议 **至少** 运行 **训练几次**，以避免不幸运的初始权重，导致模型不收敛或收敛到较差的局部最优解。

However, we can make the random weight initialization deterministic by seeding the random generator. For instance, if we set the seed to a specific value like 123, the weights will still initialize with small random values. Nonetheless, the neural network will consistently initialize with the **same small random weights**, enabling accurate reproduction of results.

但通过给随机数生成器设种子可使初始化确定：例如固定种子为 123，权重仍为小幅随机值，但每次都会得到**同一组小幅随机初值**，从而可复现实验。

Tips: 通过设置 **随机种子**，可以使得 **初始化** 权重参数是 **确定的**。

Dataset Sampling and Shuffling

数据集划分、k 折等与随机抽样相关的变异性。

When we train and evaluate machine learning models, we usually start by dividing a dataset into training and test sets. This requires random sampling since we have to decide which examples we put into a training set and which examples we put into a test set.

训练与评估模型时通常先把数据划分为训练集与测试集，这需要随机抽样来决定哪些样本进训练、哪些进测试。

In practice, we often use model evaluation techniques such as k -fold cross-validation or holdout validation. In holdout validation, we split the training set into **training**, **validation**, and **test** datasets, which are also sampling procedures influenced by randomness. Similarly, unless we use a fixed random seed, we get a different model each time we partition the dataset or tune or evaluate the model using k -fold cross-validation since the training partitions will differ.



实践中还常用 k 折交叉验证或留出法；留出法下再把数据分为 **训练**、**验证**、**测试**，同样受随机划分影响。若不固定种子，每次划分或用 k 折调参/评估时，训练子集不同，模型也会不同。

Tips:

- 在训练和评估机器学习模型时，我们通常将数据集分为 **训练集**、**验证集** 和 **测试集**。
- 这需要 **随机采样**，因为我们必须决定将哪些样本放入训练集，哪些放入验证集，哪些放入测试集。
- 除非我们使用固定的随机种子，否则每次划分数据集或使用 **k** 折交叉验证时，我们都会得到不同的模型。

Nondeterministic Algorithms

网络结构中的随机成分（如 dropout）及如何配合种子与推理模式获得确定性。

We may include random components and algorithms depending on the architecture and hyperparameter choices. A popular example of this is **dropout**.

按架构与超参选择，模型中可能包含随机成分，**dropout** 是常见例子。

Dropout works by randomly setting a fraction of a layer's units to zero during training, which helps the model learn more robust and generalized representations. This "**dropping out**" is typically applied at each training iteration with a probability p , a hyperparameter that controls the fraction of units dropped out. Typical values for p are in the range of 0.2 to 0.8.

Dropout 在训练时以概率 p 随机将一层中一部分单元置零，促使表示更稳健、泛化更好； p 控制丢弃比例，典型取值约在 0.2–0.8。

To illustrate this concept, Figure 10.2 shows a small neural network where dropout randomly drops a subset of the hidden layer nodes in each forward pass during training.

图 10.2 示意小网络在训练每次前向时 dropout 如何随机屏蔽部分隐层节点。

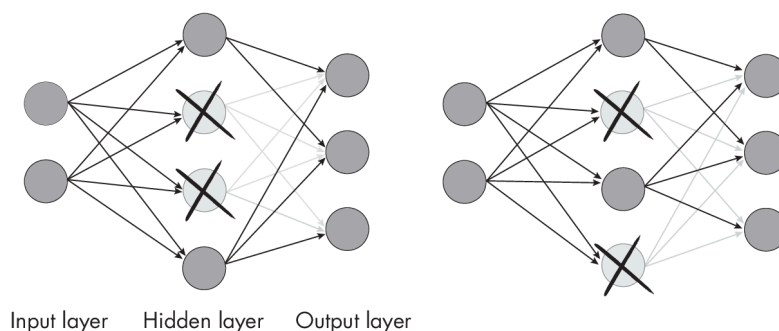


Figure 10.2

To create reproducible training runs, we must seed the random generator before training with dropout (analogous to seeding the random generator before initializing the model weights). During inference, we need to disable dropout to guarantee deterministic results. Each deep learning framework has a specific setting for that purpose -- a PyTorch example is included in the *supplementary/q10-random-sources* subfolder at

<https://github.com/rasbt/MachineLearning-QandAI-book>.

要复现含 dropout 的训练，须在训练前设种子（与权重初始化前设种子类似）。推理时应关闭 dropout 以保证确定性。各框架有对应设置；PyTorch 示例见 *supplementary/q10-random-sources*：

<https://github.com/rasbt/MachineLearning-QandAI-book>。

Tips:

- 为了创建可复现的训练运行，我们必须在训练前设置随机种子（类似于在初始化模型权重之前设置随机种子）。
- 在推理时，我们需要禁用 `dropout`，以保证结果的确定性。
- 每个深度学习框架都有特定的设置，以实现这一点。

Different Runtime Algorithms

同名运算的不同 GPU 实现（如卷积）、数值微差累积与确定性算法选项。

The most intuitive or simplest implementation of an algorithm or method is not always the best one to use in practice. For example, when training deep neural networks, we often use efficient alternatives and approximations to gain speed and resource advantages during training and inference.



理论上最直观的实现未必是实践最优；深度网络训练常用更快或更省资源的近似实现。

A popular example is the convolution operation used in convolutional neural networks. There are several possible ways to implement the convolution operation:

卷积是典型例子，实现途径有多种：

The **classic direct convolution** The common implementation of discrete convolution via an element-wise product between the input and the window, followed by summing the result to get a single number. (See Chapter [\[ch12\]](#) for a discussion of the convolution operation.)

经典直接卷积：输入与窗口逐元相乘再累加成标量。（卷积运算讨论见第 [\[ch12\]](#) 章。）

FFT-based convolution Uses **fast Fourier transform** (FFT) to convert the convolution into an element-wise multiplication in the frequency domain.

基于 FFT 的卷积：用快速傅里叶变换在频域做逐元乘法实现卷积。

Winograd-based convolution An efficient algorithm for small filter sizes (like 3×3 that reduces the number of multiplications required for the convolution.

基于 Winograd 的卷积：适用于小核（如 3×3 ），减少乘法次数。

Different convolution algorithms have different trade-offs in terms of memory usage, computational complexity, and speed. By default, libraries such as the **CUDA Deep Neural Network library** (**cuDNN**), which are used in PyTorch and TensorFlow, can choose different algorithms for performing convolution operations when running deep neural networks on GPUs. However, the deterministic algorithm choice has to be explicitly enabled. In PyTorch, for example, this can be done by setting

不同算法在内存、复杂度与速度上权衡各异。PyTorch/TensorFlow 调用的 **cuDNN** 等在 GPU 上默认可能切换卷积算法；要使用确定性算法需显式开启，例如 PyTorch 中设置



```
torch.use_deterministic_algorithms(True)
```

While these approximations yield similar results, subtle numerical differences can accumulate during training and cause the training to converge to slightly different local minima.

近似结果看似相近，但细微数值差异在训练中会累积，使收敛到的局部极小略有不同。

Tips: 算法自身也会带来随机性，特别是不同优化算法实现，本身得到的就是近似效果。

- 不同的卷积算法在 内存 使用、 计算复杂度 和 速度 方面有不同的权衡。
- 默认情况下，PyTorch和TensorFlow等库中的 `CUDA Deep Neural Network library`（`cuDNN`）可以选择不同的算法来执行卷积操作。
- 但是，确定性算法的选择必须显式启用。
- 在PyTorch中，可以通过设置 `torch.use_deterministic_algorithms(True)` 来启用确定性算法。

Hardware and Drivers

精度、硬件与库优化导致的数值差异。

Training deep neural networks on different hardware can also produce different results due to small numeric differences, even when the same algorithms are used and the same operations are executed. These differences may sometimes be due to different numeric precision for floating-point operations. However, small numeric differences may also arise due to hardware and software optimization, even at the same precision.

在不同硬件上训练即使用相同算法与运算，也可能因微小数值差异而结果不同：可能源于浮点精度不同，也可能在相同精度下仍因硬件与软件优化产生偏差。

Tips: 硬件和驱动也会带来随机性，特别是不同硬件平台，不同优化库，不同优化算法实现，本身得到的就是近似效果。

- 不同的 **数值精度**，会导致不同的结果。
- 不同的 **硬件** 和 **软件** 优化，会导致不同的结果。

For instance, different hardware platforms may have specialized optimizations or libraries that can slightly alter the behavior of deep learning algorithms. To give one example of how different GPUs can produce different modeling results, the following is a quotation from the official NVIDIA documentation: "Across different architectures, no cuDNN routines guarantee bit-wise reproducibility. For example, there is no guarantee of bit-wise reproducibility when comparing the same routine run on NVIDIA *VoltaTM* and NVIDIA *TuringTM* [. . .] and NVIDIA Ampere architecture."

例如不同平台有专门优化或库，会轻微改变数值行为。NVIDIA 官方文档说明：跨架构时 cuDNN 例程不保证按位可复现；例如在 Volta、Turing 与 Ampere 等架构上运行同一例程，无法保证按位一致。

Tips:

- 不同的硬件平台可能具有专门的优化或库，可以稍微改变深度学习算法的性能。
- 例如，不同的GPU可以产生不同的建模结果。

Randomness and Generative AI

生成式模型在推理中“有意为之”的随机性；扩散噪声与 LLM 采样策略。

Besides the various sources of randomness mentioned earlier, certain models may also exhibit random behavior during inference that we can think of as "**randomness by design**." For instance, generative image and language models may create different results for identical prompts to produce a diverse sample of results. For image models, this is often so that users can select the most accurate and aesthetically pleasing image. For language models, this is often to vary the responses, for example, in chat agents, to avoid repetition.



除前述来源外，部分模型在推理阶段会表现出可称为“**设计上的随机性**”的行为：例如生成式图像/语言模型对同一提示给出不同输出以丰富样本；图像侧便于用户挑选；语言侧（如聊天）可减少重复。

Tips:

- 除了前面提到的各种随机性来源，某些模型在**推理时**也可能表现出**随机行为**，我们可以将其视为“**设计上的随机性**”。
- 例如，生成式图像和语言模型可能会对相同的提示产生不同的结果，以产生多样化的结果样本。
- 对于图像模型，这通常是为了让用户选择最准确和最吸引人的图像。
- 对于语言模型，这通常是为了避免重复，例如在聊天代理中。

The intended randomness in generative image models during inference is often due to sampling different noise values at each step of the reverse process. In diffusion models, a noise schedule defines the noise variance added at each step of the diffusion process.

生成式图像模型推理中的有意随机常来自反向过程各步对噪声的不同采样；扩散模型用噪声调度规定每步所加噪声方差。

Tips:

- 在生成式图像模型中，推理时的随机性，通常是由于在反向过程中对不同的 **噪声值** 进行采样。
- 在扩散模型中，**噪声调度** 定义了扩散过程中添加的 **噪声方差**。

Autoregressive LLMs like GPT tend to create different outputs for the same input prompt (GPT will be discussed at greater length in Chapters [\[ch14\]](#) and [\[ch17\]](#)). The ChatGPT user interface even has a Regenerate Response button for that purpose. The ability to generate different results is due to the sampling strategies these models employ. Techniques such as top- k sampling, nucleus sampling, and temperature scaling influence the model's output by controlling the degree of randomness. This is a feature, not a bug, since it allows for diverse responses and prevents the model from producing overly deterministic or repetitive outputs. (See Chapter [\[ch09\]](#) for a more in-depth overview of generative AI and deep learning models; see Chapter [\[ch17\]](#) for more detail on autoregressive LLMs.)



自回归 LLM（如 GPT）对同一提示常产生不同输出（GPT 详见第 [ch14]、[ch17] 章）；ChatGPT 界面亦有“重新生成”按钮。差异来自 top- k 、核采样、温度等采样策略对随机程度的控制——这是特性而非缺陷，可带来多样性并避免刻板重复。（生成式模型总览见第 [ch09] 章，自回归 LLM 细节见第 [ch17] 章。）

Tips:

- 自回归语言模型（如GPT）倾向于对相同的输入提示，产生不同的输出。
- 这是因为这些模型，采用了不同的 **采样策略**。
- 例如，**top- k 采样**、**核采样** 和 **温度缩放** 等技术，通过控制随机性程度，影响模型的输出。

Top- k sampling, illustrated in Figure 10.3, works by sampling tokens from the top k most probable candidates at each step of the next-word generation process.

Top- k sampling（图 10.3）在每一步从概率最高的 k 个词元中采样下一词。

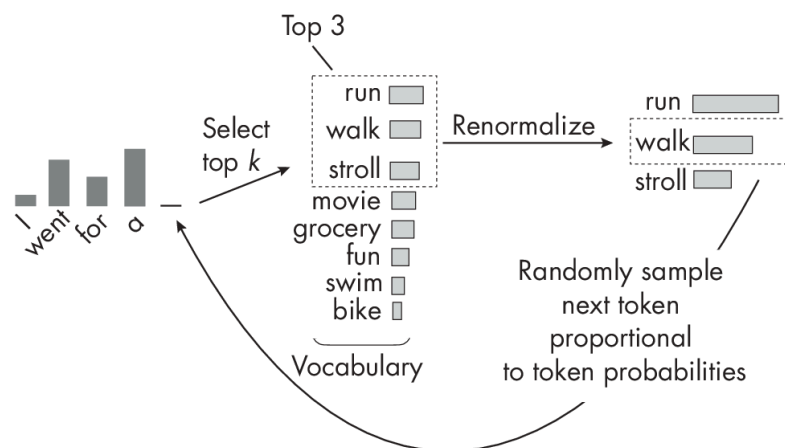


Figure 10.3

Given an input prompt, the language model produces a probability distribution over the entire vocabulary (the candidate words) for the next token. Each token in the vocabulary is assigned a probability based on the model's understanding of the context. The selected top- k tokens are then renormalized so that the probabilities sum to 1. Finally, a token is sampled from the renormalized top- k probability distribution and is appended to the input prompt. This process is



repeated for the desired length of the generated text or until a stop condition is met.

给定提示，模型为下一词元给出全词表上的概率分布；按上下文为每个候选词赋概率。取 top- k 后重归一化使和为 1，再从中采样一词并拼回提示；重复直至达到目标长度或停止条件。

Nucleus sampling (also known as *top- p sampling*), illustrated in Figure 10.4, is an alternative to top- k sampling.

核采样（top- p 采样，图 10.4）是 top- k 的替代方案。

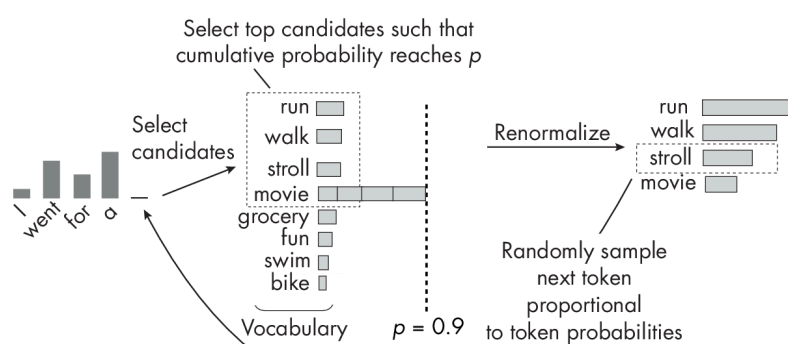


Figure 10.4

Similar to top- k sampling, the goal of nucleus sampling is to balance diversity and coherence in the output. However, nucleus and top- k sampling differ in how to select the candidate tokens for sampling at each step of the generation process. Top- k sampling selects the k most probable tokens from the probability distribution produced by the language model, regardless of their probabilities. The value of k remains fixed throughout the generation process. Nucleus sampling, on the other hand, selects tokens based on a probability threshold p , as shown in Figure 10.4. It then accumulates the most probable tokens in descending order until their cumulative probability meets or exceeds the threshold p . In contrast to top- k sampling, the size of the candidate set (nucleus) can vary at each step.

与 top- k 类似，核采样也在多样性与连贯性间折中，但候选集构造不同：top- k 固定取概率最高的 k 个， k 全程不变；核采样按阈值 p （图 10.4）从高概率往下累加直至累计概率 $\geq p$ ，故每步候选集大小可变。

Tips:



- 与top- k 采样类似，核采样的目标是平衡输出中的多样性和连贯性。
- 然而，核采样和top- k 采样在选择每个生成步骤中的候选标记时有所不同。
- **top- k 采样***从语言模型产生的概率分布中选择 **概率最高** 的 k *个标记，而核采样**则根据 **累计的概率阈值** p 选择标记。

Exercises

习题：top- k /核采样的确定性；推理时何时希望 dropout 仍随机。

10-1. Suppose we train a neural network with top- k or nucleus sampling where k and p are hyperparameter choices. Can we make the model behave deterministically during inference without changing the code?

10-1. 若在训练中使用 top- k 或核采样且 k 、 p 为超参，不改代码能否在推理时使模型行为确定？

10-2. In what scenarios might random dropout behavior during inference be desired?

10-2. 何种场景下会希望在推理阶段仍保留 dropout 的随机行为？

References

延伸阅读：模型评估、dropout 原论文、FFT/Winograd 卷积与 PyTorch/cuDNN 确定性设置。

- For more about different data sampling and model evaluation techniques, see my article: "Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning"? (2018), <https://arxiv.org/abs/1811.12808>.
- 关于数据抽样与模型评估等：见文章 "Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning" (2018), <https://arxiv.org/abs/1811.12808>。



- The paper that originally proposed the dropout technique: Nitish Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting"? (2014), <https://jmlr.org/papers/v15/srivastava14a.html>.
- 提出 dropout 的论文：Nitish Srivastava 等，"Dropout: A Simple Way to Prevent Neural Networks from Overfitting" (2014)，
<https://jmlr.org/papers/v15/srivastava14a.html>。
- A detailed paper on FFT-based convolution: Lu Chi, Borui Jiang, and Yadong Mu, "Fast Fourier Convolution"? (2020),
<https://dl.acm.org/doi/abs/10.5555/3495724.3496100>.
- FFT 卷积详解：Lu Chi、Borui Jiang、Yadong Mu，《Fast Fourier Convolution》(2020)，
<https://dl.acm.org/doi/abs/10.5555/3495724.3496100>。
- Details on Winograd-based convolution: Syed Asad Alam et al., "Winograd Convolution for Deep Neural Networks: Efficient Point Selection"? (2022),
<https://arxiv.org/abs/2201.10369>.
- Winograd 卷积：Syed Asad Alam 等，《Winograd Convolution for Deep Neural Networks: Efficient Point Selection》(2022)，
<https://arxiv.org/abs/2201.10369>。
- More information about the deterministic algorithm settings in PyTorch:
https://pytorch.org/docs/stable/generated/torch.use_deterministic_algorithms.html.
- PyTorch 确定性算法：
https://pytorch.org/docs/stable/generated/torch.use_deterministic_algorithms.html。
- For details on the deterministic behavior of NVIDIA graphics cards, see the "Reproducibility"? section of the official NVIDIA documentation:
<https://docs.nvidia.com/deeplearning/cudnn/developer-guide/index.html#reproducibility>.
- NVIDIA 显卡/cuDNN 可复现性参见官方文档“Reproducibility”：
<https://docs.nvidia.com/deeplearning/cudnn/developer-guide/index.html#reproducibility>。



Chapter 11: Calculating the Number of Parameters

如何从卷积与全连接层逐层清点参数并求和；参数规模对复杂度、数据量与内存的意义。

How do we compute the number of parameters in a convolutional neural network, and why is this information useful?

如何计算卷积神经网络中的参数个数，这些信息有何用处？

Knowing the number of parameters in a model helps gauge the model's size, which affects storage and memory requirements. The following sections will explain how to compute the convolutional and fully connected layer parameter counts.

知道参数个数有助于衡量模型体量，从而影响存储与显存需求。以下几节说明如何清点卷积层与全连接层的参数。

Tips: 模型参数的数量，是衡量模型大小（存储空间大小）的重要指标，用于估算所需的存储空间。重点关注 **卷积层** 和 **全连接层**。

How to Find Parameter Counts

通过一个具体 CNN 架构从左到右逐层累加可训练权重与偏置。

Suppose we are working with a convolutional network that has two **convolutional layers** with kernel size 5 and kernel size 3, respectively.

设想一个 CNN，含有两个 **卷积层**，核大小分别为 5 与 3。

- The first convolutional layer has 3 input channels and 5 output channels,
- and the second one has 5 input channels and 12 output channels.
- The stride of these **convolutional layers** is 1.



第一层有 3 个输入通道、5 个输出通道；第二层有 5 个输入通道、12 个输出通道；两层步长均为 1。

Furthermore, the network has two **pooling layers**,

此外网络有两个 **池化层**,

- one with a kernel size of 3 and a stride of 2,
- and another with a kernel size of 5 and a stride of 2.

其一池化核为 3、步长为 2；其二池化核为 5、步长为 2。

It also has two **fully connected hidden layers** with 192 and 128 hidden units each, where the output layer is a **classification layer** for 10 classes.

另有两个 **全连接隐层**，单元数分别为 192 与 128，输出层为 10 类的 **分类层**。

The architecture of this network is illustrated in Figure 11.1.

网络结构见图 11.1。

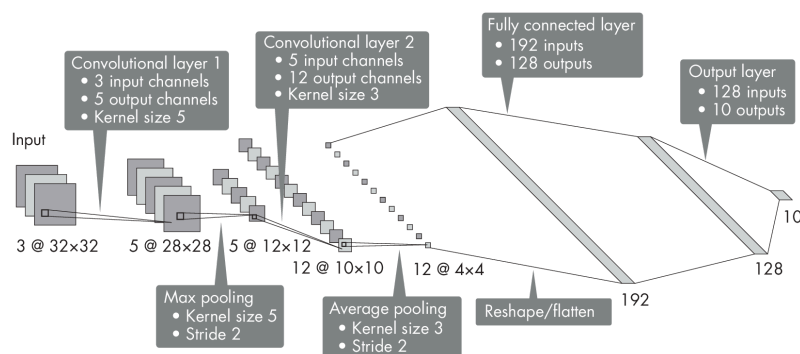


Figure 11.1

What is the number of trainable parameters in this convolutional network? We can approach this problem from left to right, computing the number of parameters for each layer and then summing up these counts to obtain the total number of parameters. Each layer's number of trainable parameters consists of weights and bias units.

该网络有多少可训练参数？可自左向右逐层计算再求和；每层的可训练参数包括权重与偏置。

Convolutional Layers

卷积核尺寸、输入/输出通道数与偏置如何决定卷积层参数量。



In a **convolutional layer**, the number of weights depends on the kernel's width and height and the number of input and output channels. The number of bias units depends on the number of output channels only. To illustrate the computation step by step, suppose we have a kernel width and height of 5, one input channel, and one output channel, as illustrated in Figure 11.2.

卷积层 的权重数取决于核高宽与输入、输出通道数；偏置个数仅由输出通道数决定。为逐步说明，设核为 5×5 、单输入单输出通道，见图 11.2。

Tips: 卷积层的参数数量，取决于 **卷积核** (kernel) 的宽度、高度、输入通道数和输出通道数。

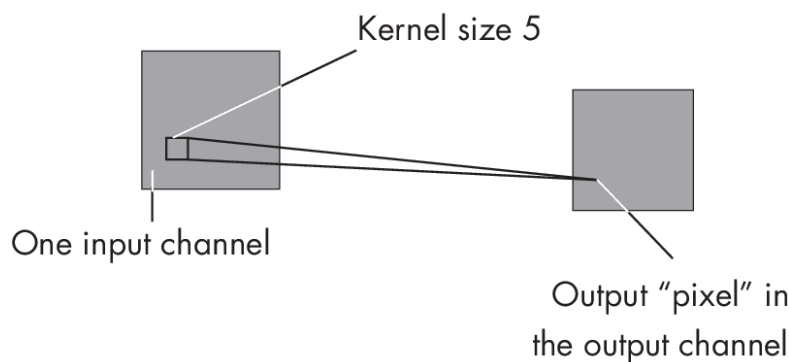


Figure 11.2

In this case, we have 26 parameters, since we have $5 \times 5 = 25$ weights via the kernel plus the bias unit. The computation to determine an output value or pixel z is $z = b + \sum_j w_j x_j$, where x_j represents an input pixel, w_j represents a weight parameter of the kernel, and b is the bias unit.

此情形共 26 个参数：核上 $5 \times 5 = 25$ 个权重加 1 个偏置。输出 $z = b + \sum_j w_j x_j$ ，其中 x_j 为输入像素， w_j 为核权重， b 为偏置。

Tips: 一个卷积核 **kernel** 的参数量，**weights** = 宽度 x 高度。

Now, suppose we have three input channels, as illustrated in Figure 11.3.

现设三输入通道，见图 11.3。

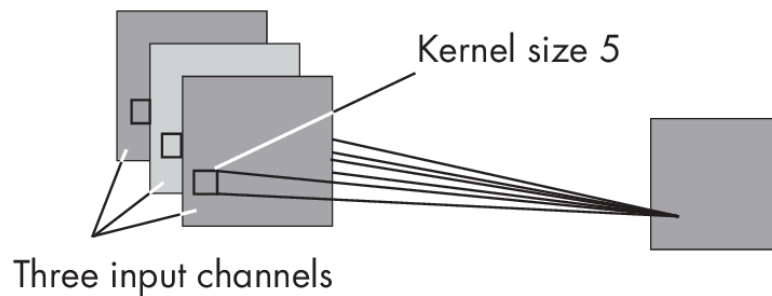


Figure 11.3

In that case, we compute the output value by performing the aforementioned operation, $\sum_j w_j x_j$, for each input channel and then add the bias unit. For three input channels, this would involve three different kernels with three sets of weights:

此时对每个输入通道各自做 $\sum_j w_j x_j$ 再相加并加偏置；三通道即三套权重：

$$z = \sum_j w_j^{(1)} x_j + \sum_j w_j^{(2)} x_j + \sum_j w_j^{(3)} x_j + b$$

Since we have three sets of weights ($w^{(1)}$, $w^{(2)}$, and $w^{(3)}$ for $j = [1, 25]$), we have $3 \times 25 + 1 = 76$ parameters in this convolutional layer.

三组权重 ($w^{(1)}$, $w^{(2)}$, $w^{(3)}$ ，各 $j = 1..25$) 故有 $3 \times 25 + 1 = 76$ 个参数。

Tips: 每个输入通道，对应的卷积核 **kernel**，都是独立的参数。

We use one kernel for each output channel, where each kernel is unique to a given output channel. Thus, if we extend the number of output channels from one to five, as shown in Figure 11.4, we extend the number of parameters by a factor of 5.

每一输出通道用各自一套核。若输出通道由 1 扩到 5（图 11.4），参数量约为原来的 5 倍。

In other words, if the kernel for one output channel has 76 parameters, the 5 kernels required for the five output channels will have $5 \times 76 = 380$ parameters.

若单输出通道需 76 个参数，五通道共 $5 \times 76 = 380$ 个参数。

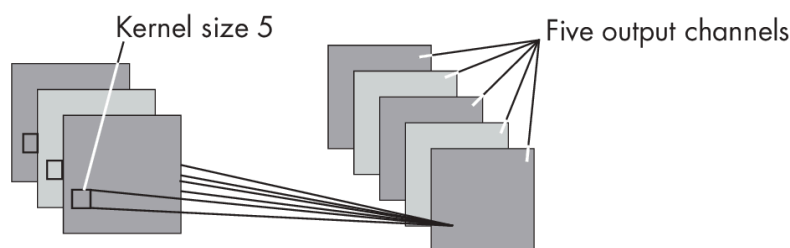


Figure 11.4

Returning to the neural network architecture illustrated in Figure [11.1](#) at the beginning of this section, we compute the number of parameters in the convolutional layers based on the kernel size and number of input and output channels. For example, the first convolutional layer has three input channels, five output channels, and a kernel size of 5. Thus, its number of parameters is $5 \times (5 \times 5 \times 3) + 5 = 380$.

回到本节开头的图 [11.1](#)：第一层 3 入 5 出、核 5，参数为 $5 \times (5 \times 5 \times 3) + 5 = 380$ 。

The second convolutional layer, with five input channels, 12 output channels, and a kernel size of 3, has $12 \times (3 \times 3 \times 5) + 12 = 552$ parameters.

第二层 5 入 12 出、核 3，参数 $12 \times (3 \times 3 \times 5) + 12 = 552$ 。

Since the **pooling layers** do not have any trainable parameters, we can count $380 + 552 = 932$ for the convolutional part of this architecture.

池化层 无可训练参数，卷积部分共 $380 + 552 = 932$ 。

Next, let's see how we can compute the number of parameters of fully connected layers.

接着看全连接层参数如何计算。

Fully Connected Layers

全连接层：权重为输入维度乘输出维度，再加输出侧偏置。

Counting the number of parameters in a **fully connected layer** is relatively straightforward. A fully connected node connects each input node to each output



node, so the number of weights is the number of inputs times the number of outputs plus the bias units added to the output. For example, if we have a fully connected layer with five inputs and three outputs, as shown in Figure 11.5, we have $5 \times 3 = 15$ weights and three bias units, that is, 18 parameters total.

全连接层 参数很直接：每个输出与所有输入相连，权重数为输入数×输出数，再加每个输出的偏置。例如 5 入 3 出（图 11.5）： $5 \times 3 = 15$ 个权重、3 个偏置，共 18。

Tips: 一个全连接层 **fully connected layer** 的参数量，**weights** = 输入节点数 × 输出节点数。

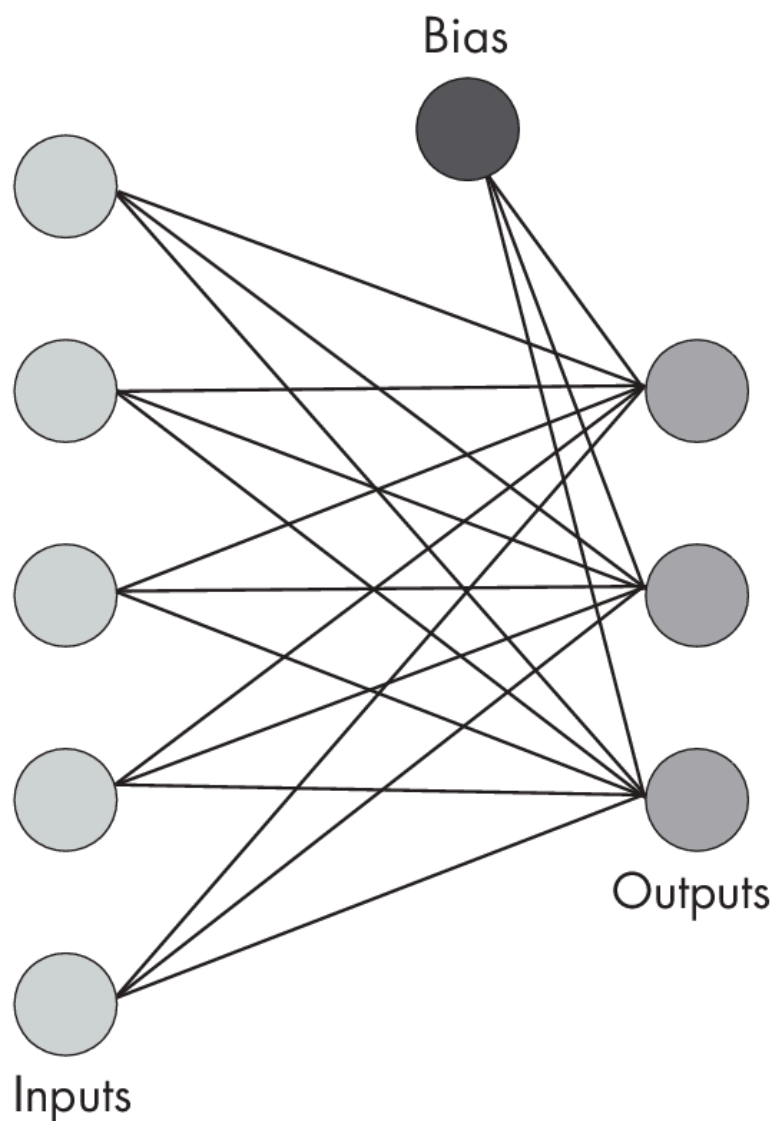


Figure 11.5



Returning once more to the neural network architecture illustrated in Figure [11.1](#), we can now calculate the parameters in the fully connected layers as follows: $192 \times 128 + 128 = 24,704$ in the first fully connected layer and $128 \times 10 + 10 = 1,290$ in the second fully connected layer, the output layer.

再回到图 [11.1](#)：第一层全连接 $192 \times 128 + 128 = 24,704$ ；第二层（输出） $128 \times 10 + 10 = 1,290$ 。

Hence, we have $24,704 + 1,290 = 25,994$ in the fully connected part of this network.

故全连接部分共 $24,704 + 1,290 = 25,994$ 。

After adding the 932 parameters from the convolutional layers and the 25,994 parameters from the fully connected layers, we can conclude that this network's total number of parameters is 26,926.

卷积部分 932 与全连接 25,994 相加，总参数量为 26,926。

As a bonus, interested readers can find PyTorch code to compute the number of parameters programmatically in the *supplementary/q11-conv-size* subfolder at <https://github.com/rasbt/MachineLearning-QandAI-book>.

补充：读者可在仓库 *supplementary/q11-conv-size* 找到用 PyTorch 编程统计参数的示例：<https://github.com/rasbt/MachineLearning-QandAI-book>。

Practical Applications

参数规模与模型复杂度、训练数据需求及硬件可承载量的关系。

Why do we care about the number of parameters at all? First, we can use this number to estimate a model's complexity. As a rule of thumb, the more parameters there are, the more training data we'll need to train the model well.

为何要关心参数数量？其一可用它估算模型复杂度：粗略地说，参数越多，通常需要更多训练数据才能学好。

Tips: 模型 **参数的数量**，是衡量模型 **复杂度** 的重要指标，用于估算所需的 **训练数据量**。



The number of parameters also lets us estimate the size of the neural network, which in turn helps us estimate whether the network can fit into GPU memory. Although the memory requirement during training often exceeds the model size due to the additional memory required for carrying out matrix multiplications and storing gradients, model size gives us a ballpark sense of whether training the model on a given hardware setup is feasible.

其二可估计网络体量，从而判断能否装入 GPU 显存。训练时实际占用常大于纯模型体积（前向乘积、梯度存储等），但模型大小仍能给出在既定硬件上是否大致可训的粗判。

Tips: 模型参数的数量，是衡量模型 **大小** 的重要指标，用于估算模型是否能 **fit** 到 GPU 的 **内存** 中。

Exercises

习题：SGD 与 Adam 需存哪些额外状态；三处 BatchNorm 增加多少参数。

11-1. Suppose we want to optimize the neural network using a plain stochastic gradient descent (SGD) optimizer or the popular Adam optimizer. What are the respective numbers of parameters that need to be stored for SGD and Adam?

11-1. 若用普通 SGD 或 Adam 优化该网络，分别需要为优化器额外存储多少组与参数同形状的变量？

11-2. Suppose we're adding three batch normalization (BatchNorm) layers: one after the first convolutional layer, one after the second convolutional layer, and another one after the first fully connected layer (we typically do not want to add BatchNorm layers to the output layer). How many additional parameters do these three BatchNorm layers add to the model?

11-2. 若在第一个卷积后、第二个卷积后以及第一个全连接后各加一个 BatchNorm（通常不在输出层加），这三个 BatchNorm 共为模型增加多少参数？





Chapter 12: Fully Connected and Convolutional Layers

在什么条件下卷积可实现与全连接相同的计算：核尺寸等于感受野或核尺寸为 1。

Under which circumstances can we replace fully connected layers with convolutional layers to perform the same computation?

在什么情况下可以用卷积层替代全连接层而保持同一计算？

Replacing fully connected layers with convolutional layers can offer advantages in terms of hardware optimization, such as by utilizing specialized hardware accelerators for convolution operations. This can be particularly relevant for edge devices.

用卷积替代全连接可借助卷积专用硬件加速器做优化，对边缘设备尤有意义。

Tips: 卷积层 替代 全连接层，有下面收益

- 卷积操作可以 硬件加速
- 这在 边缘设备 上非常关键。

There are exactly two scenarios in which fully connected layers and convolutional layers are equivalent: when the size of the convolutional filter is equal to the size of the receptive field and when the size of the convolutional filter is 1. As an illustration of these two scenarios, consider a fully connected layer with two input and four output units, as shown in Figure [12.1](#).

全连接与卷积完全等价恰有两种情形：卷积核尺寸等于感受野尺寸，或核尺寸为 1。下文用两层入、四层出的全连接为例（图 [12.1](#)）。

Tips: 全连接层 和 卷积层 在两种情况下是 等价 的：

- 当 卷积核 的大小等于 感受野 的大小。
- 当 卷积核 的大小为 1。



receptive field 感受野，在 CNN 和 RNN 中，有不同的含义。

- 在卷积神经网络 (CNN) 中，感受野是指网络中某个特定层的神经元在输入图像上映射的区域大小（像素范围）。它是 **空间** 维度上的概念。
- 在循环神经网络 (RNN) 中，感受野是 **时间** 维度上的概念。它衡量的是当前状态在时间轴上向后能追溯到多远的输入信息。

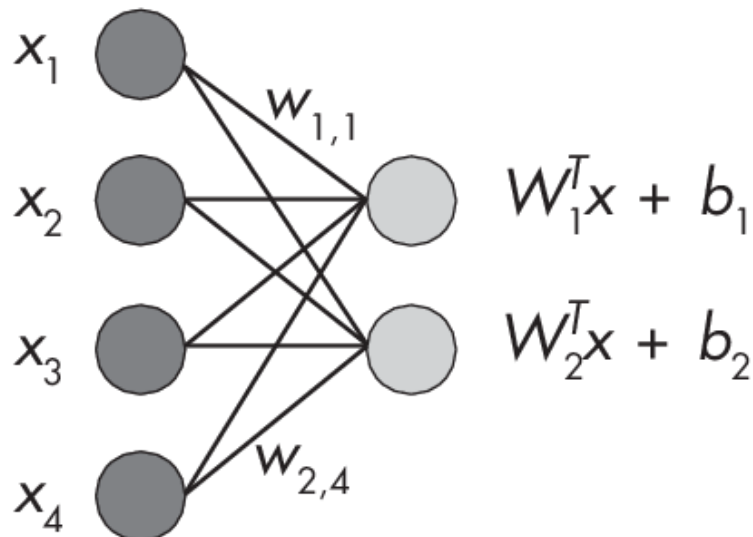


Figure 12.1

The fully connected layer in this figure consists of eight weights and two bias units. We can compute the output nodes via the following dot products:

图中该全连接层有 8 个权重与 2 个偏置，输出由下述点积得到：

Node 1

$$w_{1,1} \times x_1 + w_{1,2} \times x_2 + w_{1,3} \times x_3 + w_{1,4} \times x_4 + b_1$$

Node 2

$$w_{2,1} \times x_1 + w_{2,2} \times x_2 + w_{2,3} \times x_3 + w_{2,4} \times x_4 + b_2$$

The following two sections illustrate scenarios in which convolutional layers can be defined to produce exactly the same computation as the fully connected layer described.

下面两节给出可定义卷积层使其与上述全连接计算完全一致的两种情形。

When the Kernel and Input Sizes Are Equal



核与输入同尺寸时不滑动窗口，逐通道逐元乘再求和即点积。

Let's start with the first scenario, where the size of the convolutional filter is equal to the size of the receptive field. Recall from Chapter [\[ch11\]](#) how we compute a number of parameters in a convolutional kernel with one input channel and multiple output channels. We have a kernel size of 2×2 , one input channel, and two output channels. The input size is also 2×2 , a reshaped version of the four inputs depicted in Figure [12.2](#).

第一种情形：核尺寸等于感受野尺寸。回想第 [\[ch11\]](#) 章中单输入通道、多输出时卷积核参数的计算。这里核为 2×2 ，单输入通道、双输出通道，输入亦为 2×2 ，可由图 [12.1](#) 的四个输入reshape 而成，见图 [12.2](#)。

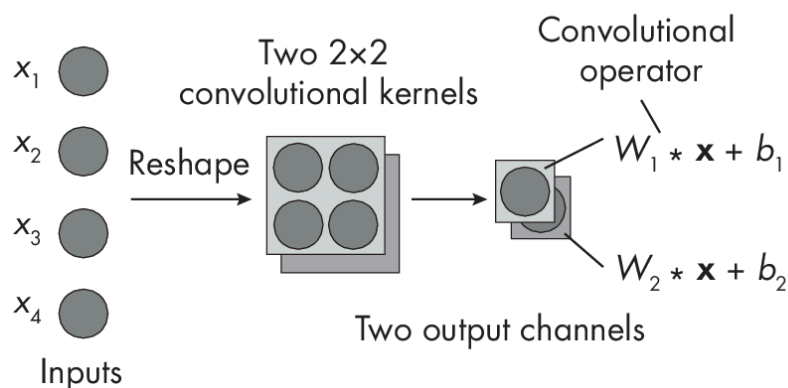


Figure 12.2

If the convolutional kernel dimensions equal the input size, as depicted in Figure [12.2](#), there is no sliding window mechanism in the convolutional layer. For the first output channel, we have the following set of weights:

若如图 [12.2](#) 所示核尺寸与输入相同，则无滑动窗口。第一输出通道权重为：

$$W_1 = \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{1,3} & w_{1,4} \end{bmatrix}$$

For the second output channel, we have the following set of weights:

第二输出通道：

$$W_2 = \begin{bmatrix} w_{2,1} & w_{2,2} \\ w_{2,3} & w_{2,4} \end{bmatrix}$$



If the inputs are organized as

输入排成矩阵

$$x = \begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix}$$

we calculate the first output channel as $o_1 = \sum_i (W_1 \times x_i) + b_1$, where the convolutional operator $*$ is equal to an element-wise multiplication. In other words, we perform an element-wise multiplication between two matrices, W_1 and x , and then compute the output as the sum over these elements; this equals the dot product in the fully connected layer. Lastly, we add the bias unit. The computation for the second output channel works analogously: $o_2 = \sum_i (W_2 \times x_i) + b_2$.

则第一输出通道 $o_1 = \sum_i (W_1 \times x_i) + b_1$ ，此处卷积算子 $*$ 等于逐元相乘后对全体元素求和，即与全连接的向量点积一致，再加偏置；第二输出通道同理： $o_2 = \sum_i (W_2 \times x_i) + b_2$ 。

As a bonus, the supplementary materials for this book include PyTorch code to show this equivalence with a hands-on example in the [supplementary/q12-fc-cnn-equivalence](https://github.com/rasbt/MachineLearning-QandAI-book) subfolder at <https://github.com/rasbt/MachineLearning-QandAI-book>.

图书配套仓库 [supplementary/q12-fc-cnn-equivalence](https://github.com/rasbt/MachineLearning-QandAI-book) 中含 PyTorch 示例演示该等价性：<https://github.com/rasbt/MachineLearning-QandAI-book>。

When the Kernel Size Is 1

把特征维当作 1×1 空间上的“多通道”，用 1×1 卷积实现全连接式混合。

The second scenario assumes that we reshape the input into an input "image" with 1×1 dimensions where the number of "color channels" equals the number of input features, as depicted in Figure [12.3](#).

第二种情形：将输入 reshape 为 1×1 的“图像”，通道数等于特征数（图 [12.3](#)）。

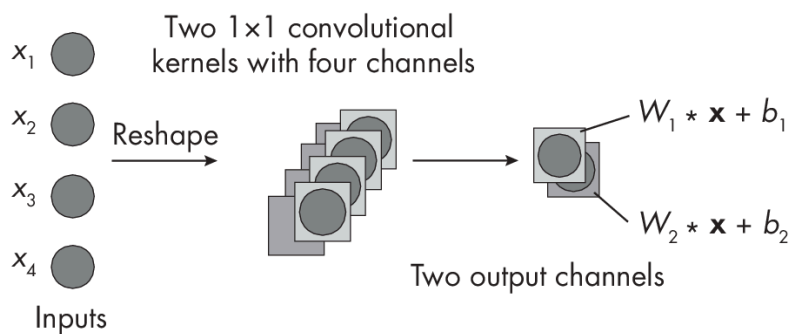


Figure 12.3

Each kernel consists of a stack of weights equal to the number of input channels. For instance, for the first output layer, the weights are

每个核在通道方向上堆叠的权重个数等于输入通道数。例如第一输出：

$$W_1 = \begin{bmatrix} w_1^{(1)} & w_1^{(2)} & w_1^{(3)} & w_1^{(4)} \end{bmatrix}$$

while the weights for the second channel are:

第二输出：

$$W_2 = \begin{bmatrix} w_2^{(1)} & w_2^{(2)} & w_2^{(3)} & w_2^{(4)} \end{bmatrix}$$

To get a better intuitive understanding of this computation, check out the illustrations in Chapter [\[ch11\]](#), which describe how to compute the parameters in a convolutional layer.

更直观理解可参考第 [\[ch11\]](#) 章中对卷积层参数计算的图示说明。

Recommendations

等价性本身在通用 CPU 上不必然提速；但与专用卷积加速器结合时有意

义。

The fact that fully connected layers can be implemented as equivalent convolutional layers does not have immediate performance or other advantages on standard computers. However, replacing fully connected layers with convolutional layers can offer advantages in combination with developing specialized hardware accelerators for convolution operations.



在普通计算机上，把全连接改写成等价卷积并不自动带来性能等好处；但若配合卷积专用硬件加速器，则可能获益。

Moreover, understanding the scenarios where fully connected layers are equivalent to convolutional layers aids in understanding the mechanics of these layers. It also lets us implement convolutional neural networks without any use of fully connected layers, if desired, to simplify code implementations.

此外，弄清等价情形有助于理解两种层的机理；若愿意，亦可实现完全不含全连接的 CNN 以简化代码。

Tips: 进一步，理解 **卷积层** 和 **全连接层** 的等价性，有助于理解这些层的机制。

此外，如果需要，我们可以实现卷积神经网络，而不使用任何全连接层，以简化代码实现。

Exercises

习题：步长增大如何破坏等价；padding 是否影响等价性。

12-1. How would increasing the stride affect the equivalence discussed in this chapter?

12-1. 增大步长会如何影响本章讨论的等价关系？

12-2. Does padding affect the equivalence between fully connected layers and convolutional layers?

12-2. padding 是否影响全连接与卷积层之间的等价性？



Chapter 13: Large Training Sets for Vision Transformers

ViT 相比 CNN 归纳偏置更少，常与大规模预训练结合；本章归纳 CNN 的空间/局部先验并解释 ViT 的数据需求与表现。

Why do vision transformers (ViTs) generally require larger training sets than convolutional neural networks (CNNs)?

为什么视觉 Transformer (ViT) 通常比卷积神经网络 (CNN) 需要更大的训练集？

Each machine learning algorithm and model encodes a particular set of assumptions or prior knowledge, commonly referred to as *inductive biases*, in its design. Some inductive biases are workarounds to make algorithms computationally more feasible, other inductive biases are based on domain knowledge, and some inductive biases are both.

每种机器学习算法与模型都会在设计中编码一组假设或先验知识，常称为归纳偏置 (inductive biases)。其中一些是为了让算法在计算上可行，一些基于领域知识，还有一些两者兼有。

CNNs and ViTs can be used for the same tasks, including image classification, object detection, and image segmentation. CNNs are mainly composed of convolutional layers, while ViTs consist primarily of multi-head attention blocks (discussed in Chapter [\[ch08\]](#) in the context of transformers for natural language inputs).

CNN 与 ViT 可用于相同任务，包括图像分类、目标检测与图像分割。CNN 主要由卷积层构成，ViT 则主要由多头注意力块组成（在自然语言输入上的 Transformer 背景见第 [\[ch08\]](#) 章）。

CNNs have more inductive biases that are hardcoded as part of the algorithmic design, so they generally require less training data than ViTs. In a sense, ViTs are given more degrees of freedom and can or must learn certain inductive biases from the data (assuming that these biases are conducive to optimizing the training objective). However, everything that needs to be learned requires more training examples.



CNN 在算法设计中硬编码了更多归纳偏置，因此通常比 ViT 需要更少训练数据。某种意义上 ViT 自由度更大，能从数据中学习某些归纳偏置（只要这些偏置有利于优化训练目标）；但凡是需要从数据里学的，都需要更多样例。

The following sections explain the main inductive biases encountered in CNNs and how ViTs work well without them.

以下几节说明 CNN 中的主要归纳偏置，以及 ViT 在没有这些偏置时如何仍能良好工作。

Inductive Biases in CNNs

CNN 通过局部连接、权值共享、层次处理等先验获得平移等变/近似不变性；这些是本章与 ViT 对比的核心。

The following are the primary inductive biases that largely define how CNNs function:

下列归纳偏置在很大程度上定义了 CNN 的工作方式：

Local connectivity In CNNs, each unit in a hidden layer is connected to only a subset of neurons in the previous layer. We can justify this restriction by assuming that neighboring pixels are more relevant to each other than pixels that are farther apart. As an intuitive example, consider how this assumption applies to the context of recognizing edges or contours in an image.

局部连接：在 CNN 中，隐层每个单元只与前一层的部分神经元相连。可基于「相邻像素比远处像素更相关」来辩护；直观上这适用于识别图像中的边缘或轮廓。

Weight sharing Via the convolutional layers, we use the same small set of weights (the kernels or filters) throughout the whole image. This reflects the assumption that the same filters are useful for detecting the same patterns in different parts of the image.

权值共享：通过卷积层，整幅图像共用同一小组权重（核或滤波器），体现「同一组滤波器可用于在图像不同位置检测同一种模式」。



Hierarchical processing CNNs consist of multiple convolutional layers to extract features from the input image. As the network progresses from the input to the output layers, low-level features are successively combined to form increasingly complex features, ultimately leading to the recognition of more complex objects and shapes. Furthermore, the convolutional filters in these layers learn to detect specific patterns and features at different levels of abstraction.

层次处理：CNN 用多层卷积从输入图像提取特征；从输入到输出，低层特征逐层组合为更复杂特征，最终识别更复杂的物体与形状；各层滤波器还在不同抽象层次学习检测特定模式与特征。

Spatial invariance CNNs exhibit the mathematical property of spatial invariance, meaning the output of a model remains consistent even if the input signal is shifted to a different location within the spatial domain. This characteristic arises from the combination of local connectivity, weight sharing, and the hierarchical architecture mentioned earlier.

空间不变性：CNN 具有空间不变性这一数学性质，即输入在空间域平移时输出仍可保持一致（在一定意义下）；它来自前述局部连接、权值共享与层次结构的结合。

The combination of local connectivity, weight sharing, and hierarchical processing in a CNN leads to spatial invariance, allowing the model to recognize the same pattern or feature regardless of its location in the input image.

局部连接、权值共享与层次处理共同带来空间不变性，使模型能在输入图像任意位置识别同一模式或特征。

Translation invariance is a specific case of spatial invariance in which the output remains the same after a shift or translation of the input signal in the spatial domain. In this context, the emphasis is solely on moving an object to a different location within an image without any rotations or alterations of its other attributes.

平移不变性是空间不变性的特例：在空间域平移输入后输出不变；此处强调仅改变物体在图像中的位置，而不旋转或改变其他属性。

In reality, convolutional layers and networks are not truly translation-invariant; rather, they achieve a certain level of translation equivariance. What is the difference between translation invariance and equivariance? *Translation*



invariance means that the output does not change with an input shift, while *translation equivariance* implies that the output shifts with the input in a corresponding manner. In other words, if we shift the input object to the right, the results will correspondingly shift to the right, as illustrated in Figure 13.1.

实际上卷积层与网络并非严格平移不变，而是达到一定程度的平移等变（translation equivariance）。区别是：平移不变指输入平移后输出不变；平移等变指输出随输入作对应平移——例如物体右移，响应也右移，见图 13.1。

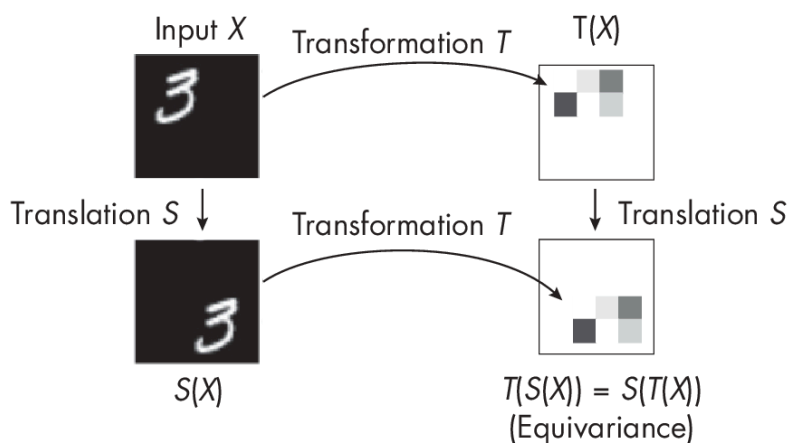


Figure 13.1

As Figure 13.1 shows, under translation invariance, we get the same output pattern regardless of the order in which we apply the operations: transformation followed by translation or translation followed by transformation.

如图 13.1 所示，在平移不变下，先变换再平移与先平移再变换得到相同输出模式。

As mentioned earlier, CNNs achieve translation equivariance through a combination of their local connectivity, weight sharing, and hierarchical processing properties. Figure 13.2 depicts a convolutional operation to illustrate the local connectivity and weight-sharing priors. This figure demonstrates the concept of translation equivariance in CNNs, in which a convolutional filter captures the input signal (the two dark blocks) irrespective of where it is located in the input.

如前所述，CNN 通过局部连接、权值共享与层次处理获得平移等变。图 13.2 用卷积操作示意局部连接与权值共享先验，并表明卷积核可捕捉输入信号（两个深色块）无论其位于输入何处。

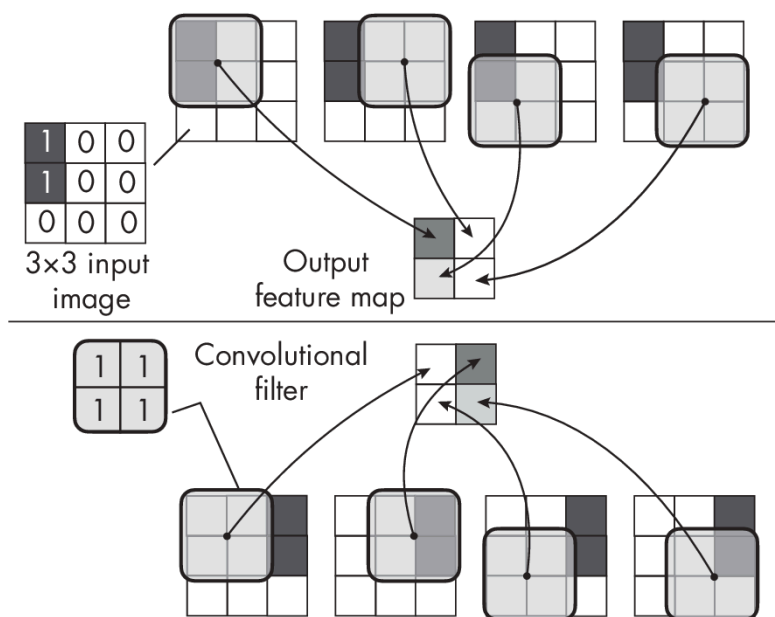


Figure 13.2

Figure 13.2 shows a 3×3 input image that consists of two nonzero pixel values in the upper-left corner (top portion of the figure) or upper-right corner (bottom portion of the figure). If we apply a 2×2 convolutional filter to these two input image scenarios, we can see that the output feature maps contain the same extracted pattern, which is on either the left (top of the figure) or the right (bottom of the figure), demonstrating the translation equivariance of the convolutional operation.

图 13.2 展示 3×3 输入，非零像素在左上（图上方）或右上（图下方）。对这两种情形施加 2×2 卷积核可见输出特征图提取到相同模式，分别位于左或右，体现卷积的平移等变。

For comparison, a fully connected network such as a multilayer perceptron lacks this spatial invariance or equivariance. To illustrate this point, picture a multilayer perceptron with one hidden layer. Each pixel in the input image is connected with each value in the resulting output. If we shift the input by one or more pixels, a different set of weights will be activated, as illustrated in Figure 13.3.

相比之下，多层感知机等全连接网络不具备这种空间不变性或等变性：单层隐层时每个输入像素与每个输出相连，输入平移一格或多格会激活不同权重，见图 13.3。

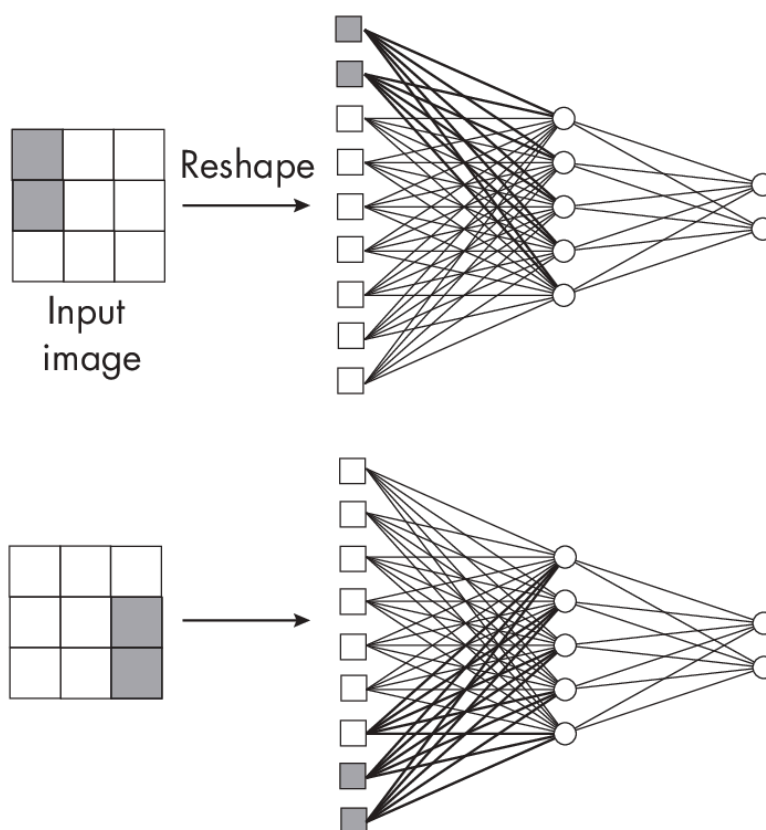


Figure 13.3

Like fully connected networks, ViT architecture (and transformer architecture in general) lacks the inductive bias for spatial invariance or equivariance. For instance, the model produces different outputs if we place the same object in two different spatial locations within an image. This is not ideal, as the semantic meaning of an object (the concept that an object represents or conveys) remains the same based on its location. Consequently, it must learn these invariances directly from the data. To facilitate learning useful patterns present in CNNs requires pretraining over a larger dataset.

与全连接网络类似，ViT（及一般 Transformer）缺乏空间不变或等变的归纳偏置：同一物体放在图像中两处可能得到不同输出，而语义上位置不应改变含义，故这些不变性须从数据中学；要达到 CNN 中那种有用模式往往要在更大数据集上预训练。

A common workaround for adding positional information in ViTs is to use relative positional embeddings (also known as *relative positional encodings*) that consider the relative distance between two tokens in the input sequence. However, while relative embeddings encode information that helps transformers keep track of the relative location of tokens, the transformer still needs to learn from the data whether and how far spatial information is relevant for the task at hand.



在 ViT 中加入位置信息的常用做法是相对位置嵌入（relative positional encodings），刻画两 token 的相对距离；但相对嵌入虽有助于记录相对次序，模型仍须从数据学习空间信息对当前任务是否、以及在多大程度上重要。

ViTs Can Outperform CNNs

参数规模与预训练数据量到位时 ViT 可超越 CNN；视觉领域常用大规模有监督预训练（如 ImageNet）而非仅靠语言模型式的无监督预训练。

The hardcoded assumptions via the inductive biases discussed in previous sections reduce the number of parameters in CNNs substantially compared to fully connected layers. On the other hand, ViTs tend to have larger numbers of parameters than CNNs, which require more training data. (Refer to Chapter [\[ch11\]](#) for a refresher on how to precisely calculate the number of parameters in fully connected and convolutional layers.)

前述硬编码假设使 CNN 相对全连接层参数更少；ViT 参数量往往更大，因而需要更多训练数据（全连接与卷积层参数计数可复习第 [\[ch11\]](#) 章）。

ViTs may underperform compared to popular CNN architectures without extensive retraining, but they can perform very well with a sufficiently large pretraining dataset. In contrast to language transformers, where unsupervised pretraining (such as self-supervised learning, discussed in Chapter [\[ch02\]](#)) is a preferred choice, vision transformers are often pretrained using large, labeled datasets like ImageNet, which provides millions of labeled images for training, and regular supervised learning.

若无大规模预训练，ViT 可能逊于常见 CNN；预训练数据足够大时则可很强。与语言 Transformer 偏好无监督/自监督预训练（见第 [\[ch02\]](#) 章）不同，ViT 常在 ImageNet 等大规模有标注数据上做常规有监督预训练。

An example of ViTs surpassing the predictive performance of CNNs, given enough data, can be observed from initial research on the ViT architecture, as shown in the paper "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale." This study compared ResNet, a type of convolutional network, with the original ViT design using different dataset sizes for pretraining. The findings also



showed that the ViT model excelled over the convolutional approach only after being pretrained on a minimum of 100 million images.

在数据足够时 ViT 超过 CNN 预测性能的例子可见于原始 ViT 论文 "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale.": 该工作比较 ResNet 与 ViT 在不同预训练规模下的表现, 并显示 ViT 仅在对至少约一亿张图像预训练后才明显优于卷积路线。

Inductive Biases in ViTs

ViT 将图像切块 (patchify) 并在块间建立全局关系; 其特征更均匀、偏全局与低频形状, 与 CNN 的纹理/高频偏置形成对照。

ViTs also possess some inductive biases. For example, vision transformers *patchify* the input image to process each input patch individually. Here, each patch can attend to all other patches so that the model learns relationships between far-apart patches in the input image, as illustrated in Figure 13.4.

ViT 也有归纳偏置: 例如将输入图像切块 (patchify) 并对每块单独处理; 每块可关注所有其他块, 从而学习图像中相距较远的块之间的关系, 见图 13.4。

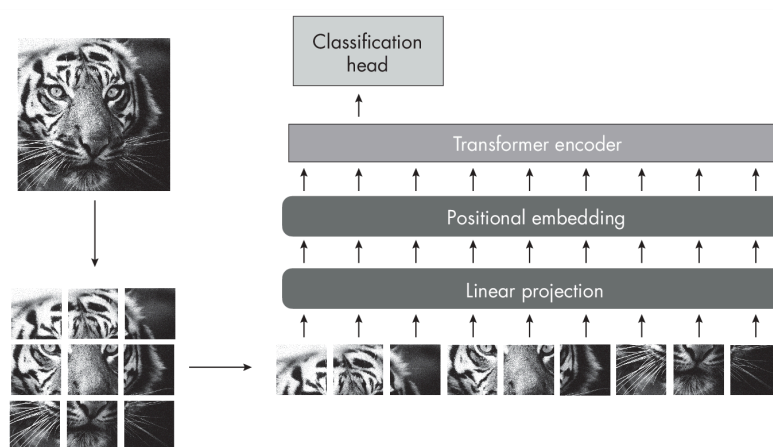


Figure 13.4

The patchify inductive bias allows ViTs to scale to larger image sizes without increasing the number of parameters in the model, which can be computationally expensive. By processing smaller patches individually, ViTs can efficiently capture spatial relationships between image regions while benefiting from the global context captured by the self-attention mechanism.



切块先验使 ViT 可扩展到大分辨率而不同比暴涨参数量（大参数量很贵）；逐块处理结合自注意力的全局上下文，能高效刻画区域间空间关系。

This raises another question: how and what do ViTs learn from the training data? ViTs learn more uniform feature representations across all layers, with self-attention mechanisms enabling early aggregation of global information. In addition, the residual connections in ViTs strongly propagate features from lower to higher layers, in contrast to the more hierarchical structure of CNNs.

进一步：ViT 从训练数据中学到什么？其各层表示更均匀，自注意力使全局信息较早汇聚；残差连接又强有力地自下而上传播特征，与 CNN 更分明的层次结构不同。

ViTs tend to focus more on global than local relationships because their self-attention mechanism allows the model to consider long-range dependencies between different parts of the input image. Consequently, the self-attention layers in ViTs are often considered low-pass filters that focus more on shapes and curvature.

ViT 更偏重全局关系，因自注意力可看输入远距离依赖；因而其自注意层常被视为更偏低频、关注形状与曲率。

In contrast, the convolutional layers in CNNs are often considered high-pass filters that focus more on texture. However, keep in mind that convolutional layers can act as both high-pass and low-pass filters, depending on the learned filters at each layer. High-pass filters detect an image's edges, fine details, and texture, while low-pass filters capture more global, smooth features and shapes. CNNs achieve this by applying convolutional kernels of varying sizes and learning different filters at each layer.

相比之下 CNN 卷积层常被视为更偏高频与纹理；但具体每层可兼具高通与低通，取决于学到的滤波器——高通抓边缘、细节与纹理，低通抓更全局、平滑的形状；CNN 通过不同核尺寸与各层不同滤波器实现。

Recommendations

实务建议：数据与算力足够可首选或混用 ViT；EfficientNetV2 等 CNN 仍省数据省显存；混合卷积与注意力的架构正在收敛「两全」。



ViTs have recently begun outperforming CNNs if enough data is available for pretraining. However, this doesn't make CNNs obsolete, as methods such as the popular EfficientNetV2 CNN architecture are less memory and data hungry.

近来只要预训练数据足够 ViT 已开始超过 CNN，但 CNN 并未过时：例如 EfficientNetV2 等仍更省显存、更省数据。

Moreover, recent ViT architectures don't rely solely on large datasets, parameter numbers, and self-attention. Instead, they have taken inspiration from CNNs and added soft convolutional inductive biases or even complete convolutional layers to get the best of both worlds.

此外最新 ViT 不只堆数据、参数与自注意力，还借鉴 CNN，引入软性卷积式归纳偏置甚至完整卷积层，以兼取两者之长。

In short, vision transformer architectures without convolutional layers generally have fewer spatial and locality inductive biases than convolutional neural networks. Consequently, vision transformers need to learn data-related concepts such as local relationships among pixels. Thus, vision transformers require more training data to achieve good predictive performance and produce acceptable visual representations in generative modeling contexts.

简言之，无卷积层的 ViT 比 CNN 更少空间与局部性先验，须从数据学习像素间局部关系等概念，因而要达到良好预测或在生成建模中得到可接受的视觉表示通常需要更多训练数据。

Exercises

习题：块大小与计算量及精度的权衡。

13-1. Consider the patchification of the input images shown in Figure 13.4. The size of the resulting patches controls a computational and predictive performance trade-off. The optimal patch size depends on the application and desired trade-off between computational cost and model performance. Do smaller patches typically result in higher or lower computational costs?

习题 13-1：考虑图 13.4 的切块：块尺寸控制计算与预测性能的权衡，最优块尺寸依应用与成本—性能取舍而定。更小的块通常带来更高还是更低的计算成



本？

13-2. Following up on the previous question, do smaller patches typically lead to a higher or lower prediction accuracy?

习题 13-2：接续上一问：更小的块通常带来更高还是更低的预测准确率？

References

本节列出 ViT 原点论文、相对位置编码、ViT 与 CNN 表征对比、EfficientNetV2，以及融入卷积的 ViT（ConViT、CvT）等文献链接。

- The paper proposing the original vision transformer model: Alexey Dosovitskiy et al., "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale"? (2020), <https://arxiv.org/abs/2010.11929>.

Dosovitskiy 等提出原始 ViT 的论文（2020），链接见上。

- A workaround for adding positional information in ViTs is to use relative positional embeddings: Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani, "Self-Attention with Relative Position Representations"? (2018), <https://arxiv.org/abs/1803.02155>.

Shaw、Uszkoreit、Vaswani 关于自注意与相对位置表示的论文（2018）。

- Residual connections in ViTs strongly propagate features from lower to higher layers, in contrast to the more hierarchical structure of CNNs: Maithra Raghu et al., "Do Vision Transformers See Like Convolutional Neural Networks?" (2021), <https://arxiv.org/abs/2108.08810>.

Raghu 等讨论 ViT 残差连接自下而上强传播特征、与 CNN 层次性的对比（2021）。

- A detailed research article covering the EfficientNetV2 CNN architecture: Mingxing Tan and Quoc V. Le, "Smaller Models and Faster Training"? (2021), <https://arxiv.org/abs/2104.00298>.

Tan、Le 关于 EfficientNetV2 CNN 的论文（2021）。



- A ViT architecture that also incorporates convolutional layers: StÃ©phane d'Ascoli et al., "ConViT: Improving Vision Transformers with Soft Convolutional Inductive Biases"? (2021), <<https://arxiv.org/abs/2103.10697>>.

d'Ascoli 等 ConViT：以软性卷积归纳偏置改进 ViT（2021）。

- Another example of a ViT using convolutional layers: Haiping Wu et al., "CvT: Introducing Convolutions to Vision Transformers"? (2021), <https://arxiv.org/abs/2103.15808>.

Wu 等 CvT：将卷积引入 ViT（2021）。



Chapter 14: The Distributional Hypothesis

分布假设：共现上下文相似的词往往语义相近；本章说明其在 Word2vec、BERT、GPT 中的角色与局限。

What is the **distributional hypothesis** in natural language processing (NLP)? Where is it used, and how far does it hold true?

自然语言处理（NLP）中的「分布假设」是什么？用在哪里？在多大程度上成立？

The **distributional hypothesis** is a linguistic theory suggesting that words occurring in the same contexts tend to have similar meanings, according to the original source, "Distributional Structure" by Zellig S. Harris. Succinctly, the more similar the meanings of two words are, the more often they appear in similar contexts.

分布假设是语言学理论：同一上下文里出现的词往往含义相近，可追溯至 Zellig S. Harris 的 "Distributional Structure"。简言之，两词含义越接近，越常出现在相似上下文中。

Tips: 分布假设 (distributional hypothesis)，也称为分布语义学 (distributional semantics)，用于描述单词在上下文中的分布模式。它认为，在 **相似的上下文** 中出现的 **单词** 往往具有 **相似的含义**。

Consider the sentence in Figure [14.1](#), for example. The words *cats* and *dogs* often occur in similar contexts, and we could replace *cats* with *dogs* without making the sentence sound awkward. We could also replace *cats* with *hamsters*, since both are mammals and pets, and the sentence would still sound plausible. However, replacing *cats* with an unrelated word such as *sandwiches* would render the sentence clearly wrong, and replacing *cats* with the unrelated word *driving* would also make the sentence grammatically incorrect.

例如图 [14.1](#) 中的句子：*cats* 与 *dogs* 常出现在相似上下文，互换后仍自然；换作 *hamsters* 也可行；换成 *sandwiches* 则语义荒谬，换成 *driving* 则语法不通。



Tips: 图1.1中的句子。*cats*和*dogs*经常出现在相似的上下文中，我们可以将*cats*替换为*dogs*，而不会让句子听起来奇怪。

- 我们也可以将*cats*替换为*hamsters*，因为它们都是哺乳动物和宠物，句子听起来仍然合理。
- 但是，如果将*cats*替换为不相关的单词*sandwiches*，句子会变得明显错误，
- 如果将*cats*替换为不相关的单词*driving*，句子也会变得语法错误。

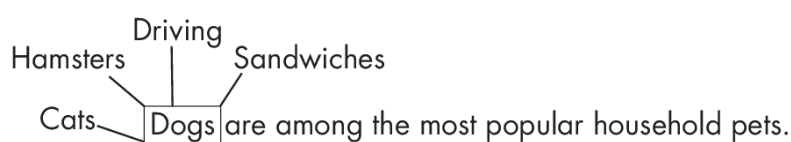


Figure 14.1

It is easy to construct counterexamples using polysemous words, that is, words that have multiple meanings that are related but not identical. For example, consider the word *bank*. As a noun, it can refer to a financial institution, the "rising ground bordering a river," the "steep incline of a hill," or a "protective cushioning rim" (according to the Merriam-Webster dictionary). It can even be a verb: to bank on something means to rely or depend on it. These different meanings have different distributional properties and may not always occur in similar contexts.

用多义词很容易构造反例：即多种相关但不完全相同的含义。例如 *bank* 可作名词指金融机构、河岸、陡坡或边缘护垫（据 Merriam-Webster），也可作动词表示指望、依赖；不同义项的分布性质不同，未必总出现在相似上下文中。

Nonetheless, the distributional hypothesis is quite useful. Word embeddings (introduced in Chapter [ch01]) such as Word2vec, as well as many large language transformer models, rely on this idea. This includes the masked language model in BERT and the next-word pretraining task used in GPT.

尽管如此分布假设仍很有用：第 [ch01] 章介绍的 Word2vec 等词嵌入，以及许多大型语言 Transformer 都依赖这一思想，包括 BERT 的掩码语言建模与 GPT 的下一词预训练任务。

Tips: 尽管存在反例，分布假设在实际应用中非常有用。

- **Word2vec** 等词嵌入（word embeddings）模型以及许多 **大型语言模型**（large language models）都基于这个概念。
- 这包括 **BERT** 中的 **掩码** 语言模型，和 **GPT** 中的 **下一个词** 预训练任务。



Word2vec, BERT, and GPT

Word2vec、BERT、GPT 三种建模范式如何用自监督目标把「共现」变成嵌入空间中的近邻关系。

The **Word2vec** approach uses a simple, two-layer neural network to encode words into embedding vectors such that the embedding vectors of similar words are both semantically and syntactically close. There are two ways to train a Word2vec model: the **continuous bag-of-words** (CBOW) approach and the **skip-gram** approach. When using CBOW, the Word2vec model learns to predict the current words by using the surrounding context words. Conversely, in the skip-gram model, Word2vec predicts the context words from a selected word. While skip-gram is more effective for infrequent words, CBOW is usually faster to train.

Word2vec 用浅层网络把词编码为嵌入，使相近词在向量空间中邻近；训练方式有 CBOW（用上下文预测中心词）与 skip-gram（由中心词预测上下文）；后者对罕见词往往更好，前者通常训练更快。

Tips: FIXME 跳字模型 没理解???

- **Word2vec** 是一种使用简单两层神经网络将单词编码为嵌入向量的方法。
- 有两种训练 Word2vec 模型的方法：**连续词袋**（CBOW）方法和 **跳字**（skip-gram）方法。
- 当使用 CBOW 时，Word2vec 模型学习通过使用周围上下文单词来预测当前单词。
- 相反，在跳字模型中，Word2vec 从选定的单词预测上下文单词。
- 尽管跳字模型对于不常见的单词更有效，但 CBOW 通常训练速度更快。

After training, word embeddings are placed within the vector space so that words with common contexts in the corpus--that is, words with semantic and syntactic



similarities--are positioned close to each other, as illustrated in Figure 14.2. Conversely, dissimilar words are located farther apart in the embedding space.

训练完成后，语料中共现多、语义与句法相近的词在向量空间中彼此靠近，反之则更远，见图 14.2。

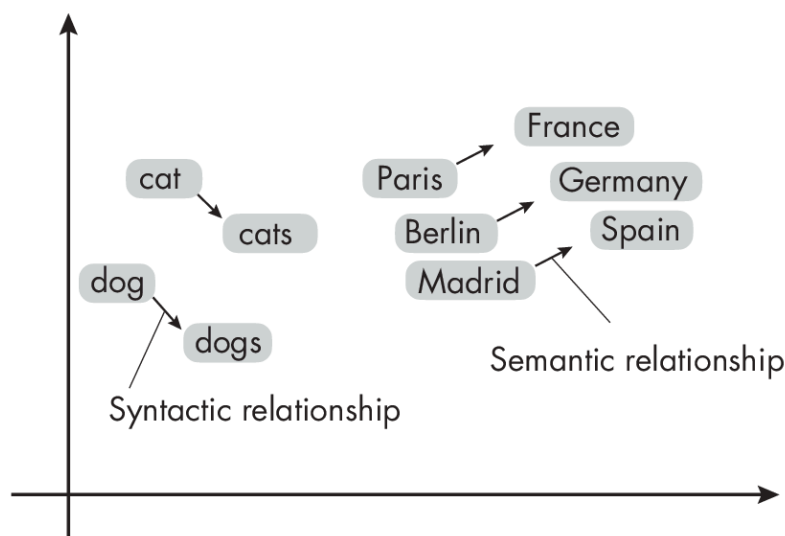


Figure 14.2

BERT is an LLM based on the transformer architecture (see Chapter [ch08]) that uses a masked language modeling approach that involves masking (hiding) some of the words in a sentence. Its task is to predict these masked words based on the other words in the sequence, as illustrated in Figure 14.3. This is a form of the self-supervised learning used to pretrain LLMs (see Chapter [ch02] for more on self-supervised learning). The pretrained model produces embeddings in which similar words (or tokens) are close in the embedding space.

BERT 是基于 Transformer（第 [ch08] 章）的大语言模型，用掩码语言建模：遮住句中部分词并据其余词预测被掩词，见图 14.3。这是预训练 LLM 常用的自监督形式（详见第 [ch02] 章）。预训练后相似词或 token 在嵌入空间中彼此接近。

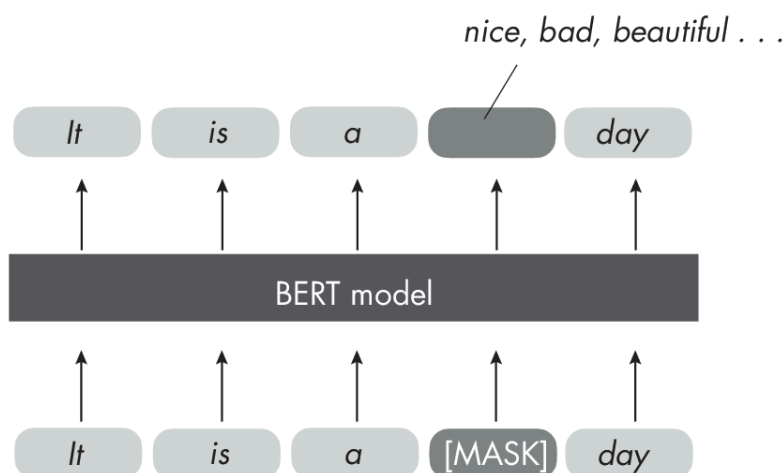


Figure 14.3

GPT, which like BERT is also an LLM based on the transformer architecture, functions as a decoder. Decoder-style models like GPT learn to predict subsequent words in a sequence based on the preceding ones, as illustrated in Figure 14.4. GPT contrasts with BERT, an encoder model, as it emphasizes predicting what follows rather than encoding the entire sequence simultaneously.

GPT 与 BERT 同为基于 Transformer 的 LLM，但作解码器用：类 GPT 的解码器风格模型根据前文预测后续词，见图 14.4。与作为编码器的 BERT 不同，GPT 强调「往后预测」而非同时对整句双向编码。

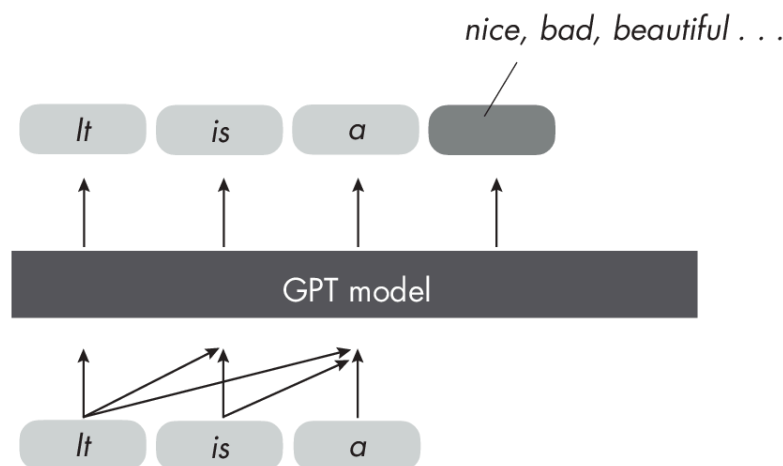


Figure 14.4

Where BERT is a bidirectional language model that considers the whole input sequence, GPT only strictly parses previous sequence elements. This means BERT is usually better suited for classification tasks, whereas GPT is more suited for text generation tasks. Similar to BERT, GPT produces high-quality contextualized word embeddings that capture semantic similarity.



BERT 双向建模整段输入；GPT 严格依赖已生成的前缀，故前者更宜分类等任务，后者更宜生成；二者都能得到捕捉语义相似性的高质量上下文化嵌入。

Tips:

- BERT 是一种 双向 语言模型，考虑 整个输入序列 。
- GPT 只严格解析 前一个序列元素 。
- 这意味着 BERT 通常更适合分类任务，而 GPT 更适合文本生成任务。
- 与 BERT 类似， GPT 产生高质量的 上下文化单词嵌入 ，捕捉语义相似性。

Does the Hypothesis Hold?

大数据上分布假设大体可用，支撑嵌入与语义技术及下游 NLP；反例存在但不妨碍其作为现代语言模型的基石。

For large datasets, the distributional hypothesis more or less holds true, making it quite useful for understanding and modeling language patterns, word relationships, and semantic meanings. For example, this concept enables techniques like word embedding and semantic analysis, which, in turn, facilitate natural language processing tasks such as text classification, sentiment analysis, and machine translation.

对大规模数据，分布假设大体成立，有助于建模语言模式、词关系与语义，并支撑词嵌入、语义分析及分类、情感分析、机器翻译等任务。

Tips:

- 对于大型数据集， 分布假设 或多或少是正确的，对于理解语言模式、单词关系和语义含义非常有用。
- 例如，这个概念启用了像 词嵌入 和 语义分析 这样的技术，这些技术反过来又促进了自然语言处理任务，如 文本分类 、 情感分析 和 机器翻译 。

In conclusion, while there are counterexamples in which the distributional hypothesis does not hold, it is a very useful concept that forms the cornerstone of



modern language transformer models.

总之，尽管存在分布假设不成立的反例，它仍是极有用的概念，构成现代语言 Transformer 模型的基石之一。

Exercises

习题：同音异义词与「类似分布假设」的其他模态。

14-1. Does the distributional hypothesis hold true in the case of homophones, or words that sound the same but have different meanings, such as *there* and *their*?

习题 14-1：对同音异义词（如 *there* 与 *their*），分布假设是否仍成立？

14-2. Can you think of another domain where a concept similar to the distributional hypothesis applies? (Hint: think of other input modalities for neural networks.)

习题 14-2：能否想到另一个领域也存在与分布假设类似的概念？（提示：神经网络的其他输入模态。）

References

本节列分布假设原文、Word2vec、BERT、GPT、上下文表示可迁移性与语言模型即知识库等相关文献链接。

- The original source describing the distributional hypothesis: Zellig S. Harris, "Distributional Structure"? (1954), <https://doi.org/10.1080/00437956.1954.11659520>.

Harris 阐述分布假设的经典文献（1954），DOI 链接见上。

- The paper introducing the Word2vec model: Tomas Mikolov et al., "Efficient Estimation of Word Representations in Vector Space"? (2013), <https://arxiv.org/abs/1301.3781>.

Mikolov 等提出 Word2vec 的论文（2013）。



- The paper introducing the BERT model: Jacob Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"? (2018), <https://arxiv.org/abs/1810.04805>.

Devlin 等 BERT 论文（2018）。

- The paper introducing the GPT model: Alec Radford and Karthik Narasimhan, "Improving Language Understanding by Generative Pre-Training"? (2018), <https://www.semanticscholar.org/paper/Improving-Language-Understanding-by-Generative-Radford-Narasimhan/cd18800a0fe0b668a1cc19f2ec95b5003d0a5035>.

Radford、Narasimhan 关于生成式预训练改进语言理解的论文（2018）。

- BERT produces embeddings in which similar words (or tokens) are close in the embedding space: Nelson F. Liu et al., "Linguistic Knowledge and Transferability of Contextual Representations"? (2019), <https://arxiv.org/abs/1903.08855>.

Liu 等讨论语言学知识与上下文表示可迁移性（2019）。

- The paper showing that GPT produces high-quality contextualized word embeddings that capture semantic similarity: Fabio Petroni et al., "Language Models as Knowledge Bases"? (2019), <https://arxiv.org/abs/1909.01066>.

Petroni 等讨论语言模型作为知识库（2019）。



Chapter 15: Data Augmentation for Text

文本侧人为扩充训练数据：同义替换、删词、换序、句序打乱、噪声、回译与 LLM 合成等；各法适用场景与风险。

How is data augmentation useful, and what are the most common augmentation techniques for text data?

数据增强有何用处？文本数据最常见增强手法有哪些？

Data augmentation is useful for artificially increasing dataset sizes to improve model performance, such as by reducing the degree of overfitting, as discussed in Chapter [\[ch05\]](#). This includes techniques often used in computer vision models, like rotation, scaling, and flipping.

数据增强通过人为扩增样本规模来改善效果（例如减轻过拟合，见第 [\[ch05\]](#) 章），计算机视觉里常用旋转、缩放、翻转等。

Tips: 数据增强（**Data Augmentation**）是一种通过增加训练数据量，来提高模型泛化能力的技术。

数据增强可以分为两类：

1. 基于**规则**的数据增强：通过规则生成新的数据，如同义词替换、单词删除、单词位置交换、句子打乱、噪声注入等。
2. 基于**模型**的数据增强：通过模型生成新的数据，如 **GAN**、**VAE** 等。

数据增强通常包括以下几种技术：

1. 同义词替换（**Synonym Replacement**）：依赖同义词典，替换同义词；建议设置替换频率和相似度阈值。
2. 单词删除（**Word Deletion**）：随机删除句子中的某些单词；建议设置删除率。
3. 单词位置交换（**Word Position Swapping**）：随机交换句子中某些单词的位置；建议设置交换率。
4. 句子打乱（**Sentence Shuffling**）：随机打乱句子中某些句子的顺序；建议设置打乱率。

5. 噪声注入 (**Noise Injection**)：在句子中添加一些随机噪声；建议设置噪声率。
6. 回译 (**Back Translation**)：将句子翻译成另一种语言，再翻译回来；建议设置翻译模型。
7. 合成数据 (**Synthetic Data**)：使用LLM生成新的数据；建议设置生成模型。



Similarly, there are several techniques for augmenting text data. The most common include synonym replacement, word deletion, word position swapping, sentence shuffling, noise injection, back translation, and text generated by LLMs. This chapter discusses each of these, with optional code examples in the *supplementary/q15-text-augment* subfolder at

<https://github.com/rasbt/MachineLearning-QandAI-book>.

类似地，文本增强常见手段包括同义替换、删词、词位交换、句序打乱、噪声、回译以及由 LLM 生成文本等；本章逐节讨论，示例代码见仓库 *supplementary/q15-text-augment* 子目录

<https://github.com/rasbt/MachineLearning-QandAI-book>。

Synonym Replacement

随机将句中词换为同义表达以扩大表述多样性；依赖词典且需注意语境可交换性。

In *synonym replacement*, we randomly choose words in a sentence -- often nouns, verbs, adjectives, and adverbs -- and replace them with synonyms. For example, we might begin with the sentence "The cat quickly jumped over the lazy dog," and then augment the sentence as follows: "The cat rapidly jumped over the idle dog."?

同义替换常在句中随机挑名词、动词、形容词、副词等换为同义词；例如由 "The cat quickly jumped over the lazy dog," 得到 "The cat rapidly jumped over the idle dog."

Synonym replacement can help the model learn that different words can have similar meanings, thereby improving its ability to understand and generate text. In practice, synonym replacement often relies on a thesaurus such as WordNet.



However, using this technique requires care, as not all synonyms are interchangeable in all contexts. Most automatic text replacement tools have settings for adjusting replacement frequency and similarity thresholds. However, automatic synonym replacement is not perfect, and you might want to apply post-processing checks to filter out replacements that might not make sense.

同义替换有助于模型学到不同词可表相似含义；实践上常依赖 WordNet 等词典，但并非所有同义词在各语境可互换，工具多有频率与相似度阈值；自动替换不完美，宜事后过滤不合理替换。

Tips: 自动 **同义词替换** 的结果并不准确，建议筛除不合适的替换结果。

Word Deletion

随机删词迫使模型在信息缺失下仍能预测，须避免删掉改变句意的关键词并控制删除率。

Word deletion is another data augmentation technique to help models learn. Unlike synonym replacement, which alters the text by substituting words with their synonyms, word deletion involves removing certain words from the text to create new variants while trying to maintain the overall meaning of the sentence. For example, we might begin with the sentence "The cat quickly jumped over the lazy dog"? and then remove the word *quickly*: "The cat jumped over the lazy dog."?

删词是另一增强手段：不删义换词而是去掉部分词以生成变体并尽量保义；例如删去 *quickly* 得到 "The cat jumped over the lazy dog."

By randomly deleting words in the training data, we teach the model to make accurate predictions even when some information is missing. This can make the model more robust when encountering incomplete or noisy data in real-world scenarios. Also, by deleting nonessential words, we may teach the model to focus on key aspects of the text that are most relevant to the task at hand.

训练时随机删词可让模型在部分信息缺失时仍能预测，提升对不完整或噪声输入的鲁棒性；删去非关键信息也有助于模型聚焦与任务最相关的部分。



However, we must be careful not to remove critical words that may significantly alter a sentence's meaning. For example, it would be suboptimal to remove the word *cat* in the previous sentence: "The quickly jumped over the lazy dog."? We must also choose the deletion rate carefully to ensure that the text still makes sense after words have been removed. Typical deletion rates might range from 10 percent to 20 percent, but this is a general guideline and could vary significantly based on the specific use case.

须避免删掉彻底改变句意的词；误删 *cat* 会得到不合句。删除率须审慎，常见约 10%–20% 仅作参考，应随任务调整。

Tips: 单词删除，有时候会删除掉一些重要的词语，导致句子不通顺，建议设置删除率。一般删除率在 10% 到 20% 之间，但具体需要根据具体任务调整。

Word Position Swapping

交换词序以弱化模型对固定语序依赖；过度打乱可破坏语法或语义。

In *word position swapping*, also known as *word shuffling* or *permutation*, the positions of words in a sentence are swapped or rearranged to create new versions of the sentence. If we begin with "The cat quickly jumped over the lazy dog,"? we might swap the positions of some words to get the following: "Quickly the cat jumped the over lazy dog."?

词位交换（词打乱/排列）通过调换词序生成新句；例如上句可变成 "Quickly the cat jumped the over lazy dog." 这类英语上可能不自然，但对增强仍可提供词与词关联的信号。

While these sentences may sound grammatically incorrect or strange in English, they provide valuable training information for data augmentation because the model can still recognize the important words and their associations with each other. However, this method has its limitations. For example, shuffling words too much or in certain ways can drastically change the meaning of a sentence or make it completely nonsensical. Moreover, word shuffling may interfere with the model's learning process, as the positional relationships between certain words can be vital in these contexts.



但过度或不当打乱会剧烈改义甚至成胡言；某些任务中词序至关重要，打乱会干扰学习。

Tips: 词序打乱，有助于模型关注词语本身及其关系，而非固定语序。但过度打乱可能导致语义丢失或句子无意义。某些任务中，词语的顺序很重要，随意打乱会影响模型学习效果。

Sentence Shuffling

在段落或文档内重排句子以学主题级结构；不适用于强时间或逻辑链文本。

In *sentence shuffling*, entire sentences within a paragraph or a document are rearranged to create new versions of the input text. By shuffling sentences within a document, we expose the model to different arrangements of the same content, helping it learn to recognize thematic elements and key concepts rather than relying on specific sentence order. This promotes a more robust understanding of the document's overall topic or category. Consequently, this technique is particularly useful for tasks that deal with document-level analysis or paragraph-level understanding, such as document classification, topic modeling, or text summarization.

句序打乱在段落或文档内重排句子，让模型见多种排列以学主题与关键概念而非死记句序，利于文档分类、主题建模、摘要等文档级或段落级任务。

In contrast to the aforementioned word-based methods (word position swapping, word deletion, and synonym replacement), sentence shuffling maintains the internal structure of individual sentences. This avoids the problem of altering word choice or order such that sentences become grammatically incorrect or change meaning entirely.

与词级方法不同，句内词序与用词不变，可避免单句被搅到语法错误或整句改义。

Sentence shuffling is useful when the order of sentences is not crucial to the overall meaning of the text. Still, it may not work well if the sentences are logically or chronologically connected. For example, consider the following paragraph: "I went to the supermarket. Then I bought ingredients to make pizza. Afterward, I



made some delicious pizza."? Reshuffling these sentences as follows disrupts the logical and temporal progression of the narrative: "Afterward, I made some delicious pizza. Then I bought ingredients to make pizza. I went to the supermarket."?

当句序对整体含义不必需时句序打乱有用；若句间有强逻辑或时间顺序则不宜，例如先购物、再买料、再做披萨的叙述被打乱会破坏叙事。

Noise Injection

字符级扰动模拟错别字与拼写问题，利于纠错类任务，亦增鲁棒性。

Noise injection is an umbrella term for techniques used to alter text in various ways and create variation in the texts. It may refer either to the methods described in the previous sections or to character-level techniques such as inserting random letters, characters, or typos, as shown in the following examples:

噪声注入统称多种扰动文本造变体的技术，可指前文方法，也可指字符级插入、删除或引入错别字等，例如：

Random character insertion "The cat qzuickly jumped over the lazy dog."?
(Inserted a *z* in the word *quickly*.)

Random character deletion "The cat quickl jumped over the lazy dog."? (Deleted *y* from the word *quickly*.)

Typo introduction "The cat qickuly jumped over the lazy dog."? (Introduced a typo in *quickly*, changing it to *qickuly*.)

随机插入字符："The cat qzuickly jumped over the lazy dog."（在 *quickly* 中插入 *z*）。随机删字母："The cat quickl jumped over the lazy dog."（删 *quickly* 中的 *y*）。引入笔误："The cat qickuly jumped over the lazy dog."

These modifications are beneficial for tasks that involve spell-checking and text correction, but they can also help make the model more robust to imperfect inputs.

这类修改利于拼写检查与纠错任务，也可增强对不完美输入的鲁棒性。

Tips: 随机字符插入、删除、错别字，有助于模型学习拼写和语法错误，但过度使用可能导致模型过拟合。



Back Translation

多语往返翻译产生表述变体；效果依赖语言对与机器翻译系统，需核验语义保持。

Back translation is one of the most widely used techniques to create variation in texts. Here, a sentence is first translated from the original language into one or more different languages, and then it is translated back into the original language. Translating back and forth often results in sentences that are semantically similar to the original sentence but have slight variations in structure, vocabulary, or grammar. This generates additional, diverse examples for training without altering the overall meaning.

回译先将句子译出外语再译回，往返往往得到语义相近但结构、词汇或语法略异的句子，在不改大意为训练集增加多样性。

For example, say we translate "The cat quickly jumped over the lazy dog"? into German. We might get "Die Katze sprang schnell Ã¼ber den faulen Hund."? We could then translate this German sentence back into English to get "The cat jumped quickly over the lazy dog."?

例如英译德再英: "The cat quickly jumped over the lazy dog." → "Die Katze sprang schnell Ã¼ber den faulen Hund." → 再英译得 "The cat jumped quickly over the lazy dog."

The degree to which a sentence changes through backtranslation depends on the languages used and the specifics of the machine translation model. In this example, the sentence remains very similar. However, in other cases or with other languages, you might see more significant changes in wording or sentence structure while maintaining the same overall meaning.

变化幅度取决于语言对与翻译模型；本例非常接近，其他情形可能出现更大措辞或结构差异而仍保大意。



This method requires access to reliable machine translation models or services, and care must be taken to ensure that the back-translated sentences retain the essential meaning of the original sentences.

需要可靠机器翻译并检查回译句是否保留原意。

Tips: 回译，通过将句子翻译成另一种语言，再翻译回来，可以生成新的数据。但回译的结果有时并不准确，需要筛选不合适的回译结果。

Synthetic Data

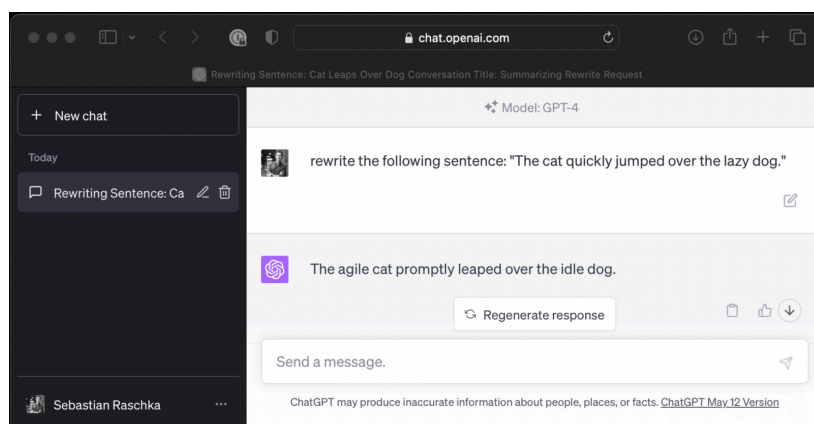
合成数据含本章诸法的总称，亦可由解码式 LLM 直接生成或改写句子。

Synthetic data generation is an umbrella term that describes methods and techniques used to create artificial data that mimics or replicates the structure of real-world data. All methods discussed in this chapter can be considered synthetic data generation techniques since they generate new data by making small changes to existing data, thus maintaining the overall meaning while creating something new.

合成数据生成泛指仿真实数据分布造样本；本章诸法都可视为在保留大意前提下对原句做小改动。

Modern techniques to generate synthetic data now also include using decoder-style LLMs such as GPT (decoder-style LLMs are discussed in more detail in Chapter [\[ch17\]](#)). We can use these models to generate new data from scratch by using "complete the sentence"? or "generate example sentences"? prompts, among others. We can also use LLMs as alternatives to back translation, prompting them to rewrite sentences as shown in Figure [1.1](#).

现代做法还包括用 GPT 等解码式 LLM（第 [\[ch17\]](#) 章）从零生成或用「续写」「生成例句」等提示；也可替代回译，按图 [1.1](#) 所示提示改写句子。



Note that an LLM, as shown in Figure 1.1, runs in a nondeterministic mode by default, which means we can prompt it multiple times to obtain a variety of rewritten sentences.

如图 1.1，LLM 默认非确定性，可多次采样提示以获得多种改写。

Recommendations

分类与情感等标注稀缺任务常用本章手法；预训练 LLM 数据已极多，增强在微调阶段更有意义。

The data augmentation techniques discussed in this chapter are commonly used in text classification, sentiment analysis, and other NLP tasks where the amount of available labeled data might be limited.

本章技术在文本分类、情感分析等标注数据有限任务中常见。

LLMs are usually pretrained on such a vast and diverse dataset that they may not rely on these augmentation techniques as extensively as in other, more specific NLP tasks. This is because LLMs aim to capture the statistical properties of the language, and the vast amount of data on which they are trained often provides a sufficient variety of contexts and expressions. However, in the fine-tuning stages of LLMs, where a pretrained model is adapted to a specific task with a smaller, task-specific dataset, data augmentation techniques might become more relevant again, mainly if the task-specific labeled dataset size is limited.

LLM 预训练语料极大，未必像专用 NLP 那样依赖增强；预训练已覆盖丰富语境与表达。但在任务数据较小的微调阶段，若标注有限，增强又可变重要。

Tips: **数据增强** 在LLM的 **预训练** 阶段可能 **不太有用**，因为LLM已经在大规模数据上预训练过了。但在 **微调阶段**，数据增强可能 **更有用**，特别是当任务特定的标注数据集较小时。



Exercises

习题：隐私与「增强反而不利」的情形。

15-1. Can the use of text data augmentation help with privacy concerns?

习题 15-1：文本数据增强能否缓解隐私方面的顾虑？

15-2. What are some instances where data augmentation may not be beneficial for a specific task?

习题 15-2：哪些情况下对特定任务做数据增强反而不利？

References

本节列 WordNet 词条库论文链接。

- The WordNet thesaurus: George A. Miller, "WordNet: A Lexical Database for English"? (1995), <https://dl.acm.org/doi/10.1145/219717.219748>.

Miller 关于 WordNet 英文词汇数据库的论文（1995）。



Chapter 16: Self-Attention

从 RNN 的 Bahdanau 跨序列注意力到同一序列上的 self-attention；弄清命名由来与和早期注意力的区别。

Where does **self-attention** get its name, and how is it different from previously developed attention mechanisms?

「自注意力」因何得名？与更早的注意力机制有何不同？

Self-attention enables a neural network to refer to other portions of the input while focusing on a particular segment, essentially allowing each part the ability to "attend" to the whole input. The original attention mechanism developed for **recurrent neural networks** (RNNs) is applied between two different sequences: the encoder and the decoder embeddings. Since the attention mechanisms used in transformer-based large language models is designed to work on all elements of the same set, it is known as *self-attention*.

自注意力让网络在关注一段输入时仍能参照其余部分，使每段都能「注意」全体输入。为 RNN 设计的原始注意力作用于两路序列：编码器与解码器嵌入。而基于 Transformer 的大语言模型所用的注意作用在同一集合的全部元素上，故称「自」注意。

Tips: **self-attention** 允许神经网络在关注特定部分时，同时参考其他部分。每个部分都能“关注”整个输入。

This chapter first discusses an earlier attention mechanism developed for RNNs, the **Bahdanau** mechanism, in order to illustrate the motivation behind developing attention mechanism. We then compare the Bahdanau mechanism to the self-attention mechanism prevalent in transformer architectures today.

本章先讨论 RNN 的 Bahdanau 注意力以说明注意力的动机，再与当今 Transformer 中主流的 self-attention 对比。

Attention in RNNs

Bahdanau 注意力：编码器—解码器两路 RNN，用动态权重把全体输入加权送入解码逐步生成；缓解单隐状态瓶颈。

One example of an attention mechanism used in **RNNs** to handle long sequences is *Bahdanau attention*. Bahdanau attention was developed to make machine learning models, particularly those used in translating languages, better at understanding long sentences. Before this type of attention, the whole input (such as a sentence in English) was squashed into a single chunk of information, and important details could get lost, especially if the sentence was long.

RNN 处理长序列的经典注意是 Bahdanau 注意力，旨在改善翻译等模型对长句的理解。此前整句常被压成单一信息，细节易丢失。

To understand the difference between regular attention and self-attention, let's begin with the illustration of the Bahdanau attention mechanism in Figure 16.1.

为理解普通注意力与自注意力的区别，先看图 16.1 的 Bahdanau 示意。

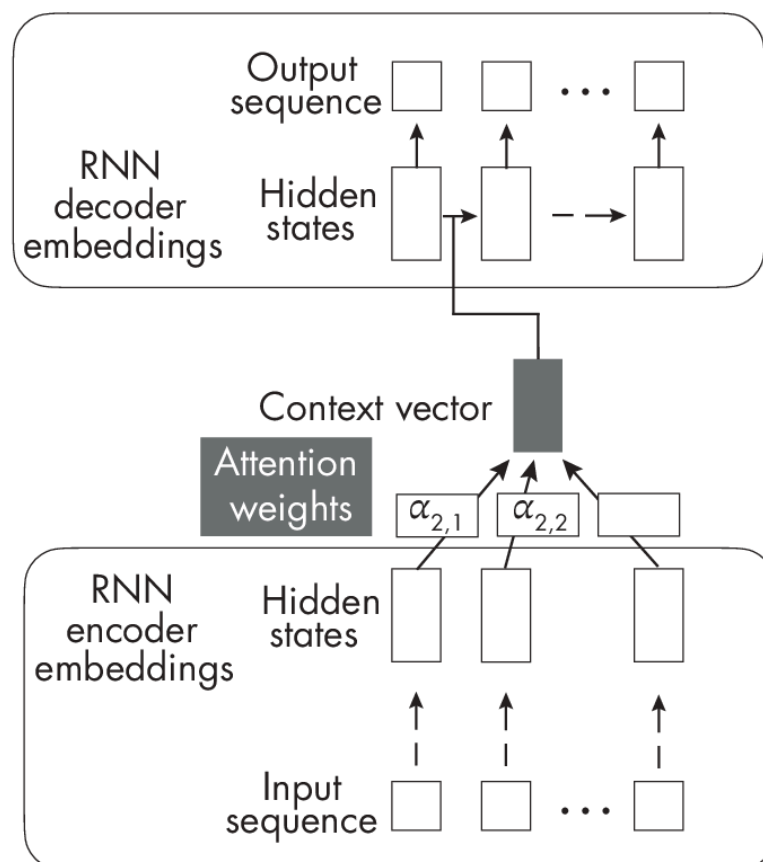


Figure 16.1



In Figure [16.1](#), the α values represent the attention weights for the second sequence element and each other element in the sequence from 1 to T . Furthermore, this original attention mechanism involves **two RNNs**.

图 [16.1](#) 中 α 表示对第二个序列位置相对位置 1 至 T 的注意权重；该原始机制涉及 **两个 RNN**。

- The RNN at the bottom, computing the attention weights, represents the encoder,
- while the RNN at the top, producing the output sequence, is a decoder.
- 下方计算注意权重的 RNN 为编码器；
- 上方产生输出序列的为解码器。

Tips:

- 原始注意力机制是应用于两个不同序列的：**编码器** 和 **解码器** 嵌入。
- 对于每个生成的输出序列元素，**解码器RNN** 基于隐藏状态和编码器生成的 **上下文向量**。
- **上下文向量** 涉及输入序列的所有元素，是所有输入元素的加权和，其中，注意力分数 (α) 表示权重系数。
- 这允许 **解码器** 访问输入序列的所有元素（上下文）。

In short, the original attention mechanism developed for RNNs is applied between two different sequences: the encoder and decoder embeddings. For each generated output sequence element, the decoder RNN at the top is based on a hidden state plus a context vector generated by the encoder. This context vector involves *all* elements of the input sequence and is a weighted sum of all input elements where the attention scores (α 's) represent the weighting coefficients. This allows the decoder to access all input sequence elements (the context) at each step. The key idea is that the attention weights (and context) may differ and change dynamically at each step.

简言之，RNN 原始注意力用在编码器与解码器两路之间：每步解码依赖隐状态与编码器给的上下文向量；该向量覆盖输入 **全体** 元素，是加权求和， α 为权重；解码器每步都能访问全序列上下文，且注意权重可步步不同、动态变化。



The motivation behind this complicated encoder-decoder design is that we cannot translate sentences word by word. This would result in grammatically incorrect outputs, as illustrated by the RNN architecture (a) in Figure 16.2.

如此设计是因为不能逐词互译，否则会产出不合语法的句子，见图 16.2(a)。

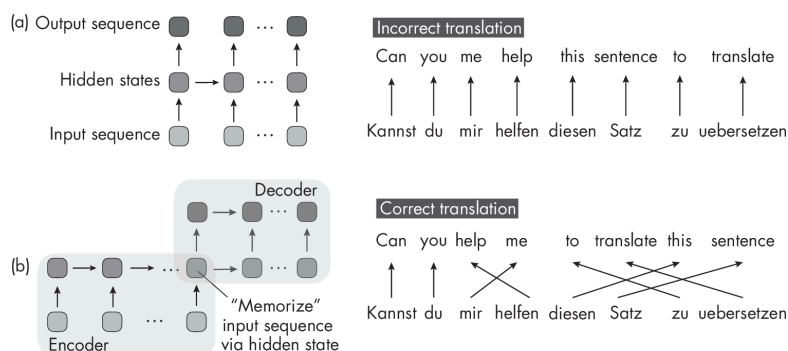


Figure 16.2

Figure 16.2 shows two different sequence-to-sequence RNN designs for sentence translation.

图 16.2 给出两种用于句到句翻译的 RNN 设计。

Figure 16.2(a) represents a regular sequence-to-sequence RNN that may be used to translate a sentence from German to English word by word.

(a) 为普通 seq2seq，可德译英逐词进行。

Figure 16.2(b) depicts an encoder-decoder RNN that first reads the whole sentence before translating it.

(b) 为编码器—解码器，先读完整句再翻译。

RNN **architecture (a)** is best suited for time series tasks in which we want to make one prediction at a time, such as predicting a given stock price day by day. For tasks like language translation, we typically opt for an encoder-decoder RNN, as in **architecture (b)** in Figure 16.2. Here, the RNN encodes the input sentence, stores it in an intermediate hidden representation, and generates the output sentence. However, this creates a bottleneck where the RNN has to memorize the whole input sentence via a single hidden state, which does not work well for longer sequences.



结构 (a) 适合时间序列逐步单点预测（如逐日股价）；翻译等多用 (b)：编码整句为中间表示再生成输出，但单隐状态要记住整句会形成瓶颈，长句差。

The bottleneck depicted in **architecture (b)** prompted the **Bahdanau** attention mechanism's original design, allowing the decoder to access all elements in the input sentence at each time step. The attention scores also give different weights to the different input elements depending on the current word that the decoder generates. For example, when generating the word *help* in the output sequence, the word *helfen* in the German input sentence may get a large attention weight, as it's highly relevant in this context.

(b) 的瓶颈催生了 Bahdanau 注意：解码器每步可访问输入全部位置，且权重随当前生成词而变；例如输出 *help* 时，德语输入中的 *helfen* 可能得大权重。

The Self-Attention Mechanism

同一序列内元素两两注意得到上下文；无需 RNN 骨干即可建强 seq2seq，引出 Transformer。

The Bahdanau attention mechanism relies on a somewhat complicated encoder-decoder design to model long-term dependencies in sequence-to-sequence language modeling tasks. Approximately three years after the Bahdanau mechanism, researchers worked on simplifying sequence-to-sequence modeling architectures by asking whether the RNN backbone was even needed to achieve good language translation performance. This led to the design of the original transformer architecture and self-attention mechanism.

Bahdanau 依赖较复杂的编码器—解码器以建模 seq2seq 长依赖；约三年后研究者问：是否必用 RNN 骨干才能做好翻译？由此出现原始 Transformer 与 self-attention。

Tips: transformer 架构，在 2017 年提出，用于解决序列到序列（sequence-to-sequence）语言建模任务中的长程依赖问题。

在 transformer 架构中，注意力机制被应用于同一序列中的所有元素，而不是像 Bahdanau 注意力机制那样涉及两个不同的序列。

与 RNN 中的注意力机制类似，上下文向量是输入序列元素的加权和，其中注意力分数 (α) 表示权重系数。

这允许解码器访问输入序列的所有元素（上下文）。

In self-attention, the attention mechanism is applied between all elements in the same sequence (as opposed to involving two sequences), as depicted in the simplified attention mechanism in Figure 16.3. Similar to the attention mechanism for RNNs, the context vector is an attention-weighted sum over the input sequence elements.

自注意中，注意作用在同一序列全体元素之间（非两序列），见图 16.3；与 RNN 注意类似，上下文仍是输入位置的加权和。

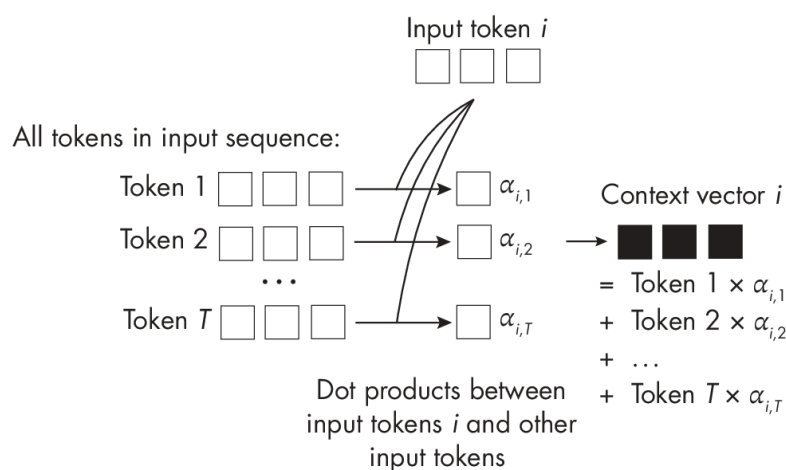


Figure 16.3

While Figure 16.3 doesn't include weight matrices, the self-attention mechanism used in transformers typically involves multiple weight matrices to compute the attention weights.

图 16.3 未画权重矩阵，而 Transformer 中实际自用注意多用多组矩阵计算注意权重。

This chapter laid the groundwork for understanding the inner workings of transformer models and the attention mechanism. The next chapter covers the different types of transformer architectures in more detail.

本章为理解 Transformer 与注意机制打基础；下一章更细讲各类 Transformer 架构。



Exercises

习题：自注意的时间/内存复杂度；除 NLP 外在视觉中的应用可能。

16-1. Considering that self-attention compares each sequence element with itself, what is the time and memory complexity of self-attention?

习题 16-1：自注意力对每个序列元素与其余元素比较，其时间与空间复杂度各如何？

16-2. We discussed self-attention in the context of natural language processing. Could this mechanism be useful for computer vision applications as well?

习题 16-2：除自然语言外，自注意是否在计算机视觉中也有用？

References

本节列原始 *scaled dot-product* 自注意、Bahdanau 对齐注意，以及自注意从零实现博文链接。

- The paper introducing the original self-attention mechanism, also known as *scaled dot-product* attention: Ashish Vaswani et al., "Attention Is All You Need"? (2017), <https://arxiv.org/abs/1706.03762>.

Vaswani 等提出 Transformer 与 *scaled dot-product* 自注意的论文（2017）。

- The Bahdanau attention mechanism for RNNs: Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate"? (2014), <https://arxiv.org/abs/1409.0473>.

Bahdanau、Cho、Bengio 关于联合对齐与翻译的 RNN 神经机器翻译（2014）。

- For more about the parameterized self-attention mechanism, check out my blog post: "Understanding and Coding the Self-Attention Mechanism of Large Language Models from Scratch"? at <https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>.

作者博文：从零理解与编码大语言模型的自注意机制（见链接）。





Chapter 17: Encoder- and Decoder-Style Transformers

本章对比 encoder-only、decoder-only 与 encoder-decoder 路线，并串起原始 Transformer、BERT/RoBERTa、GPT 与混合模型等代表工作。

What are the differences between encoder- and decoder-based language transformers?

编码器与解码器两类架构都用自注意力编码词元，但前者更适合学习可用于分类等任务的上下文 embedding，后者更偏向按步生成新文本（例如回答查询）。

Both encoder- and decoder-style architectures use the same self-attention layers to encode word tokens. The main difference is that encoders are designed to learn embeddings that can be used for various predictive modeling tasks such as classification. In contrast, decoders are designed to generate new texts, for example, to answer user queries.

两种架构都以自注意力层编码词元；区别在于编码器侧重学习可服务多种预测任务的表示，解码器侧重生成式输出。

Tips: 编码器（encoder）和解码器（decoder）架构都采用自注意力（self-attention）层来对词元进行编码。

主要区别在于：

- 编码器，侧重于**学习文本**的上下文表示（embedding），以便用于分类等各种预测任务；
- 而解码器，则专注于**生成**新的文本内容，比如用于回答用户问题等生成式任务。

This chapter starts by describing the original transformer architecture consisting of an encoder that processes input text and a decoder that produces translations. The subsequent sections then describe how models like BERT and RoBERTa utilize only the encoder to understand context and how the GPT architectures emphasize decoder-only mechanisms for text generation.

本章先回顾由处理输入的编码器与产出译文的解码器组成的原始 Transformer，再分述 BERT、RoBERTa 等只用编码器理解上下文，以及 GPT 系强调解码器生成文本的路线。



The Original Transformer

本节按图梳理原始 Transformer 的编码器与解码器栈、从分词到多子层与 Add & Norm 的数据流，以及编码表示如何送入解码器。

The original transformer architecture introduced in Chapter [ch16] was developed for English-to-French and English-to-German language translation. It utilized both an encoder and a decoder, as illustrated in Figure 17.1.

第 16 章介绍的原始 Transformer 面向英法、英德翻译，编码器与解码器并用，见图 17.1。

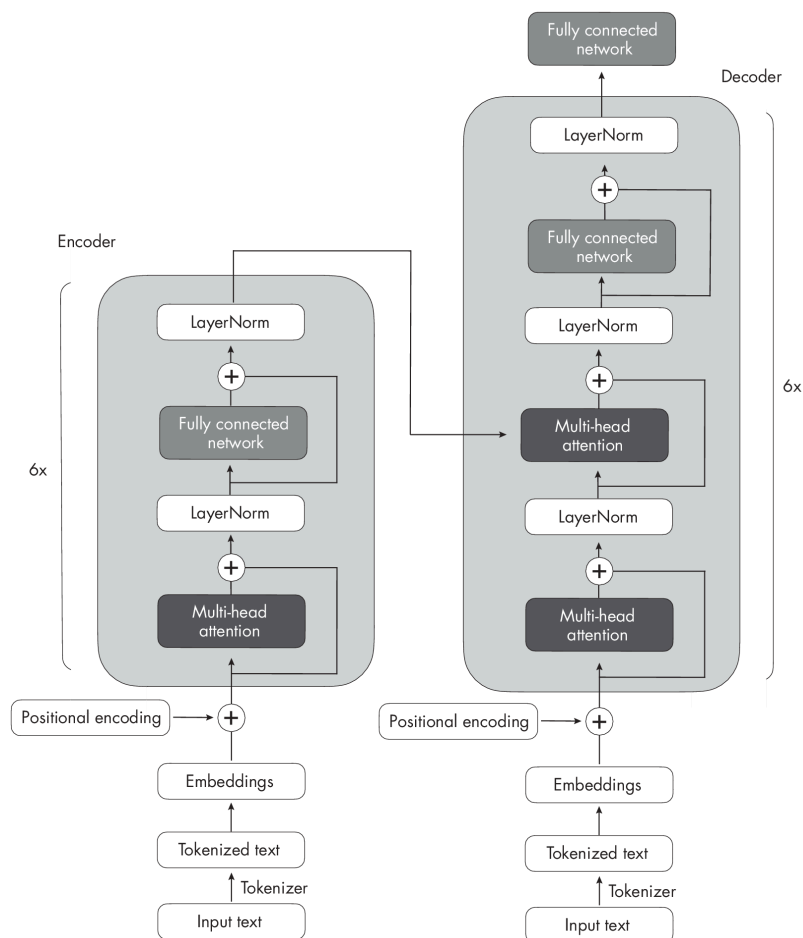


Figure 17.1



In Figure 17.1, the input text (that is, the sentences of the text to be translated) is first **tokenized** into individual word tokens, which are then encoded via an **embedding layer** before they enter the encoder part (see Chapter [ch01] for more on embeddings). After a **positional encoding vector** is added to each embedded word, the embeddings go through a **multi-head self-attention layer**. This layer is followed by an **addition step**, indicated by a plus sign (+) in Figure 17.1, which performs a **layer normalization** and adds the original embeddings via a **skip connection**, also known as a **residual** or **shortcut connection**. Following this is a LayerNorm block, short for *layernormalization*, which normalizes the activations of the previous layer to improve the stability of the neural network's training. The addition of the original embeddings and the layer normalization steps are often summarized as the **Add & Normstep**.

图中待译句先经 **tokenized** 成词元，再经 **embedding layer** 进入编码器（embedding 详见第 1 章）；为每个嵌入加上 **positional encoding vector** 后进入 **multi-head self-attention layer**，随后的 **addition step**（图中的“+”）做 **layer normalization** 并把原嵌入经 **skip connection**（残差/快捷连接）加回；接着是 LayerNorm 块，常把“加与归一”概括为 * **Add & Normstep** *。

Finally, after entering the **fully connected network** -- a small, multilayer perceptron consisting of two fully connected layers with a nonlinear activation function in between -- the outputs are again added and normalized before they are passed to a **multi-head self-attention layer** of the decoder.

再经小型 MLP 式的 **fully connected network**（两层全连接夹非线性），输出再次相加、归一后，送入解码器的 **multi-head self-attention layer**。

Tips:

- **tokenized**：将文本拆分成单词或词元（token）。
- **embedding layer**：将词元转换为向量表示。
- **positional encoding vector**：将词元的位置信息添加到向量表示中。
- **multi-head self-attention layer**：对词元进行自注意力计算。
- **addition step**：将原始向量表示和自注意力计算结果相加，也称为残差连接、跳跃连接、快捷连接。

- **layer normalization**：对每个层的激活进行归一化，以提高神经网络的稳定性。
- **Add & Normstep**：将原始向量表示和自注意力计算结果相加，并进行归一化。
- **fully connected network**：一个小的多层感知器，包含两个全连接层和一个非线性激活函数。



The decoder in Figure [17.1](#) has a similar overall structure to the encoder. The key difference is that the inputs and outputs are different: the encoder receives the input text to be translated, while the decoder generates the translated text.

解码器整体栈式结构与编码器相似，但输入输出角色不同：编码器读入待译原文，解码器生成译文。

Encoders

本节说明编码器如何抽取输入信息、输出连续表示给解码器，并概述 BERT 路线及其掩码语言建模等预训练。

The encoder part in the original transformer, as illustrated in Figure [17.1](#), is responsible for understanding and extracting the relevant information from the input text. It then outputs a continuous representation (embedding) of the input text, which is passed to the decoder. Finally, the decoder generates the translated text (target language) based on the continuous representation received from the encoder.

编码器负责理解并抽取输入中的相关信息，输出连续表示交给解码器，由后者据此生成目标语译文。

Over the years, various **encoder-only** architectures have been developed based on the encoder module of the original transformer model outlined earlier. One notable example is **BERT**, which stands for **Bidirectional Encoder Representations from Transformers**.

此后在原始编码器模块上发展出多种 **encoder-only** 架构，著名例子是 **Bidirectional Encoder Representations from Transformers**，即 **BERT**。



As noted in Chapter [\[ch14\]](#), BERT is an encoder-only architecture based on the transformer's encoder module. The BERT model is pretrained on a large text corpus using masked language modeling and next-sentence prediction tasks. Figure [17.2](#) illustrates the masked language modeling pretraining objective used in BERT-style transformers.

如第 14 章所述，BERT 是纯编码器栈，在大规模语料上用掩码语言建模与下一句预测预训练；图 17.2 示意掩码语言建模目标。

Tips: BERT 是基于掩码语言建模和预测下一句任务预训练的模型。

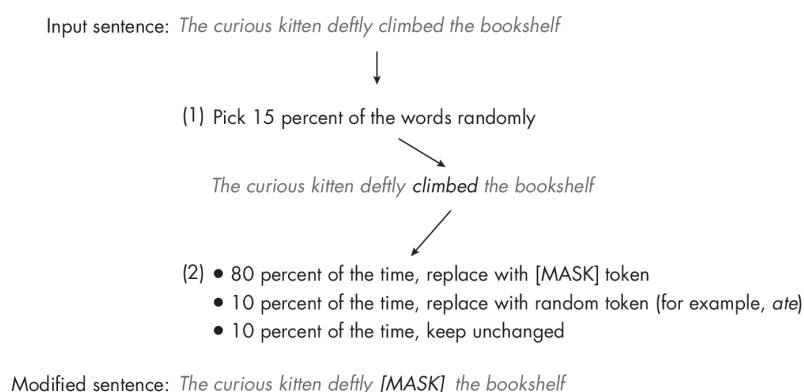


Figure 17.2

As Figure [17.2](#) demonstrates, the main idea behind masked language modeling is to mask (or replace) random word tokens in the input sequence and then train the model to predict the original masked tokens based on the surrounding context.

掩码语言建模的核心是随机遮盖或替换输入中的词元，再据上下文预测原词。

In addition to the **masked language modeling** pretraining task illustrated in Figure [17.2](#), the next-sentence prediction task asks the model to predict whether the original document's sentence order of two randomly shuffled sentences is correct. For example, say that two sentences, in random order, are separated by the [SEP] token (*SEP* is short for *separate*). The brackets are a part of the token's notation and are used to make it clear that this is a special token as opposed to a regular word in the text. BERT-style transformers also use a [CLS] token. The [CLS] token serves as a placeholder token for the model, prompting the model to return a *True* or *False* label indicating whether the sentences are in the correct order:



除掩码语言建模外，下一句预测要求模型判断两句随机打乱后是否仍为原文顺序；两句间用 [SEP] (*separate*) 分隔，方括号标明特殊符号；另用 [CLS] 作为汇总位，使模型输出二值判断两句顺序是否正确：

- "[CLS] Toast is a simple yet delicious food. [SEP] It's often served with butter, jam, or honey."?
- "[CLS] It's often served with butter, jam, or honey. [SEP] Toast is a simple yet delicious food."?

上述两条示例分别对应顺序正确与被打乱的情形（保留原英文例句）。

The masked language and next-sentence pretraining objectives allow BERT to learn rich contextual representations of the input texts, which can then be fine-tuned for various downstream tasks like sentiment analysis, question answering, and named entity recognition. It's worth noting that this pretraining is a form of self-supervised learning (see Chapter [\[ch02\]](#) for more details on this type of learning).

两项预训练目标使 BERT 学到丰富的上下文表示，可再微调用于情感分析、问答、命名实体识别等；该预训练属于自监督学习（见第 2 章）。

Tips: 值得一提的是，这种预训练方式是一种自监督学习（self-supervised learning）

RoBERTa , which stands for **Robustly Optimized BERT Approach**, is an improved version of BERT. It maintains the same overall architecture as BERT but employs several training and optimization improvements, such as larger batch sizes, more training data, and eliminating the next-sentence prediction task. These changes have resulted in RoBERTa achieving better performance on various natural language understanding tasks than BERT.

RoBERTa（**Robustly Optimized BERT Approach**）与 BERT 结构相同，但通过更大 batch、更多数据并去掉下一句预测等优化，常在多项 NLU 任务上优于 BERT。

Decoders

本节说明解码器中的因果掩码、自回归生成性质，以及 GPT 等 decoder-only 模型与“涌现”行为、上下文学习入口。



Coming back to the original transformer architecture outlined in Figure 17.1, the multi-head self-attention in the decoder is similar to the one in the encoder, but it is masked to prevent the model from attending to future positions, ensuring that the predictions for position i can depend only on the known outputs at positions less than i . As illustrated in Figure 17.3, the decoder generates the output word by word.

回到图 17.1：解码器里的多头自注意与编码器类似，但加掩码以禁止看到未来位置，使位置 i 只能依赖更小下标的已知输出；解码器由此逐词生成，见图 17.3。

Tips: 解码器中，为了防止模型关注未来的位置上信息，会进行掩码处理。

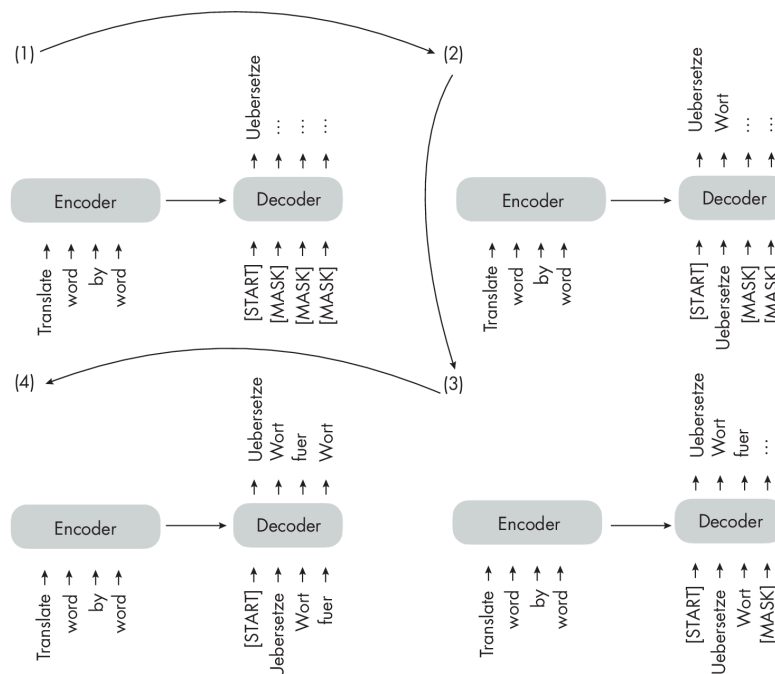


Figure 17.3

This masking (shown explicitly in Figure 17.3, although it occurs internally in the decoder's multi-head self-attention mechanism) is essential to maintaining the transformer model's autoregressive property during training and inference. This autoregressive property ensures that the model generates output tokens one at a



time and uses previously generated tokens as context for generating the next word token.

掩码（图中标出，实际发生在解码器自注意内部）保证训练与推理时的自回归性：一次只出一个词，并以前文为条件预测下一词元。

Tips: 解码器中，为了 **保持** 模型 **自回归** 的特性，会进行 **掩码** 处理。

自回归 (Autoregressive)：在生成式任务中，模型会根据之前生成的内容，预测下一个词。

FIXME: 自回归 auto-regressive 是什么意思？？？

Over the years, researchers have built upon the original encoder-decoder transformer architecture and developed several decoder-only models that have proven highly effective in various natural language processing tasks. The most notable models include the GPT family, which we briefly discussed in Chapter [ch14] and in various other chapters throughout the book. **GPT** stands for **Generative Pre-trained Transformer**. The GPT series comprises decoder-only models pretrained on large-scale unsupervised text data and fine-tuned for specific tasks such as text classification, sentiment analysis, question answering, and summarization. The GPT models, including at the time of writing GPT-2, GPT-3, and GPT-4, have shown remarkable performance in various benchmarks and are currently the most popular architecture for natural language processing.

近年研究在原始架构之上发展出多款高效的 decoder-only 模型，尤以 GPT 家族著称 (**Generative Pre-trained Transformer**)：在大规模无监督文本上预训练，再面向分类、情感、翻译式 QA、摘要等任务微调；截至书中成稿，GPT-2/3/4 等在基准上表现突出，已是 NLP 中最流行的架构之一。

One of the most notable aspects of GPT models is their **emergent** properties. **Emergent** properties are the abilities and skills that a model develops due to its next-word prediction pretraining. Even though these models were taught only to predict the next word, the pretrained models are capable of text summarization, translation, question answering, classification, and more. Furthermore, these models can perform new tasks without updating the model parameters via in-context learning, which we'll discuss in more detail in Chapter [ch18].



GPT 的突出之处在于“涌现”能力：仅经下一词预测预训练，却能摘要、翻译、问答、分类等；还可借助 in-context learning 在不改参数的情况下试新任务，第 18 章详述。

Encoder-Decoder Hybrids

本节介绍融合编码器与解码器的长处的新型混合架构（如 BART、T5）及其在理解输入并生成不同结构输出类任务中的典型用途。

Next to the traditional encoder and decoder architectures, there have been advancements in the development of new encoder-decoder models that leverage the strengths of both components. These models often incorporate novel techniques, pretraining objectives, or architectural modifications to enhance their performance in various natural language processing tasks. Some notable examples of these new encoder-decoder models include **BART** and **T5**.

在传统编解结构之外，也出现了结合两者优势的新 encoder-decoder 模型，常配合新预训练目标或结构改动；例如 **BART** 与 **T5**。

Encoder-decoder models are typically used for natural language processing tasks that involve understanding input sequences and generating output sequences, often with different lengths and structures. They are particularly good at tasks where there is a complex mapping between the input and output sequences and where it is crucial to capture the relationships between the elements in both sequences. Some common use cases for encoder-decoder models include text translation and summarization.

此类模型适合“理解一段输入、再生成另一段长度与结构可不同的输出”，在输入输出映射复杂且需对齐两侧关系时尤其有用；常见用法包括翻译与摘要。

Tips: Encoder-decoder models, 通常用于自然语言处理任务，涉及理解输入序列和生成输出序列，通常具有不同的长度和结构。在文本翻译和摘要等任务中，表现尤为出色。

Terminology

本节澄清 encoder-only / decoder-only / encoder-decoder 与 seq2seq、自回归解码等术语之间的关系与易混点。



All of these methods -- **encoder-only** , **decoder-only** , and **encoder-decoder** models -- are **sequence-to-sequence** models (often abbreviated as **seq2seq**). While we refer to BERT-style methods as "encoder-only," the description may be misleading since these methods also *decode* the embeddings into output tokens or text during pretraining. In other words, both encoder-only and decoder-only architectures perform decoding.

上述路线都属于 **sequence-to-sequence** (**seq2seq**)。称 BERT 式方法为“encoder-only”有时易误解：预训练时仍会把 embedding “解码”回词或文本，故 encoder-only 与 decoder-only 在字面上都会发生某种 decoding。

However, the encoder-only architectures, in contrast to decoder-only and encoder-decoder architectures, don't decode in an autoregressive fashion. *Autoregressive decoding* refers to generating output sequences one token at a time, conditioning each token on the previously generated tokens. Encoder-only models do not generate coherent output sequences in this manner. Instead, they focus on understanding the input text and producing task-specific outputs, such as labels or token predictions.

但 encoder-only 不像另两类那样做自回归解码：*Autoregressive decoding* 指逐步生成、每步条件于已生成词元；encoder-only 通常不依此方式产生连贯长文本，而侧重理解输入并产出标签或逐词预测等任务化输出。

Contemporary Transformer Models

本节概括三类架构在分类嵌入、强条件生成与开放生成等场景中的分工，以及产业与研究中的流行度变化。

In brief, encoder-style models are popular for learning embeddings used in classification tasks, encoder-decoder models are used in generative tasks where the output heavily relies on the input (for example, translation and summarization), and decoder-only models are used for other types of generative tasks, including Q&A. Since the first transformer architecture emerged, hundreds



of encoder-only, decoder-only, and encoder-decoder hybrids have been developed, as diagrammed in Figure 17.4.

简言之：编码器式模型常用来学分类用嵌入；encoder-decoder 适合输出强依赖输入的生成（翻译、摘要等）；decoder-only 适合包括问答在内的更开放生成。自 Transformer 问世以来已涌现大量三类变体，见图 17.4。

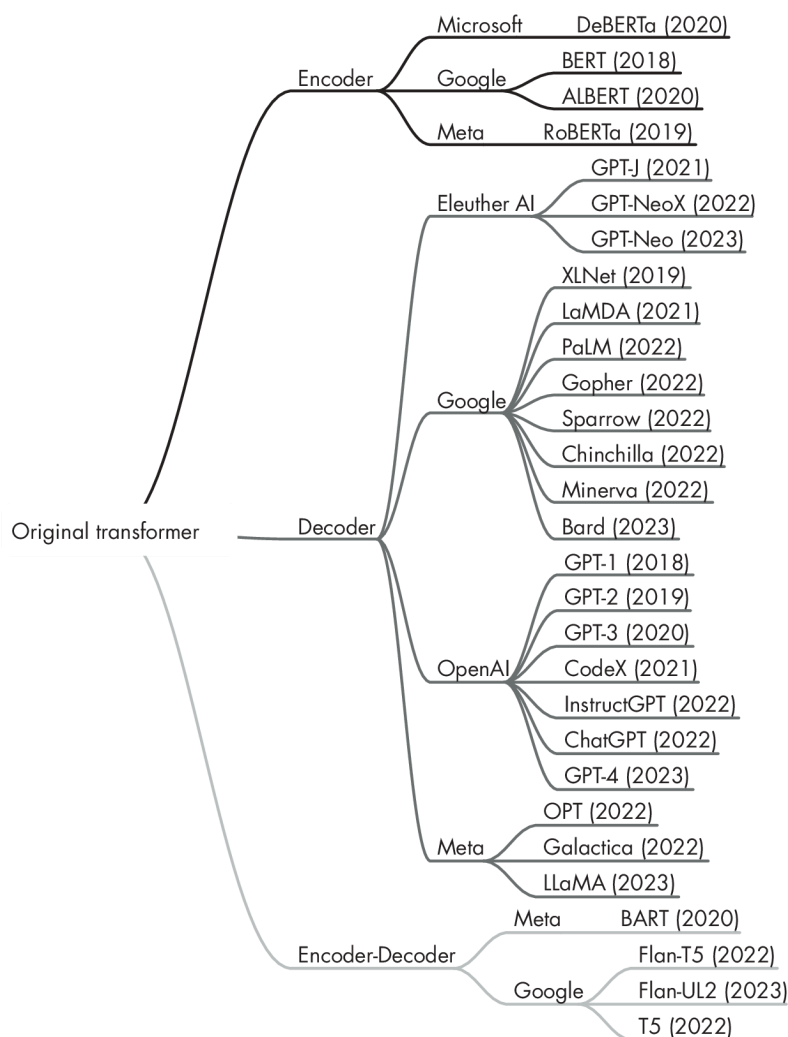


Figure 17.4

While encoder-only models have gradually become less popular, decoder-only models like GPT have exploded in popularity, thanks to breakthroughs in text generation via GPT-3, ChatGPT, and GPT-4. However, encoder-only models are still very useful for training predictive models based on text embeddings as opposed to generating texts.

encoder-only 热度相对放缓的同时，GPT 等 decoder-only 因 GPT-3、ChatGPT、GPT-4 等文本生成突破而大行其道；但若目标是基于文本嵌入做判别式建模而非长文生成，encoder-only 仍很实用。



Exercises

本节习题引导思考如何将 BERT 类模型用于分类，以及能否对 GPT 类 decoder-only 做分类微调。

17-1. As discussed in this chapter, BERT-style encoder models are pretrained using masked language modeling and next-sentence prediction pretraining objectives. How could we adopt such a pretrained model for a classification task (for example, predicting whether a text has a positive or negative sentiment)?

17-2. Can we fine-tune a decoder-only model like GPT for classification?

References

本节列出注意力、BERT、RoBERTa、BART、T5、GPT 系列等相关论文与链接。

- The Bahdanau attention mechanism for RNNs: Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate"? (2014), <https://arxiv.org/abs/1409.0473>.
- The original BERT paper, which popularized encoder-style transformers with a masked word and a next-sentence prediction pretraining objective: Jacob Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"? (2018), <https://arxiv.org/abs/1810.04805>.
- RoBERTa improves upon BERT by optimizing training procedures, using larger training datasets, and sentence prediction task: Yinhan Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach"? (2019), <https://arxiv.org/abs/1907.11692>.
- The BART encoder-decoder architecture: Mike Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension"? (2018), <https://arxiv.org/abs/1910.13461>.
- The T5 encoder-decoder architecture: Colin Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer"? (2019),



<https://arxiv.org/abs/1910.10683>.

- The paper proposing the first GPT architecture: Alec Radford et al., "Improving Language Understanding by Generative Pre-Training"? (2018),
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
 - The GPT-2 model: Alec Radford et al., "Language Models Are Unsupervised Multitask Learners"? (2019),
<https://www.semanticscholar.org/paper/Language-Models-are-Unsupervised-Multitask-Learners-Radford-Wu/9405cc0d6169988371b2755e573cc28650d14dfe>.
 - The GPT-3 model: Tom B. Brown et al., "Language Models Are Few-Shot Learners"? (2020), <https://arxiv.org/abs/2005.14165>.
-



Chapter 18: Using and Fine-Tuning Pretrained Transformers

本章从分类与下游任务出发，串讲特征提取、全量/部分微调、上下文学习与提示、索引，以及参数高效微调与 RLHF 等适配手段。

What are the different ways to use and fine-tune pretrained large language models?

使用和微调预训练大语言模型的三种方法：

1. 特征提取
2. 上下文学习（提示学习）
3. 微调部分模型参数

The three most common ways to use and fine-tune pretrained LLMs include a **feature-based approach**, **in-context prompting**, and **updating a subset of the model parameters**. First, most pretrained LLMs or language transformers can be utilized without the need for further fine-tuning. For instance, we can employ a feature-based method to train a new downstream model, such as a linear classifier, using embeddings generated by a pretrained transformer. Second, we can showcase examples of a new task within the input itself, which means we can directly exhibit the expected outcomes without requiring any updates or learning from the model. This concept is also known as *prompting*. Finally, it's also possible to fine-tune all or just a small number of parameters to achieve the desired outcomes.

常见路径包括：用冻结的 Transformer 抽特征再训下游；在提示里放示例做 in-context（即 *prompting*）；或对全部或少量参数做微调。多数预训练模型本身不必再训即可间接使用。

The following sections discuss these types of approaches in greater depth.

下文将逐类展开这些思路。

Using Transformers for Classification Tasks

本节在分类设定下比较特征提取、Fine-tuning I/II，并用图示概括效率与效果上的经验法则。



Let's start with the conventional methods for utilizing pretrained transformers: training another model on feature embeddings, fine-tuning output layers, and fine-tuning all layers. We'll discuss these in the context of classification. (We will revisit prompting later in the section "In-Context Learning, Indexing, and Prompt Tuning" on page .)

我们先在分类场景讨论三类常规用法：在 embedding 上另训模型、只微调输出层、微调全部层；提示相关方法稍后同一章另一节再谈。

In the feature-based approach, we load the pretrained model and keep it "frozen," meaning we do not update any parameters of the pretrained model. Instead, we treat the model as a feature extractor that we apply to our new dataset. We then train a downstream model on these embeddings.

特征法则是加载预训练模型并保持冻结，把它当特征提取器在新数据上跑一遍，再在特征上训练下游模型。

特征提取方法：

1. 加载预训练模型并保持冻结状态，不更新任何参数。
2. 将预训练模型视为特征提取器，应用于新数据集。
3. 在提取的特征上训练下游模型。

This downstream model can be any model like (random forests, XGBoost, and so on), but linear classifiers typically perform best. This is likely because pretrained transformers like BERT and GPT already extract high-quality, informative features from the input data. These feature embeddings often capture complex relationships and patterns, making it easy for a linear classifier to effectively separate the data into different classes. Furthermore, linear classifiers, such as logistic regression machines and support vector machines, tend to have strong regularization properties. These regularization properties help prevent overfitting when working with high-dimensional feature spaces generated by pretrained transformers. This feature-based approach is the most efficient method since it doesn't require updating the transformer model at all. Finally, the embeddings



can be precomputed for a given training dataset (since they don't change) when training a classifier for multiple training epochs.

下游可选任意模型，但线性分类器往往最好：高质量嵌入已编码复杂模式，线性边界即可分开类别，且强正则有助抑制高维过拟合；特征还可预先算好、多轮训练分类器而无需反复前向骨干。

特征提取方法的优点：

1. 不需要更新预训练模型。
2. 提取的特征通常能够捕捉复杂的关系和模式，使得线性分类器能够有效地分离数据。
3. 线性分类器通常具有较强的正则化属性，有助于防止在高维特征空间中过拟合。
4. 特征可以预先计算，用于多个训练轮次的分类器训练。

Figure 18.1 illustrates how LLMs are typically created and adopted for downstream tasks using fine-tuning. Here, a pretrained model, trained on a general text corpus, is fine-tuned to perform tasks like German-to-English translation.

图 18.1 展示从通用语料预训练到下游微调的典型流程，例如德英翻译。

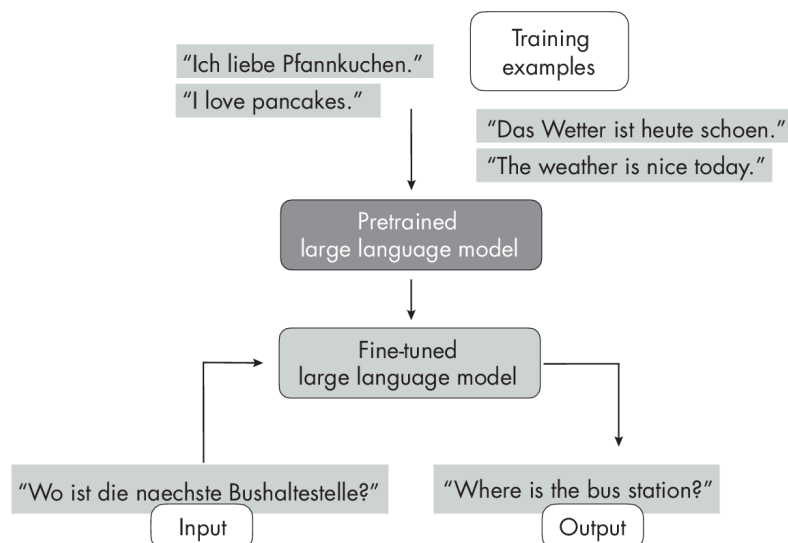


Figure 18.1

The conventional methods for fine-tuning pretrained LLMs include updating only the output layers, a method we'll refer to as *fine-tuning I*, and updating all layers, which we'll call *fine-tuning II*.



微调层面常分 *fine-tuning I*（只动输出侧）与 *fine-tuning II*（全层更新）。

微调方法，分为 2 类：

1. 只更新输出层，称为 **fine-tuning I**。
2. 更新所有层，称为 **fine-tuning II**。

Fine-tuning I is similar to the feature-based approach described earlier, but it adds one or more output layers to the LLM itself. The backbone of the LLM remains frozen, and we update only the model parameters in these new layers. Since we don't need to backpropagate through the whole network, this approach is relatively efficient regarding throughput and memory requirements.

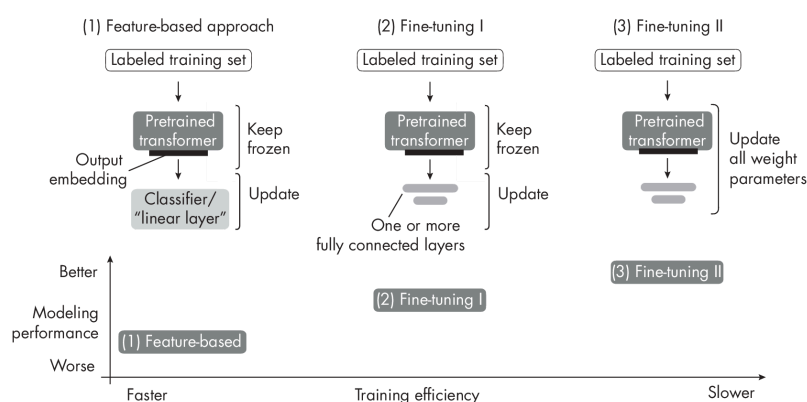
Fine-tuning I 与特征法类似，但把新输出层接在 LLM 上并只训这些层，骨干仍冻结，故反传浅、吞吐与显存相对友好。

In **fine-tuning II**, we load the model and add one or more output layers, similarly to fine-tuning I. However, instead of backpropagating only through the last layers, we update *all* layers via backpropagation, making this the most expensive approach. While this method is computationally more expensive than the feature-based approach and fine-tuning I, it typically leads to better modeling or predictive performance. This is especially true for more specialized domain-specific datasets.

fine-tuning II 同样加输出头但全网络反传，代价最高，往往在领域专用数据上效果最好。

Figure 18.2 summarizes the three approaches described in this section so far.

图 18.2 归纳本节至今的三类做法。



**Figure 18.2**

In addition to the conceptual summary of the three fine-tuning methods described in this section, Figure 18.2 also provides a rule-of-thumb guideline for these methods regarding training efficiency. Since fine-tuning II involves updating more layers and parameters than fine-tuning I, backpropagation is costlier for fine-tuning II. For similar reasons, fine-tuning II is costlier than a simpler feature-based approach.

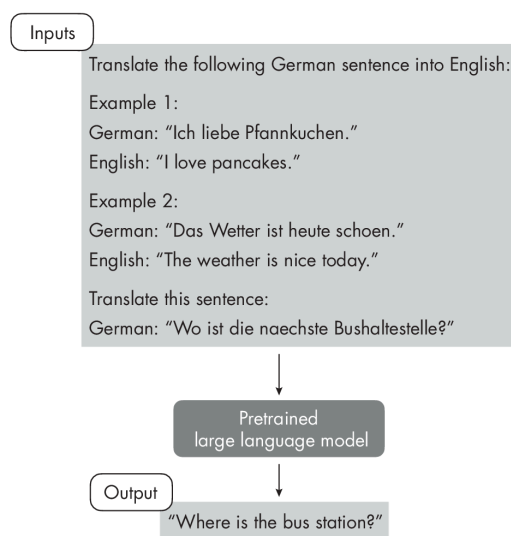
图 18.2 也给出训练开销的经验排序：fine-tuning II 更新最多因而反传最贵，其次 Fine-tuning I，特征法通常最省。

In-Context Learning, Indexing, and Prompt Tuning

本节讨论 in-context / 硬提示优化、与索引式检索增强，以及它们与全参数微调的取舍。

LLMs like GPT-2 and GPT-3 popularized the concept of **in-context learning**, often called **zero-shot** or **few-shot learning** in this context, which is illustrated in Figure 18.3.

GPT-2/3 等使 **in-context learning** 广为人知，在此语境下也常称 **zero-shot** 或 **few-shot learning**，见图 18.3。

**Figure 18.3**



As Figure [18.3](#) shows, **in-context learning** aims to provide context or examples of the task within the input or prompt, allowing the model to infer the desired behavior and generate appropriate responses. This approach takes advantage of the model's ability to learn from vast amounts of data during pretraining, which includes diverse tasks and contexts.

其核心是在提示中嵌入任务说明或示例，让模型据预训练中学到的广泛模式推断合适回复。

The definition of **few-shot learning**, considered synonymous with in-context learning-based methods, differs from the conventional approach to few-shot learning discussed in Chapter [\[ch03\]](#).

这里把 **few-shot** 与基于上下文的示范学习近乎同义，与第 3 章经典小样本学习设定有所不同。

此处讨论的 **few-shot learning** 与第 3 章讨论的 **few-shot learning** 不同。

For example, suppose we want to use in-context learning for few-shot German -- English translation using a large-scale pretrained language model like GPT-3. To do so, we provide a few examples of German -- English translations to help the model understand the desired task, as follows:

例如用 GPT-3 做德英示范翻译时，可在提示里放若干德英句对：

text

Translate the following German sentences into English:

Example 1:

German: "Ich liebe Pfannkuchen."

English: "I love pancakes."

Example 2:

German: "Das Wetter ist heute schoen."

English: "The weather is nice today."

Translate this sentence:

German: "Wo ist die naechste Bushaltestelle?"



Generally, in-context learning does not perform as well as fine-tuning for certain tasks or specific datasets since it relies on the pretrained model's ability to generalize from its training data without further adapting its parameters for the particular task at hand.

总体上它对部分任务不如微调：参数未针对当前数据再适配，全凭预训练泛化。

上下文学习在某些任务或特定数据集上可能不如微调，因为它依赖于预训练模型从其训练数据中泛化，而无需为特定任务进一步调整其参数。

However, in-context learning has its advantages. It can be particularly useful when labeled data for fine-tuning is limited or unavailable. It also enables rapid experimentation with different tasks without fine-tuning the model parameters in cases where we don't have direct access to the model or where we interact only with the model through a UI or API (for example, ChatGPT).

但在标注稀缺或只能通过 API/UI 访问模型时，它便于快速试错、无需本地训练。

上下文学习的优点：

1. 在有标签数据有限或不可用的情况下，上下文学习特别有用。
2. 在不需要直接访问模型或仅通过 UI 或 API 与模型交互的情况下，可以快速尝试不同的任务。

Related to in-context learning is the concept of **hard prompt tuning**, where *hard* refers to the non-differentiable nature of the input tokens. Where the previously described fine-tuning methods update the model parameters to better perform the task at hand, **hard prompt tuning** aims to optimize the prompt itself to achieve better performance. Prompt tuning does not modify the model parameters, but it may involve using a smaller labeled dataset to identify the best prompt formulation for the specific task.

与之相关的是 **hard prompt tuning**：离散 token 不可微，因而不改权重而改提示措辞，有时用少量标注搜更好模板。

提示词工程，即提示词微调。不改变模型参数，而是优化提示词（可能包含一小部分标签示例数据），以达到更好的性能。



For example, to improve the prompts for the previous German -- English translation task, we might try the following three prompting variations:

可尝试如下三种模板变体（占位符示例保留原貌）：

- Translate the German sentence '{german_sentence}' into English: {english_translation}
- German: '{german_sentence}' English: {english_translation}
- From German to English: '{german_sentence}' -> {english_translation}

提示词工程的优点：

1. 资源效率高，不需要更新模型参数。
2. 性能通常不如全模型微调，因为它不更新模型参数，可能限制其适应特定任务的细微差别。

Prompt tuning is a resource-efficient alternative to parameter fine-tuning. However, its performance is usually not as good as full model fine-tuning, as it does not update the model's parameters for a specific task, potentially limiting its ability to adapt to task-specific nuances. Furthermore, prompt tuning can be labor intensive since it requires either human involvement comparing the quality of the different prompts or another similar method to do so. This is often known as *hard* prompting since, again, the input tokens are not differentiable. In addition, other methods exist that propose to use another LLM for automatic prompt generation and evaluation.

硬提示省算力但常弱于全微调，且可能要人工或外挂 LLM 比较多条提示质量。

Yet another way to leverage a purely in-context learning-based approach is

indexing , illustrated in Figure [18.4](#)

纯 in-context 路线还可借 **indexing** 把外部文档搬进上下文，见图 18.4。

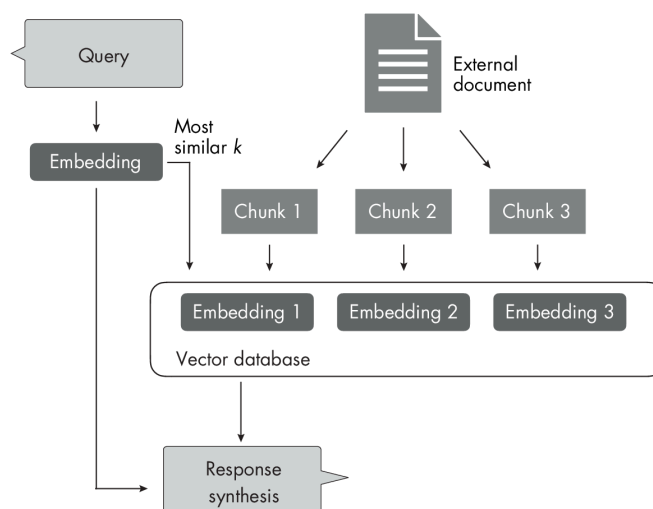


Figure 18.4

In the context of LLMs, we can think of **indexing** as a workaround based on in-context learning that allows us to turn LLMs into information retrieval systems to extract information from external resources and websites. In Figure 18.4, an indexing module parses a document or website into smaller chunks, embedded into vectors that can be stored in a vector database. When a user submits a query, the indexing module computes the vector similarity between the embedded query and each vector stored in the database. Finally, the indexing module retrieves the top k most similar embeddings to synthesize the response.

可把索引视为 in-context 的变通：切块、嵌入、入向量库，查询时取与 query 最相近的 top- k 片段拼进提示再生成。

索引，即索引模块，将文档或网站解析为更小的块 chunk，嵌入到向量中，可以存储在向量数据库中。

- 当用户提交查询时，索引模块计算嵌入查询与数据库中每个向量的相似度。
- 最后，索引模块检索与查询最相似的 k 个嵌入向量，以合成响应。

Parameter-Efficient Fine-Tuning

本节介绍软/前缀提示、Adapter 与 LoRA 等参数高效微调思路及典型实现草图。



In recent years, many methods have been developed to adapt pretrained transformers more efficiently for new target tasks. These methods are commonly referred to as **parameter-efficient fine-tuning**, with the most popular methods at the time of writing summarized in Figure 18.5.

近年大量工作致力于少更新原权重即可完成适配，统称 **parameter-efficient fine-tuning**，见图 18.5。

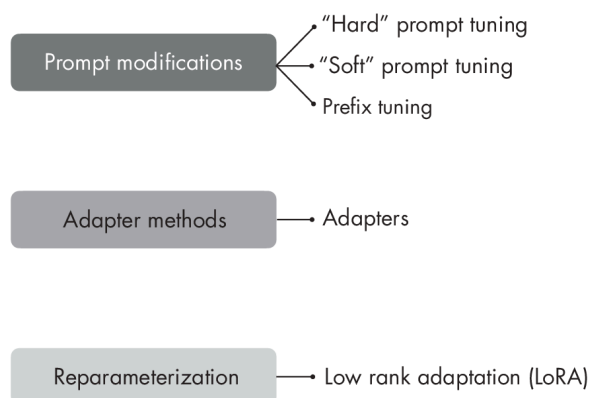


Figure 18.5

In contrast to the hard prompting approach discussed in the previous section, **soft prompting** strategies optimize embedded versions of the prompts. While in **hard prompt** tuning we modify the discrete input tokens, in **soft prompt** tuning we utilize trainable parameter tensors instead. The idea behind soft prompt tuning is to prepend a trainable parameter tensor (the "soft prompt") to the embedded query tokens. The prepended tensor is then tuned to improve the modeling performance on a target dataset using gradient descent.

与硬提示不同，**soft prompting** 在连续嵌入空间优化可训练张量，常把它拼在序列前再梯度下降。

硬提示词，调整了输入的离散 tokens；软提示词，调整了输入的 tokens 的嵌入。

- 软提示词的思路是，在输入的 tokens 前添加一个可训练的参数 tensor（即软提示词），
- 然后使用梯度下降优化这个 tensor，以提高在目标数据集上的建模性能。

In Python-like pseudocode, soft prompt tuning can be described as



下面用与 Python 语法相近的伪代码示意。

```
python

x = EmbeddingLayer(input_ids)
x = concatenate([soft_prompt_tensor, x], dim=seq_len)
output = model(x)
```

where the `soft_prompt_tensor` has the same feature dimension as the embedded inputs produced by the embedding layer. Consequently, the modified input matrix has additional rows (as if it extended the original input sequence with additional tokens, making it longer).

`soft_prompt_tensor` 与词嵌入同维，相当于在序列维上“加长”了输入。

Another popular prompt tuning method is `prefix tuning`. *Prefix tuning* is similar to soft prompt tuning, except that in prefix tuning, we prepend trainable tensors (soft prompts) to each transformer block instead of only the embedded inputs, which can stabilize the training.

`prefix tuning` 则在每个 Transformer 块前也拼可训练前缀，有时更稳。

前缀调优，在每个 transformer block 前添加一个可训练的参数 tensor。

The implementation of prefix tuning is illustrated in the following pseudocode:

实现示意伪代码如下：

```
python

def transformer_block_with_prefix(x):
    soft_prompt = FullyConnectedLayers(# Prefix
        soft_prompt)                    # Prefix
    x = concatenate([soft_prompt, x],   # Prefix
                    dim=seq_len)        # Prefix
    residual = x
    x = SelfAttention(x)
    x = LayerNorm(x + residual)
    residual = x
    x = FullyConnectedLayers(x)
```

```
x = LayerNorm(x + residual)
return x
```



Let's break Listing 18.6 into three main parts: implementing the soft prompt, concatenating the soft prompt (prefix) with the input, and implementing the rest of the transformer block.

可把 Listing 18.6 拆成三段：前缀经 MLP；与 `x` 沿 `seq_len` 拼接；其后是标准自注意—LayerNorm—前馈—残差。

First, the `soft_prompt`, a tensor, is processed through a set of fully connected layers. Second, the transformed soft prompt is concatenated with the main input, `x`. The dimension along which they are concatenated is denoted by `seq_len`, referring to the sequence length dimension. Third, the subsequent lines of code describe the standard operations in a transformer block, including self-attention, layer normalization, and feed-forward neural network layers, wrapped around residual connections.

依次：张量 `soft_prompt` 过全连接；变换后与主输入 `x` 拼接；再接常规子层。

As shown in Listing 18.6, `prefix tuning` modifies a transformer block by adding a trainable `soft prompt`. Figure 18.6 further illustrates the difference between a regular transformer block and a prefix tuning transformer block.

Listing 18.6 表明 `prefix tuning` 通过对块内注入可训练 `soft prompt` 改变块行为；图 18.6 对比普通块与前缀块。

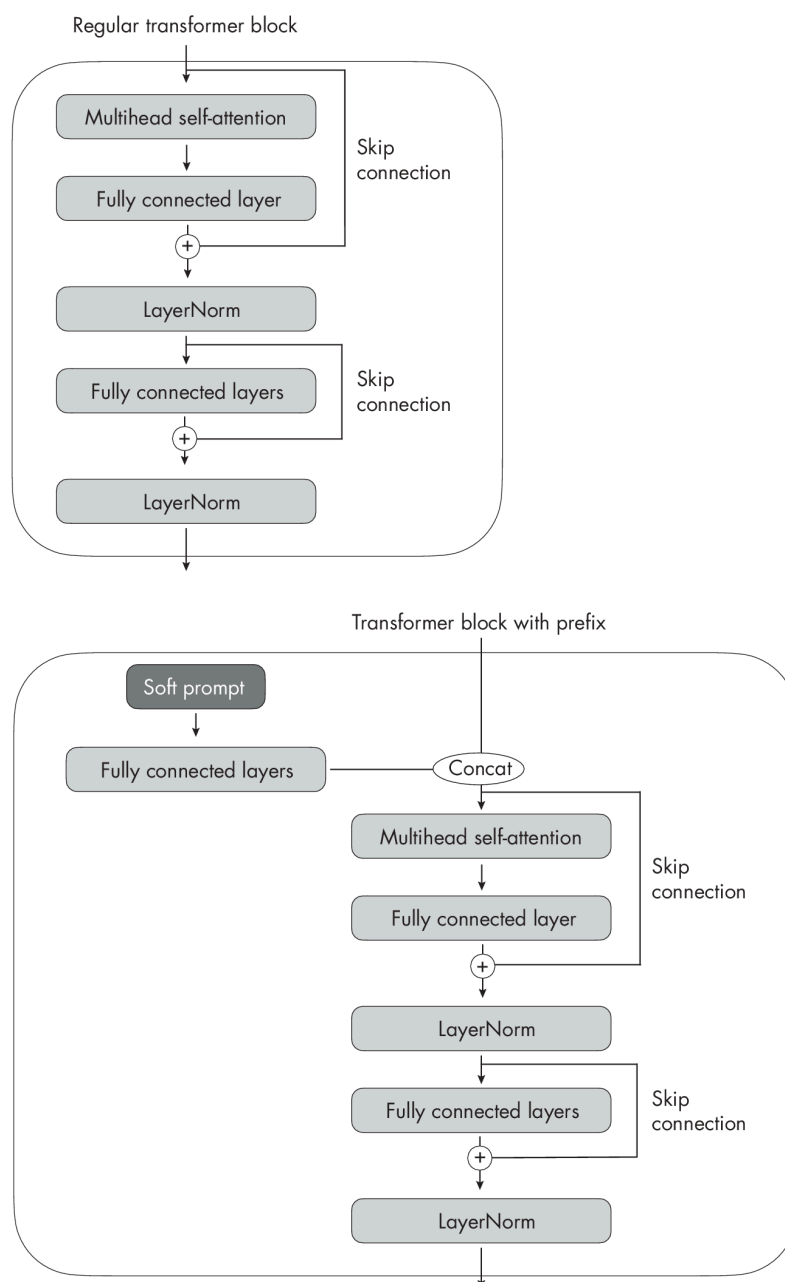


Figure 18.6

Both **soft prompt** tuning and **prefix tuning** are considered parameter efficient since they require training only the prepended parameter tensors and not the LLM parameters themselves.

二者只训前缀张量，原 LLM 权重可不更新，故属参数高效。

Adapter methods are related to **prefix tuning** in that they add additional parameters to the transformer layers. In the original adapter method, additional fully connected layers were added after the multihead self-attention and existing fully connected layers in each transformer block, as illustrated in Figure 18.7.



Adapter 在每块注意力后与 MLP 旁插入小型全连接支路，见图 18.7。

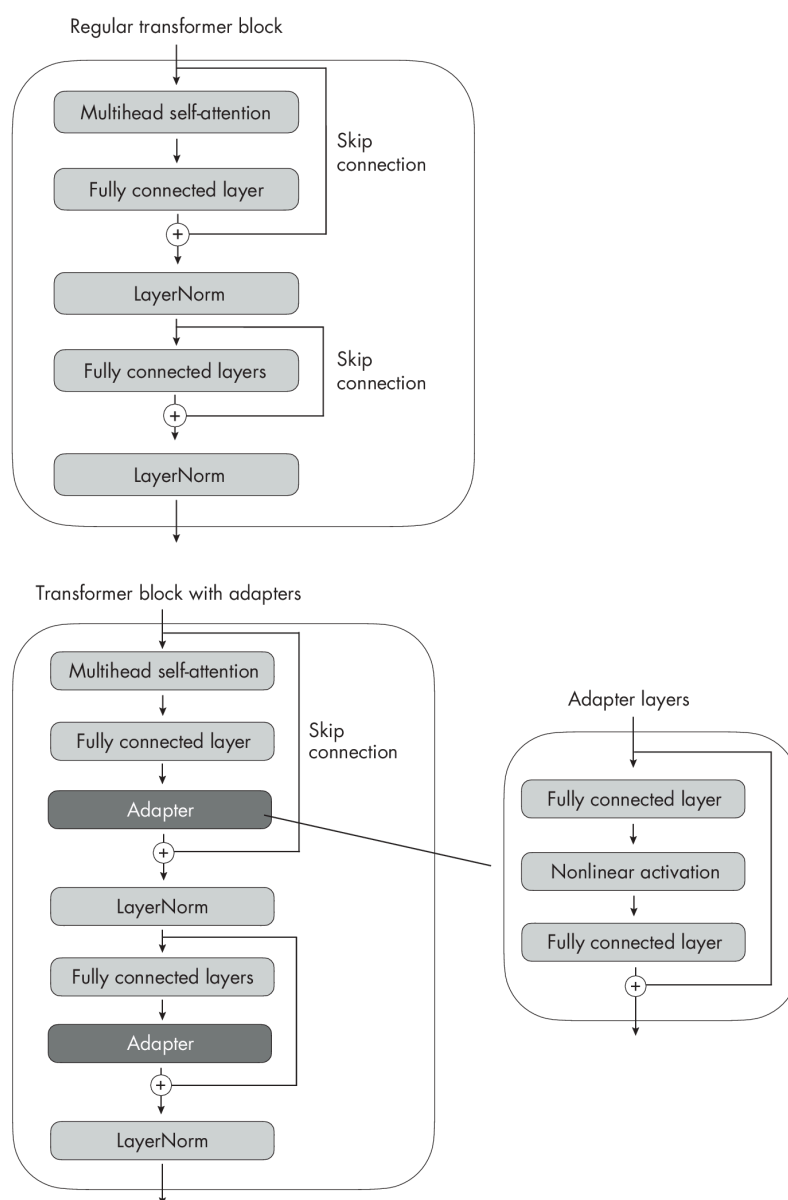


Figure 18.7

Only the new adapter layers are updated when training the LLM using the original adapter method, while the remaining transformer layers remain frozen. Since the adapter layers are usually small -- the first fully connected layer in an adapter block projects its input into a low-dimensional representation, while the second layer projects it back into the original input dimension -- this adapter method is usually considered parameter efficient.

原始 Adapter 只更新插入层，其余冻结；先投影到低维再映回原维，因而参数量小。

只会更新 adapter 层，其他层保持冻结。



In pseudocode, the original adapter method can be written as follows:

Adapter 伪代码：

```
python

def transformer_block_with_adapter(x):
    residual = x
    x = SelfAttention(x)
    x = FullyConnectedLayers(x) # Adapter
    x = LayerNorm(x + residual)
    residual = x
    x = FullyConnectedLayers(x)
    x = FullyConnectedLayers(x) # Adapter
    x = LayerNorm(x + residual)
    return x
```

Low-rank adaptation (LoRA) , another popular parameter-efficient fine-tuning method worth considering, refers to reparameterizing pretrained LLM weights using low-rank transformations. LoRA is related to the concept of **low-rank transformation** , a technique to approximate a high-dimensional matrix or dataset using a lower-dimensional representation. The lower-dimensional representation (or **low-rank approximation**) is achieved by finding a combination of fewer dimensions that can effectively capture most of the information in the original data. Popular low-rank transformation techniques include **principal component analysis** and **singular vector decomposition**.

LoRA 用低秩分解近似权重更新，与低秩近似思想一致；PCA、奇异值分解等是经典低秩工具。

低秩适应（LoRA），另一种流行的参数高效微调方法，值得考虑，指的是使用低秩变换重新参数化预训练 LLM 权重。

- LoRA 与低秩变换的概念相关，低秩变换是一种技术，使用较低维度的表示来近似高维矩阵或数据集。
- 低秩变换（或低秩近似）通过找到更少的维度组合来有效捕获原始数据中的大部分信息。
- 流行的低秩变换技术包括主成份分析和奇异值分解。



For example, suppose ΔW represents the parameter update for a weight matrix of the LLM with dimension $\mathbb{R}^{A \times B}$. We can decompose the weight update matrix into two smaller matrices: $\Delta W = W_A W_B$, where $W_A \in \mathbb{R}^{A \times h}$ and $W_B \in \mathbb{R}^{h \times B}$. Here, we keep the original weight frozen and train only the new matrices W_A and W_B .

例如将 ΔW 分解为 $W_A W_B$ ，冻结原权重只训二者。

How is this method parameter efficient if we introduce new weight matrices?

These new matrices can be very small. For example, if $A = 25$ and $B = 50$, then the size of ΔW is $25 \times 50 = 1,250$. If $h = 5$, then W_A has 125 parameters, W_B has 250 parameters, and the two matrices combined have only $125 + 250 = 375$ parameters in total.

因 h 可远小于 $\min(A, B)$ ，新增参数量可远小于满秩 ΔW 。

After learning the weight update matrix, we can then write the matrix multiplication of a fully connected layer, as shown in this pseudocode:

学好后，全连接前向可把低秩修正叠到原输出上，如：

python

```
def lora_forward_matmul(x):  
    h = x . W # Regular matrix multiplication  
    h += x . (W_A . W_B) * scalar  
    return h
```

In Listing [\[matrixMultiplication\]](#), `scalar` is a scaling factor that adjusts the magnitude of the combined result (original model output plus low-rank adaptation). This balances the pretrained model's knowledge and the new task-specific adaptation.

`scalar` 调节原输出与低秩增量之比重，平衡旧知识与任务适配。

According to the original paper introducing the LoRA method, models using LoRA perform slightly better than models using the adapter method across several task-specific benchmarks. Often, LoRA performs even better than models fine-tuned using the fine-tuning II method described earlier.

原论文显示 LoRA 在多项基准上常略胜 Adapter，有时甚至优于前文 fine-tuning II。



原始提出 LoRA 方法的论文指出，使用 LoRA 的模型在多个任务特定基准上略微优于使用适配器方法的模型。通常，LoRA 甚至比前面描述的 fine-tuning II 方法微调的模型性能更好。

Reinforcement Learning with Human Feedback

本节转向监督微调之外的 RLHF：奖励建模、PPO 以及为何用人机协同信号代替端到端在人类标注上训练。

The previous section focused on ways to make fine-tuning more efficient. Switching gears, how can we improve the modeling performance of LLMs via fine-tuning?

上一节侧重“省参数”；这里问：微调还能如何进一步提升表现？

The conventional way to adapt or fine-tune an LLM for a new target domain or task is to use a supervised approach with labeled target data. For instance, the **fine-tuning II** approach allows us to adapt a pretrained LLM and fine-tune it on a target task such as sentiment classification, using a dataset that contains texts with sentiment labels like *positive*, *neutral*, and *negative*.

常规做法是用带标注的目标数据监督微调，例如情感三分类。

监督微调，用有标签的目标数据集，训练 LLM 以适应新任务。

Supervised fine-tuning is a foundational step in training an LLM. An additional, more advanced step is **reinforcement learning with human feedback (RLHF)**, which can be used to further improve the model's alignment with human preferences. For example, ChatGPT and its predecessor, InstructGPT, are two popular examples of pretrained LLMs (GPT-3) fine-tuned using RLHF.



监督微调之上还可做 **reinforcement learning with human feedback (RLHF)** 以对齐人类偏好；ChatGPT、InstructGPT 是著名先例。

强化学习，用人类反馈，训练 LLM 以适应人类偏好。

In **RLHF**, a pretrained model is fine-tuned using a combination of supervised learning and reinforcement learning. This approach was popularized by the original ChatGPT model, which was in turn based on InstructGPT. Human feedback is collected by having humans rank or rate different model outputs, providing a reward signal. The collected reward labels can be used to train a reward model that is then used to guide the LLMs' adaptation to human preferences. The reward model is learned via supervised learning, typically using a pretrained LLM as the base model, and is then used to adapt the pretrained LLM to human preferences via additional fine-tuning. The training in this additional fine-tuning stage uses a flavor of reinforcement learning called **proximal policy optimization (PPO)**.

RLHF 混合监督与强化：人类排序或打分形成奖励信号，先训奖励模型，再以 **proximal policy optimization (PPO)** 等策略进一步优化策略模型。

在 RLHF 中，使用监督学习和强化学习相结合的方法，训练 LLM 以适应人类偏好。

- 人类反馈被收集，通过人类对不同模型输出的排序或评分，提供奖励信号。
- 收集到的奖励标签可以用于训练奖励模型，然后用于指导 LLM 适应人类偏好。
- 奖励模型通过监督学习学习，通常使用预训练 LLM 作为基础模型，然后用于进一步微调预训练 LLM 以适应人类偏好。
- 在额外的微调阶段，使用一种称为 **proximal policy optimization** 的强化学习方法进行训练。

RLHF uses a reward model instead of training the pretrained model on the human feedback directly because involving humans in the learning process would create a bottleneck since we cannot obtain feedback in realtime.

不用人类反馈直接当损失，是因为人在环无法实时、规模受限，故用奖励模型代行。

使用**奖励模型**而不是直接在人类反馈上训练预训练模型，因为涉及人类的学习过程会创建瓶颈，因为无法实时获得反馈。



Adapting Pretrained Language Models

本节收束全章：在多种适配路线之间做效率—效果权衡，并展望预训练模型用法演进。

While fine-tuning all layers of a pretrained LLM remains the gold standard for adaption to new target tasks, several efficient alternatives exist for leveraging pretrained transformers. For instance, we can effectively apply LLMs to new tasks while minimizing computational costs and resources by utilizing feature-based methods, in-context learning, or parameter-efficient fine-tuning techniques.

全层微调仍是强基准，但特征法、上下文学习与参数高效微调等可在算力受限时仍把 LLM 用到新任务上。

The three conventional methods -- feature-based approach, fine-tuning I, and fine-tuning II -- provide different computational efficiency and performance trade-offs. Parameter-efficient fine-tuning methods like soft prompt tuning, prefix tuning, and adapter methods further optimize the adaptation process, reducing the number of parameters to be updated. Meanwhile, RLHF presents an alternative approach to supervised fine-tuning, potentially improving modeling performance.

特征法 / Fine-tuning I / II 在效率与效果间取舍不同；软提示、前缀、Adapter 等进一步减需训参数；RLHF 则是另一条提升对齐与表现的路线。

三种传统方法 -- 特征基方法、微调 I 和微调 II -- 提供了不同的计算效率和性能权衡。

- 参数高效微调方法，如软提示词调优、前缀调优和适配器方法，进一步优化了适配过程，减少了需要更新的参数数量。
- RLHF 提供了一种替代监督微调的方法，可能提高建模性能。

In sum, the versatility and efficiency of pretrained LLMs continue to advance, offering new opportunities and strategies for effectively adapting these models to



a wide array of tasks and domains. As research in this area progresses, we can expect further improvements and innovations in using pretrained language models.

预训练语言模型的适配手段仍在快速演进，可期待后续新方法在任务覆盖面与效率上继续改进。

Exercises

本节习题对比 in-context 与微调适用场景，并思考参数高效微调如何缓解遗忘。

18-1. When does it make more sense to use in-context learning rather than fine-tuning, and vice versa?

18-2. In prefix tuning, adapters, and LoRA, how can we ensure that the model preserves (and does not forget) the original knowledge?

References

本节列出 GPT-2/3、提示与索引库、软/前缀提示、Adapter、LoRA、PEFT 综述、InstructGPT 与 PPO 等文献与资源。

- The paper introducing the GPT-2 model: Alec Radford et al., "Language Models Are Unsupervised Multitask Learners"? (2019), [https://www.semanticscholar.org/paper/Language-Models-are-Unsupervised - Multitask-Learners-Radford-Wu/9405cc0d6169988371b2755e573](https://www.semanticscholar.org/paper/Language-Models-are-Unsupervised-Multitask-Learners-Radford-Wu/9405cc0d6169988371b2755e573).
- The paper introducing the GPT-3 model: Tom B. Brown et al., "Language Models Are Few-Shot Learners"? (2020), <https://arxiv.org/abs/2005.14165>.
- The automatic prompt engineering method, which proposes using another LLM for automatic prompt generation and evaluation: Yongchao Zhou et al., "Large Language Models Are Human-Level Prompt Engineers"? (2023), <https://arxiv.org/abs/2211.01910>.



- LlamaIndex is an example of an indexing approach that leverages in-context learning: https://github.com/jerryliu/llama_index.
- DSPy is a popular open source library for retrieval augmentation and indexing: <https://github.com/stanfordnlp/dsp>.
- A first instance of soft prompting: Brian Lester, Rami Al-Rfou, and Noah Constant, "The Power of Scale for Parameter-Efficient Prompt Tuning"? (2021), <https://arxiv.org/abs/2104.08691>.
- The paper that first described prefix tuning: Xiang Lisa Li and Percy Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation"? (2021), <https://arxiv.org/abs/2101.00190>.
- The paper introducing the original adapter method: Neil Houlsby et al., "Parameter-Efficient Transfer Learning for NLP"? (2019) <https://arxiv.org/abs/1902.00751>.
- The paper introducing the LoRA method: Edward J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models"? (2021), <https://arxiv.org/abs/2106.09685>.
- A survey of more than 40 research papers covering parameter- efficient fine-tuning methods: Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky, "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning"? (2023), <https://arxiv.org/abs/2303.15647>.
- The InstructGPT paper: Long Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback"? (2022), <https://arxiv.org/abs/2203.02155>.
- Proximal policy optimization, which is used for reinforcement learning with human feedback: John Schulman et al., "Proximal Policy Optimization Algorithms"? (2017), <https://arxiv.org/abs/1707.06347>.



Chapter 19: Evaluating Generative Large Language Models

本章区分内生/外生指标，系统讲解困惑度、BLEU、ROUGE、BERTScore 及代理指标的定位，并附参考文献。

What are the standard metrics for evaluating the quality of text generated by large language models, and why are these metrics useful?

评估生成文本时，常需在昂贵主观的人工判决与可复现的自动分数之间做折中；Perplexity、BLEU、ROUGE、BERTScore 等即为代表性自动指标。

Perplexity , BLEU , ROUGE , and BERTScore are some of the most common evaluation metrics used in natural language processing to assess the performance of LLMs across various tasks. Although there is ultimately no way around human quality judgments, human evaluations are tedious, expensive, hard to automate, and subjective. Hence, we develop metrics to provide objective summary scores to measure progress and compare different approaches.

最终，还是需要人工评估，但是，人工评估，费时费力，且主观性较强。因此，需要开发一些客观的评估指标，来衡量模型的性能。

This chapter discusses the difference between intrinsic and extrinsic performance metrics for evaluating LLMs, and then it dives deeper into popular metrics like BLEU, ROUGE, and BERTScore and provides simple hands-on examples for illustration purposes.

本章先厘清内生（intrinsic）与外生（extrinsic）评估，再深入 BLEU、ROUGE、BERTScore 并给直观小例。

本章，主要介绍，如何使用 Perplexity , BLEU , ROUGE , and BERTScore 来评估 LLM 的性能。

Evaluation Metrics for LLMs

本节总述困惑度、BLEU、ROUGE 的含义，并用分类任务的交叉熵类比说明内生/外生指标的分工。



The **perplexity metric** is directly related to the loss function used for pretraining LLMs and is commonly used to evaluate text generation and text completion models. Essentially, it quantifies the average uncertainty of the model in predicting the next word in a given context -- the lower the perplexity, the better.

困惑度，是衡量模型在给定上下文下，预测下一个词的平均不确定性。困惑度越低，模型越好。

The **bilingual evaluation understudy (BLEU)** score is a widely used metric for evaluating the quality of machine-generated translations. It measures the overlap of n-grams between the machine-generated translation and a set of human-generated reference translations. A higher BLEU score indicates better performance, ranging from 0 (worst) to 1 (best).

BLEU 是评估机器翻译质量的常用指标，它衡量机器翻译与一组人工翻译的 n-gram 重叠程度。BLEU 得分越高，表示翻译质量越好，得分范围从 0（最差）到 1（最佳）。

The **recall-oriented understudy for gisting evaluation (ROUGE)** score is a metric primarily used for evaluating automatic summarization (and sometimes machine translation) models. It measures the overlap between the generated summary and reference summaries.

ROUGE 是评估自动摘要质量的常用指标，它衡量生成摘要与一组人工摘要的重叠程度。

We can think of perplexity as an **intrinsic metric** and BLEU and ROUGE as **extrinsic metrics**. To illustrate the difference between the two types of metrics, think of optimizing the conventional cross entropy to train an image classifier. The cross entropy is a loss function we optimize during training, but our end goal is to maximize the classification accuracy. Since classification accuracy cannot be optimized directly during training, as it's not differentiable, we



minimize the surrogate loss function like the cross entropy. Minimizing the cross entropy loss more or less correlates with maximizing the classification accuracy.

可把困惑度看作与预训练目标更贴近的内生指标，把 BLEU/ROUGE 看作更贴近下游任务表现的外生指标；这与分类里用可微的交叉熵去逼近不可微的准确率相类似。

我们可以将困惑度视为一个 **内生指标**，而 BLEU 和 ROUGE 视为 **外生指标**。

- 为了说明这两种指标之间的差异，可以考虑优化传统的交叉熵来训练图像分类器。
- 交叉熵是一个在训练过程中优化的损失函数，但我们的最终目标是最大化分类准确率。
- 由于分类准确率不能在训练过程中直接优化，因为它不是可微的，因此我们最小化替代损失函数，如交叉熵。
- 最小化交叉熵损失，与最大化分类准确率相关。

Perplexity is often used as an evaluation metric to compare the performance of different language models, but it is not the optimization target during training. BLEU and ROUGE are more related to classification accuracy, or rather precision and recall. In fact, BLEU is a precision-like score to evaluate the quality of a translated text, while ROUGE is a recall-like score to evaluate summarized texts.

困惑度常用于横向比较语言模型却未必是训练损；BLEU 更偏“译出词在参考里命中多少”的精确率味道，ROUGE 在摘要里更偏召回味道。

困惑度，通常用于评估不同语言模型的性能，但不是训练的目标。

- BLEU 和 ROUGE 与分类准确率相关，或者更准确地说，与精确率和召回率相关。
- 事实上，BLEU 是一个类似于精确率的评分，用于评估翻译文本的质量，而 ROUGE 是一个类似于召回率的评分，用于评估摘要文本的质量。

The following sections discuss the mechanics of these metrics in more detail.

下面分节展开各指标的计算要点。



Perplexity

本节把困惑度与训练中的交叉熵联系起来，给出定义、对数底的注意点，以及按 token 概率手算示例与代码位置。

Perplexity is closely related to the **cross entropy** directly minimized during training, which is why we refer to perplexity as an *intrinsic metric*.

Perplexity is defined as $2^{H(p,q)/n}$, where $H(p, q)$ is the **cross entropy** between the true distribution of words p and the predicted distribution of words q , and n is the sentence length (the number of words or tokens) used to normalize the score. As cross entropy decreases, perplexity also decreases—the lower the perplexity, the better. While we typically compute cross entropy using a natural logarithm, we calculate cross entropy and perplexity with a base-2 logarithm for the intuitive relationship to hold. (However, whether we use a base-2 or natural logarithm is only a minor implementation detail.)

定义上困惑度由交叉熵与长度归一化导出；为与比特含义一致，书中采用以 2 为底的对数，细节实现上也可换自然对数。

困惑度，与交叉熵直接相关，因此我们将其称为内生指标。FIXME: 交叉熵损失，不理解 ???

In practice, since the probability for each word in the target sentence is always 1, we compute the cross entropy as the logarithm of the probability scores returned by the model we want to evaluate. In other words, if we have the predicted probability score for each word in a sentence s , we can compute the perplexity directly as follows:

实践上对真实词位置取模型给出的概率并取对数平均，即可由公式直接得句级困惑度。

$$\text{Perplexity}(s) = 2^{-\frac{1}{n} \log_2(p(s))}$$

where s is the sentence or text we want to evaluate, such as "The quick brown fox jumps over the lazy dog," $p(s)$ is the probability scores returned by the model, and n is the number of words or tokens. For example, if the model returns the



probability scores $[0.99, 0.85, 0.89, 0.99, 0.99, 0.99, 0.99, 0.99]$, the perplexity is:

$$\begin{aligned} & 2^{-\frac{1}{8} \cdot \sum_i \log_2 p(w_i)} \\ &= 2^{-\frac{1}{8} \cdot \sum \log_2 (0.99 \times 0.85 \times 0.89 \times 0.99 \times 0.99 \times 0.99 \times 0.99 \times 0.99)} \\ &= 1.043 \end{aligned}$$

If the sentence was "The fast black cat jumps over the lazy dog," with probabilities $[0.99, 0.65, 0.13, 0.05, 0.21, 0.99, 0.99, 0.99]$, the corresponding perplexity would be 2.419.

换一组更低置信的词概率，困惑度即升高，符合“模型更不确定则分数更差”的直觉。

You can find a code implementation and example of this calculation in the *supplementary/q19-evaluation-llms* subfolder at <https://github.com/rasbt/MachineLearning-QandAI-book>.

配套实现见仓库 *supplementary/q19-evaluation-llms* 子目录。

BLEU Score

本节解释 BLEU 的参考译文设定、一元简化与 brevity penalty，并结合图 19.1；随后讨论其局限与可用场景。

BLEU is the most popular and most widely used metric for evaluating translated texts. It's used in almost all LLMs capable of translation, including popular tools such as OpenAI's Whisper and GPT models.

BLEU 几乎成为机器翻译评测的默认选项之一，许多含翻译能力的 LLM 与工具亦以其为参照。

BLEU is a reference-based metric that compares the model output to human-generated references and was first developed to capture or automate the essence of human evaluation. In short, BLEU measures the lexical overlap between the model output and the human-generated references based on a precision score.

BLEU 是一个基于精确率的指标，它衡量模型输出与人工生成参考之间的词汇重叠程度。



In more detail, as a precision-based metric, BLEU counts how many words in the generated text (candidate text) occur in the reference text divided by the candidate text length (the number of words), where the reference text is a sample translation provided by a human, for example. This is commonly done for n-grams rather than individual words, but for simplicity, we will stick to words or 1-grams. (In practice, BLEU is often computed for 4-grams.)

更细地说是对 n-gram（常为 4-gram）做裁剪计数并归一，再与 brevity penalty 等组合；此处先用 1-gram 示意。

Figure [19.1](#) demonstrates the BLEU score calculation, using the example of calculating the 1-gram BLEU score. The individual steps in Figure [19.1](#) illustrate how we compute the 1-gram BLEU score based on its individual components, the weighted precision times a brevity penalty. You can also find a code implementation of this calculation in the [supplementary/q15-text-augment](#) subfolder at <https://github.com/rasbt/MachineLearning-QandAI-book>.

图 19.1 分解 1-gram BLEU；同款计算亦可参考 [supplementary/q15-text-augment](#) 中的代码。

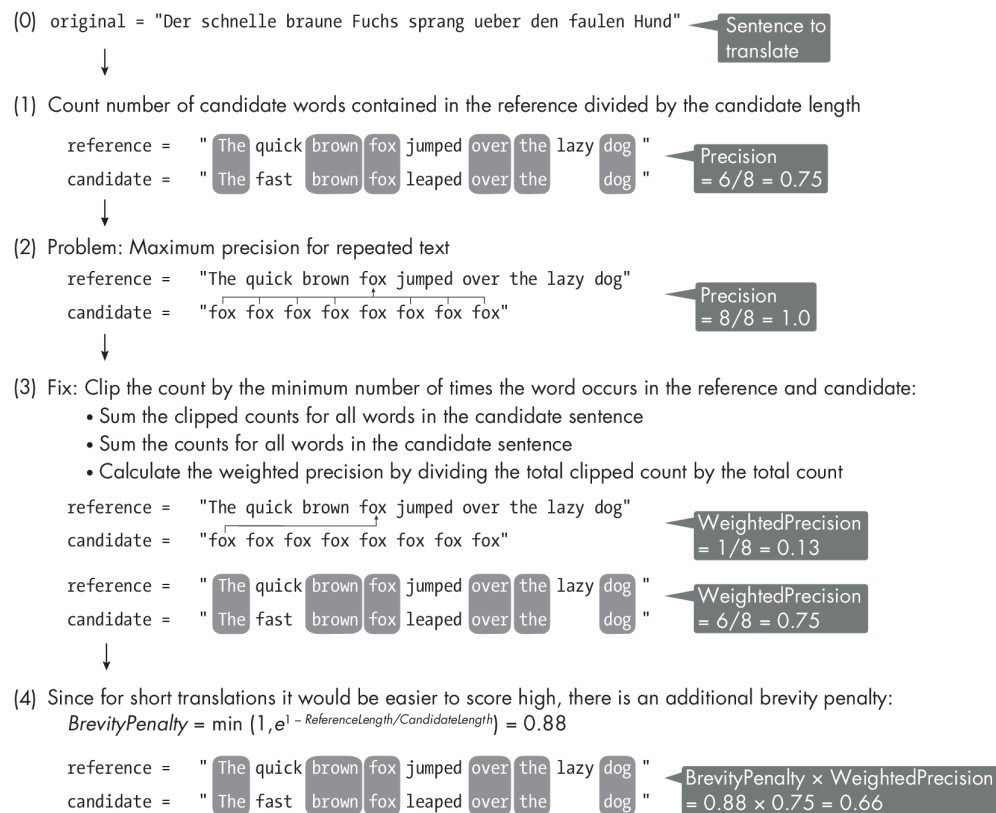


Figure 19.1

BLEU has several shortcomings, mostly owing to the fact that it measures string similarity, and similarity alone is not sufficient for capturing quality. For instance, sentences with similar words but different word orders might still score high, even though altering the word order can significantly change the meaning of a sentence and result in poor grammatical structure. Furthermore, since BLEU relies on exact string matches, it is sensitive to lexical variations and is incapable of identifying semantically similar translations that use synonyms or paraphrases. In other words, BLEU may assign lower scores to translations that are, in fact, accurate and meaningful.

BLEU 本质是字符串重叠，易忽略词序与句法，也对同义改写不敏感，可能低估事实正确的好译文。

BLEU 有以下几个缺点，主要是由于它衡量的是字符串相似度，而相似度本身不足以捕捉质量。

- 例如，具有相似单词但不同词序的句子可能得分很高，即使改变词序可能会显著改变句子的意义并导致糟糕的语法结构。
- 此外，由于 BLEU 依赖于精确的字符串匹配，它对词汇变体很敏感，无法识别使用同义词或释义的语义相似翻译。

- 换句话说，BLEU 可能会给那些实际上准确且有意义的翻译，分配较低的分。



The original BLEU paper found a high correlation with human evaluations, though this was disproven later.

早期论文称 BLEU 与人工高度相关，后续研究对此有所质疑。

原始的 BLEU 论文发现与人工评估有很高的相关性，尽管后来被证明是错误的。

Is BLEU flawed? Yes. Is it still useful? Also yes. BLEU is a helpful tool to measure or assess whether a model improves during training, as a proxy for fluency. However, it may not reliably give a correct assessment of the quality of the generated translations and is not well suited for detecting issues. In other words, it's best used as a model selection tool, not a model evaluation tool.

它仍适合作为训练曲线上的粗粒度监控或模型选择 proxy，不宜单独当作翻译“真实质量”的终审。

尽管 BLEU 有上述缺点，但它仍然是有用的，可以作为**流畅性**的代理来衡量或评估模型在训练中的改进。然而，它可能无法可靠地评估生成翻译的质量，并且不适合检测问题。换句话说，它最好用作模型选择工具，而不是模型评估工具。

At the time of writing, the most popular alternatives to BLEU are METEOR and COMET (see the References section at the end of this chapter for more details).

同期更常讨论的改进型替代指标包括 METEOR、COMET 等（见章末文献）。

目前，最流行的替代 BLEU 的指标是 METEOR 和 COMET（见本章末尾的 "METEOR 和 COMET" 部分，了解更多细节）。

ROUGE Score

本节说明 ROUGE 在摘要中的角色、与 BLEU 的“翻转”关系、F1 式现代实现及 ROUGE-N/L/S 变体与社区现状。



While BLEU is commonly used for translation tasks, ROUGE is a popular metric for scoring text summaries.

BLEU 通常用于翻译任务，但 ROUGE 是一个流行的评分文本摘要的指标。

There are many similarities between BLEU and ROUGE. The precision-based BLEU score check shows how many words in the candidate translation occur in the reference translation. The ROUGE score also takes a flipped approach, checking how many words in the reference text appear in the generated text (here typically a summarization instead of a translation); this can be interpreted as a recall-based score.

二者都做 n-gram 重叠，但 BLEU 更看候选命中参考的比例（偏精），ROUGE 常更看参考被候选覆盖多少（偏召）。

现代的实现将 ROUGE 计算为 F1 分数，这是召回率（参考文本中出现多少个词）和精确率（候选文本中出现多少个词）的调和平均值。

Modern implementations compute ROUGE as an **F1** score that is the harmonic mean of recall (how many words in the reference occur in the candidate text) and precision (how many words in the candidate text occur in the reference text). For example, Figure 19.2 shows a 1-gram ROUGE score computation (though in practice, ROUGE is often computed for bigrams, that is, 2-grams).

现实现多用 F1；图 19.2 仍用 1-gram 演示，实务上常见 2-gram。

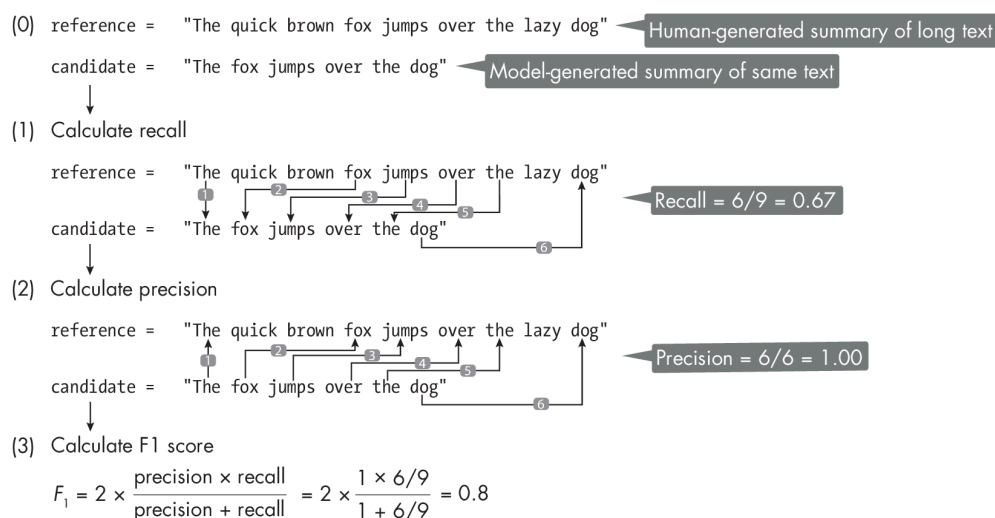


Figure 19.2



There are other ROUGE variants beyond ROUGE-1 (the F1 score--based ROUGE score for 1-grams):

- **ROUGE-N** Measures the overlap of n-grams between the candidate and reference summaries. For example, ROUGE-1 would look at the overlap of individual words (1-grams), while ROUGE-2 would consider the overlap of 2-grams (bigrams).
- **ROUGE-L** Measures the longest common subsequence (LCS) between the candidate and reference summaries. This metric captures the longest co-occurring in-order subsequence of words, which may have gaps in between them.
- **ROUGE-S** Measures the overlap of *skip-bigrams*, or word pairs with a flexible number of words in between them. It can be useful to capture the similarity between sentences with different word orderings.

除 ROUGE-1（基于 F1 的一元版）外，还有多种变体：ROUGE-N 度量 n-gram 重合；ROUGE-L 用最长公共子序列；ROUGE-S 用 skip-bigram 放松词序约束；可归为上述三大常见家族。

ROUGE shares similar weaknesses with BLEU. Like BLEU, ROUGE does not account for synonyms or paraphrases. It measures the n-gram overlap between the candidate and reference summaries, which can lead to lower scores for semantically similar but lexically different sentences. However, it's still worth knowing about ROUGE since, according to a study, *all* papers introducing new summarization models at computational linguistics conferences in 2021 used it, and 69 percent of those papers used *only* ROUGE.

ROUGE 与 BLEU 有类似的缺点。与 BLEU 一样，ROUGE 不考虑同义词或释义。它衡量候选文本和参考文本的 n-gram 重叠，这可能导致语义相似但词汇不同的句子的得分较低。然而，了解 ROUGE 仍然是有价值的，因为根据一项研究，2021 年计算语言学会议上介绍的所有新摘要模型都使用了它，其中 69% 的论文只使用了 ROUGE。

BERTScore

本节逐步给出 BERTScore：子词嵌入、逐 token 余弦对齐与平均，并比较其与 n-gram 指标及人工判决的互补关系。



Another more recently developed extrinsic metric is **BERTScore**.

另一类较新的外生自动指标是 **BERTScore**。

For readers familiar with the inception score for generative vision models, BERTScore takes a similar approach, using embeddings from a pretrained model (for more on embeddings, see Chapter [\[ch01\]](#)). Here, BERTScore measures the similarity between a candidate text and a reference text by leveraging the contextual embeddings produced by the BERT model (the encoder-style transformer discussed in Chapter [\[ch17\]](#)).

思路类似“用深度的表示比较分布”：借助第 17 章所述 BERT 式编码器得到上下文向量，再度量候选与参考的相似度（embedding 基础见第 1 章）。

The steps to compute BERTScore are as follows:

计算流程可概括为以下六步：

1. Obtain the candidate text via the LLM you want to evaluate (PaLM, LLaMA, GPT, BLOOM, and so on).
2. Tokenize the candidate and reference texts into subwords, preferably using the same tokenizer used for training BERT.
3. Use a pretrained BERT model to create the embeddings for all tokens in the candidate and reference texts.
4. Compare each token embedding in the candidate text to all token embeddings in the reference text, computing their cosine similarity.
5. Align each token in the candidate text with the token in the reference text that has the highest cosine similarity.
6. Compute the final BERTScore by taking the average similarity scores of all tokens in the candidate text.

从待评模型得到候选句，与参考句用（理想情况下与 BERT 训练一致的）分词器切成子词；用预训练 BERT 编码全体子词，再为候选每个位置在参考中找余弦最



邻并平均得分。

Figure 19.3 further illustrates these six steps. You can also find a computational example in the `subfolder/q15-text-augment` subfolder at <https://github.com/rasbt/MachineLearning-QandAI-book>.

图 19.3 逐步对应上述流程；数值例亦见 `subfolder/q15-text-augment`。

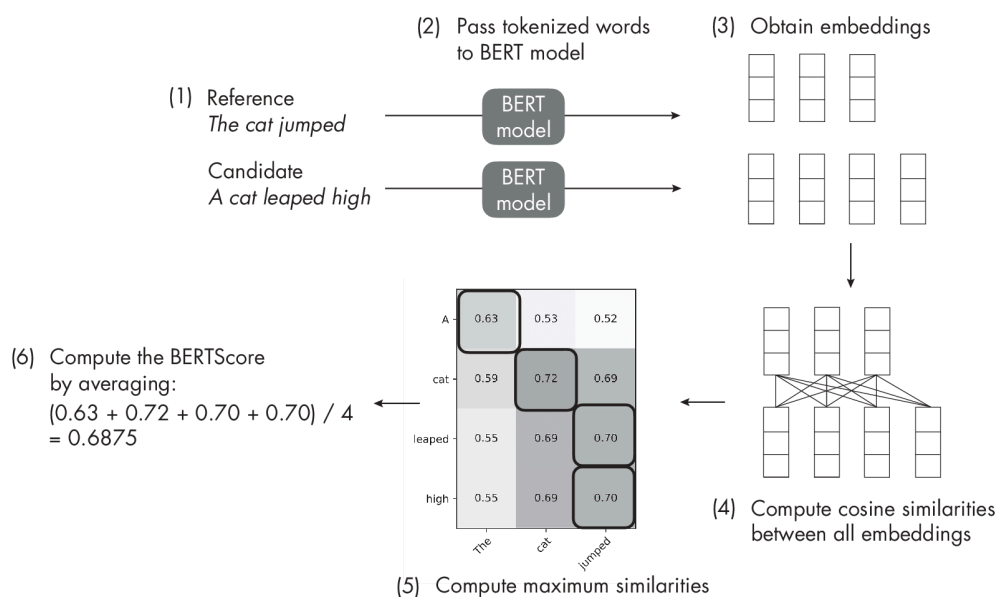


Figure 19.3

BERTScore can be used for translations and summaries, and it captures the semantic similarity better than traditional metrics like BLEU and ROUGE. However, BERTScore is more robust in paraphrasing than BLEU and ROUGE and captures semantic similarity better due to its contextual embeddings. Also, it may be computationally more expensive than BLEU and ROUGE, as it requires using a pretrained BERT model for the evaluation. While BERTScore provides a useful automatic evaluation metric, it's not perfect and should be used alongside other evaluation techniques, including human judgment.

对翻译与摘要都可用，凭借上下文嵌入对释义更友好，但要额外跑 BERT，成本高于 n-gram 指标，仍应与人工与其它自动方法并用。

Surrogate Metrics

本节收束：自动指标是对人类有用性与任务目标的代理；引用 InstructGPT 论文说明公开评测集与真实产品用法之间的错位。



All metrics covered in this chapter are surrogates or proxies to evaluate how useful the model is in terms of measuring how well the model compares to human performance for accomplishing a goal. As mentioned earlier, the best way to evaluate LLMs is to assign human raters who judge the results. However, since this is often expensive and not easy to scale, we use the aforementioned metrics to estimate model performance. To quote from the InstructGPT paper "Training Language Models to Follow Instructions with Human Feedback": "Public NLP datasets are not reflective of how our language models are used ... [They] are designed to capture tasks that are easy to evaluate with automatic metrics."

人类判决是金标准但难扩展，本章指标都是在近似这一标准；InstructGPT 一文亦指出公开 NLP 集多服务于“易自动化评估”而非真实部署形态。

这些指标只是对模型实际表现的近似衡量。正如InstructGPT论文所说，公开的NLP数据集并不能真实反映模型的实际应用场景，它们主要用于便于自动化指标评估的任务。

Besides perplexity, ROUGE, BLEU, and BERTScore, several other popular evaluation metrics are used to assess the predictive performance of LLMs.

除上述四者外，评估生成/判别式 LLM 还有大量其它流行指标可从综述与文献中延伸阅读。

Exercises

本节习题触及 BERTScore 对齐细节与非对称性等实现层面的思考题。

19-1. In step 5 of Figure [19.3](#), the cosine similarity between the two embeddings of "cat"? is not 1.0, where 1.0 indicates a maximum cosine similarity. Why is that?

19-2. In practice, we might find that the BERTScore is not symmetric. This means that switching the candidate and reference sentences could result in different BERTScores for specific texts. How could we address this?

References

本节收录 BLEU/ROUGE/BERTScore 原论文与批评文、评测综述，以及 METEOR、COMET、InstructGPT 等延伸引用。



- The paper proposing the original BLEU method: Kishore Papineni et al., "BLEU: A Method for Automatic Evaluation of Machine Translation"? (2002), <https://aclanthology.org/P02-1040/>.
- A follow-up study disproving BLEU's high correlation with human evaluations: Chris Callison-Burch, Miles Osborne, and Philipp Koehn, "Re-Evaluating the Role of BLEU in Machine Translation Research"? (2006), <https://aclanthology.org/E06-1032/>.
- The shortcomings of BLEU, based on 37 studies published over 20 years: Benjamin Marie, "12 Critical Flaws of BLEU"? (2022), https://medium.com/@bnjmn_marie/12-critical-flaws-of-bleu-1d790ccbe1b1.
- The paper proposing the original ROUGE method: Chin-Yew Lin, "ROUGE: A Package for Automatic Evaluation of Summaries"? (2004), <https://aclanthology.org/W04-1013/>.
- A survey on the usage of ROUGE in conference papers: Sebastian Gehrmann, Elizabeth Clark, and Thibault Sellam, "Repairing the Cracked Foundation: A Survey of Obstacles in Evaluation Practices for Generated Text"? (2022), <https://arxiv.org/abs/2202.06935>.
- BERTScore, an evaluation metric based on a large language model: Tianyi Zhang et al., "BERTScore: Evaluating Text Generation with BERT"? (2019), <https://arxiv.org/abs/1904.09675>.
- A comprehensive survey on evaluation metrics for large language models: Asli Celikyilmaz, Elizabeth Clark, and Jianfeng Gao, "Evaluation of Text Generation: A Survey"? (2021), <https://arxiv.org/abs/2006.14799>.
- METEOR is a machine translation metric that improves upon BLEU by using advanced matching techniques and aiming for better correlation with human judgment at the sentence level: Satanjeev Banerjee and Alon Lavie, "METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments"? (2005), <https://aclanthology.org/W05-0909/>.



- COMET is a neural framework that sets new standards for correlating machine translation quality with human judgments, using cross-lingual pretrained models and multiple types of evaluation: Ricardo Rei et al., "COMET: A Neural Framework for MT Evaluation"? (2020), <https://arxiv.org/abs/2009.09025>.
 - The InstructGPT paper: Long Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback"? (2022), <https://arxiv.org/abs/2203.02155>.
-



Chapter 20: Stateless and Stateful Training

本章在生产训练/部署语境下对比无状态（周期重训）与有状态（增量更新）两类流程，并各辅图示与习题。

What is the difference between stateless and stateful training workflows in the context of production and deployment systems?

无状态与有状态描述的是：新数据到来时，是周期性从零或近似从零重训，还是在既有权重上继续优化。

Stateless training and stateful training refer to different ways of training a production model.

Tips: 无状态训练 `stateless` 和有状态训练 `stateful`，是两种不同的训练方式；训练生产模型时，需要考虑用哪种方式。

Stateless (Re)training

本节说明无状态训练先训初版再在数据流上按窗口重训的图景，并点出与传统批量学习模型的自然契合。

In stateless training, the more conventional approach, we first train an `initial model` on the original training set and then retrain it as new data arrives. Hence, stateless training is also commonly referred to as stateless *retraining*.

常见做法是先在初始训练集上得到 `initial model`，新数据持续到达后按策略整批或按窗口再次训练，因此也称无状态 *retraining*。

Tips: 无状态训练，是先训练一个 `初始模型`，然后在新数据到达时，重新训练模型；可以简单认为是 `树状结构`，初始模型是 `父节点`、衍生出一堆 `叶子节点` 模型。



As Figure 20.1 shows, we can think of stateless retraining as a sliding window approach in which we retrain the initial model on different parts of the data from a given data stream.

可把它想象成对流式数据截取滑动窗口、周期性回到同一训练管线重训，使模型随时间对齐最新数据段。

Tips: 图示中，**初始模型** 是 **父节点**，**新模型** 是 **叶子节点**，**新模型** 是基于 **初始模型** 训练的；训练新模型时，会截取 **滑动窗口** 数据。

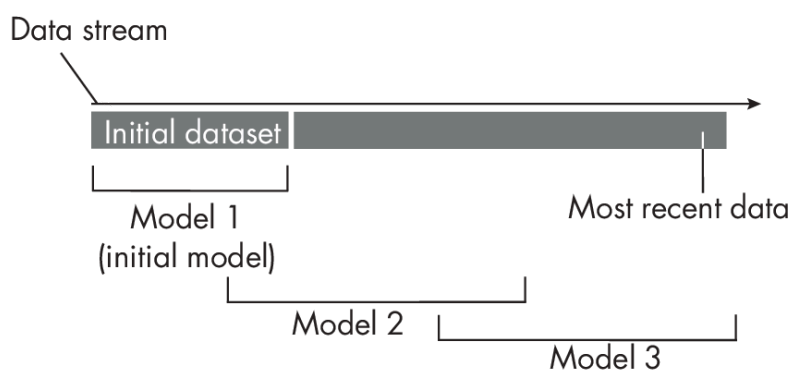


Figure 20.1

For example, to update the initial model in Figure 20.1 (Model 1) to a newer model (Model 2), we train the model on 30 percent of the initial data and 70 percent of the most recent data at a given point in time.

举例：从 Model 1 升到 Model 2 时，可在某一时刻混合 30% 早期样本与 70% 近期样本再训一版。

Stateless retraining is a straightforward approach that allows us to adapt the model to the most recent changes in the data and feature-target relationships via retraining the model from scratch in user-defined checkpoint intervals. This approach is prevalent with conventional machine learning systems that cannot be fine-tuned as part of a transfer or self-supervised learning workflow (see Chapter [ch02]).

做法直观：按固定节拍整体重训即可响应最新的特征-目标关系；它特别适合难以“增量更新权重”的传统监督管线（参见第 2 章关于迁移/自监督的讨论）。

Tips: 传统的模型，中无状态训练，比较流行，比如 **随机森林**、**梯度提升** 等，这些都是无法 **微调** 的



For example, standard implementations of tree-based models, such as random forests and gradient boosting (XGBoost, CatBoost, and LightGBM), fall into this category.

例如随机森林与各类梯度提升库的主流实现通常属于此类。

Stateful Training

本节描述有状态训练如何在初训后持续微调、并阐明其针对概念/特征/标签漂移与迁移学习“换任务”之间的本质不同。

In stateful training, we train the model on an initial batch of data and then update it periodically (as opposed to retraining it) when new data arrives.

有状态路线是首次批量训练后，新数据到来时在原参数附近继续更新，而非每次都完整重训。

Tips: 有状态的训练，可以认为是 **链式结构**，初始模型 -> 新模型 -> 新模型 -> ...，每次都基于最新模型叠加而来。

As illustrated in Figure 20.2, we do not retrain the initial model (Model1.0) from scratch; instead, we update or fine-tune it as new data arrives. This approach is particularly attractive for models compatible with transfer learning or self-supervised learning.

图 20.2 显示并不每次从 Model 1.0 彻底重算，而是在其权重上微调；这与支持迁移或自监督初始化的深度学习十分契合。



Figure 20.2

The stateful approach mimics a transfer or self-supervised learning workflow where we adopt a pretrained model for fine-tuning. However, stateful training differs fundamentally from transfer and self-supervised learning because it



updates the model to accommodate concept, feature, and label drifts. In contrast, transfer and self-supervised learning aim to adopt the model for a different classification task. For instance, in transfer learning, the target labels often differ. In self-supervised learning, we obtain the target labels from the dataset features.

表面上都像“拿一版预训练再微调”，但有状态强调同一部署任务下随时间的分布漂移（概念、特征、标注方式变化）；迁移与自监督则更常指换任务或从数据本身构造伪标签。

Tips: 有状态的训练，跟 **迁移学习**、**自监督学习**，有本质区别；有状态的训练，会更新模型，以适应概念、特征、标签的漂移；而迁移学习、自监督学习，是基于预训练模型，进行微调。

One significant advantage of stateful training is that we do not need to store data for retraining; instead, we can use it to update the model as soon as it arrives. This is particularly attractive when data storage is a concern due to privacy or resource limitations.

若不能长期囤积原始样本（合规或资源），有状态增量可以在样本流过当下立即用于更新，而不必保留全史做周期全量重训。

Tips: 有状态的训练，不需要存储数据，可以及时更新模型；这在 **隐私** 或 **资源有限** 的情况下，特别有用。

Exercises

本节通过金融日更随机森林与按月迭代大模型两题，巩固两类形态的算法与业务约束层面的选择。

20-1. Suppose we train a classifier for stock trading recommendations using a random forest model, including the moving average of the stock price as a feature. Since new stock market data arrives daily, we are considering how to update the classifier daily to keep it up to date. Should we take a stateful training or stateless retraining approach to update the classifier?

► Answer, Click to expand

20-2. Suppose we deploy a large language model (transformer) such as ChatGPT that can answer user queries. The dialogue interface includes thumbs-up and thumbs-down buttons so that users can give direct feedback based on the generated queries. While collecting the user feedback, we don't update the model immediately as new feedback arrives. However, we are planning to release a new or updated model at least once per month. Should we use stateless or stateful retraining for this model?





Chapter 21: Data-Centric AI

本章界定以数据迭代为核心的工作流，对比常见的模型驱动范式，并讨论适用场景、实践建议与练习。

What is data-centric AI, how does it compare to the conventional modeling paradigm, and how do we decide whether it's the right fit for a project?

何谓数据中心 AI，它与常规的建模范式有何不同，又应如何判断项目是否适合采用这一思路？

Data-centric AI is a paradigm or workflow in which we keep the model training procedure fixed and iterate over the dataset to improve the predictive performance of a model. The following sections define what data-centric AI means in more detail and compare it to conventional model-centric approaches.

数据中心 AI (data-centric AI) 是一种范式或工作流：固定模型训练流程，通过反复迭代数据集来提升模型的预测表现。下文将更具体地界定该含义，并把它与常见的**模型驱动 (model-centric)** 做法对照讨论。

Tips: **数据驱动 AI** 是一种范式或工作流，其中我们保持模型训练过程不变，通过**迭代数据集**来提高模型的预测性能。

Data-Centric vs. Model-Centric AI

本节对照学术发表场景中典型的模型驱动流程，解释“数据中心”这一说法的动机，说明二者如何互补，并列岀数据中心范式通常涵盖的技术要素。

In the context of data-centric AI, we can think of the conventional workflow, which is often part of academic publishing, as model-centric AI. However, in an academic research setting, we are typically interested in developing new methods (for example, neural network architectures or loss functions). Here, we consider existing benchmark datasets to compare the new method to previous approaches and determine whether it is an improvement over the status quo.



在数据中心 AI 的语境下，学界常见、以论文发表为主轴的那种流程，往往可视为**模型驱动 AI**：研究目标通常是提出新方法（例如新架构或新损失），并借助既有基准数据集与先前工作比较，判断是否真的带来提升。

Tips: 在 **数据驱动 AI** 的上下文中，我们可以将传统的学术届优化的模型视为 **模型驱动 AI**。

- 在学术研究环境中，我们通常对开发**新方法**（例如神经网络架构或损失函数）感兴趣。
- 我们考虑现有的**基准数据集**，以比较新方法与先前的方法，并确定它是否更优。

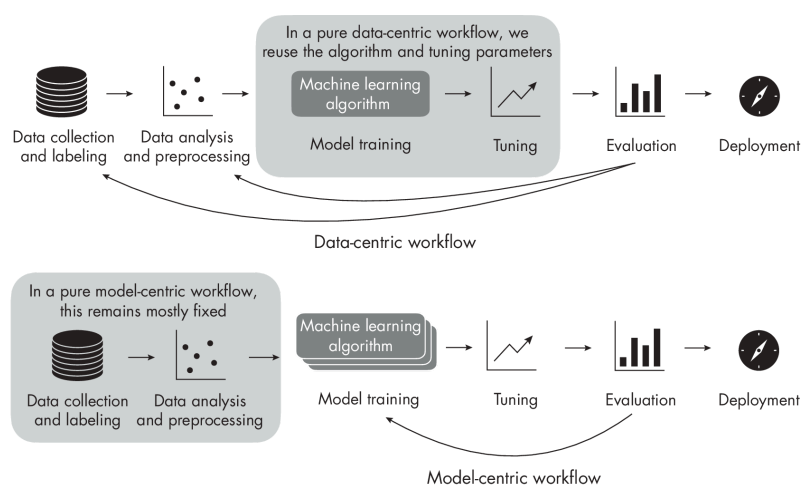


Figure 21.1

While *data-centric AI* is a relatively new term, the idea behind it is not. Many people I've spoken with say they used a data-centric approach in their projects before the term was coined. In my opinion, data-centric AI was created to make "caring about data quality"? attractive again, as data collection and curation are often considered tedious or thankless. This is analogous to how the term *deep learning* made neural networks interesting again in the early 2010s.

尽管**数据中心 AI**这个称谓较新，其背后的做法并不新鲜；许多从业者表示在术语流行前项目里就已这么干。作者认为，这一提法一定程度上是为了让“重视数据质量”重新变得有吸引力——采集与整理数据常被看作枯燥、默默无闻，却至关重要；这也类似**深度学习**一词曾在 2010 年代初让神经网络重新“被看见”。

Tips: 虽然 **数据驱动 AI** 是一个相对较新的术语，但其背后的理念并不新。



- 许多人告诉我，在他们使用 **数据驱动 AI** 之前，他们已经在项目中使用 **数据驱动 AI**。
- 在我看来，**数据驱动 AI** 的称呼，是为了让 **关注数据质量** 再次变得有吸引力，因为数据收集和整理通常被认为是繁琐或无意义的。
- 这类似于 **深度学习** 在2010年代初期让神经网络变得有趣起来。

Do we need to choose between data-centric and model-centric AI, or can we rely on both? In short, data-centric AI focuses on changing the data to improve performance, while model-centric approaches focus on modifying the model to improve performance. Ideally, we should use both in an applied setting where we want to get the best possible predictive performance. However, in a research setting or an exploratory stage of an applied project, working with too many variables simultaneously is messy. If we change both model and data at once, it's hard to pinpoint which change is responsible for the improvement.

不必二选一：数据中心路径强调改数据以提性能，模型驱动路径强调改模型以提性能；在追求极致预测表现的落地场景里，理想情况是两者兼用。但在科研或应用项目的探索期，同时动太多旋钮会让归因困难——若模型与数据一起改，很难判断提升来自哪一侧。

Tips: 我们是否需要在 **数据驱动 AI** 和 **模型驱动 AI** 之间做出选择，或者我们可以依赖两者？

- 简而言之，**数据驱动 AI** 专注于**改变数据**以提高性能，而 **模型驱动 AI** 专注于**修改模型**以提高性能。
- 理想情况下，我们应该同时使用两者。

It is important to emphasize that data-centric AI is a paradigm and workflow, not a particular technique. Data-centric AI therefore implicitly includes the following:

需要强调的是，数据中心 AI 是范式与工作流，而不是某一种孤立技巧；因此它通常隐含地包含下面几类工作：

- Analyses and modifications of training data, from outlier removal to missing data imputation
- Data synthesis and data augmentation techniques
- Data labeling and label-cleaning methods



- The classic active learning setting where a model suggests which data points to label
- 对训练数据的分析与改动：从剔除异常到缺失值填补；
- 数据合成与数据增强；
- 标注与标签清洗；
- 经典的主动学习：由模型提示下一步该标哪些样本。

Tips: 重要的是要强调，**数据驱动 AI** 是一种范式和流程，而不是一种特定的技术，隐含地包括以下内容：

- 训练数据的分析和修改，从异常值去除到缺失数据插补
- 数据合成和数据增强技术
- 数据标注和标签清理方法
- 经典主动学习设置，其中模型建议哪些数据点需要标注

We consider an approach *data centric* if we change only the data (using the methods listed here), not the other aspects of the modeling pipeline.

若我们只改动数据（采用此处列出的手段），而不动建模流水线中的其他环节，就可把这种作法视为**数据中心**。

In machine learning and AI, we often use the phrase "garbage in, garbage out" meaning that poor-quality data will result in a poor predictive model. In other words, we cannot expect a well-performing model from a low-quality dataset.

机器学习中常说“垃圾进、垃圾出”：数据质量差，模型也难好；低质数据集上很难指望得到可靠预测。

I've observed a common pattern in applied academic projects that attempt to use machine learning to replace an existing methodology. Often, researchers have only a small dataset of examples (say, hundreds of training examples). Labeling data is often expensive or considered boring and thus best avoided. In these cases, researchers spend an unreasonable amount of time trying out different machine-learning algorithms and model tuning. To resolve this issue, investing additional time or resources in labeling additional data would be worthwhile.



在试图用 ML 替换既有流程的应用型学术项目里，常见模式是：训练样本很少（例如仅数百条），标注又贵或被认为无聊于是一躲再躲，结果把大量时间耗在换算法与调参上；若能把部分精力转投到补充标注，往往更划算。

Tips:

- 通常，研究人员，只有一个 **小数据集**（例如，几百个训练样本）。
- **标注数据**通常很 **昂贵** 或被认为很 **无聊**，因此最好避免。
- 在这些情况下，研究人员**花费了不合理的时间**来尝试不同的机器学习算法和模型调优。
- 为了解决这个问题，投资额外的时间或资源来 **标注更多的数据** 将是有价值的。

The main advantage of data-centric AI is that it puts the data first so that if we invest resources to create a higher-quality dataset, all modeling approaches will benefit from it downstream.

数据中心 AI 的主要好处在于“数据优先”：一旦把资源投入到更高质量的数据集上，后续的各类建模路径都能共享这份红利。

Tips: 数据驱动 AI 的主要优势在于，它将数据放在首位，因此如果我们投资资源来创建更高质量的数据集，所有建模方法都将从中受益。

Recommendations

本节从应用型项目与方法论研究两类目标出发，说明何时优先迭代数据、何时宜固定基准数据改模型，并给出二者交替推进的节奏。

Taking a data-centric approach is often a good idea in an applied project where we want to improve the predictive performance to solve a particular problem. In this context, it makes sense to start with a modeling baseline and improve the dataset since it's often more worthwhile than trying out bigger, more expensive models.

在面向具体问题的应用项目里，若目标是提升预测效果，采用数据中心思路往往划算：先立建模基线，再优先迭代数据，常常比一味换更大、更贵模型更有



效。

If our task is to develop a new or better methodology, such as a new neural network architecture or loss function, a model-centric approach might be a better choice. Using an established benchmark dataset without changing it makes it easier to compare the new modeling approach to previous work. Increasing the model size usually improves performance, but so does the addition of training examples. Assuming small training sets ($< 2k$) for classification, extractive question answering, and multiple-choice tasks, adding a hundred examples can result in the same performance gain as adding billions of parameters.

若任务是提出新架构、新损失等**方法学**创新，模型驱动可能更合适：沿用公认基准、少动数据，便于与文献公平比较。扩大模型规模通常能涨分，增加训练样本也能；在分类、抽取式问答、选择题等任务上，若训练集很小 ($< 2k$)，有时**多加百来条样本**带来的增益可与**多加数十亿参数**相当。

In a real-world project, alternating between data-centric and model-centric modes makes a lot of sense. Investing in data quality early on will benefit all models. Once a good dataset is available, we can begin to focus on model tuning to improve performance.

真实项目里在两种模式间轮流推进很合理：早期投资数据质量能普惠所有模型；数据集相对稳定后，再侧重调参与结构优化以榨取性能。

Tips: 在实际项目中，交替使用 **数据驱动** 和 **模型驱动** 模式是有意义的。

- 早期投资于 **数据质量** 将使所有模型受益。
- 一旦有了一个好的数据集，我们可以开始专注于 **模型调优** 以提高性能。

Exercises

本节以两道情景题，检验读者能否识别数据中心界定以及数据增强是否属于仅改数据的范畴。

21-1. A recent trend is the increased use of predictive analytics in healthcare. For example, suppose a healthcare provider develops an AI system that analyzes patients' electronic health records and provides recommendations for lifestyle



changes or preventive measures. For this, the provider requires patients to monitor and share their health data (such as pulse and blood pressure) daily. Is this an example of data-centric AI?

21-1. 医疗领域越来越多地用预测分析。例如某机构开发 AI 分析电子病历并给出生活方式或预防建议，为此要求患者每日上报脉搏、血压等数据。这是否属于数据中心 AI 的例子？

21-2. Suppose we train a ResNet-34 convolutional neural network to classify images in the CIFAR-10 and ImageNet datasets. To reduce overfitting and improve classification accuracy, we experiment with data augmentation techniques such as image rotation and cropping. Is this approach data centric?

21-2. 假设用 ResNet-34 在 CIFAR-10 与 ImageNet 上做图像分类，为减轻过拟合并提准确率，尝试旋转、裁剪等数据增强。这算数据中心做法吗？

References

本节提供“样本增益可比拟扩模型”的代表性论文链接，以及用于标签与数据质量改进的 Cleanlab 开源库。

- An example of how adding more training data can benefit model performance more than an increase in model size: Yuval Kirstain et al., "A Few More Examples May Be Worth Billions of Parameters"? (2021), <https://arxiv.org/abs/2110.04374>.
- 展示“增加训练数据对模型表现的助益可超过单纯加大模型规模”的论文：Yuval Kirstain 等，《A Few More Examples May Be Worth Billions of Parameters?》(2021)，<https://arxiv.org/abs/2110.04374>。
- Cleanlab is an open source library that includes methods for improving labeling errors and data quality in computer vision and natural language processing contexts: <https://github.com/cleanlab/cleanlab>.
- Cleanlab 是开源库，提供改进标注错误与数据质量的方法，覆盖视觉与 NLP 等场景：<https://github.com/cleanlab/cleanlab>。





Chapter 22: Speeding Up Inference

本章在不改动模型架构、不牺牲精度的前提下，梳理推理加速的通用手段：并行、向量化、循环分块、算子融合与量化，并说明各自要点与边界。

What are techniques to speed up model inference through optimization without changing the model architecture or sacrificing accuracy?

在不改变模型结构、也不以牺牲精度为代价的前提下，有哪些优化手段可以加快推理（inference）？

In machine learning and AI, *model inference* refers to making predictions or generating outputs using a trained model. The main general techniques for improving model performance during inference include parallelization, vectorization, loop tiling, operator fusion, and quantization, which are discussed in detail in the following sections.

在机器学习与 AI 中，*模型推理*指用已训练模型做预测或生成输出。推理阶段常见的通用加速思路包括并行化、向量化、循环分块（loop tiling）、算子融合与量化等，下文逐节展开。

Tips:

- 优化模型**推理速度**，有多种方法，包括：**并行化**、**向量化**、**循环分块**、**算子融合**、**量化**等。
- 这些方法，将在后续章节中详细讨论。

Parallelization

本节说明如何通过批量处理同时利用硬件并行，并以图示对比串行与批量推理的瓶颈差异。

One common way to achieve better parallelization during inference is to run the model on a batch of samples rather than on a single sample at a time. This is sometimes also referred to as *batched inference* and assumes that we are



receiving multiple input samples or user inputs simultaneously or within a short time window, as illustrated in Figure 22.1.

推理阶段提升并行度的一种常见做法，是对**一批样本**而非单条依次前向传播；这也常称为**批量推理** (*batched inference*)，前提是在同一时间或很短时间窗内会汇聚多条请求，见图 22.1。

Tips: 并行化，也被称为 **批量推理**，同时或短时间窗口内，接收到多个输入样本或用户输入，模型同时处理。

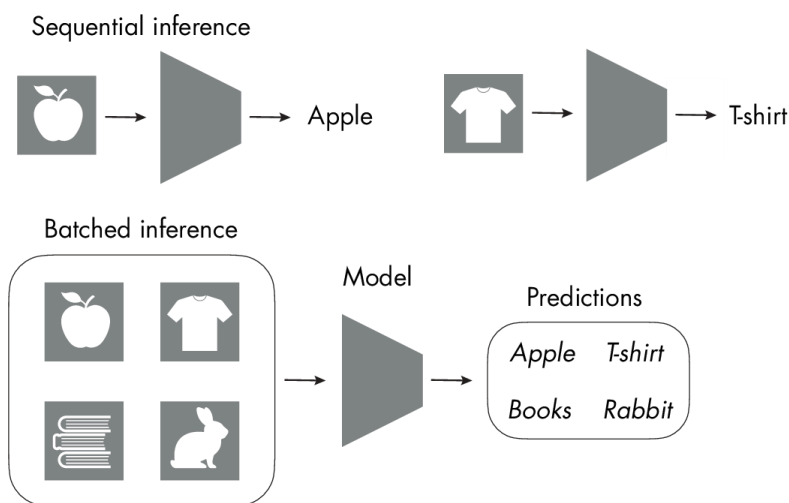


Figure 22.1

Figure 22.1 shows sequential inference processing one item at a time, which creates a bottleneck if there are several samples waiting to be classified. In batched inference, the model processes all four samples at the same time.

图 22.1 表明：逐条串行推理会在多条样本排队时形成瓶颈；而批量推理可让模型一次性处理图中的四条输入。

Vectorization

本节解释向量化的含义、与 BLAS 的关系，并说明在 TensorFlow、PyTorch 等框架中张量运算如何自动受益。

Vectorization refers to performing operations on entire data structures, such as arrays (tensors) or matrices, in a single step rather than using iterative constructs



like `for` loops. Using vectorization, multiple operations from the loop are performed simultaneously using single instruction, multiple data (SIMD) processing, which is available on most modern CPUs.

向量化指对数组（张量）、矩阵等整体一次完成运算，而不是用 `for` 循环逐元素迭代；底层往往借助 CPU 上的 SIMD（单指令多数据）把循环中的多次操作并行完成。

Tips: 向量化，也被称为 **单指令多数据**，在现代 CPU 上，可以同时处理多个数据。

This approach takes advantage of the low-level optimizations in many computing systems and often results in significant speedups. For example, it might rely on BLAS.

这种做法能吃到不少系统级低层优化，往往带来可观加速，例如可依赖 BLAS。

BLAS (which is short for *Basic Linear Algebra Subprograms*) is a specification that prescribes a set of low-level routines for performing common linear algebra operations such as vector addition, scalar multiplication, dot products, matrix multiplication, and others. Many array and deep learning libraries like NumPy and PyTorch use BLAS under the hood.

BLAS (Basic Linear Algebra Subprograms) 规定了一套底层例程，用于向量加、标量乘、点积、矩阵乘等常见线性代数运算；NumPy、PyTorch 等库在底层常会调用 BLAS。

To illustrate vectorization with an example, suppose we wanted to compute the dot product between two vectors. The non-vectorized way of doing this would be to use a `for` loop, iterating over each element of the array one by one. However, this can be quite slow, especially for large arrays. With vectorization, you can perform the dot product operation on the entire array at once, as shown in Figure [22.2](#).

举例：两向量点积若用 `for` 逐元素累加，大数组上会很慢；向量化则可对整体一次性完成点积运算，见图 [22.2](#)。



Classic for loop	<pre>x = [1.2, 2.2, 3.3, 4.4] w = [5.5, 6.6, 7.7, 8.8] output = 0. for x_j, w_j in zip(x, w): output += x_j * w_j print(output) 85.25</pre>
Vectorized implementation	<pre>import torch x = torch.tensor([1.2, 2.2, 3.3, 4.4]) w = torch.tensor([5.5, 6.6, 7.7, 8.8]) x.dot(w) tensor(85.2500)</pre>

Figure 22.2

In the context of linear algebra or deep learning frameworks like TensorFlow and PyTorch, vectorization is typically done automatically. This is because these frameworks are designed to work with multidimensional arrays (also known as *tensors*), and their operations are inherently vectorized. This means that when you perform functions using these frameworks, you automatically leverage the power of vectorization, resulting in faster and more efficient computations.

在线性代数或 TensorFlow、PyTorch 等深度学习框架里，向量化通常由框架自动完成：它们面向多维数组（*张量*）设计，算子本身即按向量化实现，因而调用高层 API 时往往已自动获得更快、更省的开销。

Loop Tiling

本节介绍循环分块如何改善缓存局部性，说明二维示例与在 Python 等高层语言中通常依赖底层库代劳的原因。

Loop tiling (also often referred to as *loop nest optimization*) is an advanced optimization technique to enhance data locality by breaking down a loop's iteration space into smaller chunks or "tiles." This ensures that once data is loaded into cache, all possible computations are performed on it before the cache is cleared.

循环分块 (*loop tiling*) (也常称循环嵌套优化) 通过把迭代空间划成较小的“块”，提升**数据局部性**：数据一旦进缓存，就尽量在同一块上把能算的都算



完，再被淘汰。

Tips: 循环分块，也被称为 **循环嵌套优化**，将循环的迭代空间分成小块，确保数据加载到缓存后，所有可能的计算都在缓存中完成，然后缓存被清除。

Figure 22.3 illustrates the concept of loop tiling for accessing elements in a two-dimensional array. In a regular **for** loop, we iterate over columns and rows one element at a time, whereas in loop tiling, we subdivide the array into smaller tiles.

图 22.3 用二维数组访问示意循环分块：常规双重循环逐元素扫；分块则把数组切成小 tile 依次处理。

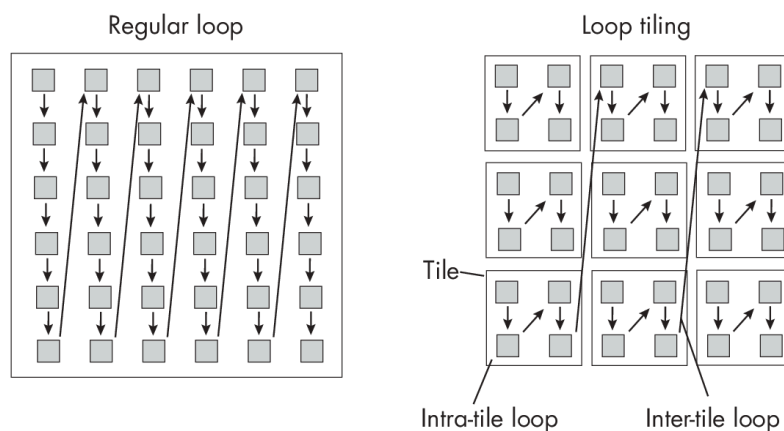


Figure 22.3

Note that in languages such as Python, we don't usually perform loop tiling, because Python and many other high-level languages do not allow control over cache memory like lower-level languages such as C and C++ do. These kinds of optimizations are often handled by underlying libraries like NumPy and PyTorch when performing operations on large arrays.

注意：在 Python 等语言里，应用层通常不会手写循环分块，因为对缓存的控制不如 C/C++ 那样直接；这类优化往往由 NumPy、PyTorch 等底层实现在大张量运算中完成。

Tips: 在 Python 等高级语言中，通常不进行循环分块，因为这些语言不提供对缓存内存的控制，如 C 和 C++ 等底层语言。这些优化通常由底层库（如 NumPy 和 PyTorch）在处理大型数组时自动处理。



Operator Fusion

本节说明算子融合如何减少循环与访存开销，并与循环分块、重参数化等概念区分与联系。

Operator fusion, sometimes called *loop fusion*, is an optimization technique that combines multiple loops into a single loop. This is illustrated in Figure 22.4, the product of an array of numbers are fused into a single loop.

算子融合 (*operator fusion*) 有时也称循环融合：把多段循环合成一段，图 22.4 示意将数组求积的多重循环并为单一循环。

Tips: 算子融合，也被称为 **循环融合**，将多个循环合并成一个循环。

<pre>numbers = [1, 2, 3, 4, 5] # First loop to calculate the sum total_sum = 0 for num in numbers: total_sum += num # Second loop to calculate the product product = 1 for num in numbers: product *= num print("Sum:", total_sum) print("Product:", product) Sum: 15 Product: 120</pre>	<pre>numbers = [1, 2, 3, 4, 5] # Single loop to calculate both # the sum AND the product total_sum = 0 product = 1 for num in numbers: total_sum += num product *= num print("Sum:", total_sum) print("Product:", product) Sum: 15 Product: 120</pre>
--	--

Figure 22.4

Operator fusion can improve the performance of a model by reducing the overhead of loop control, decreasing memory access times by improving cache performance, and possibly enabling further optimizations through vectorization. You might think this behavior of **vectorization** would be incompatible with **loop tiling**, in which we break a **for** loop into multiple loops.

算子融合可减少循环控制开销、改善缓存行为从而降访存成本，并可能为进一步向量化创造条件。乍看之下，向量化与把 **for** 拆成多段的循环分块似乎相冲突。

Tips: 算子融合，可以提高模型性能，通过减少循环控制的开销，提高缓存性能，并可能通过向量化进一步优化。



However, these techniques are actually complementary, used for different optimizations, and applicable in different situations. **Operator fusion** is about reducing the total number of loop iterations and improving data locality when the entire data fits into cache. **Loop tiling** is about improving cache utilization when dealing with larger multidimensional arrays that do not fit into cache.

但二者实为互补：融合侧重减少总迭代次数、在数据能落入缓存时改善局部性；分块则面对塞不进缓存的大张量，侧重提高缓存利用率。

Related to operator fusion is the concept of *reparameterization*, which can often also be used to simplify multiple operations into one. Popular examples include training a network with multibranch architectures that are reparameterized into single-stream architectures during inference. This reparameterization approach differs from traditional operator fusion in that it does not merge multiple operations into a single operation. Instead, it rearranges the operations in the network to create a more efficient architecture for inference. In the so-called RepVGG architecture, for example, each branch during training consists of a series of convolutions. Once training is complete, the model is reparameterized into a single sequence of convolutions.

与融合相关的是**重参数化** (*reparameterization*)：可把多步运算在推理时等价改写得更紧凑。典型例子是多分支结构训练、推理时并成单路；这与把多个算子合成“一个算子”的传统融合不同，而是**重排**计算图以得到更利于推理的结构。RepVGG 中，训练期每分支是一串卷积，训练结束后可重参为单序列卷积。

Tips: 重参数化，也被称为 **重参数化优化**，将多个操作合并成一个操作。

Quantization

本节说明量化如何降算力与存储需求、后训练量化与量化感知训练的区别，并交代为何严格“不伤精度”时量化与本章主题略有张力，以及蒸馏、剪枝等为何超出本章范围。

Quantization reduces the computational and storage requirements of machine learning models, particularly deep neural networks. This technique involves converting the floating-point numbers (technically discrete but representing continuous values within a specific range) for implementing weights and biases in



a trained neural network to more discrete, lower-precision representations such as integers. Using less precision reduces the model size and makes it quicker to execute, which can lead to significant improvements in speed and hardware efficiency during inference.

量化降低模型（尤其深度网络）的计算与存储成本：把表示权重与偏置的浮点（技术上离散但近似连续区间）映射到更低比特、更离散的表示（如整数）。精度位宽下降可缩小模型、加快执行，从而提升推理速度与硬件效率。

Tips: 量化，也被称为 **量化优化**，将浮点数转换为整数，减少模型大小和计算量，提高推理速度。

In the realm of deep learning, it has become increasingly common to quantize trained models down to 8-bit and 4-bit integers. These techniques are especially prevalent in the deployment of large language models.

在深度学习部署中，把已训练模型压到 8 bit、4 bit 整数已很常见，大语言模型场景尤为突出。

There are two main categories of quantization. In **post-training quantization**, the model is first trained normally with full-precision weights, which are then quantized after training. **Quantization-aware training**, on the other hand, introduces the quantization step during the training process. This allows the model to learn to compensate for the effects of quantization, which can help maintain the model's accuracy.

量化主要有两类：**后训练量化（post-training quantization）**先照常全精度训练，训完再量化；**量化感知训练（quantization-aware training）**则在训练过程中模拟量化，让模型学会补偿量化误差，有助于保住精度。

Tips: 量化，一般分为 2 大类：**后训练量化**、**量化感知训练**。

- 后训练量化，在训练完成后，对模型进行量化。
- 量化感知训练，在训练过程中，引入量化步骤，让模型学习量化带来的影响。

However, it's important to note that quantization can occasionally lead to a reduction in model accuracy. Since this chapter focuses on techniques to speed



up model inference *without* sacrificing accuracy, quantization is not as good a fit for this chapter as the previous categories.

需注意量化有时会带来精度损失；本章强调**不伤准确率**的加速，因而量化不如前几类那样“严丝合缝”地贴合本章设问。

Tips: 量化，可能会导致模型精度下降，因此，本章再不讨论量化。

Other techniques to improve inference speeds include knowledge distillation and pruning, discussed in Chapter [\[ch06\]](#). However, these techniques affect the model architecture, resulting in smaller models, so they are out of scope for this chapter's question.

提升推理速度还可借助知识蒸馏、剪枝等（见第 6 章），但它们会改变模型结构、得到更小的网络，故不在本章“不改架构”的限定之内。

Tips: 其他提升推理速度的策略，包括：知识蒸馏、剪枝等，之前章节已经讨论过；但是，这些策略会影响模型架构，导致模型变小，因此，也不在本章讨论范围内。

Exercises

本节习题引导思考多 GPU 推理的实际瓶颈，以及向量化与循环分块各自最理想的适用情景。

22-1. Chapter [\[ch07\]](#) covered several multi-GPU training paradigms to speed up model training. Using multiple GPUs can, in theory, also speed up model inference. However, in reality, this approach is often not the most efficient or most practical option. Why is that?

22-1. 第 [ch07](#) 章介绍过多 GPU 训练范式以加速训练；理论上多 GPU 也能加速推理，但实际中却常不是最高效或最务实的选择，原因是什么？

22-2. Vectorization and loop tiling are two strategies for optimizing operations that involve accessing array elements. What would be the ideal situation in which to use each?



22-2. 向量化和循环分块都用于优化涉及数组访问的运算；各自最理想的使用情境分别是什么？

References

本节列出 BLAS、循环分块经典论文、RepVGG 以及 LLM 8 bit / 4 bit 量化代表作等链接。

- The official BLAS website: <https://www.netlib.org/blas/>.
- BLAS 官方网站: <https://www.netlib.org/blas/>。
- The paper that proposed loop tiling: Michael Wolfe, "More Iteration Space Tiling"? (1989), <https://dl.acm.org/doi/abs/10.1145/76263.76337>.
- 提出循环分块的论文: Michael Wolfe, 《More Iteration Space Tiling?》(1989), <https://dl.acm.org/doi/abs/10.1145/76263.76337>。
- RepVGG CNN architecture merging operations in inference mode: Xiaohan Ding et al., "RepVGG: Making VGG-style ConvNets Great Again"? (2021), <https://arxiv.org/abs/2101.03697>.
- 推理阶段合并运算的 RepVGG: Ding 等, 《RepVGG: Making VGG-style ConvNets Great Again?》(2021), <https://arxiv.org/abs/2101.03697>。
- A new method for quantizing the weights in large language models down to 8-bit integer representations: Tim Dettmers et al., "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale"? (2022), <https://arxiv.org/abs/2208.07339>.
- 大语言模型权重量化到 8 bit 整数: Dettmers 等, 《LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale?》(2022), <https://arxiv.org/abs/2208.07339>。
- A new method for quantizing the weights in LLMs farther down to 4-bit integers: Elias Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers"? (2022), <https://arxiv.org/abs/2210.17323>.

- 将 LLM 权重进一步压到 4 bit 整数：Frantar 等, 《GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers?》(2022), <https://arxiv.org/abs/2210.17323>。





Chapter 23: Data Distribution Shifts

本章界定训练分布与部署后真实分布不一致的问题，分述协变量偏移、标签偏移、概念漂移与域偏移，并比较严重程度、监控要点与练习。

What are the main types of data distribution shifts we may encounter after model deployment?

模型上线之后，我们可能遇到哪些主要的**数据分布偏移（data distribution shifts）**类型？

Data distribution shifts are one of the most common problems when putting machine learning and AI models into production. In short, they refer to the differences between the distribution of data on which a model was trained and the distribution of data it encounters in the real world. Often, these changes can lead to significant drops in model performance because the model's predictions are no longer accurate.

数据分布偏移是把 ML/AI 模型投入生产时最常见的问题之一：简而言之，指**训练数据分布与线上/真实世界所见数据分布**之间的差异。此类变化常导致性能明显下滑，因为旧假设下的预测不再可靠。

Tips:数据分布偏移是生产环境中使用模型时，最常见的问题。

- 指的是，模型在**训练时所使用的数据分布**，与在**实际应用中遇到的数据分布**之间的差异。
- 通常，这些变化会导致 **模型性能** 显著下降，因为模型的预测不再准确。

There are several types of distribution shifts, some of which are more problematic than others. The most common are covariate shift, concept drift, label shift, and domain shift; all discussed in more detail in the following sections.

分布偏移有多种，棘手程度不一；最常见包括**协变量偏移、概念漂移、标签偏移与域偏移**，下文分别展开。

Tips:

- 数据分布偏移，有多种类型，其中最常见的是：协变量偏移、概念漂移、标签偏移和域偏移。
- 这些偏移类型，将在后续章节中详细讨论。



Covariate Shift

本节在 $p(x)$ 、 $p(y)$ 、 $p(y|x)$ 记号下定义协变量偏移，举例说明并点出对抗验证与重要性加权等应对思路。

Suppose $p(x)$ describes the distribution of the input data (for instance, the features), $p(y)$ refers to the distribution of the target variable (or class label distribution), and $p(y|x)$ is the distribution of the targets y given the inputs x .

设 $p(x)$ 为输入（特征）分布， $p(y)$ 为目标（或类标签边缘）分布， $p(y|x)$ 为给定输入下目标的条件分布。

Covariate shift happens when the distribution of the input data, $p(x)$, changes, but the conditional distribution of the output given the input, $p(y|x)$, remains the same.

协变量偏移 (*covariate shift*) 指：输入分布 $p(x)$ 变化，而 $p(y|x)$ 不变。

Tips:

- 协变量偏移，指的是，输入数据分布 $p(x)$ 发生变化，但输出条件分布 $p(y|x)$ 保持不变。
- 协变量 *covariate*，一般是指 特征变量，通常会影响输出结果，但并不一定是主要因素。

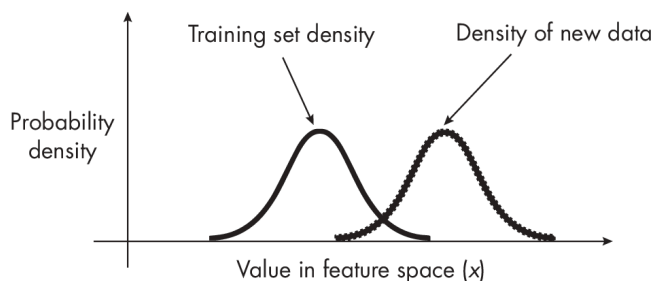


Figure 23.1



For example, suppose we trained a model to predict whether an email is spam based on specific features. Now, after we embed the email spam filter in an email client, the email messages that customers receive have drastically different features. For example, the email messages are much longer and are sent from someone in a different time zone. However, if the way those features relate to an email being spam or not doesn't change, then we have a covariate shift.

例如训练邮件反垃圾模型后，嵌入客户端；用户收件特征整体变了（更长、来自不同时区等），但若“特征如何对应是否垃圾邮件”的规则未变，则属于协变量偏移。

Covariate shift is a very common challenge when deploying machine learning models. It means that the data the model receives in a live or production environment is different from the data on which it was trained. However, because the relationship between inputs and outputs, $p(y|x)$, remains the same under covariate shift, techniques are available to adjust for it.

协变量偏移在部署中极常见：线上分布与训练集不同，但因 $p(y|x)$ 不变，仍有一类校正手段可用。

Tips:

- 协变量偏移，模型在实际应用中遇到的**数据分布**，与在训练时所使用的数据分布**不同**。
- 但是，由于输入和输出之间的关系 $p(y|x)$ 保持不变，因此有调整方法，例如：对抗验证、重要性加权等。

A common technique to detect covariate shift is *adversarial validation*, which is covered in more detail in Chapter [\[ch29\]](#). Once covariate shift is detected, a common method to deal with it is **importance weighting**, which assigns different weights to the training example to emphasize or de-emphasize certain instances during training. Essentially, instances that are more likely to appear in the test distribution are given more weight, while instances that are less likely to occur are given less weight. This approach allows the model to focus more on the instances representative of the test data during training, making it more robust to covariate shift.

检测协变量偏移的常用做法是**对抗验证** (*adversarial validation*)（详见第 29 章）。发现偏移后，常见处理是**重要性加权** (**importance weighting**)：给训练样



本不同权重，使更像在目标分布会出现的样本在训练中更受重视、更少出现的则被压低，从而让模型更贴近测试分布、提升对协变量偏移的鲁棒性。

Label Shift

本节定义标签偏移（先验漂移），给出邮件垃圾比例的例子，并说明加权损失等对策。

Label shift, sometimes referred to as *prior probability shift*, occurs when the class label distribution $p(y)$ changes, but the class-conditional distribution $p(y|x)$ remains unchanged. In other words, there is a significant change in the label distribution or target variable.

标签偏移（*label shift*）也称先验概率偏移：类别边缘分布 $p(y)$ 变，而 $p(y|x)$ 不变，即可理解为标签 / 目标变量的整体配比发生明显变化。

Tips:

- 标签偏移，指的是，标签分布 $p(y)$ 发生变化，但条件分布 $p(y|x)$ 保持不变。
- 标签偏移，通常与目标变量（或类标签分布）的变化有关。

As an example of such a scenario, suppose we trained an email spam classifier on a balanced training dataset with 50 percent spam and 50 percent non-spam email. In contrast, in the real world, only 10 percent of email messages are spam.

例如训练集里垃圾与非垃圾各占 50%，而真实世界里垃圾邮件仅占约 10%。

A common way to address label shifts is to update the model using the **weighted loss function**, especially when we have an idea of the new distribution of the labels. This is essentially a form of **importance weighting**. By adjusting the weights in the loss function according to the new label distribution, we are incentivizing the model to pay more attention to certain classes that have become more common (or less common) in the new data. This helps align the model's predictions more closely with the current reality, improving its performance on the new data.



对策之一是用**加权损失**更新模型，尤其在已知新标签分布时；本质上仍是**重要性加权**：按新分布调节各类在损失中的权重，促使模型更关注变多或变少的类别，从而更贴近当前数据。

Tips: 损失函数加权，突出重要样本分类。

Concept Drift

本节说明概念漂移即 $p(y|x)$ 的变化，并结合反垃圾例子讨论其难于处理之处。

Concept drift refers to the change in the mapping between the input features and the target variable. In other words, concept drift is typically associated with changes in the conditional distribution $p(y|x)$, such as the relationship between the inputs x and the output y .

概念漂移 (*concept drift*) 指输入与目标之间映射关系发生变化，通常体现为条件分布 $p(y|x)$ 变了，即 x 如何决定 y 的规律改变。

Tips:

- 概念漂移，指的是，输入特征与目标变量之间的映射关系发生变化。
- 概念漂移，通常与**条件分布** $p(y|x)$ 的变化有关。

Using the example of the spam email classifier from the previous section, the features of the email messages might remain the same, but *how* those features relate to whether an email is spam might change. This could be due to a new spamming strategy that wasn't present in the training data. Concept drift can be much harder to deal with than the other distribution shifts discussed so far since it requires continuous monitoring and potential model retraining.

延续邮件例子：邮件表层特征可能相仿，但“这些特征如何指向是否垃圾”的规则变了——例如出现训练期未见的新型诈骗话术。概念漂移往往比前述几类更难处理，需要持续监控并可能反复重训。



Domain Shift

本节澄清文献中域偏移与概念漂移的混用，给出联合分布视角与公式，并回到邮件例子说明为何最难应对。

The terms *domain shift* and *concept drift* are used somewhat inconsistently across the literature and are sometimes taken to be interchangeable. In reality, the two are related but slightly different phenomena. *Concept drift* refers to a change in the function that maps from the inputs to the outputs, specifically to situations where the relationship between features and target variables changes as more data is collected over time.

文献中对域偏移 (*domain shift*) 与概念漂移的用法并不完全一致，有时还被混用；实际上二者相关却略有差别。概念漂移强调从输入到输出的映射在变，即特征与目标的关系随时间推移而改变。

Tips:

- 领域偏移，通常跟概念漂移有差异。
- 领域偏移，指的是，输入数据分布 $p(x)$ 和输出条件分布 $p(y|x)$ 都发生变化。
- 领域偏移，也被称为**联合分布偏移**，因为联合分布 $p(x, y)$ 是输入和输出分布的乘积。

In *domain shift*, the distribution of inputs, $p(x)$, and the conditional distribution of outputs given inputs, $p(y|x)$, both change. This is sometimes also called *joint distribution shift* due to the joint distribution:

域偏移指 $p(x)$ 与 $p(y|x)$ 同时变化；因联合分布可分解为二者乘积，故有时也称联合分布偏移：

$$p(x, y) = p(y|x) \cdot p(x)$$

We can thus think of domain shift as a combination of both covariate shift and concept drift. In addition, since we can obtain the marginal distribution $p(y)$ by integrating over the joint distribution $p(x, y)$ over the variable x (mathematically expressed as $p(y) = \int p(x, y) dx$), covariate drift and concept shift also imply label shift. (However, exceptions may exist where the change in $p(x)$



compensates for the change in $p(y|x)$ such that $p(y)$ may not change.)
Conversely, label shift and concept drift usually also imply covariate shift.

因此域偏移可视为协变量偏移与概念漂移的“叠加”。又因 $p(y)$ 可通过对联合分布 $p(x, y)$ 关于 x 积分得到 ($p(y) = \int p(x, y) dx$)，协变量侧与概念侧的变化通常也会牵动标签边缘分布（存在 $p(x)$ 变化与 $p(y|x)$ 变化相互抵消、使 $p(y)$ 看似不变的例外）。反过来，标签偏移与概念漂移往往也意味着输入分布层面需要随之调整。

To return once more to the example of email spam classification, domain shift would mean that the features (content and structure of email) *and* the relationship between the features and target both change over time. For instance, spam email in 2023 might have different features (new types of phishing schemes, new language, and so forth), and the definition of what constitutes spam might have changed as well. This type of shift would be the most challenging scenario for a spam filter trained on 2020 data, as it would have to adjust to changes in both the input data and the target concept.

再以邮件反垃圾为例：域偏移意味着邮件的内容 / 结构特征与“何为垃圾”的定义一起随时间而变；若模型仍停留在 2020 年的数据与标签语义上，却要面对 2023 年的新模式与新规则，这是最棘手的情形。

Domain shift is perhaps the most difficult type of shift to handle, but monitoring model performance and data statistics over time can help detect domain shifts early. Once they are detected, mitigation strategies include collecting more labeled data from the target domain and retraining or adapting the model.

域偏移或许最难处理，但通过持续监控模型表现与数据统计，有望尽早发现；一旦确认，可收集目标域标注并重训或做模型自适应等缓解。

Types of Data Distribution Shifts

本节借助示意图比较各类偏移的相对“杀伤力”，强调现实语境中严重度取决于场景，并重申监控与预警的重要性。

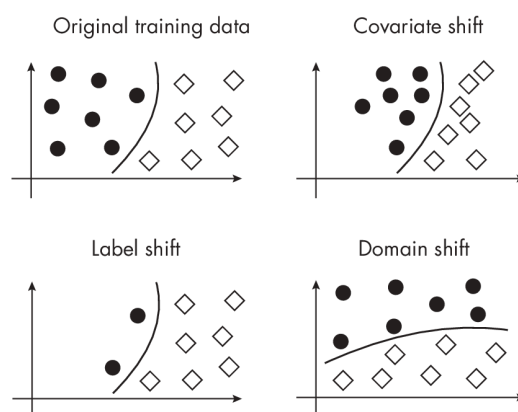


Figure 23.2

As noted in the previous sections, some types of distribution shift are more problematic than others. The least problematic among them is typically **covariate shift**. Here, the distribution of the input features, $p(x)$, changes between the training and testing data, but the conditional distribution of the output given the inputs, $p(y|x)$, remains constant. Since the underlying relationship between the inputs and outputs remains the same, the model trained on the training data can still apply, in principle, to the testing data and new data.

如前所述，不同偏移的麻烦程度不同。通常相对“温和”的是**协变量偏移**：训练与测试之间 $p(x)$ 变，而 $p(y|x)$ 不变；因输入—输出规律未改，原则上训练好的模型仍有望迁移到新数据。

The most problematic type of distribution shift is typically **joint distribution shift**, where both the input distribution $p(x)$ and the conditional output distribution $p(y|x)$ change. This makes it particularly difficult for a model to adjust, as the learned relationship from the training data may no longer hold. The model has to cope with both new input patterns and new rules for making predictions based on those patterns.

最棘手的是**联合分布偏移**： $p(x)$ 与 $p(y|x)$ 同时变，训练期学到的关系可能整体失效，模型既要适应新输入模式，也要适应基于这些模式的新预测规则。

However, the "severity" of a shift can vary widely depending on the real-world context. For example, even a covariate shift can be extremely problematic if the shift is severe or if the model cannot adapt to the new input distribution. On the other hand, a joint distribution shift might be manageable if the shift is relatively minor or if we have access to a sufficient amount of labeled data from the new distribution to retrain our model.



但现实里“严重度”取决于场景：协变量偏移若极其剧烈或模型无法适配新 $p(x)$ ，同样可能灾难性；反之，若联合偏移较温和，或能从新分布拿到足够标注重训，也未必不可控。

In general, it's crucial to monitor our models' performance and be aware of potential shifts in the data distribution so that we can take appropriate action if necessary.

总体而言，持续监控模型表现并对潜在分布变化保持警觉，才能在必要时及时采取行动。

Tips: 监控模型性能，及时发现潜在 **数据分布偏移**，非常重要。

Exercises

本节习题讨论重要性加权的局限，以及在无新标签时如何察觉偏移。

23-1. What is the big issue with importance weighting as a technique to mitigate covariate shift?

23-1. 把重要性加权作为缓解协变量偏移的手段，主要问题是什么？

23-2. How can we detect these types of shifts in real-world scenarios, especially when we do not have access to labels for the new data?

23-2. 在真实场景中，尤其当新数据没有标签时，如何检测这些偏移？

References

本节给出域偏移缓解与域适应相关的综述文献链接。

- Recommendations and pointers to advanced mitigation techniques for avoiding domain shift: Abolfazl Farahani et al., "A Brief Review of Domain Adaptation"? (2020), <https://arxiv.org/abs/2010.03978>.

- 关于规避域偏移及进阶缓解技术的建议与索引：Farahani 等, 《A Brief Review of Domain Adaptation?》(2020), <https://arxiv.org/abs/2010.03978>。
-





Chapter 24: Poisson and Ordinal Regression

本章对比泊松回归与序数回归：前者面向计数与近似泊松分布的响应，后者面向有序类别且不要求类间“间距”可度量，并配有练习。

When is it preferable to use **Poisson regression** over **Ordinal regression**, and vice versa?

在什么情况下更适合用泊松回归（Poisson regression），又在什么情况下更适合用序数回归（ordinal regression），反之亦然？

We usually use **Poisson regression** when the target variable represents count data (**positive integers**). As an example of count data, consider the number of colds contracted on an airplane or the number of guests visiting a restaurant on a given day. Besides the target variable representing counts, the data should also be Poisson distributed, which means that the mean and variance are roughly the same. (For large means, we can use a normal distribution to approximate a Poisson distribution.)

当目标变量是**计数**（正整数）时，通常用泊松回归；例如机舱内患感冒人数、某日餐厅来客数。除“取值为计数”外，数据还应大致服从**泊松分布**，即均值与方差接近（均值很大时，可用正态分布近似泊松；可参阅 [泊松分布（中文维基）](#)）。

Ordinal data is a subcategory of categorical data where the categories have a natural order, such as $1 < 2 < 3$, as illustrated in Figure [24.1](#). Ordinal data is often represented as positive integers and may look similar to count data. For example, consider the star rating on Amazon (1 star, 2 stars, 3 stars, and so on). However, ordinal regression does not make any assumptions about the distance between the ordered categories. Consider the following measure of disease severity: *severe* > *moderate* > *mild* > *none*. While we would typically map the disease severity variable to an integer representation ($4 > 3 > 2 > 1$), there is no assumption that the distance between 4 and 3 (severe and moderate) is the same as the distance between 2 and 1 (mild and none).



序数数据 (ordinal data) 是有自然顺序的分类数据的子类，例如 $1 < 2 < 3$ ，见图 24.1。序数常编码为正整数，因而*表面上像计数；例如电商星级（1星、2星.....）。但序数回归不要求**相邻等级之间的“距离”有意义或可比。再以疾病严重程度 重 > 中 > 轻 > 无 为例：即便映射成 $4 > 3 > 2 > 1$ ，也*不假设**“4与3的差距”等于“2与1的差距”。



Figure 24.1

In short, we use Poisson regression for count data. We use Ordinal regression when we know that certain outcomes are "higher" or "lower" than others, but we are not sure how much or if it even matters.

简言之：计数且近似泊松用泊松回归；当只知结果有高下先后、却不清楚间隔多大、甚至间隔是否重要时，用序数回归更合适。

Exercises

本节通过进球预测与观影偏好排序两题，巩固泊松与序数两类响应的判别。

24-1. Suppose we want to predict the number of goals a soccer player will score in a particular season. Should we solve this problem using ordinal regression or Poisson regression?

24-1. 若要预测某球员一季进球个数，该用序数回归还是泊松回归？

24-2. Suppose we ask someone to sort the last three movies they have watched based on their order of preference. Ignoring the fact that this dataset is a tad too small for machine learning, which approach would be best suited for this kind of data?

24-2. 若让某人按偏好对最近看过的三部电影排序（先忽略样本量太小这件事），哪种建模思路更贴切？





Chapter 25: Confidence Intervals

介绍评估分类器性能的置信区间构造思路：正态近似、训练集 bootstrap、测试集预测 bootstrap，以及更换随机种子重训；并说明各自假设与代价。

What are the different ways to construct confidence intervals for machine learning classifiers?

There are several ways to construct **confidence intervals** for machine learning models, depending on the model type and the nature of your data. For instance, some methods are computationally expensive when working with deep neural networks and are thus more suitable to less resource-intensive machine learning models. Others require larger datasets to be reliable.

为机器学习模型构造置信区间（confidence intervals）的途径有多种，具体取决于模型类型与数据特性。例如，某些方法在面对深度神经网络时计算代价很高，因此更适合较轻量的模型；另一些则需要较大样本才能可靠。

The following are the most common methods for constructing confidence intervals:

- Constructing normal approximation intervals based on a test set
- Bootstrapping training sets
- Bootstrapping the test set predictions
- Confidence intervals from retraining models with different random seeds

上述四类依次为：单次划分测试集上的**正态近似区间**；**对训练集**做 bootstrap（多轮训练/评估）；**对测试集预测**做 bootstrap；以及**用不同随机种子多次重训**再汇总精度。

Before reviewing these in greater depth, let's briefly review the definition and interpretation of confidence intervals.

在深入逐项讨论之前，先简要回顾置信区间的定义与解释方式。



Defining Confidence Intervals

澄清置信区间的统计含义及其在机器学习里与“总体泛化准确率”的对应关系，并图示常见展示方式。

A **confidence interval** is a type of method to estimate an unknown population parameter. A **population parameter** is a specific measure of a statistical population, for example, a mean (average) value or proportion. By "specific" measure, I mean there is a single, exact value for that parameter for the entire population. Even though this value may not be known and often needs to be estimated from a sample, it is a fixed and definite characteristic of the population. A *statistical population*, in turn, is the complete set of items or individuals we study.

置信区间用于估计未知的**总体参数**。**总体参数**是对整个统计总体某一数量的刻画，例如均值或比例；“具体”的意思是：对整个总体而言该参数只有一个确定的真值——即便未知且常需用样本估计，它仍是总体固定不变的特征；而**统计总体**就是我们研究的全体对象。

In a machine learning context, the population could be considered the entire possible set of instances or data points that the model may encounter, and the parameter we are often most interested in is the true generalization accuracy of our model on this population.

在机器学习语境下，可把总体理解为模型未来可能遇到的全部实例；而我们最关心的参数之一，往往是模型在该总体上的**真实泛化准确率**。

The accuracy we measure on the test set estimates the true generalization accuracy. However, it's subject to random error due to the specific sample of test instances we happened to use. This is where the concept of a confidence interval comes in. A 95 percent confidence interval for the generalization accuracy gives us a range in which we can be reasonably sure that the true generalization accuracy lies.

测试集上算出的准确率是对真实泛化准确率的估计，但会因我们恰好抽到的测试样本而带有随机波动；置信区间正是用来刻画这种不确定性：例如 95% 置信区间给出一个范围，使我们有理由认为真实泛化准确率落在其中。



For instance, if we take 100 different data samples and compute a 95 percent confidence interval for each sample, approximately 95 of the 100 confidence intervals will contain the true population value (such as the generalization accuracy), as illustrated in Figure 25.1.

例如重复抽样 100 次并各算一条 95% 置信区间，则大约其中 95 条会包含真实总体值（例如泛化准确率），如图 25.1 所示。

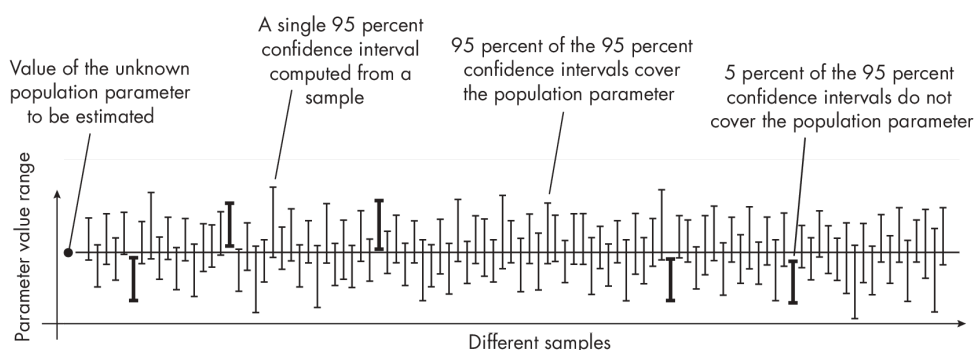


Figure 25.1

More concretely, if we were to draw 100 different representative test sets from the population (for instance, the entire possible set of instances that the model may encounter) and compute the 95 percent confidence interval for the generalization accuracy from each test set, we would expect about 95 of these intervals to contain the true generalization accuracy.

更具体地说，若从总体中抽取 100 个有代表性的测试集并分别构造 95% 泛化准确率置信区间，我们预期约有 95 个区间包含真实泛化准确率。

We can display confidence intervals in several ways. It is common to use a bar plot representation where the top of the bar represents the parameter value (for example, model accuracy) and the whiskers denote the upper and lower levels of the confidence interval (left chart of Figure 25.2). Alternatively, the confidence intervals can be shown without bars, as in the right chart of Figure 25.2.

可视化上可以用多种方式呈现置信区间：常见做法是用柱状图顶端表示参数点估计（例如准确率），用误差棒（whiskers）表示区间上下界（见 Figure 25.2 左图）；也可以不画柱体，只画区间（右图）。

This visualization is functionally useful in a number of ways. For instance, when confidence intervals for two model performances do *not* overlap, it's a strong visual indicator that the performances are significantly different. Take the example of statistical significance tests, such as t-tests: if two 95 percent confidence



intervals do not overlap, it strongly suggests that the difference between the two measurements is statistically significant at the 0.05 level.

这种可视化在比较模型时很实用：若两条性能置信区间**完全不重叠**，通常强烈提示两者差异显著；这与显著性检验的直觉一致——两个 95% 区间若不重叠，往往意味着在 0.05 水平上差异显著。

On the other hand, if two 95 percent confidence intervals overlap, we cannot automatically conclude that there's no significant difference between the two measurements. Even when confidence intervals overlap, there can still be a statistically significant difference.

反之，若两个 95% 区间**重叠**，也不能自动推出“无显著差异”；重叠时仍可能存在统计上显著的差别。

Alternatively, to provide more detailed information about the exact quantities, we can use a table view to express the confidence intervals. The two common notations are summarized in Table [25.1](#).

若需要更精确的数值表达，也可用表格给出区间；两种常见记法见 Table [25.1](#)。

Table 25.1

模型编号	置信区间（±表示法）	置信区间（下限, 上限）
1	89.1% ± 1.7%	89.1% (87.4%, 90.8%)
2	79.5% ± 2.2%	79.5% (77.3%, 81.7%)
3	95.2% ± 1.6%	95.2% (93.6%, 96.8%)

The $\pm$ notation is often preferred if the confidence interval is *symmetric*, meaning the upper and lower endpoints are equidistant from the estimated parameter. Alternatively, the lower and upper confidence intervals can be written explicitly.

当区间关于点估计**对称**时，常用 $\pm$ 记法；否则可显式写出上下界。

The Methods

分述四种常用构造方式：适用场景、公式要点与局限。



The following sections describe the four most common methods of constructing confidence intervals.

下面各节概述构造置信区间的四种最常见做法。

Method 1: Normal Approximation Intervals

单次训练-测试划分下用正态近似与标准误估计准确率区间；省事但依赖分布假定且看不到划分间波动。

The normal approximation interval involves generating the confidence interval from a single train-test split. It is often considered the simplest and most traditional method for computing confidence intervals. This approach is especially appealing in the realm of deep learning, where training models is computationally costly. It's also desirable when we are interested in evaluating a specific model, instead of models trained on various data partitions like in k -fold cross-validation.

正态近似区间在单次 train-test 划分下即可得到置信区间，常被视为最简单、最“传统”的做法；特别适合深度学习中训练代价高的场景；当你想评估“这一版具体模型”、而不是像在 k 折交叉验证那样比较多种划分时也很有吸引力。

How does it work? In short, the formula for calculating the confidence interval for a predicted parameter (for example, the sample mean, denoted as $\bar{x}$), assuming a normal distribution, is expressed as $\bar{x} \pm z \times SE$.

思路简述如下：在给定近似正态的假定下，对某估计量（如样本均值 $\bar{x}$ ），置信区间为 $\bar{x} \pm z \times SE$ 。

In this formula, z represents the z -score, which indicates a particular value's number of standard deviations from the mean in a standard normal distribution. SE represents the standard error of the predicted parameter (in this case, the sample mean).

其中 z 为标准正态下对应置信水平的临界值所用 z -score； SE 为该估计量（此处为样本均值）的**标准误**。

Most readers will be familiar with **z-score** tables that are usually found in the back of introductory statistics textbooks. However, a more convenient and



preferred way to obtain `z-scores` is to use functions like SciPy's `stats.zscore` function, which computes the `z-scores` for given confidence levels.

教科书附录常附 z 表；更便利的是像 SciPy `stats.norm.ppf` 等方式按置信水平求得临界值。（注意：原书示例提到 `stats.zscore` ——它通常用于对样本逐元素标准化；求置信区间的临界倍数时，一般用逆 CDF/ `ppf` 类函数更贴切。）

For our scenario, the sample mean, denoted as $\bar{x}$, corresponds to the test set accuracy, ACC_{test} , a measure of successful predictions in the context of a binomial proportion confidence interval.

对本章场景，样本均值可取为测试准确率 ACC_{test} ，可视作二项比例的正态近似问题中的成功比例估计。

The standard error can be calculated under a normal approximation as follows:

$$SE = \sqrt{\frac{1}{n} ACC_{\text{test}} (1 - ACC_{\text{test}})}$$

In this equation, n signifies the size of the test set. Substituting the standard error back into the previous formula, we obtain the following:

$$ACC_{\text{test}} \pm z \sqrt{\frac{1}{n} ACC_{\text{test}} (1 - ACC_{\text{test}})}$$

在正常近似下，标准误由前一式给出； n 为测试集样本量；将 SE 代回 $\bar{x} \pm z \cdot SE$ 即得测试准确率置信区间的常见写法。

Additional code examples to implement this method can also be found in the *supplementary/q25_confidence-intervals* subfolder in the supplementary code repository at <https://github.com/rasbt/MachineLearning-QandAI-book>. While the normal approximation interval method is very popular due to its simplicity, it has some downsides. First, the normal approximation may not always be accurate, especially for small sample sizes or for data that is not normally distributed. In such cases, other methods of computing confidence intervals may be more accurate. Second, using a single train-test split does not provide information about the variability of the model performance across different splits of the data. This can be an issue if the performance is highly dependent on the specific split used, which may be the case if the dataset is small or if there is a high degree of variability in the data.



配套代码见 [supplementary/q25_confidence-intervals](#)（同上链接）。该方法简单好用，但要注意：近似失准常见于小样本与强偏态情形；单次划分也无法反映换划分导致的性能起伏——数据稀缺或划分敏感时需配合其他途径。

Method 2: Bootstrapping Training Sets

通过对数据有放回抽样生成多轮训练/测试划分，用准确率分布的分位数构造区间；更“分布中立”但更贵且仍受限于所见数据。

Confidence intervals serve as a tool for approximating unknown parameters. However, when we are restricted to just one estimate, such as the accuracy derived from a single test set, we must make certain assumptions to make this work. For example, when we used the normal approximation interval described in the previous section, we assumed normally distributed data, which may or may not hold.

置信区间用于逼近未知总体参数；若只有单次测试集上的准确率点估计，往往就要依赖额外假定——例如前文正态近似所要求的分布性质。

In a perfect scenario, we would have more insight into our test set sample distribution. However, this would require access to many independent test datasets, which is typically not feasible. A workaround is the bootstrap method, which resamples existing data to estimate the sampling distribution.

理想情况下我们希望直接了解测试性能的抽样分布；但独立测试集往往不可得，于是用 bootstrap 在有放回抽样中估计该分布。

In practice, when the test set is large enough, the normal distribution approximation will hold, thanks to the central limit theorem. This theorem states that the sum (or average) of a large number of independent, identically distributed random variables will approach a normal distribution, regardless of the underlying distribution of the individual variables. It is difficult to specify what constitutes a large-enough test set. However, under stronger assumptions than those of the central limit theorem, we can at least estimate the rate of convergence to the normal distribution using the Berry-Esseen theorem, which gives a more quantitative estimate of how quickly the convergence in the central limit theorem occurs.



测试集足够大时，许多统计量会因中心极限定理而近似正态，但“多大算够大”并无统一标尺；Berry-Esseen 定理在给定更强的矩条件下，可对收敛速率给出更可量化的界限。

In a machine learning context, we can take the original dataset and draw a random sample *with replacement*. If the dataset has size n and we draw a random sample with replacement of size n , this implies that some data points will likely be duplicated in this new sample, whereas other data points are not sampled at all. We can then repeat this procedure for multiple rounds to obtain multiple training and test sets. This process is known as *out-of-bag bootstrapping*, illustrated in Figure 25.4.

在机器学习场景中，可先对数据集做容量为 n 的有放回抽样；新样本中会重复出现一些点也会漏掉一些点；重复多轮可得到多套训练-测试划分，即 **out-of-bag bootstrap**（图 25.4）。

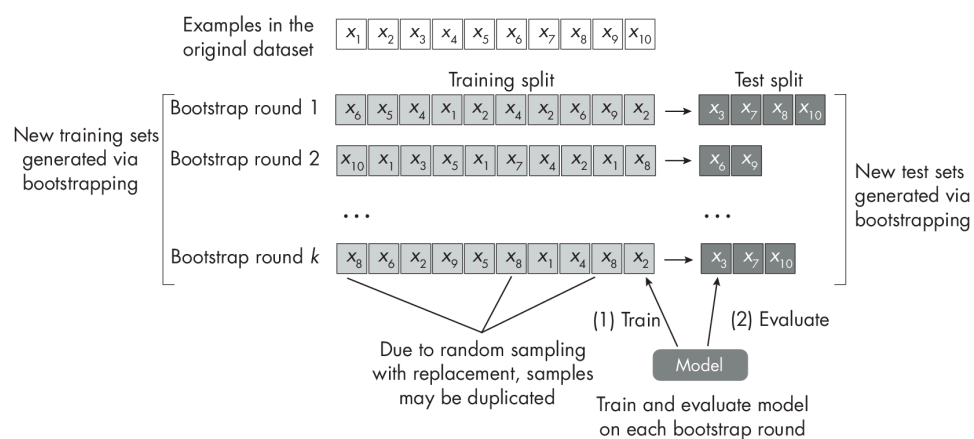


Figure 25.4

Suppose we constructed k training and test sets. We can now take each of these splits to train and evaluate the model to obtain k test set accuracy estimates. Considering this distribution of test set accuracy estimates, we can take the range between the 2.5th and 97.5th percentile to obtain the 95 percent confidence interval, as illustrated in Figure 25.5.

若有 k 套划分，可训练 k 个模型并得到 k 个测试准确率；再在这组经验分布上取第 2.5 与第 97.5 百分位，得到近似的 95% 区间（图 25.5）。

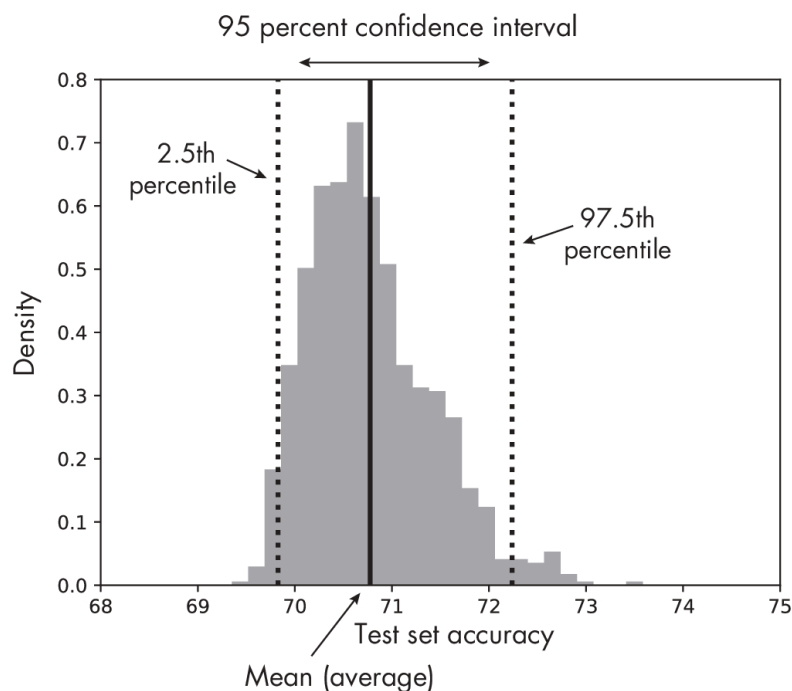


Figure 25.5

Unlike the normal approximation interval method, we can consider this out-of-bag bootstrap approach to be more agnostic to the specific distribution. Ideally, if the assumptions for the normal approximation are satisfied, both methodologies would yield identical outcomes.

与正态近似相比，该方法对总体分布形态的依赖更弱；若正态近似的条件成立，两种路径在理想情况下应给出相近结论。

Since bootstrapping relies on resampling the existing data used in those bootstrap rounds, its downside is that it doesn't bring in any new information that could be available in a broader population or unseen data. Therefore, it may not always be able to generalize the performance of the model to new, unseen data.

由于 bootstrap 只复用“已观测到的数据”，并不会补充来自更广总体的全新信息，因此对外推到完全未见场景仍需谨慎解读。

Note that we are using the bootstrap sampling approach in this chapter instead of obtaining the train-test splits via k -fold cross-validation, because of the bootstrap's theoretical grounding via the central limit theorem discussed earlier. There are also more advanced out-of-bag bootstrap methods, such as the .632 and .632+ estimates, which are reweighting the accuracy estimates.



本章采用自助划分而非 k 折，部分是为了与前面讨论的极限理论叙事衔接；工程上还存在 .632、.632+ 等对 out-of-bag 误差做加权修正的更精细版本。

Method 3: Bootstrapping Test Set Predictions

固定一套已训模型，仅对测试集做 bootstrap；省重训但看不到训练数据微小变动带来的不稳定。

An alternative approach to bootstrapping training sets is to bootstrap test sets. The idea is to train the model on the existing training set as usual and then to evaluate the model on bootstrapped test sets, as illustrated in Figure 25.6. After obtaining the test set performance estimates, we can then apply the percentile method described in the previous section.

与“重抽样训练集”相对，这一做法固定一套训练数据先训好模型，再对测试集做有放回抽样；得到多条性能序列后同样可用分位数法构造区间（图 25.6）。

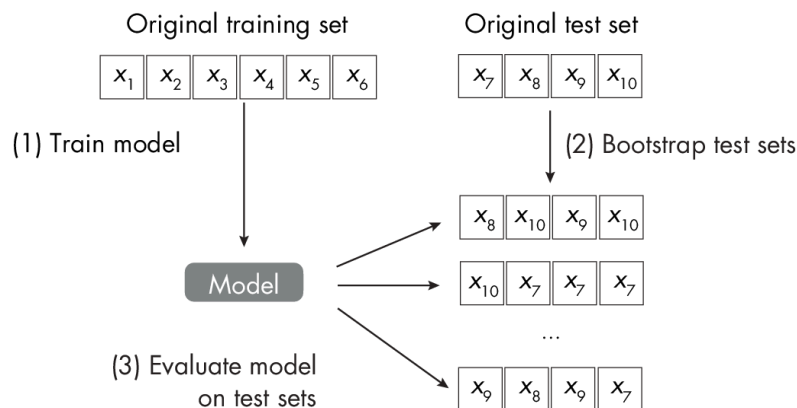


Figure 25.6

Contrary to the prior bootstrap technique, this method uses a trained model and simply resamples the test set (instead of the training sets). This approach is especially appealing for evaluating deep neural networks, as it doesn't require retraining the model on the new data splits. However, a disadvantage of this approach is that it doesn't assess the model's variability toward small changes in the training data.

其优点是评估深度模型时无需为每个 bootstrap 副本重训，计算上更省事；缺点是它**不包含**训练集扰动所带来的模型变异，只适合回答“对这一固定学到的模型，测试抽样不确定性如何”这一类问题。



Method 4: Retraining Models with Different Random Seeds

汇总多次随机种子实验的均值与标准误，用 t 临界值近似区间；最贵但利于理解深度模型稳定性。

In deep learning, models are commonly retrained using various random seeds since some random weight initializations may lead to much better models than others. How can we build a confidence interval from these experiments? If we assume that the sample means follow a normal distribution, we can employ a previously discussed method where we calculate the confidence interval around a sample mean, denoted as $\bar{x}$, as follows:

$$\bar{x} \pm z \cdot \text{SE}$$

深度学习中常会更换随机种子重训同一结构；若把这些重复试验得到的均值视作近似服从正态的样本均值，可先写出上述 $\bar{x} \pm z \cdot \text{SE}$ 的范式。

Since in this context we often work with a relatively modest number of samples (for instance, models from 5 to 10 random seeds), assuming a t distribution is deemed more suitable than a normal distribution. Therefore, we substitute the z value with a t value in the preceding formula. (As the sample size increases, the t distribution tends to look more like the standard normal distribution, and the critical values [z and t] become increasingly similar.)

但实际重复次数往往很少（例如 5-10 个种子），用 **t 分布** 临界值替代 z 更谨慎；当样本量增大时， t 与标准正态的临界值会逐渐接近。

Furthermore, if we are interested in the average accuracy, denoted as $\overline{\text{ACC}}_{\text{test}}$, we consider $\text{ACC}_{\text{test}, j}$ corresponding to a unique random seed j as a sample. The number of random seeds we evaluate would then constitute the sample size n . As such, we would calculate:

$$\overline{\text{ACC}}_{\text{test}} \pm t \cdot \text{SE}$$

若关注的是各随机种子下测试准确率 $\text{ACC}_{\text{test}, j}$ 的平均水平，就把它当作 r 次重复试验的观测，形式上仍写成 $\overline{\text{ACC}}_{\text{test}} \pm t \cdot \text{SE}$ 。

Here, SE is the standard error, calculated as $\text{SE} = \text{SD} / \sqrt{n}$, while



$$\overline{\text{ACC}}_{\text{test}} = \frac{1}{r} \sum_{j=1}^r \text{ACC}_{\text{test}, j}$$

is the average accuracy, which we compute over the r random seeds. The standard deviation SD is calculated as follows:

$$\text{SD} = \sqrt{\frac{\sum_{j=1}^r (\text{ACC}_{\text{test}, j} - \overline{\text{ACC}}_{\text{test}})^2}{r - 1}}$$

其中 $\text{SE} = \text{SD}/\sqrt{n}$; $\overline{\text{ACC}}_{\text{test}}$ 为 r 个随机种子结果的样本均值; SD 则按上面最后一式对所有 $\text{ACC}_{\text{test}, j}$ 做无偏样本标准差估计。

To summarize, calculating the confidence intervals using various random seeds is another effective alternative. However, it is primarily beneficial for deep learning models. It proves to be costlier than both the normal approximation approach (method 1) and bootstrapping the test set (method 3), as it necessitates retraining the model. On the bright side, the outcomes derived from disparate random seeds provide us with a robust understanding of the model's stability.

综上，多随机种子汇总是一条对深度模型特别自然的补充工具：它比方法 1 和方法 3 更“贵”（要反复训练），但能同时反映随机性带来的性能散布，从而更直接地刻画训练稳定性。

Recommendations

用一张对照表收束四种路径：计算代价、分布假设与对“稳定性/划分敏感度”的诊断价值。

Each possible method for constructing confidence intervals has its unique advantages and disadvantages. The normal approximation interval is cheap to compute but relies on the normality assumption about the distribution. The out-of-bag bootstrap is agnostic to these assumptions but is substantially more expensive to compute. A cheaper alternative is bootstrapping the test only, but this involves bootstrapping a smaller dataset and may be misleading for small or nonrepresentative test set sizes. Lastly, constructing confidence intervals from different random seeds is expensive but can give us additional insights into the model's stability.



一条实用对照是：正态近似最省算力但强依赖分布假定；全集 out-of-bag bootstrap 对分布较“超脱”但更耗时；只对测试 bootstrap 在中间地带，却会低估训练扰动引发的差异；汇总多种子主要服务深度模型的**稳定性画像**，但需要反复训练买单。

Exercises

练习题：置信水平与区间宽窄；更高效地评估 bootstrap 测试集准确率。

25-1. As mentioned earlier, the most common choice of confidence level is 95 percent confidence intervals. However, 90 percent and 99 percent are also common. Are 90 percent confidence intervals smaller or wider than 95 percent confidence intervals, and why is this the case?

25-2. In the discussion of Method 3 bootstrapping the test set earlier in this chapter, we created test sets by bootstrapping and then applied the already trained model to compute the test set accuracy on each of these datasets. Can you think of a method or modification to obtain these test accuracies more efficiently?

90% 与 95% 置信区间的宽窄关系需要从“覆盖率—区间长度”的取舍来理解；25-2 则引导你把“重复在 bootstrap 测试集上算准确率”与更省算的近似或加权估计联系起来。

References

延伸阅读：置信区间可视化误区、二项比例区间、中心极限与 Berry-Esseen、以及 .632 bootstrap 系列文献。

- A detailed discussion of the pitfalls of concluding statistical significance from nonoverlapping confidence intervals: Martin Krzywinski and Naomi Altman, "Error Bars" (2013), <https://www.nature.com/articles/nmeth.2659>.
- A more detailed explanation of the binomial proportion confidence interval: https://en.wikipedia.org/wiki/Binomial_proportion_confidence_interval.



- For a detailed explanation of normal approximation intervals, see Section 1.7 of my article: "Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning" (2018), <https://arxiv.org/abs/1811.12808>.
- Additional information on the central limit theorem for independent and identically distributed random variables:
https://en.wikipedia.org/wiki/Central_limit_theorem.
- For more on the Berry-Esseen theorem: https://en.wikipedia.org/wiki/Berry-Esseen_theorem.
- The .632 bootstrap addresses a pessimistic bias of the regular out-of-bag bootstrapping approach: Bradley Efron, "Estimating the Error Rate of a Prediction Rule: Improvement on Cross-Validation" (1983),
<https://www.jstor.org/stable/2288636>.
- The .632+ bootstrap corrects an optimistic bias introduced in the .632 bootstrap: Bradley Efron and Robert Tibshirani, "Improvements on Cross-Validation: The .632+ Bootstrap Method" (1997),
<https://www.jstor.org/stable/2965703>.
- A deep learning research paper that discusses bootstrapping the test set predictions: Benjamin Sanchez-Lengeling et al., "Machine Learning for Scent: Learning Generalizable Perceptual Representations of Small Molecules" (2019),
<https://arxiv.org/abs/1910.10685>.



Chapter 26: Confidence Intervals vs. Conformal Predictions

对照置信区间（总体参数）与预测区间 / conformal prediction（单点预测不确定性），说明覆盖保证、分布假设与用法差异。

What are the differences between confidence intervals and conformal predictions, and when do we use one over the other?

Confidence intervals and conformal predictions are both statistical methods to estimate the range of plausible values for an unknown population parameter. As discussed in Chapter [\[ch25\]](#), a confidence interval quantifies the level of confidence that a population parameter lies within an interval. For instance, a 95 percent confidence interval for the mean of a population means that if we were to take many samples from the population and calculate the 95 percent confidence interval for each sample, we would expect the true population mean (average) to lie within these intervals 95 percent of the time. Chapter [\[ch25\]](#) covered several techniques for applying this method to estimate the prediction performance of machine learning models. Conformal predictions, on the other hand, are commonly used for creating prediction intervals, which are designed to cover a true outcome with a certain probability.

二者都用于刻画“未知量可能落在何处”，但对象不同：置信区间针对**总体参数**（例如真实泛化准确率），其频率学解释是在重复抽样下覆盖真值的比例；conformal prediction 更直接服务**单条预测**的输出集合/区间，并给定“真实结果落在集合内”的控制概率。第 25 章的方法主要把置信区间用于评估模型层面的性能；本章则把 conformal prediction 宽泛地看作构造**预测区间（prediction interval）**的一类现代工具。

This chapter briefly explains what a prediction interval is and how it differs from confidence intervals, and then it explains how conformal predictions are, loosely speaking, a method for constructing prediction intervals.

本章先澄清预测区间相对置信区间回答的是哪一类不确定性，再说明 conformal prediction 如何在此基础上构造带有限样本覆盖保证的预测集合。



Confidence Intervals and Prediction Intervals

用“人群平均身高 vs. 某个人的身高”类比，区分面向总体的参数区间与面向单次输出的预测区间。

Whereas a confidence interval focuses on parameters that characterize a population as a whole, a *prediction interval* provides a range of values for a single predicted target value. For example, consider the problem of predicting people's heights. Given a sample of 10,000 people from the population, we might conclude that the mean (average) height is 5 feet, 7 inches. We might also calculate a 95 percent confidence interval for this mean, ranging from 5 feet, 6 inches to 5 feet, 8 inches.

对总体均值而言，我们只关心样本均值及其不确定性：它回答“平均身高是否在某一范围内”，而非某个具体个体的身高轨迹。

A *prediction interval*, however, is concerned with estimating not the height of the population but the height of an individual person. For example, given a weight of 185 pounds, a given person's prediction interval may fall between 5 feet 8 inches and 6 feet.

预测区间则要覆盖**单次观测**：若已知体重等信息，我们更关心某位具体人士身高可能出现的范围，这已经与均值置信区间截然不同。

In a machine learning model context, we can use confidence intervals to estimate a population parameter such as the accuracy of a model (which refers to the performance on all possible prediction scenarios). In contrast, a prediction interval estimates the range of output values for a single given input example.

在机器学习里同理：准确率置信区间刻画的是“整条决策规则在整个总体上的表现”；预测区间 / 集合说的是“对这一输入向量，单次输出取值可能散落何处”。

Prediction Intervals and Conformal Predictions

归纳：经典预测区间常与模型族/分布假定绑定；conformal 路线更一般，但可能要付更多算力。



Both conformal predictions and prediction intervals are statistical techniques that estimate uncertainty for individual model predictions, but they do so in different ways and under different assumptions.

二者都服务单点预测的不确定性，但 assumptions 与接口不同。

While prediction intervals often assume a particular data distribution and are tied to a specific type of model, conformal prediction methods are distribution free and can be applied to any machine learning algorithm.

传统预测区间往往依附于指定模型与噪声/分布模型；conformal prediction 在较弱的可交换等工作条件下可包装任意黑盒分类器或回归器。

In short, we can think of conformal predictions as a more flexible and generalizable form of prediction intervals. However, conformal predictions often require more computational resources than traditional methods for constructing prediction intervals, which involve resampling or permutation techniques.

可以把 conformal prediction 理解为“更通用的预测区间族”；代价常是额外的校准集、排序与集合构造步骤，算力与实现复杂度往往更高。

Prediction Regions, Intervals, and Sets

统一术语：回归用区间、分类用集合，二者都可称 prediction region。

In the context of conformal prediction, the terms *prediction interval*, *prediction set*, and *prediction region* are used to denote the plausible outputs for a given instance. The type of term used depends on the nature of the task.

conformal prediction 文献里会交替使用“区间 / 集合 / 区域”，本质是模型认为与观测相容的输出范围；叫法随任务类型而变。

In regression tasks where the output is a continuous variable, a *prediction interval* provides a range within which the true value is expected to fall with a certain level



of confidence. For example, a model might predict that the price of a house is between \$200,000 and \$250,000.

回归场景下输出连续，预测结果常给出一个**实数区间**，并控制该区间覆盖真值的经验频率。

In classification tasks, where the output is a discrete variable (the class labels), a *prediction set* includes all class labels that are considered plausible predictions for a given instance. For example, a model might predict that an image depicts either a cat, dog, or bird.

分类场景下标签离散，输出往往是一个**类别集合**（prediction set），必要时允许多标并列以换取覆盖保证。

Prediction region is a more general term that can refer to either a prediction interval or a prediction set. It describes the set of outputs considered plausible by the model.

prediction region 是上两者的统称：强调“输出空间中一块允许区域”，而不限定连续或离散。

Computing Conformal Predictions

走一遍训练—校准—测试流程：定义非一致性分数、在校准集上取分位阈值、再对新点生成预测集合。

Now that we've introduced the difference between confidence intervals and prediction regions and learned how conformal prediction methods are related to prediction intervals, how exactly do conformal predictions work?

In short, conformal prediction methods provide a framework for creating prediction regions, sets of potential outcomes for a prediction task. Given the assumptions and methods used to construct them, these regions are designed to contain the true outcome with a certain probability.

若从实现角度翻译上一段设问：关键在于把分类器输出映射成可比的非一致性分数，再借校准集的经验分位数决定“保哪些标签”。



其核心是构造**非一致性分数（nonconformity score）**：分数越大表示该标签与模型输出越“不协调”；阈值则直接读自校准样本的排序。

For classifiers, a prediction region for a given input is a set of labels such that the set contains the true label with a given confidence (typically 95 percent), as illustrated in Figure 26.1.

对分类任务，预测区域就是一组标签；在正确设定下它以高概率覆盖真实类（见图 26.1）。

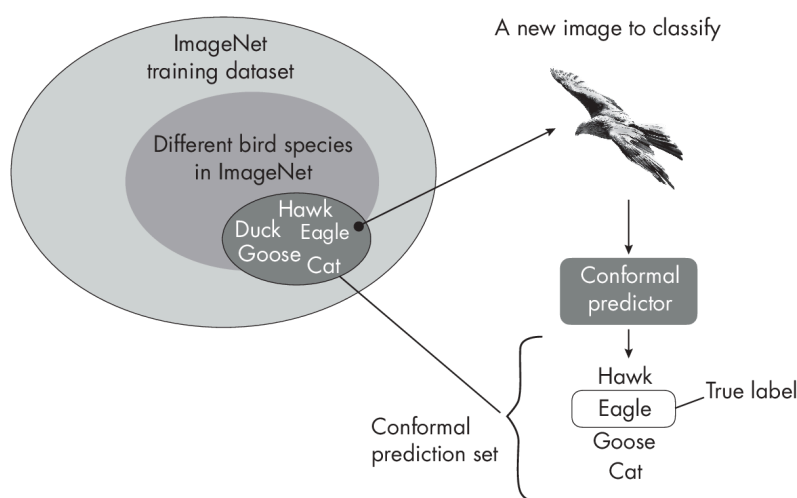


Figure 26.1

As depicted in Figure 26.1, the ImageNet dataset consists of a subset of bird species. Some bird species in ImageNet belong to one of the following classes: *hawk*, *duck*, *eagle*, or *goose*. ImageNet also contains other animals, for example, cats. For a new image to classify (here, an eagle), the conformal prediction set consists of classes such that the true label, *eagle*, is contained within this set with 95 percent probability. Often, this includes closely related classes, such as *hawk* and *goose* in this case. However, the prediction set can also include less closely related class labels, such as *cat*.

图 26.1 的示例说明：即便真实类是 *eagle*，为换取 95% 的有限样本覆盖保证，预测集合也可能同时包含语义相近的 *hawk/goose*，甚至在某些分数配置下纳入远类 *cat*——这是“以集合换覆盖”的直接体现。

To sketch the concept of computing prediction regions step by step, let's suppose we train a machine learning classifier for images. Before the model is trained, the dataset is typically split into three parts: a training set, a calibration set, and a test set. We use the training set to train the model and the calibration set to obtain the



parameters for the conformal prediction regions. We can then use the test set to assess the performance of the conformal predictor. A typical split ratio might be 60 percent training data, 20 percent calibration data, and 20 percent test data.

标准流程先把数据划成训练 / 校准 / 测试：训练集拟合模型；校准集单独用来定阈值（不参与训练）；测试集再汇报 conformal predictor 的经验覆盖与集合大小等指标。常见比例例如 60%/20%/20%。

The first step after training the model on the training set is to define a *nonconformity measure*, a function that assigns a numeric score to each instance in the calibration set based on how "unusual" it is. This could be based on the distance to the classifier's decision boundary or, more commonly, 1 minus the predicted probability of a class label. The higher the score is, the more unusual the instance is.

第一步是定义非一致性度量：常见取法之一是对真类用 $1 - \hat{p}(y | x)$ ，分数越大说明模型越“不信”该标注。

Before using conformal predictions for new data points, we use the nonconformity scores from the calibration set to compute a quantile threshold. This threshold is a probability level such that, for example, 95 percent of the instances in the calibration set (if we choose a 95 percent confidence level) have nonconformity scores below this threshold. This threshold is then used to determine the prediction regions for new instances, ensuring that the predictions are calibrated to the desired confidence level.

阈值一般取校准分数的适当分位数（并常做有限样本修正），使得在可交换性假定下，对新点的预测集合在目标水平上有有限样本覆盖。

Once we have the threshold value, we can compute prediction regions for new data. Here, for each possible class label (each possible output of your classifier) for a given instance, we check whether its nonconformity score is below the threshold. If it is, then we include it in the prediction set for that instance.

阈值确定后，对每个候选标签计算其“假设为真类”时的非一致性分数；凡低于阈值的标签都收入该实例的预测集合。

A Conformal Prediction Example

用一个三分类与 *score method* 演示：从概率到非一致性分数，再到预测集合。



Let's illustrate this process of making conformal predictions with an example using a simple conformal prediction method known as the *score method*. Suppose we train a classifier on a training set to distinguish between three species of birds: sparrows, robins, and hawks. Suppose the predicted probabilities for a calibration dataset are as follows:

- Sparrow [0.95, 0.9, 0.85, 0.8, 0.75]
- Robin [0.7, 0.65, 0.6, 0.55, 0.5]
- Hawk [0.4, 0.35, 0.3, 0.25, 0.2]

以下为 *score method* 第一步：列出校准集上各样本对真类的 softmax，再进入非一致性分数与阈值。

As depicted here, we have a calibration set consisting of 15 examples, five for each of the three classes. Note that a classifier returns three probability scores for each training example: one probability corresponding to each of the three classes (*Sparrow*, *Robin*, and *Hawk*). Here, however, we've selected only the probability for the true class label. For example, we may obtain the values [0.95, 0.02, 0.03] for the first calibration example with the true label *Sparrow*. In this case, we kept only 0.95.

这里每个校准样本只保留真类对应的 softmax 概率，共 15 个标量，来自三类各 5 个例子。

Next, after we obtain the previous probability scores, we can compute the nonconformity score as 1 minus the probability, as follows:

- Sparrow [0.05, 0.1, 0.15, 0.2, 0.25]
- Robin [0.3, 0.35, 0.4, 0.45, 0.5]
- Hawk [0.6, 0.65, 0.7, 0.75, 0.8]

即对每个真类概率取 $1 - p$ 作为非一致性分数。

Considering a confidence level of 0.95, we now select a threshold such that 95 percent of these nonconformity scores fall below that threshold. Based on the nonconformity scores in this example, this threshold is 0.8. We can then use this threshold to construct the prediction sets for new instances we want to classify.



若要控制约 95% 的校准覆盖，需要相应地上调阈值；本例中演示选取阈值为 0.8（示意用），用于之后筛选标签。

Now suppose we have a new instance (a new image of a bird) that we want to classify. We calculate the nonconformity score of this new bird image, assuming it belongs to each bird species (class label) in the training set:

- Sparrow 0.26
- Robin 0.45
- Hawk 0.9

In this case, the *Sparrow* and *Robin* nonconformity scores fall below the threshold of 0.8. Thus, the prediction set for this input is [*Sparrow*, *Robin*]. In other words, this tells us that, on average, the true class label is included in the prediction set 95 percent of the time.

当阈值为 0.8 时，*Sparrow* 与 *Robin* 的非一致性分数低于阈值，故进入预测集合；新图像上逐类“反事实”算分正对应上面列出的 0.26/0.45/0.9。严格实现需按校准分布计算分位数，这里仅示意流程。

A hands-on code example implementing the score method can be found in the *supplementary/q26_conformal-prediction* subfolder at <https://github.com/rasbt/MachineLearning-QandAI-book>.

可参考作者仓库目录 *supplementary/q26_conformal-prediction* 的可运行脚本，把本节步骤对齐到代码。

The Benefits of Conformal Predictions

强调有限样本覆盖、与模型无关的通用性，以及与传统概率输出相比的松紧取舍。

In contrast to using class-membership probabilities returned from classifiers, the major benefits of conformal prediction are its theoretical guarantees and its generality. Conformal prediction methods don't make any strong assumptions about the distribution of the data or the model being used, and they can be applied in conjunction with any existing machine learning algorithm to provide confidence measures for predictions.



与直接解读 softmax 相比，conformal prediction 的卖点是：在标准可交换性条件下给出**有限样本覆盖**，而不是仅依赖渐近或启发式阈值；并且几乎不限制底层模型结构。

Confidence intervals have asymptotic coverage guarantees, which means that the coverage guarantee holds in the limit as the sample (test set) size goes to infinity. This doesn't necessarily mean that confidence intervals work for only very large sample sizes, but rather that their properties are more firmly guaranteed as the sample size increases. Confidence intervals therefore rely on asymptotic properties, meaning that their guarantees become more robust as the sample size grows.

许多经典置信区间构造依赖渐近正态等性质：样本越大，名义覆盖往往越接近理论值，但有限样本下仍可能有偏差。

In contrast, conformal predictions provide finite-sample guarantees, ensuring that the coverage probability is achieved for any sample size. For example, if we specify a 95 percent confidence level for a conformal prediction method and generate 100 calibration sets with corresponding prediction sets, the method will include the true class label for 95 out of the 100 test points. This holds regardless of the size of the calibration sets.

conformal prediction 在理想设定下对**有限校准规模**即可给出覆盖控制（实现细节需按具体算法校正分位数）；这与“靠增大测试集让渐近保证更紧”的置信区间叙事形成对照。

While conformal prediction has many advantages, it does not always provide the tightest possible prediction intervals. Sometimes, if the underlying assumptions of a specific classifier hold, that classifier's own probability estimates might offer tighter and more informative intervals.

代价是集合可能偏“松”：若模型概率经过良好校准并且类间结构简单，直接使用模型概率未必比 conformal 集合更宽，但需要额外验证。

Recommendations

选型建议：要比较模型还是用模型——分别对应置信区间与 conformal / 预测区间。



A confidence interval tells us about our level of uncertainty about the model's properties, such as the prediction accuracy of a classifier. A prediction interval or conformal prediction output tells us about the level of uncertainty in a specific prediction from the model. Both are very important in understanding the reliability and performance of our model, but they provide different types of information.

简短建议：若想回答“这个模型整体上有多准”，用第 25 章的置信区间类工具；若要在部署后标记“哪些单点决策必须人工复核”，则 conformal prediction 更对口。

For example, a confidence interval for the prediction accuracy of a model can be helpful for comparing and evaluating models and for deciding which model to deploy. On the other hand, a prediction interval can be helpful for using a model in practice and understanding its predictions. For instance, it can help identify cases where the model is unsure and may need additional data, human oversight, or a different approach.

例如：离线比较多个候选模型可看准确率置信区间是否重叠；在线上则可用预测集合大小激增作为触发人工审核的信号。

Exercises

练习题：集合大小的语义；回归场景下的置信区间与 conformal。

26-1. Prediction set sizes can vary between instances. For example, we may encounter a prediction set size of 1 for a given instance and for another, a set size of 3. What does the prediction set size tell us?

26-2. Chapters [ch25] and [ch26] focused on classification methods. Could we use conformal prediction and confidence intervals for regression too?

26-1 让你把“集合元素个数”解释为模型在该点的把握程度（以及风险暴露）；26-2 提示回归同样可构造参数不确定性与 conformalized 区间两条并行路线。

References

工具与文献入口：MAPIE、Molnar 小书、以及 conformal prediction 资源汇总。



- MAPIE is a popular library for conformal predictions in Python:
<https://mapie.readthedocs.io/>.
 - For more on the score method used in this chapter: Christoph Molnar, *Introduction to Conformal Prediction with Python* (2023),
<https://christophmolnar.com/books/conformal-prediction/>.
 - In addition to the score method, several other variants of conformal prediction methods exist. For a comprehensive collection of conformal prediction literature and resources, see the Awesome Conformal Prediction page: <https://github.com/valeman/awesome-conformal-prediction>.
-



Chapter 27: Proper Metrics

距离要满足非负、同一性、对称与三角不等式才算“proper metric”；本章用 MSE 与交叉熵逐项对照这些公理。

What are the three properties of a distance function that make it a *proper metric*?

Metrics are foundational to mathematics, computer science, and various other scientific domains. Understanding the fundamental properties that define a good distance function to measure distances or differences between points or datasets is important. For instance, when dealing with functions like loss functions in neural networks, understanding whether they behave like proper metrics can be instrumental in knowing how optimization algorithms will converge to a solution.

度量与“长得像距离的函数”是许多算法分析的共同语言；当我们在神经网络里选一个损失函数时，追问它是否具有 metric 三公理，有助于判断优化曲面是否具备某些几何正则性。

Tips:

- **Metrics** 度量，是 **数学**、**计算机科学** 和各种其他科学领域的基础。
- 理解定义良好的 **距离函数** 的关键属性，对于测量点或数据集之间的距离或差异至关重要。
- 例如，在处理神经网络中的损失函数时，了解它们是否表现出良好的距离函数属性，对于了解优化算法如何收敛到解决方案至关重要。
- **距离函数**，具有三个关键属性：**非负性**、**对称性** 和 **三角不等式**。

This chapter analyzes two commonly utilized loss functions, the **mean squared error** and the **cross-entropy loss**, to demonstrate whether they meet the criteria for proper metrics.

以下内容分别检查 **mean squared error** 与 **cross-entropy loss** 是否真的能当作严格意义下的度量。

Tips: 本章节，分析了两个常用的损失函数，**均方误差** 和 **交叉熵损失**，来演示它们是否符合良好的度量标准。



The Criteria

三元组形式写出 metric 三公理，并图示欧氏距离与三角不等式的几何直觉。

To illustrate the criteria of a proper metric, consider two vectors or points $\mathbf{v}$ and $\mathbf{w}$, and their distance $d(\mathbf{v}, \mathbf{w})$, as shown in Figure 27.1.

若以欧氏平面上的两点为例（图 27.1），可把“距离函数”视作给每对向量赋一个标量值的映射 $d(\cdot, \cdot)$ 。

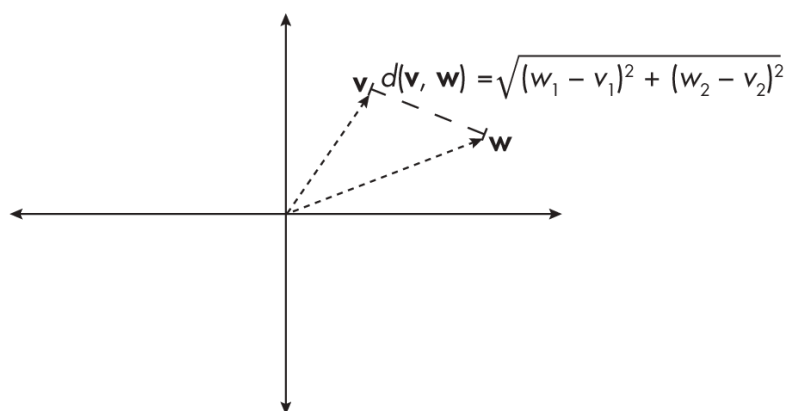


Figure 27.1

The criteria of a proper metric are the following:

- The distance between two points is always **non-negative**, $d(\mathbf{v}, \mathbf{w}) \geq 0$, and can be 0 only if the two points are identical, that is, $\mathbf{v} = \mathbf{w}$.
- The distance is **symmetric**; for instance, $d(\mathbf{v}, \mathbf{w}) = d(\mathbf{w}, \mathbf{v})$.
- The distance function satisfies the **triangle inequality** for any three points: $\mathbf{v}$, $\mathbf{w}$, $\mathbf{x}$, meaning:

$$d(\mathbf{v}, \mathbf{w}) \leq d(\mathbf{v}, \mathbf{x}) + d(\mathbf{x}, \mathbf{w})$$

在数学分析的 metric space 语境里，这组条件概括为：非负且零距等价于重合、对称、以及任意三点上的三角不等式。

Tips: 距离函数，具有三个关键属性：**非负性**、**对称性** 和 **三角不等式**。



To better understand the triangle inequality, think of the points as vertices of a triangle. If we consider any triangle, the sum of two of the sides is always larger than the third side, as illustrated in Figure 27.2.

把三点看成三角形顶点：两边长度之和总不小于第三边，这便是三角不等式的几何图像（图 27.2）。

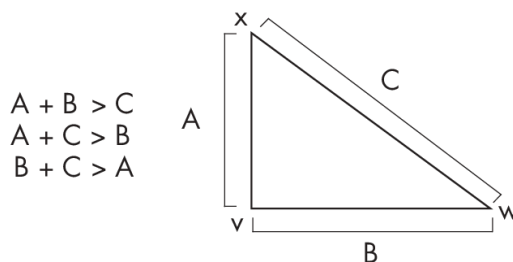


Figure 27.2

Consider what would happen if the triangle inequality depicted in Figure 27.2 weren't true. If the sum of the lengths of sides AB and BC was shorter than AC, then sides AB and BC would not meet to form a triangle; instead, they would fall short of each other. Thus, the fact that they meet and form a triangle demonstrates the triangle inequality.

若破坏该不等式，相当于两条边无法“接住”第三点，几何上就拼不出封闭的三角形。

The Mean Squared Error

阐明 $(y - \hat{y})^2$ 为何满足前两式公理却往往破坏三角不等式；开平方回到欧氏度量可恢复三公理。

The **mean squared error (MSE)** loss computes the squared Euclidean distance between a target variable y and a predicted target value $\hat{y}$:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

上式在样本维上把逐点平方误差平均；索引 i 遍历各数据点。



The index i denotes the i th data point in the dataset or sample. Is this loss function a proper metric?

为聚焦 SE 是否在单点层面构成 metric，下面把 MSE 退回到其逐点核函数。

For simplicity's sake, we will consider the **squared error (SE)** loss between two data points (though the following insights also hold for the MSE). As shown in the following equation, the SE loss quantifies the squared difference between the predicted and actual values for a single data point, while the MSE loss averages these squared differences over all data points in a dataset:

$$\text{SE}(y, \hat{y}) = (y - \hat{y})^2$$

the distance between two points is always **non-negative**. Since we are raising the difference to the power of 2, it cannot be negative.

平方保证了非负性，这是第一条公理的“半边”。

How about the second criterion, that the distance can be 0 only if the two points are identical? Due to the subtraction in the SE, it is intuitive to see that it can be 0 only if the prediction matches the target variable, $y = \hat{y}$. As with the first criterion, we can use the square to confirm that SE satisfies the second criterion: we have $(y - \hat{y})^2 = (\hat{y} - y)^2$.

同时， $(y - \hat{y})^2 = 0$ 当且仅当 $y = \hat{y}$ ，因而满足“重合则零距离”的判别性；又因为平方，天然对称 $\text{SE}(y, \hat{y}) = \text{SE}(\hat{y}, y)$ 。

At first glance, it seems that the squared error loss also satisfies the third criterion, the **triangle inequality**. Intuitively, you can check this by choosing three arbitrary numbers, here 1, 2, 3:

- $(1 - 2)^2 \leq (1 - 3)^2 + (2 - 3)^2$
- $(1 - 3)^2 \leq (1 - 2)^2 + (2 - 3)^2$
- $(2 - 3)^2 \leq (1 - 2)^2 + (1 - 3)^2$

用 (1, 2, 3) 只是快速获得直觉；它并不能证明一般情形。

However, there are values for which this is not true. For example, consider the values $a = 0$, $b = 2$, and $c = 1$. This gives us $d(a, b) = 4$, $d(a, c) = 1$, and



$d(b, c) = 1$, such that we have the following scenario, which violates the triangle inequality:

取 $a = 0, b = 2, c = 1$, 若强行令 $d(x, y) = (x - y)^2$, 则有 $d(a, b) = 4$ 、 $d(a, c) = d(b, c) = 1$, 进而出现违背三角不等式的一组关系:

- $(0 - 2)^2 \not\leq (0 - 1)^2 + (2 - 1)^2$
- $(2 - 1)^2 \leq (0 - 1)^2 + (0 - 2)^2$
- $(0 - 1)^2 \leq (0 - 2)^2 + (1 - 2)^2$

若坚持把 $(x - y)^2$ 当作距离就会得到上述第一条失败的不等式, 因此平方差不满足三角不等式。

Since it does not satisfy the triangle inequality via the example above, we conclude that the (mean) squared error loss is not a proper metric.

However, if we change the squared error into the **root-squared error**

$$\sqrt{(y - \hat{y})^2}$$

the triangle inequality can be satisfied:

对标量而言, 这等价于取 $|y - \hat{y}|$, 与一维欧氏距离一致, 因而满足三角不等式; 例如

$$\sqrt{(0 - 2)^2} \leq \sqrt{(0 - 1)^2} + \sqrt{(2 - 1)^2}$$

You might be familiar with the **L2** distance or Euclidean distance, which is known to satisfy the triangle inequality. These two distance metrics are equivalent to the root-squared error when considering two scalar values.

在回归里可把根式视作“差分矢量”上的 ℓ_2 度量, 它已经满足本节开头列出的三公理。

Tips:

- 如果将 **平方误差** 改为 **平方根误差**, 则三角不等式可以满足。
- 平方根误差, 是平方误差的平方根。



The Cross-Entropy Loss

说明交叉熵作为“失真测度”在训练中大有用处，但并非度量空间上的合法距离。

Tips: 交叉熵损失，是衡量两个概率分布之间距离的损失函数。

FIXME ??? 不理解

Cross entropy is used to measure the distance between two probability distributions. In machine learning contexts, we use the discrete cross-entropy loss (CE) between class label y and the predicted probability $\hat{p}$ when we train logistic regression or neural network classifiers on a dataset consisting of n training examples:

$$\text{CE}(\mathbf{y}, \mathbf{p}) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \times \log(p^{(i)})$$

此即训练循环里见到的平均交叉熵；讨论是否度量时我们常把它分解成逐样本因子。

Is this loss function a proper metric? Again, for simplicity's sake, we will look at the cross-entropy function (H) between only two data points:

$$H(y, p) = -y \times \log(p)$$

其中 $y \in [0, 1]$ 、 $p \in (0, 1]$ ；多类 softmax 只是把该 scalar 核对角化到向量情形，反例精神不变。

The cross-entropy loss satisfies one part of the first criterion: the distance is always non-negative because the probability score is a number in the range $[0, 1]$. Hence, $\log(p)$ ranges between $-\infty$ and 0. The important part is that the H function includes a negative sign. Hence, the cross entropy ranges between 0 and $+\infty$ and thus satisfies one aspect of the first criterion shown above.

在取值合法时 $H(y, p) \geq 0$ ，满足“距离非负”的直觉要求。

However, the cross-entropy loss is not 0 for two identical points. For example, $H(0.9, 0.9) = -0.9 \times \log(0.9) = 0.095$.



但它无法做到“完全一致则距离为零”：即便 $y = p = 0.9$ ，交叉熵仍为正，违背了 metric 对所要求的强同一性判别。

The second criterion shown above is also violated by the cross-entropy loss because the loss is not symmetric: $-y \times \log(p) \neq -p \times \log(y)$. Let's illustrate this with a concrete, numeric example:

- If $y = 1$ and $p = 0.5$, then $-1 \times \log(0.5) = 0.693$.
- If $y = 0.5$ and $p = 1$, then $-0.5 \times \log(1) = 0$.

简单数值即可看到互换 y, p 会改变损失的数值层级，对称性荡然无存。

Finally, the cross-entropy loss does not satisfy the triangle inequality, $H(r, p) \geq H(r, q) + H(q, p)$. Let's illustrate this with an example as well. Suppose we choose $r = 0.9$, $p = 0.5$, and $q = 0.4$. We have:

- $H(0.9, 0.5) = 0.624$
- $H(0.9, 0.4) = 0.825$
- $H(0.4, 0.5) = 0.277$

若以距离的标准三角不等式 $d(r, p) \leq d(r, q) + d(q, p)$ 来核验，可把 $d \equiv H$ 逐项代入；下面英文给出本书原述不等式方向的数值现象。

As you can see, $0.624 \geq 0.825 + 0.277$ does not hold here.

本书在此方向检验 H ，是为了强调它并不能像度量那样沿三角路径“次可加”。工程上可把交叉熵理解为 Bregman 散度的特例，而不可直接当作 $\mathbb{R}^2$ 上的路程长度。

In conclusion, while the **cross-entropy loss** is a useful loss function for training neural networks via (stochastic) gradient descent, it is not a proper distance metric, as it does not satisfy any of the three criteria.

因此：交叉熵是出色的训练目标，却不满足本节列出的三条 metric 准则（实际上三条都可构造反例）。

Tips:

- 交叉熵损失，是训练逻辑回归或神经网络分类器时，用于衡量两个概率分布之间距离的损失函数，这种损失函数在训练过程中，可以引导模型



学习到更好的概率分布。

- 但是，交叉熵损失，不是良好的度量标准，因为它不满足三角不等式。

Exercises

MAE vs RMSE 是否构成 metric；比较两者作为误差度量的实务取舍。

27-1. Suppose we consider using the mean absolute error (MAE) as an alternative to the root mean square error (RMSE) for measuring the performance of a machine learning model, where

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$$

and

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}$$

However, a colleague argues that the MAE is not a proper distance metric in metric space because it involves an absolute value, so we should use the RMSE instead. Is this argument correct?

27-2. Based on your answer to the previous question, would you say that the MAE is better or is worse than the RMSE?

27-1 让你检验同事“绝对值 $\Rightarrow$ 不是 metric $\Rightarrow$ 必选 RMSE”的推理链条； ℓ_1 距离其实满足三公理——真正需要辨析的是 RMSE 作为复合泛函是否与逐点 $|y - \hat{y}|$ 同型。



Chapter 28: The k in k -Fold Cross-Validation

澄清 k -fold 中 k 对训练相似度、方差—偏差与算力开销的影响，并给出按目标（近似最终模型 vs. 探查数据敏感度）选型与两步法的思路。

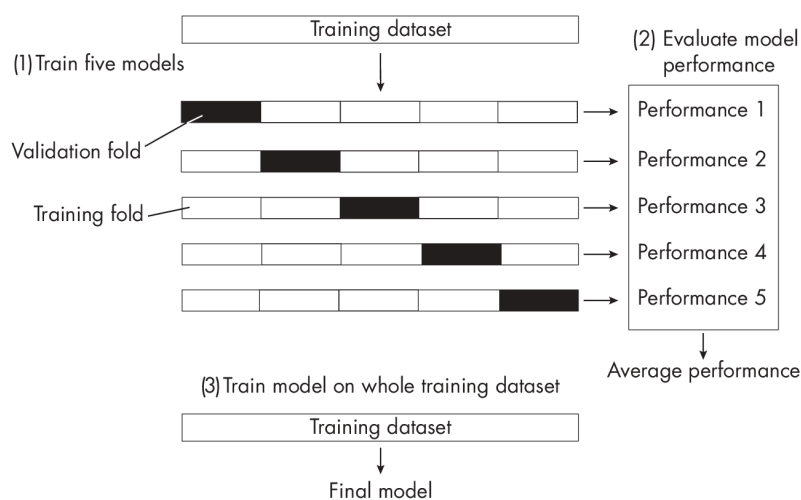
k -fold cross-validation is a common choice for evaluating machine learning classifiers because it lets us use all training data to simulate how well a machine learning algorithm might perform on new data. What are the advantages and disadvantages of choosing a large k ?

Selecting a larger k means each fold leaves out fewer training points per round—so successive models resemble a full-data fit—but it also repeats training more often and yields smaller validation shards that can fluctuate sharply.

可把问题拆成三件事：相邻折之间训练集的相似程度、单次训练消耗的样本体量与迭代次数（算力账单）、以及很小的验证片段如何扰动指标的方差。

We can think of k -fold cross-validation as a workaround for model evaluation when we have limited data. In machine learning model evaluation, we care about the generalization performance of our model, that is, how well it performs on new data. In k -fold cross-validation, we use the training data for model selection and evaluation by partitioning it into k validation rounds and folds. If we have k folds, we have k iterations, leading to k different models, as illustrated in Figure [28.1](#).

可把 k -折理解为：在同一份候选训练数据上轮转留出，从而在有限样本下粗略回答“换新数据会发生什么”。形式上，每轮都只留一块作验证并用其余折重训整套管线，迭代 k 次即得 k 条性能轨迹（见图 [28.1](#)）。

**Figure 28.1**

Using k -fold cross-validation, we usually evaluate the performance of a particular hyperparameter configuration by computing the average performance over the k models. This performance reflects or approximates the performance of a model trained on the complete training dataset after evaluation.

对某一固定超参向量，我们常把 k 折得分的算术平均视作其泛化性能的代理——经验上近似于“若在完整训练数据上重新训练后再测”会如何。

The following sections cover the trade-offs of selecting values for k in k -fold cross-validation and address the challenges of large k values and their computational demands, especially in deep learning contexts. We then discuss the core purposes of k and how to choose an appropriate value based on specific modeling needs.

下面分两节展开：其一是 k 与相似度—方差—算力的张力；其二结合建模目的讨论如何挑 k 以及是否与调参拆分。

Trade-offs in Selecting Values for k

论述大 k 使模型彼此更像全量训练版本、提高单次训练体量，却会压缩验证折并放大估计方差图景的复杂性。

If k is too large, the training sets are too similar between the different rounds of cross-validation. The k models are thus very similar to the model we obtain by training on the whole training set. In this case, we can still leverage the advantage of k -fold cross-validation: evaluating the performance for the entire training set



via the held-out validation fold in each round. (Here, we obtain the training set by concatenating all $k - 1$ training folds in a given iteration.) However, a disadvantage of a large k is that it is more challenging to analyze how the machine learning algorithm with the particular choice of hyperparameter setting behaves on different training datasets.

k 很大时，每轮只差一点点样本，学到的决策边界高度相关：好处是均值分数更贴近“用几乎全数据训练”；坏处是你更难从分数分布里读出“换一批训练数据会发生什么”的诊断信息——因为训练折几乎总是同一分布的高重叠子集。

Besides the issue of too-similar datasets, running k -fold cross-validation with a large value of k is also computationally more demanding. A larger k is more expensive since it increases both the number of iterations and the training set size at each iteration. This is especially problematic if we work with relatively large models that are expensive to train, such as contemporary deep neural networks.

算力维度上，大 k 同时推高**迭代次数**和**每轮训练样本量**：对深度模型尤其刺耳，因为要重复训练一整套优化。

A common choice for k is typically 5 or 10, for practical and historical reasons. A study by Ron Kohavi (see References at the end of this chapter) found that $k = 10$ offers a good bias and variance trade-off for classical machine learning algorithms, such as decision trees and naive Bayes classifiers, on a handful of small datasets.

实务上常以 $k \in \{5, 10\}$ 为默认脚注式选择；Ron Kohavi 的经典研究（参考文献）在多款小数据集与传统学习器上演示 $k=10$ 常落在可用偏差-方差折中点附近。

For example, in 10-fold cross-validation, we use 9/10 (90 percent) of the data for training in each round, whereas in 5-fold cross-validation, we use only 4/5 (80 percent) of the data, as shown in Figure [28.2](#).

例如在 10 折场景里单轮训练的样本量是分母的 90%，而 5 折只占 80%；图 [28.2](#) 给出直观对比。

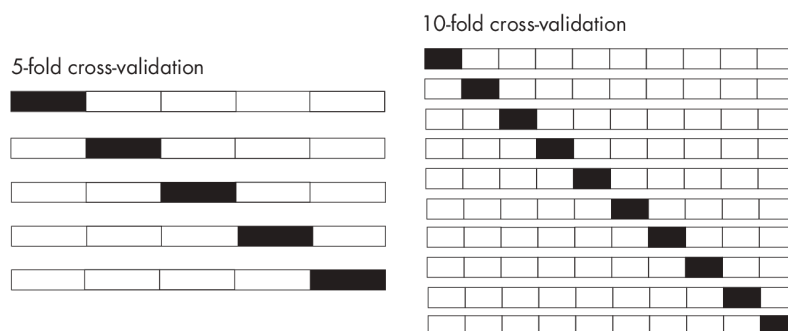


Figure 28.2

However, this does not mean large training sets are bad, since they can reduce the pessimistic bias of the performance estimate (mostly a good thing) if we assume that the model training can benefit from more training data. (See Figure 5.1 on page for an example of a learning curve.)

更大的训练折意味着单轮估计更乐观地贴近“全数据”性能，只要模型仍能从额外样本里受益；学习曲线（第 5 章图 5.1）展示了这种随训练量变化的偏差。

In practice, both a very small and a very large k may increase variance. For instance, a larger k makes the training folds more similar to each other since a smaller proportion is left for the held-out validation sets. Since the training folds are more similar, the models in each round will be more similar. In practice, we may observe that the variance of the held-out validation fold scores is more similar for larger values of k . On the other hand, when k is large, the validation sets are small, so they may contain more random noise or be more susceptible to quirks of the data, leading to more variation in the validation scores across the different folds. Even though the models themselves are more similar (since the training sets are more similar), the validation scores may be more sensitive to the particulars of the small validation sets, leading to higher variance in the overall cross-validation score.

实务解读要小心：极小或极大的 k 都可能推高“你看到的那条汇总分数”的方差图景——极大 k 时模型彼此酷似，验证折却非常小，单次验证可能对偶然样本极度敏感（尽管模型差异不大）。这也是为何不能只看均值而忽略分数散布。

Determining Appropriate Values for k

把“我们想回答的问题”排到前面：是想逼近最终全数据模型，还是想扫描训练抽样敏感度；顺带提示嵌套交叉验证与留出测试的重要性。



When deciding upon an appropriate value of k , we are often guided by computational performance and conventions. However, it's worthwhile to define the purpose and context of using k -fold cross-validation. For example, if we care primarily about approximating the predictive performance of the final model, using a large k makes sense. This way, the training folds are very similar to the combined training dataset, yet we still get to evaluate the model on all data points via the validation folds.

若想回答“我几乎用满训练样本时模型有多好”，偏大 k 更合拍：训练折高度接近全集，却仍能通过轮换验证让所有点各当一次留出观察。

On the other hand, if we care to evaluate how sensitive a given hyperparameter configuration and training pipeline is to different training datasets, then choosing a smaller number for k makes more sense.

若更关心管线对数据扰动的敏感度，偏小 k 让各轮训练样本差异更明显，更可读出鲁棒画像。

Since most practical scenarios consist of two steps -- tuning hyperparameters and evaluating the performance of a model -- we can also consider a two-step procedure. For instance, we can use a smaller k during hyperparameter tuning. This will help speed up the hyperparameter search and probe the hyperparameter configurations for robustness (in addition to the average performance, we can also consider the variance as a selection criterion). Then, after hyperparameter tuning and selection, we can increase the value of k to evaluate the model.

工程上可把“搜寻超参”（需要快且要扫风险）与“定妆评估”（可以更慢更稳）拆分：前半段用小 k 兼看均值与方差，后半段再以更大 k 复核——但要牢记：若始终啃同一份数据，仍然会掺入选型偏差。

However, reusing the same dataset for model selection and evaluation introduces biases, and it is usually better to use a separate test set for model evaluation. Also, **nested cross-validation** may be preferred as an alternative to k -fold cross-validation.

更稳妥的路线是：**留出独立测试集**专司最终打分，或对“选模型 + 报告泛化”使用嵌套交叉验证（nested cross-validation），以降低把同一噪声多次刷进指标里的风险。（中文读者可参考公开的 nested CV 讲义，例如 <https://ljalphabetagitbooks.io/python/content/nested.html> 的示意流程。）



Exercises

LOOCV 与病态准确率的讨论；以及 k -折除调参评估外的其他用途枚举。

28-1. Suppose we want to provide a model with as much training data as possible. We consider using *leave-one-out cross-validation (LOOCV)*, a special case of k -fold cross-validation where k is equal to the number of training examples, such that the validation folds contain only a single data point. A colleague mentions that LOOCV is defective for discontinuous loss functions and performance measures such as classification accuracy. For instance, for a validation fold consisting of only one example, the accuracy is always either 0 (0 percent) or 1 (100 percent). Is this really a problem?

28-2. This chapter discussed model selection and model evaluation as two use cases of k -fold cross-validation. Can you think of other use cases?

练习题 28-1 引导你辨析：离散指标在小留出上的极端取值如何进入整体方差公式；是否真的“坏”要看你要优化的风险与是否要光滑 surrogate。28-2 则鼓励联想到特征筛选、校准集构造、以及与 bootstrap 组合的汇报策略等延伸用途。

References

模型评估综述与奠定 $k = 5/10$ 常识的 Kohavi (1995) 原始论文入口。

- For a longer and more detailed explanation of why and how to use k -fold cross-validation, see my article: "Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning" (2018), <https://arxiv.org/abs/1811.12808>.
- The paper that popularized the recommendation of choosing $k = 5$ and $k = 10$: Ron Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection" (1995), <https://dl.acm.org/doi/10.5555/1643031.1643047>.



Chapter 29: Training and Test Set Discordance

本章讨论「测试集表现明显优于训练集」时的排查路径：先查代码与评估流程，再分析分布差异与对抗验证，并简述缓解策略与练习。

Suppose we train a model that performs much better on the test dataset than on the training dataset. Since a similar model configuration previously worked well on a similar dataset, we suspect something might be unusual with the data. What are some approaches for looking into training and test set discrepancies, and what strategies can we use to mitigate these issues?

假设我们训练出的模型在测试集上表现远好于训练集；在相似配置曾在相似数据上工作正常的前提下，我们怀疑数据本身异常。可以如何排查训练集与测试集之间的不一致，又有哪些缓解策略？

Before investigating the datasets in more detail, we should check for technical issues in the **data loading** and **evaluation** code. For instance, a simple sanity check is to temporarily replace the test set with the training set and to reevaluate the model. In this case, we should see identical training and test set performances (since these datasets are now identical). If we notice a discrepancy, we likely have a bug in the code; in my experience, such bugs are frequently related to incorrect shuffling or inconsistent (often missing) data normalization.

在进一步细查数据集之前，应先检查**数据加载与评估**代码：例如暂时用训练集顶替测试集再评估，此时训练与测试指标应完全一致；若不一致则多半有bug，常见原因是洗牌不当或归一化前后不一致（甚至遗漏归一化）。

在进一步检查数据集之前，我们应该检查数据加载和评估代码中的技术问题。

- 例如，一个简单的健全性检查是暂时将测试集替换为训练集，并重新评估模型。
- 在这种情况下，我们应该看到训练和测试集的性能相同（因为这些数据集现在相同）。
- 如果我们注意到差异，我们代码可能有bug；根据经验，这种错误，通常是数据洗牌不均匀或数据归一化不一致（通常缺失）。



If the test set performance is much better than the training set performance, we can rule out overfitting. More likely, there are substantial differences in the training and test data distributions. These distributional differences may affect both the features and the targets. Here, plotting the target or label distributions of training and test data is a good idea. For example, a common issue is that the test set is missing certain class labels if the dataset was not shuffled properly before splitting it into training and test data. For small tabular datasets, it is also feasible to compare feature distributions in the training and test sets using histograms.

若测试明显优于训练，可基本排除过拟合，更可能是两集分布差异大（特征与目标均可受影响）。宜画标签/目标分布；若划分前未充分洗牌，测试集可能缺失某些类别。小型表格数据也可用直方图比较特征分布。

Looking at feature distributions is a good approach for tabular data, but this is trickier for image and text data. A relatively easy and more general approach to check for discrepancies between training and test sets is adversarial validation.

对表格数据看特征分布很有效；对图像与文本更难。更通用的一招是对抗验证（adversarial validation）。

Adversarial validation , illustrated in

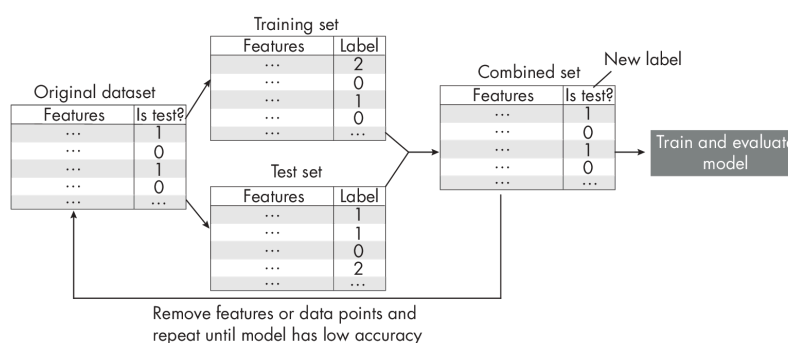


Figure 29.1

is a technique to identify the degree of similarity between the training and test data. We first merge the training and test sets into a single dataset, and then we create a binary target variable that distinguishes between training and test data. For instance, we can use a new *Is test?* label where we assign the label 0 to training data and the label 1 to test data. We then use *k*-fold cross-validation or repartition the dataset into a training set and a test set and train a machine learning model as usual. Ideally, we want the model to perform poorly, indicating that the training and test data distributions are similar. On the other hand, if the



model performs well in predicting the *Is test?* label, it suggests a discrepancy between the training and test data that we need to investigate further.

对抗验证（见图）用于衡量训练与测试数据的相似度：合并两集并构造二元目标（如 **Is test?**：训练为 0、测试为 1），再按 *k-fold* 或重划分训练分类器。理想情况下模型应几乎无法区分来源；若能较好预测「是否来自测试集」，则提示两集分布不一致，需要进一步排查。

对抗验证，用于识别训练和测试数据之间的相似程度。

- 我们首先将训练和测试集合并为一个数据集，然后创建一个二元目标变量，用于区分训练和测试数据。
- 例如，我们可以使用一个新的 *Is test?* 标签，将标签 0 分配给训练数据，将标签 1 分配给测试数据。
- 然后，我们使用 *k-fold* 交叉验证或重新划分数据集为训练集和测试集，并像往常一样训练机器学习模型。
- 理想情况下，我们希望模型表现不佳，表明训练和测试数据分布相似。
- 另一方面，如果模型在预测 *Is test?* 标签时表现良好，则表明训练和测试数据之间存在差异，我们需要进一步调查。

What mitigation techniques should we use if we detect a training-test set discrepancy using adversarial validation? If we're working with a tabular dataset, we can remove features one at a time to see if this helps address the issue, as spurious features can sometimes be highly correlated with the target variable. To implement this strategy, we can use sequential feature selection algorithms with an updated objective. For example, instead of maximizing classification accuracy, we can minimize classification accuracy. For cases where removing features is not so trivial (such as with image and text data), we can also investigate whether removing individual training instances that are different from the test set can address the discrepancy issue.

若对抗验证发现不一致，可尝试：对表格数据逐个删除特征（伪相关特征有时与目标强相关），并用顺序特征选择配合「最小化准确率」等反向目标；对图像/文本则可考虑删除与测试分布差异过大的训练样本。

如果我们使用对抗验证检测到训练-测试集差异，我们应该使用什么缓解技术？如果我们使用表格数据集，我们可以一次删除一个特征，看看是否有助于解决这个问题，因为虚假特征有时与目标变量高度相关。为了实现这

个策略，我们可以使用顺序特征选择算法，并更新目标函数。例如，我们不再最大化分类准确率，而是最小化分类准确率。对于图像和文本数据，我们也可以研究是否删除与测试集不同的训练实例是否有助于解决差异问题。



Exercises

本节两道思考题：对抗预测任务的合理基线；以及训练集远大于测试集时的类别不平衡与应对。

29-1. What is a good performance baseline for the adversarial prediction task?

29-1. 对抗预测任务中，什么样的性能基线是合理的？

29-2. Since training datasets are often bigger than test datasets, adversarial validation often results in an imbalanced prediction problem (with a majority of examples labeled as *Is test?* being false instead of true). Is this an issue, and if so, how can we mitigate that?

29-2. 训练集通常比测试集大，对抗验证会得到类别极不平衡的「是否来自测试集」二分类问题（多数样本为 false）。这是否构成问题？若是，如何缓解？



Chapter 30: Limited Labeled Data

本章系统梳理监督学习中标注数据稀缺时的主要路径（增标、自助式扩充、迁移与自监督、主动学习、小样本与元学习、弱/半监督、自训练、多任务与多模态、归纳偏置等），并给出选用建议、练习与文献。

Suppose we plot a learning curve (as shown in Figure 5.1 on page , for example) and find the machine learning model overfits and could benefit from more training data. What are some different approaches for dealing with limited labeled data in supervised machine learning settings?

若学习曲线（例如 Figure 5.1）显示模型过拟合、仍可从更多数据中受益，在监督学习设定下应对「标注有限」有哪些不同思路？

学习曲线（Learning Curve）是机器学习中用于评估模型性能随训练数据量变化趋势的图表。它通常用于诊断模型是否存在过拟合或欠拟合问题。

In lieu of collecting more data, there are several methods related to regular supervised learning that we can use to improve model performance in limited labeled data regimes.

在无法立即收集更多数据时，仍有一类贴近常规监督学习的方法，用于在标注稀缺时提升模型表现。

除了收集更多数据，还有几种方法，用于改进标签数据有限时的模型性能。

Improving Model Performance with Limited Labeled Data

本节作为总领，引出后续各子节：在训练数据有限时，多种学习范式如何提供帮助。

The following sections explore various machine learning paradigms that help in scenarios where training data is limited.



下文将分节讨论在训练数据有限时有所帮助的各类机器学习范式。

Labeling More Data

本节讨论通过收集更多标注样本直接提升效果，以及其在成本、算力与可及性上的现实制约。

Collecting additional training examples is often the best way to improve the performance of a model (a learning curve is a good diagnostic for this). However, this is often not feasible in practice, because acquiring high-quality data can be costly, computational resources and storage might be insufficient, or the data may be hard to access.

收集更多带标签样本往往是提升模型性能的首选（学习曲线对此是很好的诊断工具）。但实践中常受限于高质量数据昂贵、算力与存储不足，或数据难以获取。

收集更多训练数据，通常是提高模型性能的最佳方法（学习曲线是诊断此问题的一个很好的指标）。

然而，这在实践中通常不可行，因为获取高质量数据可能很昂贵，计算资源和存储空间可能不足，或者数据可能难以获取。

Bootstrapping the Data

本节联系第 5、21 章，说明用增强或合成样本「扩充」数据以及提升数据质量的路径。

Similar to the techniques for reducing overfitting discussed in Chapter [\[ch05\]](#), it can be helpful to "bootstrap" the data by generating modified (augmented) or artificial (synthetic) training examples to boost the performance of the predictive model. Of course, improving the quality of data can also lead to the improved predictive performance of a model, as discussed in Chapter [\[ch21\]](#).

与第 5 章中讨论的缓解过拟合技术类似，可通过生成经修改（增强）或人工（合成）的训练样本来「扩充」数据，以提升预测模型表现。当然，提升数据



质量同样能改善预测性能，见第 21 章。

与第 5 章讨论的减少过拟合的技术类似，可以通过生成修改（增强）或人工（合成）训练示例来“引导”数据，以提高预测模型的性能。

当然，提高数据质量也可以提高模型的预测性能，如第 21 章所述。

Transfer Learning

本节说明在大规模通用数据上预训练再在目标任务上微调的基本流程，并对比深度网络与树模型在可更新参数上的差异。

Transfer learning describes training a model on a general dataset (for example, ImageNet) and then fine-tuning the pretrained target dataset (for example, a dataset consisting of different bird species), as outlined in Figure 30.1.

迁移学习指先在大规模通用数据（如 ImageNet）上训练，再在目标任务数据（如某鸟类细粒度数据集）上微调预训练模型；流程见 Figure 30.1。

迁移学习描述了在通用数据集（例如 ImageNet）上训练模型，然后对预训练的目标数据集（例如包含不同鸟类物种的数据集）进行微调，如图 1.1 所示。

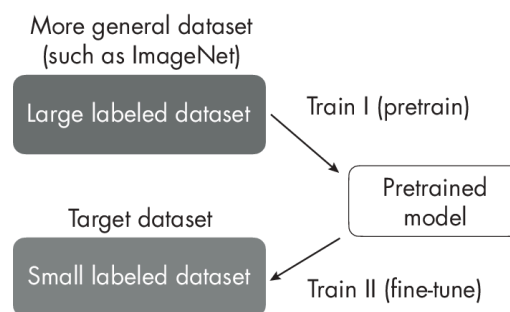


Figure 30.1

Transfer learning is usually done in the context of deep learning, where model weights can be updated. This is in contrast to tree-based methods, since most decision tree algorithms are nonparametric models that do not support iterative training or parameter updates.



迁移学习通常出现在深度学习中，以便迭代更新权重。相对地，多数树模型为非参数、难以像神经网络那样不断迭代更新参数。

迁移学习，通常用于深度学习场景，其中可以更新模型权重。这与基于树的方法形成对比，因为大多数决策树算法是非参数模型，不支持迭代训练或参数更新。

Self-Supervised Learning

本节说明自监督如何从数据本身构造辅助任务，及其与「无监督预训练」称谓的关系，并给出语言与视觉上的典型例子及章节引用。

Similar to transfer learning, in self-supervised learning, the model is pretrained on a different task before being fine-tuned to a target task for which only limited data exists. However, self-supervised learning usually relies on label information that can be directly and automatically extracted from unlabeled data. Hence, self-supervised learning is also often called **unsupervised pretraining**.

与迁移学习类似，自监督学习先在另一任务上预训练，再对仅有少量标注的目标任务微调。不同之处在于辅助标签通常可直接从无标注数据中自动构造，因此也常被称为**无监督预训练**。

与迁移学习类似，在自监督学习中，模型在不同的任务上进行预训练，然后针对目标任务进行微调，而目标任务只有有限的数据。

然而，自监督学习通常依赖于可以直接从无标签数据中自动提取的标签信息。因此，自监督学习也经常被称为**无监督预训练**。

Common examples of self-supervised learning include the *next word* (used in GPT, for example) or *masked word* (used in BERT, for example) pretraining tasks in language modeling, covered in more detail in Chapter [\[ch17\]](#). Another intuitive example from computer vision includes *inpainting*: predicting the missing part of an image that was randomly removed, illustrated in Figure [30.2](#).

自监督常见例包括语言模型中的 *next word*（如 GPT）或 *masked word*（如 BERT）预训练任务，详见第 17 章；在视觉上，*inpainting*（预测被随机挖去的图像区域）亦很典型，见 Figure [30.2](#)。

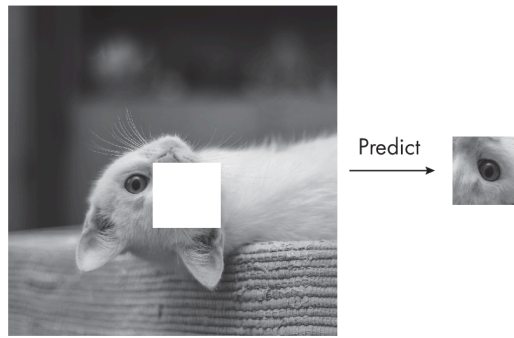


Figure 30.2

For more detail on self-supervised learning, see Chapter [\[ch02\]](#).

更多自监督学习背景见第 2 章。

Active Learning

本节说明主动学习如何通过优先级策略挑选待标注点，并借助 oracle 在有限标注预算下提升模型。

In active learning, illustrated in Figure [30.3](#), we typically involve manual labelers or users for feedback during the learning process. However, instead of labeling the entire dataset up front, active learning includes a prioritization scheme for suggesting unlabeled data points for labeling to maximize the machine learning model's performance.

在主动学习（Figure [30.3](#)）中，训练过程会引入标注者或用户反馈；但并非一次性标全量数据，而是按优先级挑选最值得标注的未标注点，以最大化模型性能。

在主动学习中，如图 1.3 所示，我们通常涉及手动标签器或用户在训练过程中提供反馈。

然而，与提前标记整个数据集不同，主动学习包括一个优先级方案，用于建议未标记的数据点进行标记，以最大化机器学习模型的性能。

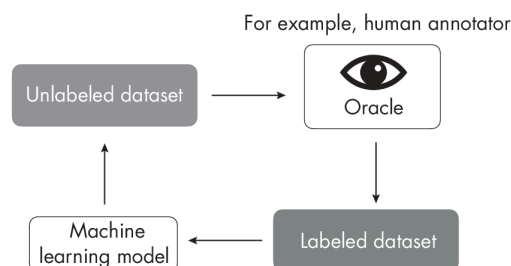


Figure 30.3

The term **active learning** refers to the fact that the model actively selects data for labeling. For example, the simplest form of active learning selects data points with high prediction uncertainty for labeling by a human annotator (also referred to as an *oracle*).

主动学习强调模型主动挑选待标注数据。例如最简策略是优先标注模型预测不确定度高的样本，由人工标注者（亦称 *oracle*）完成标注。

Few-Shot Learning

本节概述 **few-shot**、**zero-shot** 及基于提示的大模型示例，并指向第 3 章延伸阅读。

In a **few-shot** learning scenario, we often deal with extremely small datasets that include only a handful of examples per class. In research contexts, 1-shot(one example per class) and 5-shot (five examples per class) learning scenarios are very common. An extreme case of few-shot learning is **zero-shot** learning, where no labels are provided. Popular examples of zero-shot learning include GPT-3 and related language models, where the user has to provide all the necessary information via the input prompt, as illustrated in Figure 30.4.

在 **few-shot** 设置下，每类往往只有极少样本；研究中常见 1-shot、5-shot。**zero-shot** 更进一步，不提供目标任务标注，仅靠提示词与模型先验；GPT-3 等即是一例，见 Figure 30.4。

在少样本学习场景中，我们通常处理包含每个类别只有少量示例的极端情况。

在研究上下文中，1-shot（每个类别一个示例）和5-shot（每个类别五个示例）学习场景非常常见。



小样本学习的极端情况，是**零样本学习**，其中没有提供标签。

零样本学习的流行示例包括 GPT-3 和相关语言模型，其中用户必须通过输入提示提供所有必要信息，如图 1.4 所示。

Zero-shot classification with ChatGPT

For more detail on few-shot learning, see Chapter [\[ch03\]](#).

少样本学习的更多细节见第 3 章。

Meta-Learning

本节区分「学会学习」式的元学习与从数据集提取 meta-features 的另一分支，并说明其与少样本学习及特征表示的关系。

Meta-learning involves developing methods that determine how machine learning algorithms can best learn from data. We can therefore think of meta-learning as "learning to learn." The machine learning community has developed several approaches for meta-learning. Within the machine learning community, the term *meta-learning* doesn't just represent multiple subcategories and approaches; it is also occasionally employed to describe related yet distinct processes, leading to nuances in its interpretation and application.

元学习（meta-learning）研究如何设计方法，使机器学习算法更有效地从数据中学习，可概括为「学会学习」。社区内该术语涵盖多类子方向，有时也用于相近却不尽相同的流程，因而解读上存在细微差别。

Meta-learning is one of the main subcategories of few-shot learning. Here, the focus is on learning a good feature extraction module, which converts support and query images into vector representations. These vector representations are optimized for determining the predicted class of the query example via comparisons with the training examples in the support set. (This form of meta-learning is illustrated in Chapter [\[ch03\]](#) on page .) Another branch of meta-learning unrelated to the few-shot learning approach is focused on extracting metadata (also called *meta-features*) from datasets for supervised learning tasks, as illustrated in Figure [30.5](#). The meta-features are descriptions of the dataset



itself. For example, these can include the number of features and statistics of the different features (kurtosis, range, mean, and so on).

元学习是少样本学习的主要子类之一：此处重点学习良好的特征提取模块，把 support/query 图像映射为向量，并通过与支持集样本比较来预测 query 类别（见第 3 章）。另一支与少样本路线无关的元学习则关注从数据集中抽取 *meta-features* 以辅助监督学习任务，见 Figure 30.5；meta-features 描述数据集自身，例如特征个数与各特征的峰度、范围、均值等统计量。

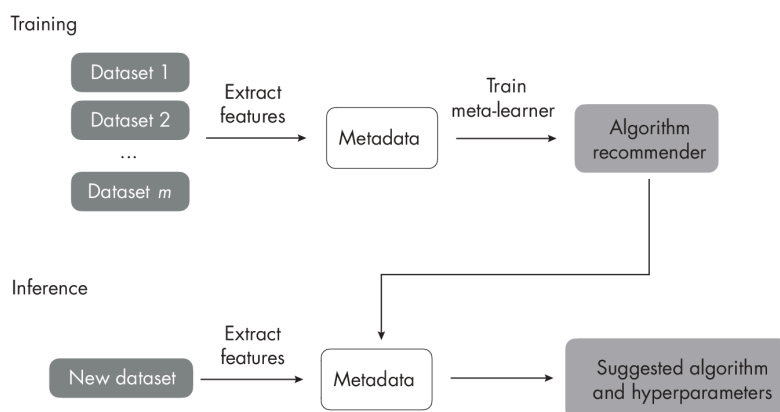


Figure 30.5

The extracted meta-features provide information for selecting a machine learning algorithm for the dataset at hand. Using this approach, we can narrow down the algorithm and hyperparameter search spaces, which helps reduce overfitting when the dataset is small.

抽取的 meta-features 可为当前数据集选择合适算法提供依据，从而缩小算法与超参搜索空间，在小数据上也有助于抑制过拟合。

Weakly Supervised Learning

本节说明弱监督如何借助外部、往往含噪的标注函数为无标签数据生成标签，并引出 PU-learning 子类。

Weakly supervised learning, illustrated in Figure 30.6, involves using an external label source to generate labels for an unlabeled dataset. Often, the labels created by a weakly supervised labeling function are more noisy or inaccurate than those produced by a human or domain expert, hence the term *weakly* supervised. We can develop or adopt a rule-based classifier to create the labels in weakly



supervised learning; these rules usually cover only a subset of the unlabeled dataset.

弱监督学习（Figure 30.6）用外部标签源为无标注数据生成标签；此类标签往往比人工或专家标注更噪或不完全，故称为 *weakly supervised*。可用规则分类器覆盖无标签数据的子集来产生标签。

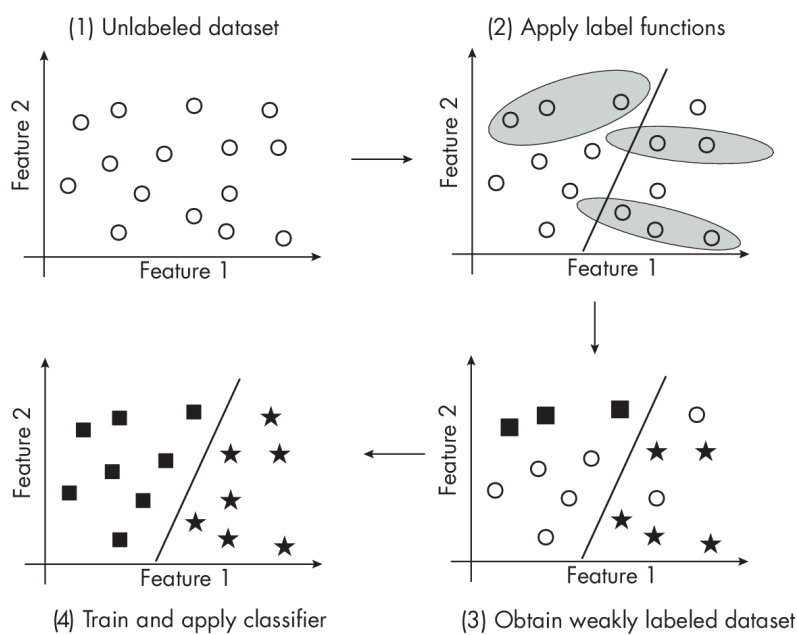


Figure 30.6

Let's return to the example of email spam classification from Chapter [ch23] to illustrate a rule-based approach for data labeling. In weak supervision, we could design a rule-based classifier based on the keyword *SALE* in the email subject header line to identify a subset of spam emails. Note that while we may use this rule to label certain emails as spam positive, we should not apply this rule to label emails without *SALE* as non-spam. Instead, we should either leave those unlabeled or apply a different rule to them.

回到第 23 章的垃圾邮件例子：可用主题含 *SALE* 的规则把一部分邮件标为垃圾邮件；但不能据此把不含 *SALE* 的邮件一律标为非垃圾，而应留空或换用其他规则。

There is a subcategory of weakly supervised learning referred to as PU-learning. In *PU-learning*, which is short for *positive-unlabeled learning*, we label and learn only from positive examples.

弱监督的一个子类是 PU-learning (*positive-unlabeled learning*)：仅在正例与未标注数据上学习与标注。



Semi-Supervised Learning

本节对比半监督与弱监督在「如何造标签」上的差异，并说明二者可组合使用的情形。

Semi-supervised learning is closely related to weakly supervised learning: it also involves creating labels for unlabeled instances in the dataset. The main difference between these two methods lies in *how* we create the labels. In weak supervision, we create labels using an external labeling function that is often noisy, inaccurate, or covers only a subset of the data. In semi-supervision, we do not use an external label function; instead, we leverage the structure of the data itself. We can, for example, label additional data points based on the density of neighboring labeled data points, as illustrated in Figure 30.7.

半监督与弱监督都涉及为未标注样本造标签，差别在于如何造：弱监督依赖外部、常含噪且未必全覆盖的标注函数；半监督则利用数据自身结构（例如依据已标注邻域密度）为更多点赋伪标签，见 Figure 30.7。

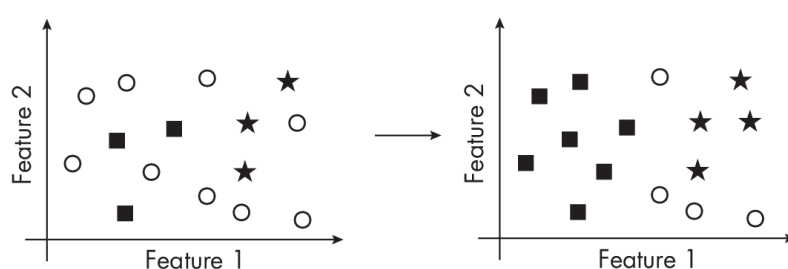


Figure 30.7

While we can apply weak supervision to an entirely unlabeled dataset, semi-supervised learning requires at least a portion of the data to be labeled. In practice, it is possible first to apply weak supervision to label a subset of the data and then to use semi-supervised learning to label instances that were not captured by the labeling functions.

弱监督可施于完全无标签数据；半监督则至少需要一部分已标注样本。实践中可先弱监督标出一部分，再对规则未覆盖的样本用半监督补标。

Thanks to their close relationship, semi-supervised learning is sometimes referred to as a subcategory of weakly supervised learning, and vice versa.

二者关系密切，文献中有时把半监督视为弱监督的子类，反之亦然。



Self-Training

本节说明自训练如何用（伪）模型为数据打伪标签，及其与弱监督、半监督的联系，并关联知识蒸馏。

Self-training falls somewhere between semi-supervised learning and weakly supervised learning. For this technique, we train a model to label the dataset or adopt an existing model to do the same. This model is also referred to as a *pseudo-labeler*.

自训练介于半监督与弱监督之间：先训练或选用一个模型为数据集打伪标签，该模型亦称 *pseudo-labeler*。

Self-training does not guarantee accurate labels and is thus related to weakly supervised learning. Moreover, while we use or adopt a machine learning model for this pseudo-labeling, self-training is also related to semi-supervised learning.

伪标签未必准确，因而与弱监督相关；同时又借助机器学习模型生成标签，故也与半监督相近。

An example of self-training is knowledge distillation, discussed in Chapter [\[ch06\]](#).

自训练的一例是第 6 章讨论的知识蒸馏。

Multi-Task Learning

本节介绍多任务学习中的辅助任务、硬/软参数共享及其作为归纳偏置的作用。

Multi-task learning trains neural networks on multiple, ideally related tasks. For example, if we are training a classifier to detect spam emails, spam classification is the main task. In multi-task learning, we can add one or more related tasks for the model to solve, referred to as *auxiliary tasks*. For the spam email example, an auxiliary task could be classifying the email's topic or language.

多任务学习在多个（最好相关的）任务上联合训练网络。例如垃圾邮件检测为主任务时，可把邮件主题或语种分类等作为 *auxiliary tasks*（辅助任务）。



Typically, multi-task learning is implemented via multiple loss functions that have to be optimized simultaneously, with one loss function for each task. The auxiliary tasks serve as an inductive bias, guiding the model to prioritize hypotheses that can explain multiple tasks. This approach often results in models that perform better on unseen data. There are two subcategories of multi-task learning: multi-task learning with hard parameter sharing and multi-task learning with soft parameter sharing. Figure 30.8 illustrates the difference between these two methods.

实现上通常为每个任务各设一个损失并同时优化；辅助任务提供归纳偏置，使模型偏好能同时解释多任务的假设，常改善未见数据上的表现。多任务又分硬参数共享与软参数共享，见 Figure 30.8。

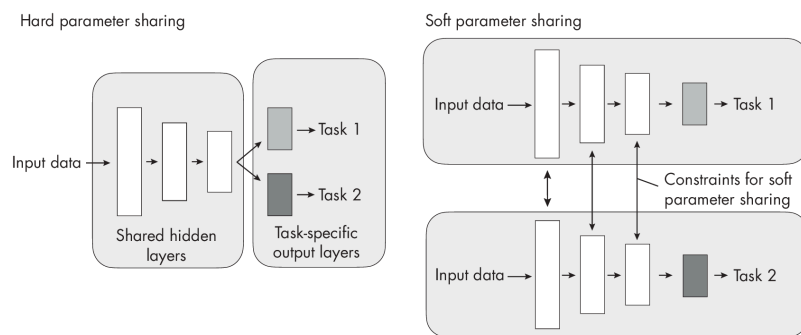


Figure 30.8

In *hard* parameter sharing, as shown in Figure 30.8, only the output layers are task specific, while all the tasks share the same hidden layers and neural network backbone architecture. In contrast, *soft* parameter sharing uses separate neural networks for each task, but regularization techniques such as distance minimization between parameter layers are applied to encourage similarity among the networks.

Hard 参数共享中各任务共享隐藏层与骨干，仅输出头任务相关；*soft* 参数共享则为每任务各用一套网络，并通过层间距离正则等鼓励参数相似。

Multimodal Learning

本节区分多任务与多模态输入，介绍匹配损失与联合编码器/单模块（如 VideoBERT）等做法，并讨论直接优化下游目标的情形。



While multi-task learning involves training a model with multiple tasks and loss functions, multimodal learning focuses on incorporating multiple types of input data.

多任务强调多损失、多任务；多模态则强调融合多种输入模态。

Common examples of multimodal learning are architectures that take both image and text data as input (though multimodal learning is not restricted to only two modalities and can be used for any number of input modalities). Depending on the task, we may employ a matching loss that forces the embedding vectors between related images and text to be similar, as shown in Figure 30.9. (See Chapter [ch01] for more on embedding vectors.)

常见做法是同时输入图像与文本（也可多于两种模态）；可用匹配损失拉近相关图文嵌入，见 Figure 30.9（嵌入概念见第 1 章）。

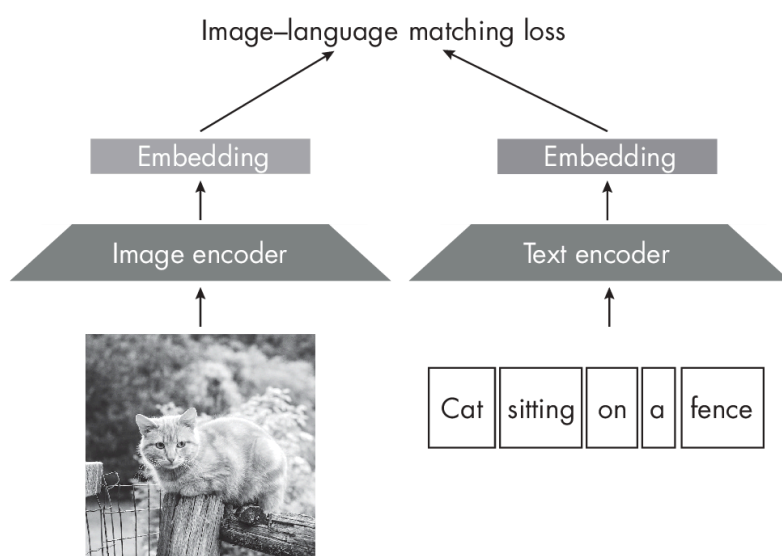


Figure 30.9

Figure 30.9 shows image and text encoders as separate components. The image encoder can be a convolutional backbone or a vision transformer, and the language encoder can be a recurrent neural network or language transformer. However, it's common nowadays to use a single transformer-based module that can simultaneously process image and text data. For example, the VideoBERT model has a joint module that processes both video and text for action classification and video captioning.

图中将图像编码器与文本编码器分开；图像侧可用 CNN 或 ViT，语言侧可用 RNN 或语言 Transformer。如今更常见单一 Transformer 同时处理图文；



VideoBERT 即用联合模块同时处理视频与文本以做动作分类与视频描述。

Optimizing a matching loss, as shown in Figure 30.9, can be useful for learning embeddings that can be applied to various tasks, such as image classification or summarization. However, it is also possible to directly optimize the target loss, like classification or regression, as Figure 30.10 illustrates.

优化匹配损失可学到通用嵌入，用于分类、摘要等多种下游；也可如图 Figure 30.10 所示，直接优化分类或回归等监督目标。

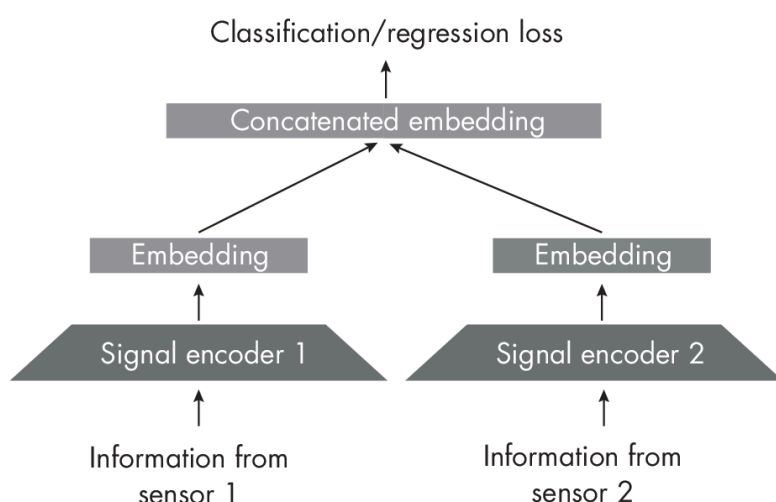


Figure 30.10

Figure 30.10 shows data being collected from two different sensors. One could be a thermometer and the other could be a video camera. The signal encoders convert the information into embeddings (sharing the same number of dimensions), which are then concatenated to form the input representation for the model.

Figure 30.10 展示来自不同传感器（如温度计与摄像头）的数据：各模态经编码器映射到同维嵌入后拼接，作为模型输入表示。

Intuitively, models that combine data from different modalities generally perform better than unimodal models because they can leverage more information. Moreover, recent research suggests that the key to the success of multimodal learning is the improved quality of the latent space representation.

直觉上，融合多模态通常优于单模态，因为可利用更多信息；近期工作认为多模态成功的关键在于潜空间表示质量的提升。



Inductive Biases

本节说明更强归纳偏置（如 CNN 相对 ViT）如何降低数据需求，并指向第 13 章。

Choosing models with stronger inductive biases can help lower data requirements by making assumptions about the structure of the data. For example, due to their inductive biases, convolutional networks require less data than vision transformers, as discussed in Chapter [\[ch13\]](#).

选择归纳偏置更强的模型，可通过对数据结构的先验假设降低样本需求；例如 CNN 通常比 ViT 更省数据，见第 13 章。

Recommendations

本节用决策图（Figure 30.11）概括在多种技术之间如何按情境取舍，并提醒与过拟合相关章节的衔接。

Of all these techniques for reducing data requirements, how should we decide which ones to use in a given situation?

在诸多降低数据需求的技术中，应如何按情境选择？

Techniques like collecting more data, data augmentation, and feature engineering are compatible with all the methods discussed in this chapter. Multi-task learning and multimodal inputs can also be used with the learning strategies outlined here. If the model suffers from overfitting, we should also include techniques discussed in Chapters [\[ch05\]](#) and [\[ch06\]](#).

收集更多数据、数据增强与特征工程可与本章各范式组合；多任务与多模态输入亦可并用。若仍存在过拟合，还应结合第 5、6 章的技术。

But how can we choose between active learning, few-shot learning, transfer learning, self-supervised learning, semi-supervised learning, and weakly supervised learning? Deciding which supervised learning technique(s) to try is highly context dependent. You can use the diagram in Figure [30.11](#) as a guide to choosing the best method for your particular project.



在主动学习、少样本、迁移、自监督、半监督与弱监督之间如何选，高度依赖上下文；可用 Figure 30.11 的流程图作项目内导航。

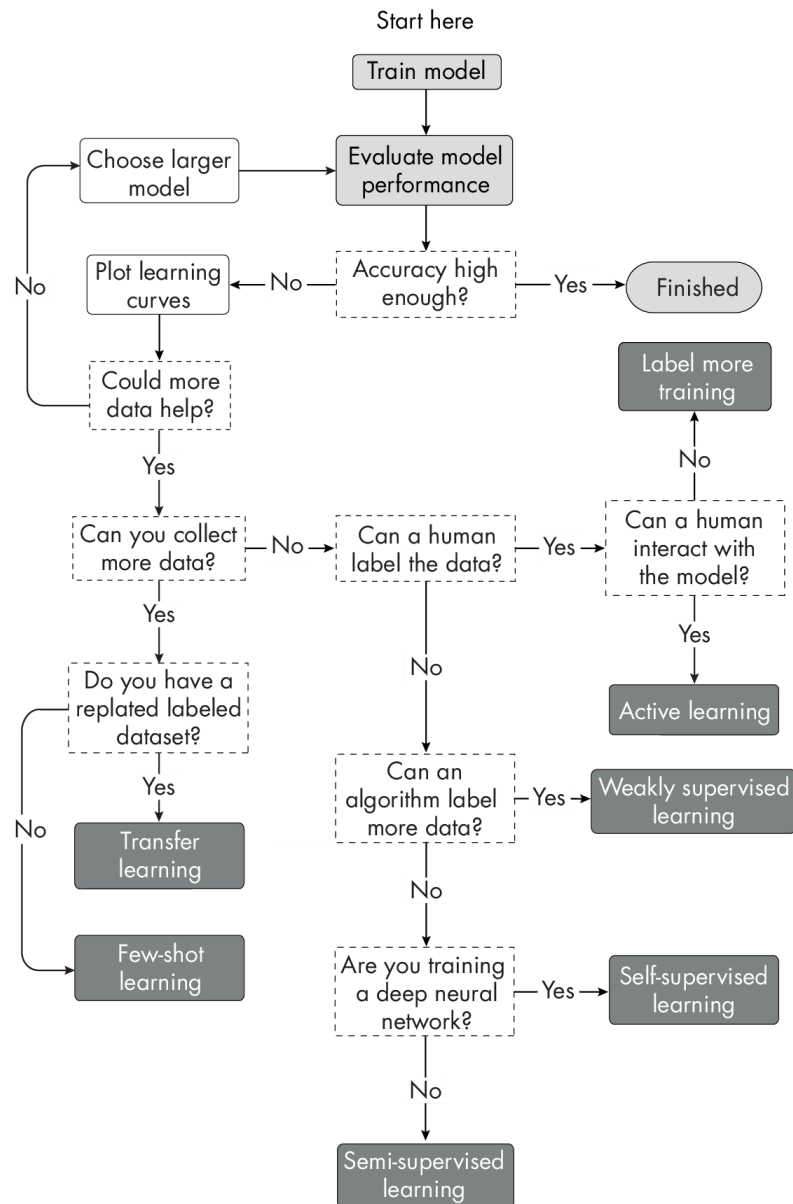


Figure 30.11

Note that the dark boxes in Figure 30.11 are not terminal nodes but arc back to the second box, "Evaluate model performance"; additional arrows were omitted to avoid visual clutter.

注意 Figure 30.11 中深色框并非终端，而是回连到「Evaluate model performance」等节点；为简洁省略了部分箭头。

Exercises

本节为两道综合题：制造缺陷检测场景下自监督/迁移的用法，以及主动学习中在深度网络过自信时替代置信度的思路。



30-1. Suppose we are given the task of constructing a machine learning model that utilizes images to detect manufacturing defects on the outer shells of tablet devices similar to iPads. We have access to millions of images of various computing devices, including smartphones, tablets, and computers, which are not labeled; thousands of labeled pictures of smartphones depicting various types of damage; and hundreds of labeled images specifically related to the target task of detecting manufacturing defects on tablet devices. How could we approach this problem using self-supervised learning or transfer learning?

30-1. 假设要用图像检测类 iPad 平板外壳制造缺陷：有数百万未标注的各型设备图、数千张带标注的手机损伤图、以及数百张与平板缺陷直接相关的标注图。如何用自监督学习或迁移学习来建模？

30-2. In active learning, selecting difficult examples for human inspection and labeling is often based on confidence scores. Neural networks can provide such scores by using the logistic sigmoid or softmax function in the output layer to calculate class-membership probabilities. However, it is widely recognized that deep neural networks exhibit overconfidence on out-of-distribution data, rendering their use in active learning ineffective. What are some other methods to obtain confidence scores using deep neural networks for active learning?

30-2. 主动学习常按置信度挑选难例；深度网络可用 sigmoid/softmax 给出类概率，但在分布外数据上往往过自信，使该策略失效。还有哪些为深度网络获取「置信度」以用于主动学习的方法？

References

本节列出增量决策树、多任务学习、VideoBERT、多模态表示理论、主动学习综述与过自信相关论文链接。

- While decision trees for incremental learning are not commonly implemented, algorithms for training decision trees in an iterative fashion do exist:
https://en.wikipedia.org/wiki/Incremental_decision_tree.



- 增量训练决策树的算法条目：
https://en.wikipedia.org/wiki/Incremental_decision_tree。
- Models trained with multi-task learning often outperform models trained on a single task: Rich Caruana, "Multitask Learning"? (1997),
<https://doi.org/10.1023%2FA%3A1007379606734>.
- 多任务学习经典论文 Caruana (1997):
<https://doi.org/10.1023%2FA%3A1007379606734>。
- A single transformer-based module that can simultaneously process image and text data: Chen Sun et al., "VideoBERT: A Joint Model for Video and Language Representation Learning"? (2019), <https://arxiv.org/abs/1904.01766>.
- VideoBERT 联合视频与文本 (2019): <https://arxiv.org/abs/1904.01766>。
- The aforementioned research suggesting the key to the success of multimodal learning is the improved quality of the latent space representation: Yu Huang et al., "What Makes Multi-Modal Learning Better Than Single (Provably)"? (2021), <https://arxiv.org/abs/2106.04538>.
- 多模态优于单模态的表示视角 Huang 等 (2021):
<https://arxiv.org/abs/2106.04538>。
- For more information on active learning: Zhen et al., "A Comparative Survey of Deep Active Learning"? (2022), <https://arxiv.org/abs/2203.13450>.
- 深度主动学习综述 (2022): <https://arxiv.org/abs/2203.13450>。
- For a more detailed discussion on how out-of-distribution data can lead to overconfidence in deep neural networks: Anh Nguyen, Jason Yosinski, and Jeff Clune, "Deep Neural Networks Are Easily Fooled: High Confidence Predictions for Unrecognizable Images"? (2014), <https://arxiv.org/abs/1412.1897>.
- 深度网络对不可识别图像给出高置信度 (2014):
<https://arxiv.org/abs/1412.1897>。

